



TRADINGPRO

Bilag 4 - Kodeindeksering

Ozan Korkmaz

Vejleder: Flemming Sørensen

1205hf26086per

03-09-2026

Indholdsfortegnelse

1. Formål	3
2. Omfang	3
4. Komplet backend-kode	7
5. Komplet frontend-kode.....	92
6. Komplet testkode	196

1. Formål

Dette bilag gør det muligt at finde den faktiske implementation bag hovedrapportens forklaringer. Tekniske nøgleord og kodeidentifikatorer er naturligvis ikke oversat, fordi de er en del af programmeringssproget eller bibliotekets API.

2. Omfang

Del	Kildekodefiler	Indhold
Backend	91	C#, controllers, services, repositories, DTO, models, hubs, background/runtime
Frontend	47	React/TypeScript, components, services, hooks og types
Tests	10	xUnit testklasser og test helpers

3.1 Backend - filindeks

Fil	Kort forklaring
TradingApp/Controllers/AiAnalysisController.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
TradingApp/Controllers/AlertController.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
TradingApp/Controllers/AuthController.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
TradingApp/Controllers/CandlesController.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
TradingApp/Controllers/ChartDrawingsController.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
TradingApp/Controllers/ExchangesController.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
TradingApp/Controllers/SymbolsController.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
TradingApp/Controllers/SystemHealthController.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
TradingApp/Controllers/UsersController.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
TradingApp/Controllers/WatchlistsController.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
TradingApp/Program.cs	Applikationens startpunkt. Registrerer dependency injection, authentication/authorization, database, controllers, SignalR og background services og bygger HTTP-pipelinen.
TradingAppLibrary/BackgroundServices/HistoricalSyncBackgroundService.cs	Behandler historical-sync jobs fra køen, eksempelvis når et nyt symbol aktiveres efter applikationsstart.
TradingAppLibrary/BackgroundServices/MarketDataSyncBackgroundService.cs	Kører marked-data synkronisering i baggrunden og koordinerer initial historical sync samt realtime streams for aktive exchanges.
TradingAppLibrary/Constants/ExchangeCodes.cs	Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.
TradingAppLibrary/DTO/AiAnalysisDtos.cs	DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.
TradingAppLibrary/DTO/AlertDtos.cs	DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.
TradingAppLibrary/DTO/AuthDtos.cs	DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.
TradingAppLibrary/DTO/CandleDtos.cs	DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.
TradingAppLibrary/DTO/ChartDrawingDtos.cs	DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.
TradingAppLibrary/DTO/ExchangeDtos.cs	DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.
TradingAppLibrary/DTO/ForgotPasswordDto.cs	DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.
TradingAppLibrary/DTO/ResetPasswordDto.cs	DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.
TradingAppLibrary/DTO/SymbolDtos.cs	DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.
TradingAppLibrary/DTO/SystemHealthDtos.cs	DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.
TradingAppLibrary/DTO/UserPreferencesDto.cs	DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

Fil	Kort forklaring
TradingAppLibrary/DTO/WatchlistDtos.cs	DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.
TradingAppLibrary/Data/AppDbContext.cs	EF Core DbContext. Samler TradingPros entiteter og konfigurerer relationer, nøgler, filtre og centrale database-regler.
TradingAppLibrary/Data/DbInitializer.cs	Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.
TradingAppLibrary/Data/PasswordHasher.cs	Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.
TradingAppLibrary/Enums/UserRole.cs	Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.
TradingAppLibrary/Hubs/MarketDataHub.cs	SignalR Hub for realtime data. Organiserer browserklienter i bruger- og symbolgrupper, så events kan sendes målrettet.
TradingAppLibrary/Interfaces/IAiAnalysisService.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/IAuthService.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/ICandleRepository.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/ICandleService.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/ICandleValidator.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/IChartDrawingService.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/IEmailService.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/IExchangeService.cs	Fælles kontrakt for exchange-adaptore. Binance og OKX implementerer samme operationer, så resten af backend kan arbejde exchange-uafhængigt.
TradingAppLibrary/Interfaces/IExchangeServiceFactory.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/IHistoricalSyncQueue.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/IMarketDataRuntimeState.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/IMarketDataStreamControl.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/IMarketDataSyncService.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/ISymbolRepository.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/ISymbolService.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/ISystemHealthService.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/ITokenService.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/IUserRepository.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/IUserService.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/IWatchlistRepository.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Interfaces/IWatchlistService.cs	Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.
TradingAppLibrary/Mappings/AlertMappingExtensions.cs	Mapping extensions, som konverterer mellem database-entiteter og DTO'er på en ensartet måde.
TradingAppLibrary/Mappings/CandleMappingExtensions.cs	Mapping extensions, som konverterer mellem database-entiteter og DTO'er på en ensartet måde.
TradingAppLibrary/Mappings/SymbolMappingExtensions.cs	Mapping extensions, som konverterer mellem database-entiteter og DTO'er på en ensartet måde.
TradingAppLibrary/Mappings/UserMappingExtensions.cs	Mapping extensions, som konverterer mellem database-entiteter og DTO'er på en ensartet måde.
TradingAppLibrary/Mappings/WatchlistMappingExtensions.cs	Mapping extensions, som konverterer mellem database-entiteter og DTO'er på en ensartet måde.
TradingAppLibrary/Models/Alert.cs	Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.
TradingAppLibrary/Models/Candle.cs	Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.
TradingAppLibrary/Models/ChartDrawing.cs	Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.

Fil	Kort forklaring
TradingAppLibrary/Models/EmailSettings.cs	Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.
TradingAppLibrary/Models/Exchange.cs	Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.
TradingAppLibrary/Models/JwtSettings.cs	Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.
TradingAppLibrary/Models/Symbol.cs	Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.
TradingAppLibrary/Models/User.cs	Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.
TradingAppLibrary/Models/UserPreferences.cs	Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.
TradingAppLibrary/Models/Watchlist.cs	Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.
TradingAppLibrary/Models/WatchlistItem.cs	Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.
TradingAppLibrary/Repositories/CandleRepository.cs	Databaseadgang for candles. Håndterer 1m-tabellen, TimescaleDB views for større timeframes, endTime-pagination og candle-opdateringer.
TradingAppLibrary/Repositories/SymbolRepository.cs	Repository-implementation med databaseoperationer for området. Holder databaseadgang adskilt fra controllers og forretningslogik.
TradingAppLibrary/Repositories/UserRepository.cs	Repository-implementation med databaseoperationer for området. Holder databaseadgang adskilt fra controllers og forretningslogik.
TradingAppLibrary/Repositories/WatchlistRepository.cs	Repository-implementation med databaseoperationer for området. Holder databaseadgang adskilt fra controllers og forretningslogik.
TradingAppLibrary/Runtime/HistoricalSyncQueue.cs	Runtime-state, queue eller stream-control, som hjælper background services med at koordinere market-data arbejde.
TradingAppLibrary/Runtime/MarketDataRuntimeState.cs	Runtime-state, queue eller stream-control, som hjælper background services med at koordinere market-data arbejde.
TradingAppLibrary/Runtime/MarketDataStreamControl.cs	Runtime-state, queue eller stream-control, som hjælper background services med at koordinere market-data arbejde.
TradingAppLibrary/Services/AiAnalysisService.cs	Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.
TradingAppLibrary/Services/AuthService.cs	Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.
TradingAppLibrary/Services/BinanceService.cs	Adapter til Binance REST- og WebSocket-API. Oversætter symbols og candles til TradingPros fælles DTO/modeller.
TradingAppLibrary/Services/CandleService.cs	Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.
TradingAppLibrary/Services/ChartDrawingService.cs	Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.
TradingAppLibrary/Services/ExchangeServiceFactory.cs	Finder den korrekte IExchangeService ud fra exchange-koden og holder exchange-valget ude af den øvrige forretningslogik.
TradingAppLibrary/Services/MarketDataSyncService.cs	Central market-data service for symbol-sync, historical download, realtime behandling, validering, databasegemning og SignalR-publicering.
TradingAppLibrary/Services/OkxService.cs	Adapter til OKX REST- og WebSocket-API. Håndterer OKX-specifikke endpoints og symbolformat og oversætter dem til fælles modeller.
TradingAppLibrary/Services/SmtEmailService.cs	Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.
TradingAppLibrary/Services/SymbolService.cs	Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.
TradingAppLibrary/Services/SystemHealthService.cs	Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.
TradingAppLibrary/Services/TokenService.cs	Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.
TradingAppLibrary/Services/UserService.cs	Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.
TradingAppLibrary/Services/WatchlistService.cs	Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.
TradingAppLibrary/Validation/CandleValidationResult.cs	Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.
TradingAppLibrary/Validators/CandleValidator.cs	Valideringsklasse, som kontrollerer market-data før de accepteres og gemmes.

3.2 Frontend - filindeks

Fil	Kort forklaring
src/App.tsx	Frontendens overordnede React-komponent. Samler navigation, valgt exchange/symbol, paneler og centrale states.

Fil	Kort forklaring
src/Components/AIAnalysisPanel.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Components/Chart/CandlestickChart.tsx	Den centrale chart-komponent. Viser OHLC-data og håndterer historical loading, live-opdateringer og chart-interaktion.
src/Components/Chart/DrawingLayer.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Components/ChartPane.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Components/ChartSettingsModal.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Components/Common/ProtectedRoute.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Components/CreateAlertModal.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Components/HeaderNavbar.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Components/HelpCenterModal.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Components/Indicators/IndicatorsModal.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Components/LanguageSwitcher.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Components/ProfileModal.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Components/TriggeredAlertToast.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Components/Watchlist.tsx	React-komponent for en afgrænset del af brugergrænsefladen.
src/Context/ThemeContext.tsx	Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.
src/Pages/AdminPanel.tsx	Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.
src/Pages/AuthPage.tsx	Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.
src/Services/aiApi.ts	Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.
src/Services/alertService.ts	Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.
src/Services/api.ts	Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.
src/Services/authService.ts	Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.
src/Services/candleApi.ts	Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.
src/Services/chartDrawingService.ts	Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.
src/Services/exchangeApi.ts	Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.
src/Services/preferenceService.ts	Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.
src/Services/signalrService.ts	Frontendens SignalR-klient. Håndterer forbindelse, reconnect, symbolgrupper og reference counting mellem eksempelvis chart og watchlist.
src/Services/symbolApi.ts	Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.
src/Services/systemHealthService.ts	Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.
src/Services/userService.ts	Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.
src/Services/watchlistService.ts	Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.
src/Types/ai.ts	TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.
src/Types/alert.ts	TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.
src/Types/auth.ts	TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.
src/Types/candle.ts	TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.
src/Types/chartDrawing.ts	TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.
src/Types/chartSettings.ts	TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.
src/Types/indicator.ts	TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.
src/Types/preferences.ts	TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.
src/Types/symbol.ts	TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.
src/Types/systemHealth.ts	TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.

Fil	Kort forklaring
src/Types/watchlist.ts	TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.
src/Utils/indicatorCalculators.ts	Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.
src/hooks/useAlerts.ts	Genbrugelig React hook, der samler state og side effects for denne funktion.
src/hooks/useUserPreferences.ts	Genbrugelig React hook, der samler state og side effects for denne funktion.
src/i18n.ts	Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.
src/main.tsx	Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.

3.3 Tests - filindeks

Fil	Kort forklaring
Controllers/CandlesControllerTests.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
Controllers/ExchangesControllerTests.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
Controllers/SymbolsControllerTests.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
Controllers/WatchlistsControllerTests.cs	ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.
Helpers/FakeExchangeServiceFactory.cs	Test-double for exchange factory. Gør det muligt at kontrollere exchange-relateret controllerlogik uden at kontakte Binance eller OKX.
Helpers/TestDbContextFactory.cs	Opretter en isoleret EF Core InMemory-database for hver test. Det gør testene uafhængige af den rigtige PostgreSQL/TimescaleDB-database.
Repositories/CandleRepositoryTests.cs	Tester repository-logik for candle-rækkefølge, lazy loading med endTime og update-vs-duplicate adfærd.
Repositories/SymbolRepositoryTests.cs	Tester normalisering, aktive symbols, exchange-filtrering og case-insensitive symbolopslag.
Repositories/UserRepositoryTests.cs	Tester login-opslag, soft delete, preferences-upsert og filtrering af slettede brugere.
Repositories/WatchlistRepositoryTests.cs	Tester indlæsning af watchlist-relationer, fjernelse af enkelte symbols og sletning af tilhørende items.

4. Komplet backend-kode

B1. TradingApp/Controllers/AiAnalysisController.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingApp.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class AiAnalysisController : ControllerBase
    {
        private readonly IAiAnalysisService _aiAnalysisService;

        public AiAnalysisController(IAiAnalysisService aiAnalysisService)
        {
            _aiAnalysisService = aiAnalysisService;
        }

        /// <summary>
        /// Opretter et AI-baseret teknisk analyseresumé ud fra det valgte symbol og interval.
        /// POST: api/aianalysis/analyze
        /// </summary>
        [HttpPost("analyze")]
        public async Task<ActionResult<AiAnalysisResponseDto>> AnalyzeMarket([FromBody] AiAnalysisRequestDto request)
        {
            if (!ModelState.IsValid)
            {
                return BadRequest(ModelState);
            }

            var result = await _aiAnalysisService.AnalyzeMarketAsync(request);
            return Ok(result);
        }
    }
}
```

```
}  
}
```

B2. TradingApp/Controllers/AlertController.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```
using Microsoft.AspNetCore.Authorization;  
using Microsoft.AspNetCore.Mvc;  
using System.Security.Claims;  
using TradingAppLibrary.DTO;  
using TradingAppLibrary.Interfaces;  
  
namespace TradingApp.Controllers;  
  
[Route("api/alerts")]  
[ApiController]  
[Authorize]  
public class AlertController : ControllerBase  
{  
    private readonly ICandleService _candleService;  
  
    public AlertController(ICandleService candleService)  
    {  
        _candleService = candleService;  
    }  
  
    // Henter user id.  
    private string GetUserId()  
    {  
        return User.FindFirst(ClaimTypes.NameIdentifier)?.Value  
            ?? User.FindFirst("id")?.Value  
            ?? User.FindFirst("sub")?.Value  
            ?? string.Empty;  
    }  
  
    /// <summary>  
    /// Henter alle alarmer for den bruger, der er logget ind.  
    /// GET: api/alerts  
    /// </summary>  
    [HttpGet]  
    public async Task<ActionResult<IEnumerable<AlertResponseDto>>> GetMyAlerts()  
    {  
        var userId = GetUserId();  
  
        if (string.IsNullOrEmpty(userId))  
            return Unauthorized(new { message = "Yetkisiz işlem." });  
  
        return Ok(await _candleService.GetUserAlertsAsync(userId));  
    }  
  
    /// <summary>  
    /// Henter aktive alarmer for et Symbolld.  
    /// GET: api/alerts/active?symbolld=10  
    /// </summary>  
    [HttpGet("active")]  
    public async Task<ActionResult<IEnumerable<AlertResponseDto>>> GetActiveAlertsBySymbol(  
        [FromQuery] int symbolld)  
    {  
        if (symbolld <= 0)  
            return BadRequest(new { message = "Geçerli bir symbolld zorunludur." });  
  
        return Ok(await _candleService.GetActiveAlertsBySymbolAsync(symbolld));  
    }  
  
    /// <summary>  
    /// Opretter en ny prisalarm.  
    /// POST: api/alerts  
    /// </summary>  
    [HttpPost]  
    public async Task<ActionResult<AlertResponseDto>> CreateAlert(  
        [FromBody] CreateAlertDto dto)  
    {  
        var userId = GetUserId();  
  
        if (string.IsNullOrEmpty(userId))  
            return Unauthorized(new { message = "Yetkisiz işlem." });  
  
        if (dto == null || dto.Symbolld <= 0 || dto.TargetPrice <= 0)  
        {  

```



```

        return BadRequest(new
        {
            message = "Geçerli symbolld ve hedef fiyat zorunludur."
        });
    }

    try
    {
        var createdAlert = await _candleService.CreateAlertAsync(userId, dto);

        return CreatedAtAction(
            nameof(GetMyAlerts),
            new { id = createdAlert.Id },
            createdAlert);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

/// <summary>
/// Sletter alarmer permanent.
/// DELETE: api/alerts/{id}
/// </summary>
[HttpDelete("{id}")]
public async Task<ActionResult> DeleteAlert(string id)
{
    var userId = GetUserId();

    if (string.IsNullOrEmpty(userId))
        return Unauthorized(new { message = "Yetkisiz işlem." });

    var success = await _candleService.DeleteAlertAsync(id, userId);

    if (!success)
    {
        return NotFound(new
        {
            message = "Alarm bulunamadı veya bu işlem için yetkiniz yok."
        });
    }

    return Ok(new { message = "Alarm başarıyla silindi." });
}

/// <summary>
/// Aktiverer eller deaktiverer alarmer.
/// PATCH: api/alerts/{id}/toggle
/// </summary>
[HttpPatch("{id}/toggle")]
public async Task<ActionResult> ToggleAlert(string id)
{
    var userId = GetUserId();

    if (string.IsNullOrEmpty(userId))
        return Unauthorized(new { message = "Yetkisiz işlem." });

    var success = await _candleService.ToggleAlertAsync(id, userId);

    if (!success)
    {
        return NotFound(new
        {
            message = "Alarm bulunamadı veya güncellenemedi."
        });
    }

    return Ok(new { message = "Alarm durumu başarıyla güncellendi." });
}
}

```

B3. TradingApp/Controllers/AuthController.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```

using Microsoft.AspNetCore.Mvc;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

```

```
namespace TradingApp.Controllers;

[Route("api/[controller]")]
[ApiController]
public class AuthController : ControllerBase
{
    private readonly IAuthService _authService;

    public AuthController(IAuthService authService)
    {
        _authService = authService;
    }

    [HttpPost("register")]
    // Opretter en ny bruger efter validering af de indsendte oplysninger.
    public async Task<IActionResult> Register([FromBody] RegisterDto request)
    {
        if (!ModelState.IsValid) return BadRequest(ModelState);
        try
        {
            var result = await _authService.RegisterAsync(request);
            return result == null
                ? BadRequest(new { message = "Kayıt işlemi gerçekleştirilemedi." })
                : Ok(result);
        }
        catch (InvalidOperationException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
        catch
        {
            return StatusCode(500, new { message = "Kayıt sırasında beklenmeyen bir sunucu hatası oluştu." });
        }
    }

    [HttpPost("login")]
    // Validerer loginoplysninger og opretter et token for en gyldig bruger.
    public async Task<IActionResult> Login([FromBody] LoginDto request)
    {
        if (!ModelState.IsValid) return BadRequest(ModelState);
        try
        {
            var result = await _authService.LoginAsync(request);
            return result == null
                ? Unauthorized(new { message = "Geçersiz kullanıcı adı/e-posta veya şifre." })
                : Ok(result);
        }
        catch (InvalidOperationException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
    }

    [HttpPost("forgot-password")]
    // Behandler forgot password.
    public async Task<IActionResult> ForgotPassword([FromBody] ForgotPasswordDto request, CancellationToken cancellationToken)
    {
        await _authService.RequestPasswordResetAsync(request, cancellationToken);

        // Vi afslører ikke, om kontoen findes.
        return Ok(new
        {
            message = "Bu bilgilerle eşleşen bir hesap varsa şifre sıfırlama bağlantısı gönderildi."
        });
    }

    [HttpPost("reset-password")]
    // Validerer reset-tokenet og gemmer brugerens nye password.
    public async Task<IActionResult> ResetPassword([FromBody] ResetPasswordDto request, CancellationToken cancellationToken)
    {
        var success = await _authService.ResetPasswordAsync(request, cancellationToken);
        if (!success)
        {
            return BadRequest(new
            {
                message = "Şifre sıfırlama bağlantısı geçersiz veya süresi dolmuş. Lütfen yeni bir bağlantı isteyin."
            });
        }

        return Ok(new { message = "Şifreniz başarıyla güncellendi. Yeni şifrenizle giriş yapabilirsiniz." });
    }
}
```

B4. TradingApp/Controllers/CandlesController.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingApp.Controllers;

[Route("api/[controller]")]
[ApiController]
[Authorize]
public class CandlesController : ControllerBase
{
    private readonly ICandleService _candleService;

    public CandlesController(ICandleService candleService)
    {
        _candleService = candleService;
    }

    /// <summary>
    /// Henter candle-data ud fra SymbolId.
    /// GET: api/candles?symbolId=10&interval=1h&limit=100&endTime=1710000000000
    /// </summary>
    [HttpGet]
    public async Task<ActionResult<IEnumerable<CandleResponseDto>>> GetCandles(
        [FromQuery] int symbolId,
        [FromQuery] string interval = "1m",
        [FromQuery] int limit = 1000,
        [FromQuery] long? endTime = null)
    {
        if (symbolId <= 0)
            return BadRequest(new { message = "Geçerli bir symbolId zorunludur." });

        if (string.IsNullOrEmpty(interval))
            return BadRequest(new { message = "Interval parametresi zorunludur." });

        limit = Math.Clamp(limit, 1, 1000);

        var candles = await _candleService.GetCandlesAsync(
            symbolId,
            interval,
            limit,
            endTime);

        return Ok(candles);
    }
}
```

B5. TradingApp/Controllers/ChartDrawingsController.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingApp.Controllers;

[ApiController]
[Route("api/chart-drawings")]
[Authorize]
public class ChartDrawingsController : ControllerBase
{
    private readonly IChartDrawingService _chartDrawingService;

    public ChartDrawingsController(IChartDrawingService chartDrawingService)
    {
        _chartDrawingService = chartDrawingService;
    }

    [HttpGet]
    // Henter den relevante operation.
}
```

```
public async Task<ActionResult<IReadOnlyList<ChartDrawingResponseDto>>> Get(
    [FromQuery] int symbolId,
    [FromQuery] string interval,
    CancellationToken cancellationToken)
{
    if (symbolId <= 0 || string.IsNullOrEmpty(interval))
        return BadRequest(new { message = "symbolId ve interval zorunludur." });

    return Ok(await _chartDrawingService.GetAsync(
        GetCurrentUser(), symbolId, interval, cancellationToken));
}

[HttpPost]
// Opretter den relevante operation.
public async Task<ActionResult<ChartDrawingResponseDto>> Create(
    [FromBody] CreateChartDrawingDto dto,
    CancellationToken cancellationToken)
{
    try
    {
        var created = await _chartDrawingService.CreateAsync(
            GetCurrentUser(), dto, cancellationToken);

        return CreatedAtAction(nameof(Get), new { symbolId = created.SymbolId, interval = created.Interval }, created);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return NotFound(new { message = ex.Message });
    }
}

[HttpPut("{drawingId:long}")]
// Opdaterer den relevante operation.
public async Task<ActionResult<ChartDrawingResponseDto>> Update(
    long drawingId,
    [FromBody] UpdateChartDrawingDto dto,
    CancellationToken cancellationToken)
{
    try
    {
        var updated = await _chartDrawingService.UpdateAsync(
            GetCurrentUser(), drawingId, dto, cancellationToken);

        return updated == null
            ? NotFound(new { message = "Çizim bulunamadı." })
            : Ok(updated);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

[HttpDelete("{drawingId:long}")]
// Sletter den relevante operation.
public async Task<ActionResult> Delete(long drawingId, CancellationToken cancellationToken)
{
    var deleted = await _chartDrawingService.DeleteAsync(
        GetCurrentUser(), drawingId, cancellationToken);

    return deleted
        ? NoContent()
        : NotFound(new { message = "Çizim bulunamadı." });
}

[HttpDelete]
// Sletter all.
public async Task<ActionResult> DeleteAll(
    [FromQuery] int symbolId,
    [FromQuery] string interval,
    CancellationToken cancellationToken)
{
    if (symbolId <= 0 || string.IsNullOrEmpty(interval))
        return BadRequest(new { message = "symbolId ve interval zorunludur." });

    var deletedCount = await _chartDrawingService.DeleteAllAsync(
        GetCurrentUser(), symbolId, interval, cancellationToken);
}
```

```

        return Ok(new { deletedCount });
    }

    // Henter current user id.
    private int GetCurrentUserId()
    {
        var userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier)?.Value
            ?? User.FindFirst("sub")?.Value
            ?? User.FindFirst("id")?.Value;

        if (string.IsNullOrEmpty(userIdClaim) || !int.TryParse(userIdClaim, out var userId))
            throw new UnauthorizedAccessException("Geçersiz veya bulunamayan kullanıcı kimliği.");

        return userId;
    }
}

```

B6. TradingApp/Controllers/ExchangesController.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using TradingAppLibrary.Data;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Models;

namespace TradingApp.Controllers;

[Route("api/[controller]")]
[ApiController]
public class ExchangesController : ControllerBase
{
    private readonly AppDbContext _dbContext;
    private readonly IExchangeServiceFactory _exchangeServiceFactory;

    public ExchangesController(
        AppDbContext dbContext,
        IExchangeServiceFactory exchangeServiceFactory)
    {
        _dbContext = dbContext;
        _exchangeServiceFactory = exchangeServiceFactory;
    }

    /// <summary>
    /// Offentlig terminalliste: kun aktive exchanges.
    /// GET: api/exchanges
    /// </summary>
    [HttpGet]
    public async Task<ActionResult<IEnumerable<ExchangeResponseDto>>> GetActiveExchanges(
        CancellationToken cancellationToken)
    {
        return Ok(await QueryExchanges()
            .Where(e => e.IsActive)
            .ToListAsync(cancellationToken));
    }

    /// <summary>
    /// Administrationsliste: aktive og inaktive exchanges.
    /// GET: api/exchanges/admin
    /// </summary>
    [HttpGet("admin")]
    [Authorize(Roles = "Admin,SuperAdmin")]
    public async Task<ActionResult<IEnumerable<ExchangeResponseDto>>> GetAllExchanges(
        CancellationToken cancellationToken)
    {
        return Ok(await QueryExchanges().ToListAsync(cancellationToken));
    }

    /// <summary>
    /// Opretter kun en exchange i databasen, når der findes en tilsvarende backend-adapter.
    /// POST: api/exchanges
    /// </summary>
    [HttpPost]
    [Authorize(Roles = "Admin,SuperAdmin")]
    public async Task<ActionResult<ExchangeResponseDto>> CreateExchange(
        [FromBody] CreateExchangeDto dto,
        CancellationToken cancellationToken)
    {

```

```
var code = NormalizeCode(dto.Code);
var name = dto.Name?.Trim() ?? string.Empty;

if (string.IsNullOrEmpty(code) || string.IsNullOrEmpty(name))
    return BadRequest(new { message = "Exchange code and name are required." });

if (code.Length > 20 || name.Length > 50)
    return BadRequest(new { message = "Exchange code or name is too long." });

if (!HasIntegration(code, out var integrationError))
    return BadRequest(new { message = integrationError });

if (await _dbContext.Exchanges.AnyAsync(e => e.Code == code, cancellationTokens))
    return Conflict(new { message = $"Exchange code '{code}' already exists." });

var exchange = new Exchange
{
    Code = code,
    Name = name,
    IsActive = dto.IsActive
};

_dbContext.Exchanges.Add(exchange);
await _dbContext.SaveChangesAsync(cancellationTokens);

var result = ToDto(exchange);
return CreatedAtAction(nameof(GetAllExchanges), new { id = exchange.Id }, result);
}

/// <summary>
/// Opdaterer exchange-navn og status. Ændring af kode blokeres, når der allerede findes symboler.
/// PUT: api/exchanges/{id}
/// </summary>
[HttpPut("{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin")]
public async Task<ActionResult<ExchangeResponseDto>> UpdateExchange(
    int id,
    [FromBody] UpdateExchangeDto dto,
    CancellationToken cancellationToken)
{
    var exchange = await _dbContext.Exchanges
        .FirstOrDefaultAsync(e => e.Id == id, cancellationToken);

    if (exchange == null)
        return NotFound(new { message = "Exchange not found." });

    var code = NormalizeCode(dto.Code);
    var name = dto.Name?.Trim() ?? string.Empty;

    if (string.IsNullOrEmpty(code) || string.IsNullOrEmpty(name))
        return BadRequest(new { message = "Exchange code and name are required." });

    if (!HasIntegration(code, out var integrationError))
        return BadRequest(new { message = integrationError });

    if (!exchange.Code.Equals(code, StringComparison.OrdinalIgnoreCase))
    {
        var hasSymbols = await _dbContext.Symbols
            .AnyAsync(s => s.ExchangeId == exchange.Id, cancellationToken);

        if (hasSymbols)
        {
            return Conflict(new
            {
                message = "Exchange code cannot be changed after symbols have been synchronized. Create a new exchange instead."
            });
        }

        if (await _dbContext.Exchanges.AnyAsync(e => e.Id != id && e.Code == code, cancellationTokens))
            return Conflict(new { message = $"Exchange code '{code}' already exists." });

        exchange.Code = code;
    }

    exchange.Name = name;
    exchange.IsActive = dto.IsActive;
    await _dbContext.SaveChangesAsync(cancellationTokens);

    return Ok(ToDto(exchange));
}
```

```

/// <summary>
/// Sletter kun en exchange permanent, når den ikke har tilknyttede symboler.
/// Exchanges med markedshistorik bør deaktiveres i stedet for at blive slettet.
/// DELETE: api/exchanges/{id}
/// </summary>
[HttpDelete("{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin")]
public async Task<ActionResult> DeleteExchange(int id, CancellationToken cancellationToken)
{
    var exchange = await _dbContext.Exchanges
        .FirstOrDefaultAsync(e => e.Id == id, cancellationToken);

    if (exchange == null)
        return NotFound(new { message = "Exchange not found." });

    var hasSymbols = await _dbContext.Symbols
        .AnyAsync(s => s.ExchangeId == id, cancellationToken);

    if (hasSymbols)
    {
        return Conflict(new
        {
            message = "This exchange already has symbols/history and cannot be deleted safely. Set it inactive instead."
        });
    }

    _dbContext.Exchanges.Remove(exchange);
    await _dbContext.SaveChangesAsync(cancellationToken);
    return NoContent();
}

// Behandler query exchanges.
private IQueryable<ExchangeResponseDto> QueryExchanges() =>
    _dbContext.Exchanges
        .AsNoTracking()
        .OrderBy(e => e.Name)
        .Select(e => new ExchangeResponseDto(e.Id, e.Code, e.Name, e.IsActive));

// Kontrollerer om integration.
private bool HasIntegration(string code, out string? error)
{
    try
    {
        _exchangeServiceFactory.GetExchange(code);
        error = null;
        return true;
    }
    catch (NotSupportedException)
    {
        error = $"No backend market-data adapter is registered for '{code}'.";
        return false;
    }
}

// Normaliserer code.
private static string NormalizeCode(string? code) =>
    (code ?? string.Empty).Trim().ToUpperInvariant();

// Behandler to dto.
private static ExchangeResponseDto ToDto(Exchange exchange) =>
    new(exchange.Id, exchange.Code, exchange.Name, exchange.IsActive);
}

```

B7. TradingApp/Controllers/SymbolsController.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingApp.Controllers;

[Route("api/[controller]")]
[ApiController]
public class SymbolsController : ControllerBase
{
    private readonly ISymbolService _symbolService;

```

```
private readonly IMarketDataSyncService _marketDataSyncService;

public SymbolsController(
    ISymbolService symbolService,
    IMarketDataSyncService marketDataSyncService)
{
    _symbolService = symbolService;
    _marketDataSyncService = marketDataSyncService;
}

/// <summary>
/// Henter alle symboler eller de symboler, der matcher query-parameteren.
/// Hvis exchangeId angives, søges der kun på den pågældende exchange.
/// GET: api/symbols?query=BTC&exchangeId=1
/// </summary>
[HttpGet]
public async Task<ActionResult<IEnumerable<SymbolResponseDto>>> GetSymbols(
    [FromQuery] string? query = null,
    [FromQuery] int? exchangeId = null)
{
    if (exchangeId.HasValue && exchangeId.Value <= 0)
        return BadRequest(new { message = "Geçersiz exchangeId." });

    if (exchangeId.HasValue)
    {
        var result = string.IsNullOrEmpty(query)
            ? await _symbolService.GetSymbolsByExchangeAsync(exchangeId.Value)
            : await _symbolService.SearchSymbolsAsync(exchangeId.Value, query);

        return Ok(result);
    }

    var symbols = string.IsNullOrEmpty(query)
        ? await _symbolService.GetAllSymbolsAsync()
        : await _symbolService.SearchSymbolsAsync(query);

    return Ok(symbols);
}

/// <summary>
/// Henter ét symbol ud fra SymbolId.
/// GET: api/symbols/10
/// </summary>
[HttpGet("{symbolId:int}")]
public async Task<ActionResult<SymbolResponseDto>> GetSymbol(int symbolId)
{
    var symbol = await _symbolService.GetSymbolByIdAsync(symbolId);

    if (symbol == null)
        return NotFound(new { message = "Sembol bulunamadı." });

    return Ok(symbol);
}

/// <summary>
/// Synkroniserer symbolerne for den angivne exchange med det eksterne API.
/// POST: api/symbols/sync/BINANCE
/// </summary>
[HttpPost("sync/{exchangeCode}")]
[Authorize(Roles = "Admin,SuperAdmin")]
public async Task<ActionResult> SyncSymbols(
    string exchangeCode,
    CancellationToken cancellationToken)
{
    if (string.IsNullOrEmpty(exchangeCode))
        return BadRequest(new { message = "Exchange kodu zorunludur." });

    await _marketDataSyncService.SyncExchangeSymbolsAsync(
        exchangeCode.Trim().ToUpperInvariant(),
        cancellationToken);

    return Ok(new
    {
        message = $"{{exchangeCode.Trim().ToUpperInvariant()}} sembolleri başarıyla senkronize edildi."
    });
}

/// <summary>
/// Henter de aktive symboler på den angivne exchange.
/// GET: api/symbols/active?exchangeId=1
/// </summary>
```



```

[HttpGet("active")]
public async Task<ActionResult<IEnumerable<SymbolResponseDto>>> GetActiveSymbols(
    [FromQuery] int exchangeId)
{
    if (exchangeId <= 0)
        return BadRequest(new { message = "Geçerli bir exchangeId zorunludur." });

    return Ok(await _symbolService.GetActiveSymbolsAsync(exchangeId));
}

/// <summary>
/// Opdaterer aktiv/inaktiv-status ud fra SymbolId.
/// PUT: api/symbols/10/status
/// </summary>
[HttpPut("{symbolId:int}/status")]
[Authorize(Roles = "Admin,SuperAdmin")]
public async Task<ActionResult> ToggleStatus(
    int symbolId,
    [FromBody] ToggleStatusDto dto)
{
    var result = await _symbolService.UpdateSymbolStatusAsync(
        symbolId,
        dto.IsActive);

    if (result == null)
        return NotFound(new { message = "Sembol bulunamadı." });

    var message = result.IsActive && result.HistoricalSyncQueued
        ? "Sembol aktif edildi. Geçmiş veriler arka planda senkronize ediliyor."
        : result.IsActive
            ? "Sembol aktif durumda."
            : "Sembol pasif edildi ve realtime aboneliği güncelleniyor.";

    return Ok(new
    {
        message,
        symbolId = result.SymbolId,
        exchangeCode = result.ExchangeCode,
        symbolName = result.SymbolName,
        isActive = result.IsActive,
        historicalSyncQueued = result.HistoricalSyncQueued,
        realtimeRestartRequested = result.RealtimeRestartRequested
    });
}
}

// Skifter status for status dto.
public record ToggleStatusDto(bool IsActive);

```

B8. TradingApp/Controllers/SystemHealthController.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingApp.Controllers;

[ApiController]
[Route("api/admin/system-health")]
[Authorize(Roles = "Admin,SuperAdmin")]
public sealed class SystemHealthController : ControllerBase
{
    private readonly ISystemHealthService _systemHealthService;

    public SystemHealthController(ISystemHealthService systemHealthService)
    {
        _systemHealthService = systemHealthService;
    }

    [HttpGet]
    [ProducesResponseType(typeof(SystemHealthDto), StatusCodes.Status200OK)]
    // Henter den relevante operation.
    public async Task<ActionResult<SystemHealthDto>> Get(
        CancellationToken cancellationToken)
    {
        var health = await _systemHealthService.GetHealthAsync(cancellationToken);
        return Ok(health);
    }
}

```

```
}  
}
```

B9. TradingApp/Controllers/UsersController.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```
using Microsoft.AspNetCore.Authorization;  
using Microsoft.AspNetCore.Http;  
using Microsoft.AspNetCore.Mvc;  
using System.Security.Claims;  
using TradingAppLibrary.DTO;  
using TradingAppLibrary.Enums;  
using TradingAppLibrary.Interfaces;  
using TradingAppLibrary.Mappings;  
using TradingAppLibrary.Models;  
  
namespace TradingApp.Controllers  
{  
    [Route("api/[controller]")]  
    [ApiController]  
    public class UsersController : ControllerBase  
    {  
        private readonly IUserService _userService;  
        private readonly IAuthService _authService;  
  
        public UsersController(IUserService userService, IAuthService authService)  
        {  
            _userService = userService;  
            _authService = authService;  
        }  
  
        /// <summary>  
        /// Den bruger, der er logget ind, henter sine egne profiloplysninger.  
        /// GET: api/users/me  
        /// </summary>  
        [HttpGet("me")]  
        [Authorize] // Alle brugere, der er logget ind, har adgang (User, Admin, SuperAdmin).  
        public async Task<ActionResult<UserManagementDto>> GetMyProfile()  
        {  
            int userId = GetCurrentUserFromClaims();  
  
            var user = await _userService.GetUserByIdAsync(userId);  
            if (user == null)  
                return NotFound(new { message = "Kullanıcı bulunamadı." });  
  
            return Ok(user.ToDto());  
        }  
  
        /// <summary>  
        /// Den bruger, der er logget ind, opdaterer sine egne profiloplysninger (fornavn, efternavn og e-mail).  
        /// PUT: api/users/me  
        /// </summary>  
        [HttpPut("me")]  
        [Authorize]  
        public async Task<ActionResult<UserManagementDto>> UpdateMyProfile([FromBody] UserManagementDto dto)  
        {  
            if (dto == null || string.IsNullOrEmpty(dto.Email))  
            {  
                return BadRequest(new { message = "E-posta alanı zorunludur." });  
            }  
  
            int userId = GetCurrentUserFromClaims();  
  
            var user = await _userService.GetUserByIdAsync(userId);  
            if (user == null)  
                return NotFound(new { message = "Kullanıcı bulunamadı." });  
  
            user.FirstName = dto.FirstName?.Trim() ?? "";  
            user.LastName = dto.LastName?.Trim() ?? "";  
            user.Email = dto.Email.Trim().ToLower();  
  
            await _userService.UpdateUserAsync(user);  
  
            return Ok(user.ToDto());  
        }  
  
        /// <summary>  
        /// Den bruger, der er logget ind, angiver en ny adgangskode efter validering af den nuværende adgangskode.  
        /// POST: api/users/change-password
```

```
/// </summary>
[HttpPost("change-password")]
[Authorize]
public async Task<ActionResult> ChangePassword([FromBody] ChangePasswordDto dto)
{
    if (dto == null || string.IsNullOrEmpty(dto.CurrentPassword) || string.IsNullOrEmpty(dto.NewPassword))
    {
        return BadRequest(new { message = "Mevcut şifre ve yeni şifre boş bırakılamaz." });
    }

    if (dto.NewPassword.Length < 6)
    {
        return BadRequest(new { message = "Yeni şifre en az 6 karakter olmalıdır." });
    }

    int userId = GetCurrentUserFromClaims();

    bool isSuccess = await _userService.ChangePasswordAsync(userId, dto.CurrentPassword, dto.NewPassword);
    if (!isSuccess)
    {
        return BadRequest(new { message = "Mevcut şifreniz hatalı veya geçersiz." });
    }

    return Ok(new { message = "Şifreniz başarıyla değiştirildi." });
}

/// <summary>
/// Henter de gemte chart- og brugerfladeindstillinger for den bruger, der er logget ind.
/// GET: api/users/preferences
/// </summary>
[HttpGet("preferences")]
[Authorize]
public async Task<ActionResult> GetPreferences()
{
    int userId = GetCurrentUserFromClaims();

    var preferences = await _userService.GetUserPreferencesAsync(userId.ToString());
    return Ok(preferences);
}

/// <summary>
/// Gemmer eller opdaterer brugerens aktuelle chart- og brugerfladeindstillinger (upsert).
/// PUT: api/users/preferences
/// </summary>
[HttpPut("preferences")]
[Authorize]
public async Task<ActionResult> SavePreferences([FromBody] UserPreferencesDto dto)
{
    if (dto == null)
    {
        return BadRequest(new { message = "Tercih verisi boş olamaz." });
    }

    int userId = GetCurrentUserFromClaims();

    await _userService.SaveUserPreferencesAsync(userId.ToString(), dto);
    return Ok(new { message = "Kullanıcı tercihleri başarıyla güncellendi." });
}

/// <summary>
/// Viser alle brugere (kun Admin og SuperAdmin).
/// GET: api/users
/// </summary>
[HttpGet]
[Authorize(Roles = "Admin,SuperAdmin")]
public async Task<ActionResult<IEnumerable<UserManagementDto>>> GetAllUsers()
{
    var users = await _userService.GetAllUsersAsync();

    return Ok(users.ToList());
}

/// <summary>
/// Opdaterer brugerens rolle (kun SuperAdmin).
/// PUT: api/users/assign-role
/// </summary>
[HttpPut("assign-role")]
[Authorize(Roles = nameof(UserRole.SuperAdmin))]
public async Task<ActionResult> AssignRole([FromBody] UserManagementDto request)
{
    var user = await _userService.GetUserByIdAsync(request.Id);
    if (user == null)
```

```
        return NotFound(new { message = "Kullanıcı bulunamadı." });

    if (!Enum.TryParse<UserRole>(request.Role, true, out var parsedRole) || !Enum.IsDefined(typeof(UserRole), parsedRole))
    {
        return BadRequest(new { message = "Geçersiz rol türü. Geçerli roller: User, Admin, SuperAdmin" });
    }

    user.Role = parsedRole.ToString();
    await _userService.UpdateUserAsync(user);

    return Ok(new { message = $"{user.Username} kullanıcısının yeni rolü '{user.Role}' olarak güncellendi." });
}

/// <summary>
/// Læser den aktive brugers id fra HTTP-contexten (JWT-claim).
/// </summary>
private int GetCurrentUserIdFromClaims()
{
    var userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier)?.Value
        ?? User.FindFirst("sub")?.Value
        ?? User.FindFirst("id")?.Value;

    if (string.IsNullOrEmpty(userIdClaim) || !int.TryParse(userIdClaim, out int userId))
    {
        throw new UnauthorizedAccessException("Geçersiz veya bulunamayan kullanıcı kimliği.");
    }

    return userId;
}

/// <summary>
/// Den bruger, der er logget ind, markerer sin egen konto som slettet (soft delete).
/// DELETE: api/users/me
/// </summary>
[HttpDelete("me")]
[Authorize]
public async Task<ActionResult> DeleteMyAccount()
{
    int userId = GetCurrentUserIdFromClaims();

    bool isSuccess = await _authService.RequestAccountDeletionAsync(userId);
    if (!isSuccess)
    {
        return NotFound(new { message = "Kullanıcı bulunamadı veya hesap zaten silinmiş." });
    }

    return Ok(new
    {
        message = "Hesabınız başarıyla silinmek üzere işaretlendi. 30 gün içinde tekrar giriş yaparsanız hesabınız otomatik olarak kurtarılacaktır."
    });
}

/// <summary>
/// Sletter brugeren permanent fra databasen (kun SuperAdmin).
/// DELETE: api/users/{id}
/// </summary>
[HttpDelete("{id:int}")]
[Authorize(Roles = "Admin, SuperAdmin")]
public async Task<ActionResult> HardDeleteUser(int id)
{
    int currentUserId = GetCurrentUserIdFromClaims();

    if (id == currentUserId)
    {
        return BadRequest(new { message = "Kendi hesabınızı bu panelden silemezsiniz." });
    }

    try
    {
        bool isDeleted = await _userService.HardDeleteUserAsync(id);
        if (!isDeleted)
        {
            return NotFound(new { message = "Silinmek istenen kullanıcı bulunamadı." });
        }

        return Ok(new { message = "Kullanıcı veritabanından kalıcı olarak silindi." });
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}
```

```
        catch (Exception)
        {
            return StatusCode(500, new { message = "Kullanıcı silinirken bir sunucu hatası oluştu." });
        }
    }
}
```

B10. TradingApp/Controllers/WatchlistsController.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingApp.Controllers;

[Route("api/[controller]")]
[ApiController]
[Authorize]
public class WatchlistsController : ControllerBase
{
    private readonly IWatchlistService _watchlistService;

    public WatchlistsController(IWatchlistService watchlistService)
    {
        _watchlistService = watchlistService;
    }

    /// <summary>
    /// Henter watchlists for den bruger, der er logget ind.
    /// GET: api/watchlists
    /// </summary>
    [HttpGet]
    public async Task<ActionResult<IEnumerable<WatchlistResponseDto>>> GetMyWatchlists()
    {
        var userId = GetCurrentUserid();
        return Ok(await _watchlistService.GetUserWatchlistsAsync(userId));
    }

    /// <summary>
    /// Opretter en ny watchlist.
    /// POST: api/watchlists
    /// Body: "Mine favoritter"
    /// </summary>
    [HttpPost]
    public async Task<ActionResult<WatchlistResponseDto>> CreateWatchlist(
        [FromBody] string name)
    {
        if (string.IsNullOrEmpty(name))
            return BadRequest(new { message = "Liste adı boş olamaz." });

        var userId = GetCurrentUserid();

        var createdList = await _watchlistService.CreateWatchlistAsync(
            userId,
            name);

        return Ok(createdList);
    }

    /// <summary>
    /// Tilføjer et Symbolld til en watchlist.
    /// POST: api/watchlists/items
    /// </summary>
    [HttpPost("items")]
    public async Task<ActionResult> AddSymbolToWatchlist(
        [FromBody] AddWatchlistItemDto dto)
    {
        if (dto.WatchlistId <= 0 || dto.Symbolld <= 0)
        {
            return BadRequest(new
            {
                message = "Geçerli watchlistId ve symbolld zorunludur."
            });
        }
    }
}
```

```

var success = await _watchlistService.AddSymbolToWatchlistAsync(dto);

if (!success)
{
    return BadRequest(new
    {
        message = "İzleme listesi veya sembol bulunamadı."
    });
}

return Ok(new
{
    message = "Sembol izleme listesine başarıyla eklendi.",
    dto.SymbolId
});
}

/// <summary>
/// Fjerner et SymbolId fra en watchlist.
/// DELETE: api/watchlists/1/items/10
/// </summary>
[HttpDelete("{watchlistId:int}/items/{symbolId:int}")]
public async Task<ActionResult> RemoveSymbolFromWatchlist(
    int watchlistId,
    int symbolId)
{
    var success = await _watchlistService.RemoveSymbolFromWatchlistAsync(
        watchlistId,
        symbolId);

    if (!success)
    {
        return NotFound(new
        {
            message = "İzleme listesi veya silinecek sembol bulunamadı."
        });
    }

    return Ok(new
    {
        message = "Sembol izleme listesinden çıkarıldı.",
        symbolId
    });
}

// Henter current user id.
private int GetCurrentUserid()
{
    var userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier)?.Value
    ?? User.FindFirst("sub")?.Value
    ?? User.FindFirst("id")?.Value;

    if (string.IsNullOrEmpty(userIdClaim) ||
        !int.TryParse(userIdClaim, out var userId))
    {
        throw new UnauthorizedAccessException(
            "Geçersiz veya bulunamayan kullanıcı kimliği.");
    }

    return userId;
}
}

```

B11. TradingApp/Program.cs

Applikationens startpunkt. Registrerer dependency injection, authentication/authorization, database, controllers, SignalR og background services og bygger HTTP-pipelinen.

```

using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi;
using System.Security.Claims;
using System.Text;
using System.Text.Json;
using TradingAppLibrary.BackgroundServices;
using TradingAppLibrary.Data;
using TradingAppLibrary.Hubs;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Models;
using TradingAppLibrary.Repositories;
using TradingAppLibrary.Services;

```

```
using TradingAppLibrary.Validators;

var builder = WebApplication.CreateBuilder(args);

// -----
// 1. DATABASE (PostgreSQL / TimescaleDB)
// -----
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")));

// -----
// 2. HTTP-KLIENTER TIL EXCHANGES
// -----
builder.Services.AddHttpClient<BinanceService>(client =>
{
    client.BaseAddress = new Uri("https://api.binance.com");
    client.Timeout = TimeSpan.FromSeconds(30);
});

builder.Services.AddHttpClient<OkxService>(client =>
{
    client.BaseAddress = new Uri(builder.Configuration["Exchanges:OKX:RestBaseUrl"] ?? "https://www.okx.com");
    client.Timeout = TimeSpan.FromSeconds(30);
});

builder.Services.AddTransient<IExchangeService>(serviceProvider =>
    serviceProvider.GetRequiredService<BinanceService>());
builder.Services.AddTransient<IExchangeService>(serviceProvider =>
    serviceProvider.GetRequiredService<OkxService>());

builder.Services.AddScoped<IExchangeServiceFactory, ExchangeServiceFactory>();
builder.Services.AddScoped<IMarketDataSyncService, MarketDataSyncService>();
builder.Services.AddSingleton<ICandleValidator, CandleValidator>();
builder.Services.AddSingleton<IMarketDataRuntimeState, TradingAppLibrary.Runtime.MarketDataRuntimeState>();
builder.Services.AddSingleton<IHistoricalSyncQueue, TradingAppLibrary.Runtime.HistoricalSyncQueue>();
builder.Services.AddSingleton<IMarketDataStreamControl, TradingAppLibrary.Runtime.MarketDataStreamControl>();
builder.Services.AddScoped<ISystemHealthService, SystemHealthService>();

// -----
// 3. APPLIKATIONSSERVICES
// -----
builder.Services.Configure<EmailSettings>(builder.Configuration.GetSection("EmailSettings"));
builder.Services.AddScoped<IEmailService, SmtplibEmailService>();
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<ICandleService, CandleService>();
builder.Services.AddScoped<ISymbolService, SymbolService>();
builder.Services.AddScoped<IWatchlistService, WatchlistService>();
builder.Services.AddScoped<IAiAnalysisService, AiAnalysisService>();
builder.Services.AddScoped<ITokenService, TokenService>();
builder.Services.AddScoped<IUserService, UserService>();
builder.Services.AddScoped<IChartDrawingService, ChartDrawingService>();

// -----
// 4. REPOSITORIES
// -----
builder.Services.AddScoped<ICandleRepository, CandleRepository>();
builder.Services.AddScoped<ISymbolRepository, SymbolRepository>();
builder.Services.AddScoped<IWatchlistRepository, WatchlistRepository>();
builder.Services.AddScoped<IUserRepository, UserRepository>();

// -----
// 5. BAGGRUNDSSERVICE TIL MARKET DATA FRA FLERE EXCHANGES
// -----
builder.Services.AddHostedService<MarketDataSyncBackgroundService>();
builder.Services.AddHostedService<HistoricalSyncBackgroundService>();

// -----
// 6. SIGNALR + JSON
// -----
builder.Services.AddSignalR()
    .AddJsonProtocol(options =>
    {
        options.PayloadSerializerOptions.PropertyNamingPolicy = JsonNamingPolicy.CamelCase;
        options.PayloadSerializerOptions.DictionaryKeyPolicy = JsonNamingPolicy.CamelCase;
    });

// -----
// 7. CORS
// -----
builder.Services.AddCors(options =>
{
    options.AddPolicy("CorsPolicy", policy =>
```

```
{
    policy.WithOrigins(
        "http://localhost:3000",
        "http://localhost:5173",
        "http://localhost:4200",
        "http://localhost:50085")
        .AllowAnyHeader()
        .AllowAnyMethod()
        .AllowCredentials();
    });
});

// -----
// 8. JWT-AUTENTICATING + SIGNALR-TOKEN
// -----
var jwtSettings = builder.Configuration.GetSection("JwtSettings");
builder.Services.Configure<JwtSettings>(jwtSettings);

var secretKey = jwtSettings["Secret"] ?? "SuperSecretKey_TradingApp_2026_123456";

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = jwtSettings["Issuer"],
        ValidAudience = jwtSettings["Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secretKey)),
        RoleClaimType = ClaimTypes.Role,
        NameClaimType = ClaimTypes.Name
    };

    options.Events = new JwtBearerEvents
    {
        OnMessageReceived = context =>
        {
            var accessToken = context.Request.Query["access_token"];
            var path = context.HttpContext.Request.Path;

            if (!string.IsNullOrEmpty(accessToken) &&
                path.StartsWithSegments("/hubs/market-data"))
            {
                context.Token = accessToken;
            }

            return Task.CompletedTask;
        }
    };
});

// -----
// 9. CONTROLLERS / REST JSON
// -----
builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.PropertyNamingPolicy = JsonNamingPolicy.CamelCase;
        options.JsonSerializerOptions.DictionaryKeyPolicy = JsonNamingPolicy.CamelCase;
    });

builder.Services.AddEndpointsApiExplorer();

// -----
// 10. SWAGGER / OPENAPI
// -----
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "TradingApp API",
        Version = "v1"
    });

    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
```



```

{
    Name = "Authorization",
    Type = SecuritySchemeType.Http,
    Scheme = "bearer",
    BearerFormularat = "JWT",
    In = ParameterLocation.Header,
    Description = "JWT token girin."
});

c.AddSecurityRequirement(_ => new OpenApiSecurityRequirement
{
    {
        new OpenApiSecuritySchemeReference("Bearer"),
        new List<string>()
    }
});
});

var app = builder.Build();

// -----
// HTTP-PIPELINE
// -----
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(c =>
    {
        c.SwaggerEndpoint("/swagger/v1/swagger.json", "Trading API v1");
        c.RoutePrefix = "swagger";
    });
}

app.UseHttpsRedirection();
app.UseCors("CorsPolicy");
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();
app.MapHub<MarketDataHub>("/hubs/market-data");

// -----
// OPRET SUPERADMIN-STARTDATA
// -----
using (var scope = app.Services.CreateScope())
{
    var authService = scope.ServiceProvider.GetRequiredService<IAuthService>();
    var userService = scope.ServiceProvider.GetRequiredService<IUserService>();

    await DbInitializer.SeedSuperAdminAsync(authService, userService);
}

app.Run();

```

B12.

TradingAppLibrary/BackgroundServices/HistoricalSyncBackgroundService.cs

Behandler historical-sync jobs fra køen, eksempelvis når et nyt symbol aktiveres efter applikationsstart.

```

using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using TradingAppLibrary.Interfaces;

namespace TradingAppLibrary.BackgroundServices;

public sealed class HistoricalSyncBackgroundService : BackgroundService
{
    private readonly IHistoricalSyncQueue _queue;
    private readonly IServiceScopeFactory _scopeFactory;
    private readonly IMarketDataRuntimeState _runtimeState;
    private readonly IMarketDataStreamControl _streamControl;
    private readonly ILogger<HistoricalSyncBackgroundService> _logger;

    public HistoricalSyncBackgroundService(
        IHistoricalSyncQueue queue,
        IServiceScopeFactory scopeFactory,
        IMarketDataRuntimeState runtimeState,
        IMarketDataStreamControl streamControl,
        ILogger<HistoricalSyncBackgroundService> logger)
    {
        _queue = queue;
        _scopeFactory = scopeFactory;
        _runtimeState = runtimeState;
        _streamControl = streamControl;
        _logger = logger;
    }

```

```
{
    _queue = queue;
    _scopeFactory = scopeFactory;
    _runtimeState = runtimeState;
    _streamControl = streamControl;
    _logger = logger;
}

// Kører den relevante operation.
protected override async Task ExecuteAsync(Cancellation_token stoppingToken)
{
    _logger.LogInformation("Historical sync queue service started.");

    while (!stoppingToken.IsCancellationRequested)
    {
        HistoricalSyncJob job;
        try
        {
            job = await _queue.DequeueAsync(stoppingToken);
        }
        catch (OperationCanceledException) when (stoppingToken.IsCancellationRequested)
        {
            break;
        }

        try
        {
            _runtimeState.SetHistoricalSyncRunning(job.ExchangeCode, true);

            using var scope = _scopeFactory.CreateScope();
            var syncService = scope.ServiceProvider.GetRequiredService<IMarketDataSyncService>();

            _logger.LogInformation(
                "Queued historical sync started. Exchange={Exchange}, SymbolId={SymbolId}",
                job.ExchangeCode,
                job.SymbolId);

            await syncService.SyncHistoricalCandlesAsync(
                job.SymbolId,
                "1m",
                stoppingToken);

            _runtimeState.ClearError(job.ExchangeCode);

            _logger.LogInformation(
                "Queued historical sync completed. Exchange={Exchange}, SymbolId={SymbolId}",
                job.ExchangeCode,
                job.SymbolId);
        }
        catch (OperationCanceledException) when (stoppingToken.IsCancellationRequested)
        {
            break;
        }
        catch (Exception ex)
        {
            _runtimeState.RecordError(job.ExchangeCode, ex.Message);
            _logger.LogError(
                ex,
                "Queued historical sync failed. Exchange={Exchange}, SymbolId={SymbolId}",
                job.ExchangeCode,
                job.SymbolId);
        }
        finally
        {
            _runtimeState.SetHistoricalSyncRunning(job.ExchangeCode, false);
            _queue.Complete(job.SymbolId);

            // Genopbyg exchange's WebSocket-abonnement, når backfill er færdig.
            // Det betyder, at et nyaktiveret symbol først tilsluttes realtime, når historikken er udfyldt.
            _streamControl.RequestRestart(job.ExchangeCode);
        }
    }
}
```

B13.**TradingAppLibrary/BackgroundServices/MarketDataSyncBackgroundService.cs**

Kører market-data synkronisering i baggrunden og koordinerer initial historical sync samt realtime streams for aktive exchanges.

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using TradingAppLibrary.Data;
using TradingAppLibrary.Interfaces;

namespace TradingAppLibrary.BackgroundServices;

public class MarketDataSyncBackgroundService : BackgroundService
{
    private readonly IServiceScopeFactory _scopeFactory;
    private readonly IMarketDataRuntimeState _runtimeState;
    private readonly IMarketDataStreamControl _streamControl;
    private readonly ILogger<MarketDataSyncBackgroundService> _logger;

    public MarketDataSyncBackgroundService(
        IServiceScopeFactory scopeFactory,
        IMarketDataRuntimeState runtimeState,
        IMarketDataStreamControl streamControl,
        ILogger<MarketDataSyncBackgroundService> logger)
    {
        _scopeFactory = scopeFactory;
        _runtimeState = runtimeState;
        _streamControl = streamControl;
        _logger = logger;
    }

    // Kører den relevante operation.
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        _logger.LogInformation("Market data sync service started.");

        // Historisk backfill kan tage lang tid. Realtidsstreams må ikke vente på den.
        var initialSyncTask = InitialSyncAsync(stoppingToken);
        var realtimeTask = StartRealtimeStreamsAsync(stoppingToken);

        await Task.WhenAll(initialSyncTask, realtimeTask);
    }

    // Behandler initial sync.
    private async Task InitialSyncAsync(CancellationToken cancellationToken)
    {
        try
        {
            using var scope = _scopeFactory.CreateScope();
            var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
            var syncService = scope.ServiceProvider.GetRequiredService<IMarketDataSyncService>();

            var exchanges = await db.Exchanges
                .Where(e => e.IsActive)
                .Select(e => e.Code)
                .ToListAsync(cancellationToken);

            foreach (var exchangeCode in exchanges)
            {
                cancellationToken.ThrowIfCancellationRequested();

                try
                {
                    _logger.LogInformation("Syncing symbols for {Exchange}", exchangeCode);
                    await syncService.SyncExchangeSymbolsAsync(exchangeCode, cancellationToken);

                    _runtimeState.SetHistoricalSyncRunning(exchangeCode, true);
                    _logger.LogInformation("Syncing historical candles for {Exchange}", exchangeCode);
                    await syncService.SyncExchangeHistoricalDataAsync(exchangeCode, "1m", cancellationToken);

                    _runtimeState.ClearError(exchangeCode);
                }
                catch (OperationCanceledException) when (cancellationToken.IsCancellationRequested)
                {
                    throw;
                }
            }
        }
    }
}
```

```

    }
    catch (Exception ex)
    {
        _runtimeState.RecordError(exchangeCode, ex.Message);
        _logger.LogError(ex, "Initial market data sync failed for {Exchange}.", exchangeCode);
    }
    finally
    {
        _runtimeState.SetHistoricalSyncRunning(exchangeCode, false);
    }
}
}
catch (OperationCanceledException) when (cancellationToken.IsCancellationRequested)
{
}
catch (Exception ex)
{
    _logger.LogError(ex, "Initial market data sync failed.");
}
}

// Starter realtime-streamen for aktive symboler på den valgte exchange.
private async Task StartRealtimeStreamsAsync(CancellationToken cancellationToken)
{
    using var scope = _scopeFactory.CreateScope();
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();

    var exchanges = await db.Exchanges
        .Where(e => e.IsActive)
        .Select(e => e.Code)
        .ToListAsync(cancellationToken);

    await Task.WhenAll(
        exchanges.Select(code => RunExchangeStreamAsync(code, cancellationToken)));
}

// Kører exchange stream.
private async Task RunExchangeStreamAsync(
    string exchangeCode,
    CancellationToken cancellationToken)
{
    while (!cancellationToken.IsCancellationRequested)
    {
        using var streamCancellation = CancellationTokenSource.CreateLinkedTokenSource(cancellationToken);
        _streamControl.Register(exchangeCode, streamCancellation);

        var restartRequested = false;

        try
        {
            _runtimeState.SetRealtimeConnected(exchangeCode, false);

            using var scope = _scopeFactory.CreateScope();
            var syncService = scope.ServiceProvider.GetRequiredService<IMarketDataSyncService>();

            _logger.LogInformation("Starting realtime stream for {Exchange}", exchangeCode);
            await syncService.StartRealtimeStreamAsync(
                exchangeCode,
                "1m",
                streamCancellation.Token);

            _runtimeState.SetRealtimeConnected(exchangeCode, false);
        }
        catch (OperationCanceledException) when (
            !cancellationToken.IsCancellationRequested &&
            streamCancellation.IsCancellationRequested)
        {
            restartRequested = true;
            _runtimeState.SetRealtimeConnected(exchangeCode, false);
            _logger.LogInformation(
                "Realtime stream restart requested for {Exchange} because active symbols changed.",
                exchangeCode);
        }
        catch (OperationCanceledException) when (cancellationToken.IsCancellationRequested)
        {
            _runtimeState.SetRealtimeConnected(exchangeCode, false);
            break;
        }
        catch (Exception ex)
        {
            _runtimeState.SetRealtimeConnected(exchangeCode, false);
            _runtimeState.RecordError(exchangeCode, ex.Message);
            _logger.LogError(ex, "Realtime stream failed for {Exchange}", exchangeCode);
        }
    }
}

```

```

    }
    finally
    {
        _streamControl.Unregister(exchangeCode, streamCancellation);
    }

    if (cancellationToken.IsCancellationRequested)
        break;

    await Task.Delay(
        restartRequested ? TimeSpan.FromMilliseconds(250) : TimeSpan.FromSeconds(5),
        cancellationToken);
}
}
}

```

B14. TradingAppLibrary/Constants/ExchangeCodes.cs

Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.

```
namespace TradingAppLibrary.Constants;
```

```

public static class ExchangeCodes
{
    public const string Binance = "BINANCE";
    public const string Bybit = "BYBIT";
    public const string Okx = "OKX";
}

```

B15. TradingAppLibrary/DTO/AiAnalysisDtos.cs

DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

```
namespace TradingAppLibrary.DTO;
```

```

// Behandler ai analysis request dto.
public record AiAnalysisRequestDto(int SymbolId, string Interval, string? Language = null);
// Behandler ai analysis response dto.
public record AiAnalysisResponseDto(int SymbolId, string Symbol, string Exchange, string AnalysisText, DateTime GeneratedAt);

```

B16. TradingAppLibrary/DTO/AlertDto.cs

DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

```
namespace TradingAppLibrary.DTO;
```

```

public class CreateAlertDto
{
    public int SymbolId { get; set; }
    public decimal TargetPrice { get; set; }
    public string Condition { get; set; } = "CROSSES_UP";
    public string? Note { get; set; }
}

public class AlertResponseDto
{
    public string Id { get; set; } = string.Empty;
    public string UserId { get; set; } = string.Empty;
    public int SymbolId { get; set; }
    public string Symbol { get; set; } = string.Empty;
    public string Exchange { get; set; } = string.Empty;
    public decimal TargetPrice { get; set; }
    public string Condition { get; set; } = string.Empty;
    public bool IsTriggered { get; set; }
    public bool IsActive { get; set; }
    public string? Note { get; set; }
    public DateTime CreatedAt { get; set; }
}

```

B17. TradingAppLibrary/DTO/AuthDtos.cs

DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

```
namespace TradingAppLibrary.DTO;
```

```

// Opretter en ny bruger efter validering af de indsendte oplysninger.
public record RegisterDto(

```

```
string FirstName,
string LastName,
string Username,
string Email,
string Password
);

// Identifier: e-mail eller brugernavn.
// Email-feltet bevares af hensyn til bagudkompatibilitet med ældre klienter.
public record LoginDto(
    string? Identifier,
    string Password,
    string? Email = null
);

// Behandler auth response dto.
public record AuthResponseDto(
    string Token,
    string Username,
    string Email,
    string Role,
    string FirstName,
    string LastName
);

// Behandler user management dto.
public record UserManagementDto(
    int Id,
    string Username,
    string Email,
    string Role,
    string FirstName,
    string LastName,
    bool IsDeleted,
    DateTime? DeletionRequestedAt
);

// Behandler change password dto.
public record ChangePasswordDto(
    string CurrentPassword,
    string NewPassword
);
```

B18. TradingAppLibrary/DTO/CandleDtos.cs

DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

```
namespace TradingAppLibrary.DTO;

// Behandler candle response dto.
public record CandleResponseDto(
    long Time,
    decimal Open,
    decimal High,
    decimal Low,
    decimal Close,
    decimal Volume,
    int SymbolId,
    string Symbol,
    string Exchange,
    string Interval,
    bool IsClosed = true);
```

B19. TradingAppLibrary/DTO/ChartDrawingDtos.cs

DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

```
using System.Text.Json;

namespace TradingAppLibrary.DTO;

// Opretter chart drawing dto.
public record CreateChartDrawingDto(
    int SymbolId,
    string Interval,
    string DrawingType,
    JsonElement Data,
    bool IsVisible = true);
```

```
// Opdaterer chart drawing dto.
public record UpdateChartDrawingDto(
    JsonElement Data,
    bool IsVisible = true);

// Behandler chart drawing response dto.
public record ChartDrawingResponseDto(
    long Id,
    int SymbolId,
    string ExchangeCode,
    string SymbolName,
    string Interval,
    string DrawingType,
    JsonElement Data,
    bool IsVisible,
    DateTime CreatedAt,
    DateTime UpdatedAt);
```

B20. TradingAppLibrary/DTO/ExchangeDtos.cs

DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

```
namespace TradingAppLibrary.DTO;

// Behandler exchange response dto.
public record ExchangeResponseDto(int Id, string Code, string Name, bool IsActive);
// Behandler exchange symbol dto.
public record ExchangeSymbolDto(string Name, string BaseAsset, string QuoteAsset, bool IsActive);
// Behandler exchange candle dto.
public record ExchangeCandleDto(long Time, decimal Open, decimal High, decimal Low, decimal Close, decimal Volume, string Symbol, string Interval, bool IsClosed);

// Opretter exchange dto.
public record CreateExchangeDto(string Code, string Name, bool IsActive = true);
// Opdaterer exchange dto.
public record UpdateExchangeDto(string Code, string Name, bool IsActive);
```

B21. TradingAppLibrary/DTO/ForgotPasswordDto.cs

DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

```
namespace TradingAppLibrary.DTO;

// Behandler forgot password dto.
public record ForgotPasswordDto(string Identifier);
```

B22. TradingAppLibrary/DTO/ResetPasswordDto.cs

DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

```
namespace TradingAppLibrary.DTO;

// Validerer reset-tokenet og gemmer brugerens nye password.
public record ResetPasswordDto(
    int UserId,
    string Token,
    string NewPassword
);
```

B23. TradingAppLibrary/DTO/SymbolDtos.cs

DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

```
namespace TradingAppLibrary.DTO;

// Behandler symbol response dto.
public record SymbolResponseDto(
    int Id,
    int ExchangeId,
    string ExchangeCode,
    string ExchangeName,
    string Name,
    string BaseAsset,
    string QuoteAsset,
    bool IsActive);
```

```
// Behandler symbol status update result dto.
public record SymbolStatusUpdateResultDto(
    int SymbolId,
    string ExchangeCode,
    string SymbolName,
    bool IsActive,
    bool HistoricalSyncQueued,
    bool RealtimeRestartRequested);
```

B24. TradingAppLibrary/DTO/SystemHealthDtos.cs

DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

```
namespace TradingAppLibrary.DTO;

public sealed class SystemHealthDto
{
    public string Status { get; init; } = "Unknown";
    public bool DatabaseHealthy { get; init; }
    public bool TimescaleHealthy { get; init; }
    public bool CandlesHypertableHealthy { get; init; }
    public int ActiveExchangeCount { get; init; }
    public int ActiveSymbolCount { get; init; }
    public DateTime? LastCandleTimeUtc { get; init; }
    public double? LastCandleAgeSeconds { get; init; }
    public long ValidationErrorCount { get; init; }
    public long ValidationWarningCount { get; init; }
    public DateTime StartedAtUtc { get; init; }
    public TimeSpan Uptime { get; init; }
    public DateTime CheckedAtUtc { get; init; }
    public IReadOnlyList<ExchangeHealthDto> Exchanges { get; init; } = Array.Empty<ExchangeHealthDto>();
}

public sealed class ExchangeHealthDto
{
    public string ExchangeCode { get; init; } = string.Empty;
    public string ExchangeName { get; init; } = string.Empty;
    public bool IsActive { get; init; }
    public int ActiveSymbolCount { get; init; }
    public DateTime? LastCandleTimeUtc { get; init; }
    public double? LastCandleAgeSeconds { get; init; }
    public bool HistoricalSyncRunning { get; init; }
    public DateTime? HistoricalSyncStartedAtUtc { get; init; }
    public DateTime? HistoricalSyncCompletedAtUtc { get; init; }
    public bool RealtimeConnected { get; init; }
    public DateTime? RealtimeConnectedAtUtc { get; init; }
    public DateTime? LastRealtimeMessageAtUtc { get; init; }
    public string? LastError { get; init; }
    public DateTime? LastErrorAtUtc { get; init; }
    public long ValidationErrorCount { get; init; }
    public long ValidationWarningCount { get; init; }
    public string Status { get; init; } = "Unknown";
}
```

B25. TradingAppLibrary/DTO/UserPreferencesDto.cs

DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

```
using System;
using System.Collections.Generic;
using System.Text;

namespace TradingAppLibrary.DTO
{
    public class IndicatorConfigDto
    {
        public string Id { get; set; } = string.Empty;
        public string Type { get; set; } = string.Empty;
        public string Name { get; set; } = string.Empty;
        public bool Enabled { get; set; }
        public string Color { get; set; } = "#2962ff";
        public int? Period { get; set; }
        public double? StdDev { get; set; }
        public int? FastPeriod { get; set; } // Til MACD.
        public int? SlowPeriod { get; set; } // Til MACD.
        public int? SignalPeriod { get; set; } // Til MACD.
    }

    public class ChartVisualSettingsDto
```



```

{
    // Candle-farver og visuelle indstillinger
    public string UpColor { get; set; } = "#089981";
    public string DownColor { get; set; } = "#23645";
    public bool ShowBorders { get; set; } = false;
    public bool ShowWicks { get; set; } = true;

    // Gitter og lærred
    public bool ShowGrid { get; set; } = true;
    public string GridColorDark { get; set; } = "#1e222d";
    public string GridColorLight { get; set; } = "#f0f3fa";

    // Prislinje
    public bool ShowPriceLine { get; set; } = true;

    // Alarmer og lyd
    public bool SoundEnabled { get; set; } = true;
}

public class UserPreferencesDto
{
    // Temavalg
    public string Theme { get; set; } = "dark";
    public string Language { get; set; } = "da";
    public string LastSymbol { get; set; } = "BTCUSDT";
    public string LastInterval { get; set; } = "15m";
    public string ChartType { get; set; } = "candles";
    public string LayoutMode { get; set; } = "1x1";
    public List<IndicatorConfigDto> Indicators { get; set; } = new();
    public ChartVisualSettingsDto VisualSettings { get; set; } = new();
}
}

```

B26. TradingAppLibrary/DTO/WatchlistDtos.cs

DTO-definitioner for dette område. Begrænser hvilke data der sendes mellem lag og API i stedet for at eksponere hele databasemodellen.

```

namespace TradingAppLibrary.DTO;

// Tilføjer watchlist item dto.
public record AddWatchlistItemDto(int WatchlistId, int SymbolId);
// Behandler watchlist item response dto.
public record WatchlistItemResponseDto(int Id, int SymbolId, string Symbol, string Exchange, DateTime AddedAt);
// Behandler watchlist response dto.
public record WatchlistResponseDto(int Id, string Name, List<WatchlistItemResponseDto> Items);

```

B27. TradingAppLibrary/Data/AppDbContext.cs

EF Core DbContext. Samler TradingPros entiteter og konfigurerer relationer, nøgler, filtre og centrale database-regler.

```

using Microsoft.EntityFrameworkCore;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Data;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<User> Users => Set<User>();
    public DbSet<Exchange> Exchanges => Set<Exchange>();
    public DbSet<Symbol> Symbols => Set<Symbol>();
    public DbSet<Candle> Candles => Set<Candle>();
    public DbSet<Watchlist> Watchlists => Set<Watchlist>();
    public DbSet<WatchlistItem> WatchlistItems => Set<WatchlistItem>();
    public DbSet<UserPreferences> UserPreferences => Set<UserPreferences>();
    public DbSet<Alert> Alerts => Set<Alert>();
    public DbSet<ChartDrawing> ChartDrawings => Set<ChartDrawing>();

    // Behandler model creating.
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<User>(entity =>
        {
            entity.HasKey(u => u.Id);
            entity.HasIndex(u => u.Email).IsUnique();
            entity.HasIndex(u => u.Username).IsUnique();
            entity.Property(u => u.Username).IsRequired().HasMaxLength(50);
        });
    }
}

```

```
entity.Property(u => u.Email).IsRequired().HasMaxLength(100);
entity.Property(u => u.PasswordHash).IsRequired();
entity.Property(u => u.Role).HasMaxLength(20);
entity.Property(u => u.FirstName).HasMaxLength(50);
entity.Property(u => u.LastName).HasMaxLength(50);
entity.Property(u => u.IsDeleted).HasDefaultValue(false);
entity.Property(u => u.DeletionRequestedAt).IsRequired(false);
entity.Property(u => u.PasswordResetTokenHash).HasMaxLength(64).IsRequired(false);
entity.Property(u => u.PasswordResetTokenExpiresAtUtc).IsRequired(false);
entity.HasQueryFilter(u => !u.IsDeleted);
});

modelBuilder.Entity<UserPreferences>(entity =>
{
    entity.HasKey(p => p.Id);
    entity.HasIndex(p => p.UserId).IsUnique();
    entity.Property(p => p.UserId).IsRequired();
    entity.Property(p => p.SettingsJson).IsRequired();
    entity.Property(p => p.UpdatedAt).HasDefaultValueSql("CURRENT_TIMESTAMP");
});

modelBuilder.Entity<Exchange>(entity =>
{
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Code).IsRequired().HasMaxLength(20);
    entity.Property(e => e.Name).IsRequired().HasMaxLength(50);
    entity.Property(e => e.IsActive).HasDefaultValue(true);
    entity.HasIndex(e => e.Code).IsUnique();

    entity.HasData(new Exchange
    {
        Id = 1,
        Code = "BINANCE",
        Name = "Binance",
        IsActive = true
    });
});

modelBuilder.Entity<Symbol>(entity =>
{
    entity.HasKey(s => s.Id);
    entity.Property(s => s.Name).IsRequired().HasMaxLength(30);
    entity.Property(s => s.BaseAsset).IsRequired().HasMaxLength(20);
    entity.Property(s => s.QuoteAsset).IsRequired().HasMaxLength(20);
    entity.Property(s => s.IsActive).HasDefaultValue(false);

    entity.HasOne(s => s.Exchange)
        .WithMany(e => e.Symbols)
        .HasForeignKey(s => s.ExchangeId)
        .OnDelete(DeleteBehavior.Restrict);

    entity.HasIndex(s => new { s.ExchangeId, s.Name }).IsUnique();
    entity.HasIndex(s => new { s.ExchangeId, s.IsActive });
});

modelBuilder.Entity<Candle>(entity =>
{
    entity.HasKey(c => new { c.Id, c.OpenTime });
    entity.Property(c => c.Id).ValueGeneratedOnAdd();
    entity.Property(c => c.Interval).IsRequired().HasMaxLength(10);

    entity.HasOne(c => c.Symbol)
        .WithMany(s => s.Candles)
        .HasForeignKey(c => c.SymbolId)
        .OnDelete(DeleteBehavior.Restrict);

    entity.HasIndex(c => new { c.SymbolId, c.Interval, c.OpenTime }).IsUnique();
});

modelBuilder.Entity<Alert>(entity =>
{
    entity.HasKey(a => a.Id);
    entity.Property(a => a.UserId).IsRequired();
    entity.Property(a => a.TargetPrice).HasColumnType("decimal(18,8)");
    entity.Property(a => a.Condition).IsRequired().HasMaxLength(20);
    entity.Property(a => a.Note).HasMaxLength(255);
    entity.Property(a => a.IsTriggered).HasDefaultValue(false);
    entity.Property(a => a.IsActive).HasDefaultValue(true);
    entity.Property(a => a.CreatedAt).HasDefaultValueSql("CURRENT_TIMESTAMP");

    entity.HasOne(a => a.Symbol)
        .WithMany()
```

```

        .HasForeignKey(a => a.SymbolId)
        .OnDelete(DeleteBehavior.Restrict);

entity.HasIndex(a => new { a.SymbolId, a.IsActive, a.IsTriggered });
entity.HasIndex(a => a.UserId);
});

modelBuilder.Entity<Watchlist>(entity =>
{
    entity.HasKey(w => w.Id);
    entity.Property(w => w.Name).IsRequired().HasMaxLength(50);
    entity.HasOne(w => w.User)
        .WithMany(u => u.Watchlists)
        .HasForeignKey(w => w.UserId)
        .OnDelete(DeleteBehavior.Cascade);
    entity.HasQueryFilter(w => !w.User.IsDeleted);
});

modelBuilder.Entity<WatchlistItem>(entity =>
{
    entity.HasKey(wi => wi.Id);
    entity.HasOne(wi => wi.Watchlist)
        .WithMany(w => w.Items)
        .HasForeignKey(wi => wi.WatchlistId)
        .OnDelete(DeleteBehavior.Cascade);

    entity.HasOne(wi => wi.Symbol)
        .WithMany()
        .HasForeignKey(wi => wi.SymbolId)
        .OnDelete(DeleteBehavior.Restrict);

    entity.HasIndex(wi => new { wi.WatchlistId, wi.SymbolId }).IsUnique();
    entity.HasQueryFilter(wi => !wi.Watchlist.User.IsDeleted);
});

modelBuilder.Entity<ChartDrawing>(entity =>
{
    entity.HasKey(d => d.Id);
    entity.Property(d => d.Interval).IsRequired().HasMaxLength(10);
    entity.Property(d => d.DrawingType).IsRequired().HasMaxLength(30);
    entity.Property(d => d.DataJson).IsRequired().HasColumnType("jsonb");
    entity.Property(d => d.IsVisible).HasDefaultValue(true);
    entity.Property(d => d.CreatedAt).HasDefaultValueSql("CURRENT_TIMESTAMP");
    entity.Property(d => d.UpdatedAt).HasDefaultValueSql("CURRENT_TIMESTAMP");

    entity.HasOne(d => d.User)
        .WithMany(u => u.ChartDrawings)
        .HasForeignKey(d => d.UserId)
        .OnDelete(DeleteBehavior.Cascade);

    entity.HasOne(d => d.Symbol)
        .WithMany(s => s.ChartDrawings)
        .HasForeignKey(d => d.SymbolId)
        .OnDelete(DeleteBehavior.Cascade);

    entity.HasIndex(d => new { d.UserId, d.SymbolId, d.Interval });
    entity.HasQueryFilter(d => !d.User.IsDeleted);
});
}
}

```

B28. TradingAppLibrary/Data/DbInitializer.cs

Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.

```

using System;
using System.Collections.Generic;
using System.Text;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Enums;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Data
{
    public static class DbInitializer
    {
        // Behandler seed super admin.
        public static async Task SeedSuperAdminAsync(IAuthService authService, IUserService userService)
        {
            const string superAdminEmail = "superadmin@tradingapp.com";

```

```
// 1. Kontroller, om der findes en SuperAdmin.
var existingUser = await userService.GetUserByEmailAsync(superAdminEmail);

if (existingUser == null)
{
    // 2. Gem brugeren (fornavn, efternavn, username, e-mail og password).
    var registerDto = new RegisterDto("Super", "Admin", "superadmin", superAdminEmail, "123456789");
    var authResponse = await authService.RegisterAsync(registerDto);

    if (authResponse != null)
    {
        // 3. Hent brugeren igen, og sæt rollen til SuperAdmin.
        var createdUser = await userService.GetUserByEmailAsync(superAdminEmail);
        if (createdUser != null)
        {
            createdUser.Role = UserRole.SuperAdmin.ToString();
            await userService.UpdateUserAsync(createdUser);
        }
    }
}
else if (existingUser.Role != UserRole.SuperAdmin.ToString())
{
    // Hvis brugeren allerede findes, men stadig har rollen User, ændres rollen til SuperAdmin.
    existingUser.Role = UserRole.SuperAdmin.ToString();
    await userService.UpdateUserAsync(existingUser);
}
}
```

B29. TradingAppLibrary/Data/PasswordHasher.cs

Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.

```
using System;
using System.Collections.Generic;
using System.Security.Cryptography;
using System.Text;
using Microsoft.AspNetCore.Cryptography.KeyDerivation;

namespace TradingAppLibrary.Data
{
    public class PasswordHasher
    {
        // Hasher password.
        public static string HashPassword(string password)
        {
            byte[] salt = RandomNumberGenerator.GetBytes(128 / 8);
            string hashed = Convert.ToBase64String(KeyDerivation.Pbkdf2(
                password: password,
                salt: salt,
                prf: KeyDerivationPrf.HMACSHA256,
                iterationCount: 100000,
                numBytesRequested: 256 / 8));

            return $"{Convert.ToBase64String(salt)}:{hashed}";
        }

        // Kontrollerer password.
        public static bool VerifyPassword(string password, string storedHash)
        {
            var parts = storedHash.Split(':');
            if (parts.Length != 2) return false;

            byte[] salt = Convert.FromBase64String(parts[0]);
            string hashToVerify = Convert.ToBase64String(KeyDerivation.Pbkdf2(
                password: password,
                salt: salt,
                prf: KeyDerivationPrf.HMACSHA256,
                iterationCount: 100000,
                numBytesRequested: 256 / 8));

            return parts[1] == hashToVerify;
        }
    }
}
```

B30. TradingAppLibrary/Enums/UserRole.cs

Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.

```
using System;
using System.Collections.Generic;
using System.Text;

namespace TradingAppLibrary.Enums
{
    public enum UserRole
    {
        User = 1,
        Admin = 2,
        SuperAdmin = 3
    }
}
```

B31. TradingAppLibrary/Hubs/MarketDataHub.cs

SignalR Hub for realtime data. Organiserer browserklienter i bruger- og symbolgrupper, så events kan sendes målrettet.

```
using Microsoft.AspNetCore.SignalR;
using System.Security.Claims;

namespace TradingAppLibrary.Hubs;

public class MarketDataHub : Hub
{
    // Henter symbol group.
    public static string GetSymbolGroup(string exchangeCode, string symbolName) =>
        $"{exchangeCode.Trim().ToUpperInvariant()};{symbolName.Trim().ToUpperInvariant()}";

    // Abonnerer på to symbol.
    public async Task SubscribeToSymbol(string exchangeCode, string symbolName)
    {
        if (!string.IsNullOrEmpty(exchangeCode) && !string.IsNullOrEmpty(symbolName))
            await Groups.AddToGroupAsync(Context.ConnectionId, GetSymbolGroup(exchangeCode, symbolName));
    }

    // Afmelder from symbol.
    public async Task UnsubscribeFromSymbol(string exchangeCode, string symbolName)
    {
        if (!string.IsNullOrEmpty(exchangeCode) && !string.IsNullOrEmpty(symbolName))
            await Groups.RemoveFromGroupAsync(Context.ConnectionId, GetSymbolGroup(exchangeCode, symbolName));
    }

    // Behandler connected.
    public override async Task OnConnectedAsync()
    {
        var userId = Context.UserIdentifier
            ?? Context.User?.FindFirst(ClaimTypes.NameIdentifier)?.Value
            ?? Context.User?.FindFirst("id")?.Value;

        if (!string.IsNullOrEmpty(userId))
            await Groups.AddToGroupAsync(Context.ConnectionId, $"User_{userId}");

        await base.OnConnectedAsync();
    }

    // Behandler disconnected.
    public override async Task OnDisconnectedAsync(Exception? exception)
    {
        var userId = Context.UserIdentifier
            ?? Context.User?.FindFirst(ClaimTypes.NameIdentifier)?.Value
            ?? Context.User?.FindFirst("id")?.Value;

        if (!string.IsNullOrEmpty(userId))
            await Groups.RemoveFromGroupAsync(Context.ConnectionId, $"User_{userId}");

        await base.OnDisconnectedAsync(exception);
    }
}
```

B32. TradingAppLibrary/Interfaces/IAiAnalysisService.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using System;
using System.Collections.Generic;
using System.Text;
using TradingAppLibrary.DTO;

namespace TradingAppLibrary.Interfaces
```

```
{
    public interface IAiAnalysisService
    {
        Task<AiAnalysisResponseDto> AnalyzeMarketAsync(AiAnalysisRequestDto request);
    }
}
```

B33. TradingAppLibrary/Interfaces/IAuthService.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using TradingAppLibrary.DTO;

namespace TradingAppLibrary.Interfaces;

public interface IAuthService
{
    Task<AuthResponseDto?> RegisterAsync(RegisterDto registerDto);
    Task<AuthResponseDto?> LoginAsync(LoginDto loginDto);
    Task RequestPasswordResetAsync(ForgotPasswordDto request, CancellationToken cancellationToken = default);
    Task<bool> ResetPasswordAsync(ResetPasswordDto request, CancellationToken cancellationToken = default);
    Task<bool> RequestAccountDeletionAsync(int userId);
    Task<IEnumerable<UserManagementDto>> GetAllUsersAsync();
    Task LogoutAsync();
}
```

B34. TradingAppLibrary/Interfaces/ICandleRepository.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Interfaces;

public interface ICandleRepository
{
    Task<IEnumerable<Candle>> GetCandlesAsync(int symbolId, string interval, int limit = 1000, long? endTime = null);
    Task<Candle?> GetFirstCandleAsync(int symbolId, string interval);
    Task<DateTime?> GetFirstMissingOpenTimeAsync(int symbolId, string interval);
    Task AddOrUpdateCandleAsync(Candle candle);
    Task AddOrUpdateRangeAsync(IEnumerable<Candle> candles);
    Task AddRangeAsync(IEnumerable<Candle> candles);
    Task<List<Alert>> GetAlertsByUserIdAsync(string userId);
    Task<List<Alert>> GetActiveAlertsBySymbolAsync(int symbolId);
    Task<Alert?> GetAlertByIdAsync(string id);
    Task<Alert> CreateAlertAsync(Alert alert);
    Task<bool> DeleteAlertAsync(string id, string userId);
    Task UpdateAlertAsync(Alert alert);
}
```

B35. TradingAppLibrary/Interfaces/ICandleService.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using TradingAppLibrary.DTO;

namespace TradingAppLibrary.Interfaces;

public interface ICandleService
{
    Task<IEnumerable<CandleResponseDto>> GetCandlesAsync(int symbolId, string interval, int limit = 1000, long? endTime = null);
    Task<List<AlertResponseDto>> GetUserAlertsAsync(string userId);
    Task<List<AlertResponseDto>> GetActiveAlertsBySymbolAsync(int symbolId);
    Task<AlertResponseDto?> GetAlertByIdAsync(string alertId, string userId);
    Task<AlertResponseDto> CreateAlertAsync(string userId, CreateAlertDto dto);
    Task<bool> DeleteAlertAsync(string alertId, string userId);
    Task<bool> ToggleAlertAsync(string alertId, string userId);
    Task<List<AlertResponseDto>> CheckAndTriggerAlertsAsync(int symbolId, decimal currentPrice);
}
```

B36. TradingAppLibrary/Interfaces/ICandleValidator.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using TradingAppLibrary.DTO;
using TradingAppLibrary.Validation;

namespace TradingAppLibrary.Interfaces;

public interface ICandleValidator
```

```
{
    CandleValidationResult Validate(ExchangeCandleDto candle, string expectedSymbol, string expectedInterval);
    CandleBatchValidationResult ValidateBatch(IReadOnlyList<ExchangeCandleDto> candles, string expectedSymbol, string expectedInterval);
}
```

B37. TradingAppLibrary/Interfaces/IChartDrawingService.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using TradingAppLibrary.DTO;

namespace TradingAppLibrary.Interfaces;

public interface IChartDrawingService
{
    Task<IReadOnlyList<ChartDrawingResponseDto>> GetAsync(int userId, int symbolId, string interval, CancellationToken cancellationToken = default);
    Task<ChartDrawingResponseDto> CreateAsync(int userId, CreateChartDrawingDto dto, CancellationToken cancellationToken = default);
    Task<ChartDrawingResponseDto?> UpdateAsync(int userId, long drawingId, UpdateChartDrawingDto dto, CancellationToken cancellationToken = default);
    Task<bool> DeleteAsync(int userId, long drawingId, CancellationToken cancellationToken = default);
    Task<int> DeleteAllAsync(int userId, int symbolId, string interval, CancellationToken cancellationToken = default);
}
```

B38. TradingAppLibrary/Interfaces/IEmailService.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
namespace TradingAppLibrary.Interfaces;

public interface IEmailService
{
    Task SendPasswordResetAsync(string email, string firstName, string resetUrl, CancellationToken cancellationToken = default);
}
```

B39. TradingAppLibrary/Interfaces/IExchangeService.cs

Fælles kontrakt for exchange-adapttere. Binance og OKX implementerer samme operationer, så resten af backend kan arbejde exchange-uafhængigt.

```
using TradingAppLibrary.DTO;

namespace TradingAppLibrary.Interfaces;

public interface IExchangeService
{
    string ExchangeCode { get; }
    DateTime HistoricalDataStartUtc { get; }

    Task<IReadOnlyList<ExchangeSymbolDto>> GetSymbolsAsync(CancellationToken cancellationToken = default);

    Task<IReadOnlyList<string>> GetTopPopularSymbolsAsync(int topCount = 50, CancellationToken cancellationToken = default);

    Task<IReadOnlyList<ExchangeCandleDto>> GetHistoricalCandlesAsync(
        string symbol,
        string interval,
        int limit = 500,
        long? startTime = null,
        CancellationToken cancellationToken = default);

    Task StreamRealTimeCandlesAsync(
        IReadOnlyCollection<string> symbols,
        string interval,
        Func<ExchangeCandleDto, Task> onCandleReceived,
        CancellationToken cancellationToken);
}
```

B40. TradingAppLibrary/Interfaces/IExchangeServiceFactory.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
namespace TradingAppLibrary.Interfaces;

public interface IExchangeServiceFactory
{
    IExchangeService GetExchange(string exchangeCode);
}
```

B41. TradingAppLibrary/Interfaces/IHistoricalSyncQueue.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
namespace TradingAppLibrary.Interfaces;

public interface IHistoricalSyncQueue
{
    bool TryEnqueue(int symbolId, string exchangeCode);
    ValueTask<HistoricalSyncJob> DequeueAsync(CancellationToken cancellationToken);
    void Complete(int symbolId);
    bool IsPendingOrRunning(int symbolId);
}

// Behandler historical sync job.
public sealed record HistoricalSyncJob(int SymbolId, string ExchangeCode);
```

B42. TradingAppLibrary/Interfaces/IMarketDataRuntimeState.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using TradingAppLibrary.Runtime;

namespace TradingAppLibrary.Interfaces;

public interface IMarketDataRuntimeState
{
    DateTime StartedAtUtc { get; }

    void SetHistoricalSyncRunning(string exchangeCode, bool running);
    void SetRealtimeConnected(string exchangeCode, bool connected);
    void TouchRealtimeMessage(string exchangeCode);
    void RecordValidationError(string exchangeCode, long count = 1);
    void RecordValidationWarning(string exchangeCode, long count = 1);
    void RecordError(string exchangeCode, string error);
    void ClearError(string exchangeCode);

    MarketDataRuntimeSnapshot GetSnapshot();
}
```

B43. TradingAppLibrary/Interfaces/IMarketDataStreamControl.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
namespace TradingAppLibrary.Interfaces;

public interface IMarketDataStreamControl
{
    void Register(string exchangeCode, CancellationTokenSource streamCancellation);
    void Unregister(string exchangeCode, CancellationTokenSource streamCancellation);
    void RequestRestart(string exchangeCode);
}
```

B44. TradingAppLibrary/Interfaces/IMarketDataSyncService.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
namespace TradingAppLibrary.Interfaces;

public interface IMarketDataSyncService
{
    Task SyncExchangeSymbolsAsync(string exchangeCode, CancellationToken cancellationToken = default);
    Task SyncHistoricalCandlesAsync(int symbolId, string interval = "1m", CancellationToken cancellationToken = default);
    Task SyncExchangeHistoricalDataAsync(string exchangeCode, string interval = "1m", CancellationToken cancellationToken = default);
    Task StartRealtimeStreamAsync(string exchangeCode, string interval, CancellationToken cancellationToken);
}
```

B45. TradingAppLibrary/Interfaces/ISymbolRepository.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Interfaces;

public interface ISymbolRepository
{
    Task<IEnumerable<Symbol>> GetAllAsync();
    Task<IEnumerable<Symbol>> GetAllActiveSymbolsAsync();
}
```



```
Task<IEnumerable<Symbol>> GetAllActiveSymbolsAsync(int exchangeId);
Task<IEnumerable<Symbol>> GetSymbolsByExchangeAsync(int exchangeId);
Task<IEnumerable<Symbol>> SearchAsync(string query);
Task<IEnumerable<Symbol>> SearchAsync(int exchangeId, string query);
Task<Symbol?> GetByIdAsync(int id);
Task<Symbol?> GetByNameAsync(int exchangeId, string name);
Task AddAsync(Symbol symbol);
Task AddRangeAsync(IEnumerable<Symbol> symbols);
Task SaveChangesAsync();
}
```

B46. TradingAppLibrary/Interfaces/ISymbolService.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using TradingAppLibrary.DTO;

namespace TradingAppLibrary.Interfaces;

public interface ISymbolService
{
    Task<IEnumerable<SymbolResponseDto>> GetAllSymbolsAsync();
    Task<IEnumerable<SymbolResponseDto>> GetSymbolsByExchangeAsync(int exchangeId);
    Task<IEnumerable<SymbolResponseDto>> SearchSymbolsAsync(string query);
    Task<IEnumerable<SymbolResponseDto>> SearchSymbolsAsync(int exchangeId, string query);
    Task<SymbolResponseDto?> GetSymbolByIdAsync(int symbolId);
    Task<List<SymbolResponseDto>> GetActiveSymbolsAsync(int exchangeId);
    Task<SymbolStatusUpdateResultDto?> UpdateSymbolStatusAsync(int symbolId, bool isActive);
}
```

B47. TradingAppLibrary/Interfaces/ISystemHealthService.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using TradingAppLibrary.DTO;

namespace TradingAppLibrary.Interfaces;

public interface ISystemHealthService
{
    Task<SystemHealthDto> GetHealthAsync(CancellationToken cancellationToken = default);
}
```

B48. TradingAppLibrary/Interfaces/ITokenService.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using System;
using System.Collections.Generic;
using System.Text;

namespace TradingAppLibrary.Interfaces
{
    public interface ITokenService
    {
        string GenerateToken(int userId, string username, string role);
    }
}
```

B49. TradingAppLibrary/Interfaces/IUserRepository.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Interfaces;

public interface IUserRepository
{
    Task<User?> GetByIdAsync(int id);
    Task<User?> GetByEmailAsync(string email);
    Task<User?> GetByUsernameAsync(string username);
    Task<User?> GetByLoginIdentifierAsync(string identifier);
    Task<IEnumerable<User>> GetAllUsersAsync();
    Task AddAsync(User user);
    Task UpdateAsync(User user);
    Task DeleteAsync(int id);
    Task<UserPreferences?> GetPreferencesByUserIdAsync(string userId);
    Task SavePreferencesAsync(UserPreferences preferences);
    Task<bool> HardDeleteUserAsync(int userId);
}
```

B50. TradingAppLibrary/Interfaces/IUserService.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using System;
using System.Collections.Generic;
using System.Text;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Interfaces
{
    public interface IUserService
    {
        Task<User?> GetUserByIdAsync(int id);
        Task<User?> GetUserByEmailAsync(string email);
        Task<User?> GetUserByUsernameAsync(string username);
        Task<IEnumerable<User>> GetAllUsersAsync();
        Task UpdateUserAsync(User user);
        Task DeleteUserAsync(int id);
        Task<UserPreferencesDto?> GetUserPreferencesAsync(string userId);
        Task SaveUserPreferencesAsync(string userId, UserPreferencesDto dto);
        Task<bool> ChangePasswordAsync(int userId, string currentPassword, string newPassword);
        Task<bool> HardDeleteUserAsync(int userId);
    }
}
```

B51. TradingAppLibrary/Interfaces/IWatchlistRepository.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Interfaces;

public interface IWatchlistRepository
{
    Task<IEnumerable<Watchlist>> GetWatchlistsByUserIdAsync(int userId);
    Task<Watchlist?> GetByIdAsync(int id);
    Task AddAsync(Watchlist watchlist);
    Task AddItemAsync(WatchlistItem item);
    Task RemoveItemAsync(int watchlistId, int symbolId);
    Task DeleteAsync(int id);
}
```

B52. TradingAppLibrary/Interfaces/IWatchlistService.cs

Interface/kontrakt for service eller repository. Reducerer kobling og gør dependency injection og mocking lettere.

```
using TradingAppLibrary.DTO;

namespace TradingAppLibrary.Interfaces;

public interface IWatchlistService
{
    Task<IEnumerable<WatchlistResponseDto>> GetUserWatchlistsAsync(int userId);
    Task<WatchlistResponseDto> CreateWatchlistAsync(int userId, string name);
    Task<bool> AddSymbolToWatchlistAsync(AddWatchlistItemDto dto);
    Task<bool> RemoveSymbolFromWatchlistAsync(int watchlistId, int symbolId);
}
```

B53. TradingAppLibrary/Mappings/AlertMappingExtensions.cs

Mapping extensions, som konverterer mellem database-entiteter og DTO'er på en ensartet måde.

```
using TradingAppLibrary.DTO;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Mappings;

public static class AlertMappingExtensions
{
    // Behandler to response dto.
    public static AlertResponseDto ToResponseDto(this Alert alert) => new()
    {
        Id = alert.Id,
        UserId = alert.UserId,
        SymbolId = alert.SymbolId,
        Symbol = alert.Symbol?.Name ?? string.Empty,
        Exchange = alert.Symbol?.Exchange?.Code ?? string.Empty,
        TargetPrice = alert.TargetPrice,
    }
}
```

```

    Condition = alert.Condition,
    IsTriggered = alert.IsTriggered,
    IsActive = alert.IsActive,
    Note = alert.Note,
    CreatedAt = alert.CreatedAt
};

// Behandler to response dtos.
public static List<AlertResponseDto> ToResponseDtos(this IEnumerable<Alert> alerts) =>
    alerts?.Select(alert => alert.ToResponseDto()).ToList() ?? new List<AlertResponseDto>();

// Behandler to entity.
public static Alert ToEntity(this CreateAlertDto dto, string userId, int symbolId, string condition) => new()
{
    Id = Guid.NewGuid().ToString(),
    UserId = userId,
    SymbolId = symbolId,
    TargetPrice = dto.TargetPrice,
    Condition = condition,
    Note = string.IsNullOrEmpty(dto.Note) ? null : dto.Note.Trim(),
    IsActive = true,
    IsTriggered = false,
    CreatedAt = DateTime.UtcNow
};
}

```

B54. TradingAppLibrary/Mappings/CandleMappingExtensions.cs

Mapping extensions, som konverterer mellem database-entiteter og DTO'er på en ensartet måde.

```

using TradingAppLibrary.DTO;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Mappings;

public static class CandleMappingExtensions
{
    // Behandler to entity.
    public static Candle ToEntity(this ExchangeCandleDto source, int symbolId) => new()
    {
        SymbolId = symbolId,
        Interval = source.Interval.Trim().ToLowerInvariant(),
        OpenTime = DateTimeOffset.FromUnixTimeMilliseconds(source.Time).UtcDateTime,
        Open = source.Open,
        High = source.High,
        Low = source.Low,
        Close = source.Close,
        Volume = source.Volume
    };

    // Behandler to response dto.
    public static CandleResponseDto ToResponseDto(
        this Candle candle,
        Symbol symbol,
        string? interval = null,
        bool isClosed = true)
    {
        var utcTime = candle.OpenTime.Kind == DateTimeKind.Unspecified
            ? DateTime.SpecifyKind(candle.OpenTime, DateTimeKind.Utc)
            : candle.OpenTime.ToUniversalTime();

        return new CandleResponseDto(
            new DateTimeOffset(utcTime).ToUnixTimeMilliseconds(),
            candle.Open,
            candle.High,
            candle.Low,
            candle.Close,
            candle.Volume,
            symbol.Id,
            symbol.Name,
            symbol.Exchange?.Code ?? string.Empty,
            (interval ?? candle.Interval).Trim().ToLowerInvariant(),
            isClosed);
    }

    // Behandler to response dto.
    public static CandleResponseDto ToResponseDto(
        this ExchangeCandleDto candle,
        int symbolId,
        string symbolName,
        string exchangeCode) => new(
            candle.Time,
            candle.Open,

```

```

        candle.High,
        candle.Low,
        candle.Close,
        candle.Volume,
        symbolId,
        symbolName,
        exchangeCode,
        candle.Interval,
        candle.IsClosed);
    }

```

B55. TradingAppLibrary/Mappings/SymbolMappingExtensions.cs

Mapping extensions, som konverterer mellem database-entiteter og DTO'er på en ensartet måde.

```

using TradingAppLibrary.DTO;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Mappings;

public static class SymbolMappingExtensions
{
    // Behandler to response dto.
    public static SymbolResponseDto ToResponseDto(this Symbol symbol) => new(
        symbol.Id,
        symbol.ExchangeId,
        symbol.Exchange?.Code ?? string.Empty,
        symbol.Exchange?.Name ?? string.Empty,
        symbol.Name,
        symbol.BaseAsset,
        symbol.QuoteAsset,
        symbol.IsActive);

    // Behandler to response dtos.
    public static IEnumerable<SymbolResponseDto> ToResponseDtos(this IEnumerable<Symbol> symbols) =>
        symbols?.Select(symbol => symbol.ToResponseDto()) ?? Enumerable.Empty<SymbolResponseDto>();

    // Behandler to entity.
    public static Symbol ToEntity(this ExchangeSymbolDto source, int exchangeId) => new()
    {
        ExchangeId = exchangeId,
        Name = source.Name.Trim().ToUpperInvariant(),
        BaseAsset = source.BaseAsset.Trim().ToUpperInvariant(),
        QuoteAsset = source.QuoteAsset.Trim().ToUpperInvariant(),
        IsActive = false
    };

    // Opdaterer from exchange.
    public static void UpdateFromExchange(this Symbol target, ExchangeSymbolDto source)
    {
        target.Name = source.Name.Trim().ToUpperInvariant();
        target.BaseAsset = source.BaseAsset.Trim().ToUpperInvariant();
        target.QuoteAsset = source.QuoteAsset.Trim().ToUpperInvariant();

        // IsActive bevarer bevidst.
        // Det angiver, om applikationen skal indsamle data for symbolet.
    }
}

```

B56. TradingAppLibrary/Mappings/UserMappingExtensions.cs

Mapping extensions, som konverterer mellem database-entiteter og DTO'er på en ensartet måde.

```

using System;
using System.Collections.Generic;
using System.Text;
using TradingAppLibrary.Data;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Enums;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Mappings
{
    public static class UserMappingExtensions
    {
        // 1. User-entitet -> UserManagementDto
        public static UserManagementDto ToDto(this User user)
        {
            if (user == null) return null!;

            return new UserManagementDto(
                user.Id,

```

```

        user.Username ?? "",
        user.Email ?? "",
        user.Role ?? UserRole.User.ToString(),
        user.FirstName ?? "",
        user.LastName ?? "",
        user.IsDeleted,
        user.DeletionRequestedAt
    );
}

// 2. IEnumerable<User> -> IEnumerable<UserManagementDto>
public static IEnumerable<UserManagementDto> ToDtoList(this IEnumerable<User> users)
{
    return users?.Select(u => u.ToDto()) ?? Enumerable.Empty<UserManagementDto>();
}

// 3. User-entitet + token -> AuthResponseDto
public static AuthResponseDto ToAuthResponseDto(this User user, string token)
{
    if (user == null) return null!;

    return new AuthResponseDto(
        Token: token,
        Username: user.Username ?? "",
        Email: user.Email ?? "",
        Role: user.Role ?? UserRole.User.ToString(),
        FirstName: user.FirstName ?? "",
        LastName: user.LastName ?? ""
    );
}

// 4. RegisterDto -> User-entitet (ved registrering)
public static User ToEntity(this RegisterDto dto)
{
    if (dto == null) return null!;

    return new User
    {
        FirstName = dto.FirstName?.Trim() ?? "",
        LastName = dto.LastName?.Trim() ?? "",
        Username = dto.Username?.Trim() ?? "",
        Email = dto.Email?.Trim().ToLower() ?? "",
        PasswordHash = PasswordHasher.HashPassword(dto.Password),
        Role = UserRole.User.ToString(),
        CreatedAt = System.DateTime.UtcNow,
        IsDeleted = false
    };
}

// 5. UserProfile/UserManagementDto -> opdatering af eksisterende User-entitet
public static void UpdateFromDto(this User user, UserManagementDto dto)
{
    if (user == null || dto == null) return;

    user.FirstName = dto.FirstName?.Trim() ?? "";
    user.LastName = dto.LastName?.Trim() ?? "";
    user.Email = dto.Email?.Trim().ToLower() ?? "";
}
}
}

```

B57. TradingAppLibrary/Mappings/WatchlistMappingExtensions.cs

Mapping extensions, som konverterer mellem database-entiteter og DTO'er på en ensartet måde.

```

using TradingAppLibrary.DTO;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Mappings;

public static class WatchlistMappingExtensions
{
    // Behandler to response dto.
    public static WatchlistItemResponseDto ToResponseDto(this WatchlistItem item) => new(
        item.Id,
        item.SymbolId,
        item.Symbol?.Name ?? string.Empty,
        item.Symbol?.Exchange?.Code ?? string.Empty,
        item.AddedAt);

    // Behandler to response dto.
    public static WatchlistResponseDto ToResponseDto(this Watchlist watchlist) => new(

```

```

    watchlist.Id,
    watchlist.Name,
    watchlist.Items?.Select(item => item.ToResponseDto()).ToList() ?? new List<WatchlistItemResponseDto>());

// Behandler to response dtos.
public static IEnumerable<WatchlistResponseDto> ToResponseDtos(this IEnumerable<Watchlist> watchlists) =>
    watchlists?.Select(watchlist => watchlist.ToResponseDto()) ?? Enumerable.Empty<WatchlistResponseDto>();

// Behandler to entity.
public static WatchlistItem ToEntity(this AddWatchlistItemDto dto) => new()
{
    WatchlistId = dto.WatchlistId,
    SymbolId = dto.SymbolId,
    AddedAt = DateTime.UtcNow
};
}

```

B58. TradingAppLibrary/Models/Alert.cs

Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace TradingAppLibrary.Models;

[Table("Alerts")]
public class Alert
{
    [Key]
    public string Id { get; set; } = Guid.NewGuid().ToString();

    [Required]
    public string UserId { get; set; } = string.Empty;

    public int SymbolId { get; set; }
    public Symbol Symbol { get; set; } = null!;

    [Column(TypeName = "decimal(18,8)")]
    public decimal TargetPrice { get; set; }

    [Required, MaxLength(20)]
    public string Condition { get; set; } = "CROSSES_UP";

    public bool IsTriggered { get; set; }
    public bool IsActive { get; set; } = true;

    [MaxLength(255)]
    public string? Note { get; set; }

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

```

B59. TradingAppLibrary/Models/Candle.cs

Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.

```

namespace TradingAppLibrary.Models;

public class Candle
{
    public long Id { get; set; }
    public int SymbolId { get; set; }
    public Symbol Symbol { get; set; } = null!;
    public string Interval { get; set; } = "1m";
    public DateTime OpenTime { get; set; }
    public decimal Open { get; set; }
    public decimal High { get; set; }
    public decimal Low { get; set; }
    public decimal Close { get; set; }
    public decimal Volume { get; set; }
}

```

B60. TradingAppLibrary/Models/ChartDrawing.cs

Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.

```

namespace TradingAppLibrary.Models;

```

```
public class ChartDrawing
{
    public long Id { get; set; }
    public int UserId { get; set; }
    public User User { get; set; } = null!;
    public int SymbolId { get; set; }
    public Symbol Symbol { get; set; } = null!;
    public string Interval { get; set; } = "1h";
    public string DrawingType { get; set; } = string.Empty;
    public string DataJson { get; set; } = "{}";
    public bool IsVisible { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
}
```

B61. TradingAppLibrary/Models/EmailSettings.cs

Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.

```
namespace TradingAppLibrary.Models;

public class EmailSettings
{
    public bool Enabled { get; set; }
    public string Host { get; set; } = string.Empty;
    public int Port { get; set; } = 587;
    public bool EnableSsl { get; set; } = true;
    public string Username { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
    public string FromEmail { get; set; } = string.Empty;
    public string FromName { get; set; } = "TradingPro";
    public string FrontendBaseUrl { get; set; } = "http://localhost:5173";
    public int ResetTokenExpiryMinutes { get; set; } = 30;
}
```

B62. TradingAppLibrary/Models/Exchange.cs

Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.

```
namespace TradingAppLibrary.Models;

public class Exchange
{
    public int Id { get; set; }
    public string Code { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public bool IsActive { get; set; } = true;
    public ICollection<Symbol> Symbols { get; set; } = new List<Symbol>();
}
```

B63. TradingAppLibrary/Models/JwtSettings.cs

Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.

```
using System;
using System.Collections.Generic;
using System.Text;

namespace TradingAppLibrary.Models
{
    public class JwtSettings
    {
        public string Secret { get; set; } = string.Empty;
        public string Issuer { get; set; } = string.Empty;
        public string Audience { get; set; } = string.Empty;
        public int ExpiryMinutes { get; set; } = 60;
    }
}
```

B64. TradingAppLibrary/Models/Symbol.cs

Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.

```
namespace TradingAppLibrary.Models;

public class Symbol
{
    public int Id { get; set; }
    public int Exchangeld { get; set; }
    public Exchange Exchange { get; set; } = null!;
    public string Name { get; set; } = string.Empty;
    public string BaseAsset { get; set; } = string.Empty;
}
```

```
public string QuoteAsset { get; set; } = string.Empty;
public bool IsActive { get; set; } = false;
public ICollection<Candle> Candles { get; set; } = new List<Candle>();
public ICollection<ChartDrawing> ChartDrawings { get; set; } = new List<ChartDrawing>();
}
```

B65. TradingAppLibrary/Models/User.cs

Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.

```
using System;
using System.Collections.Generic;
using System.Text;

namespace TradingAppLibrary.Models
{
    public class User
    {
        public int Id { get; set; }
        public string FirstName { get; set; } = string.Empty;
        public string LastName { get; set; } = string.Empty;
        public string Username { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string PasswordHash { get; set; } = string.Empty;
        public string Role { get; set; } = "User";
        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
        public bool IsDeleted { get; set; } = false;
        public DateTime? DeletionRequestedAt { get; set; }
        public string? PasswordResetTokenHash { get; set; }
        public DateTime? PasswordResetTokenExpiresAtUtc { get; set; }
        public ICollection<Watchlist> Watchlists { get; set; } = new List<Watchlist>();
        public ICollection<ChartDrawing> ChartDrawings { get; set; } = new List<ChartDrawing>();
    }
}
```

B66. TradingAppLibrary/Models/UserPreferences.cs

Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Text;

namespace TradingAppLibrary.Models
{
    [Table("UserPreferences")]
    public class UserPreferences
    {
        [Key]
        public string Id { get; set; } = Guid.NewGuid().ToString();

        [Required]
        public string UserId { get; set; } = string.Empty;

        // Gemmer alle chart-, indikator- og layoutindstillinger som JSON.
        [Required]
        public string SettingsJson { get; set; } = "{}";

        public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
    }
}
```

B67. TradingAppLibrary/Models/Watchlist.cs

Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.

```
using System;
using System.Collections.Generic;
using System.Text;

namespace TradingAppLibrary.Models
{
    public class Watchlist
    {
        public int Id { get; set; }
        public string Name { get; set; } = "Favorilerim";
        public int UserId { get; set; }
        public User User { get; set; } = null!;
        public ICollection<WatchlistItem> Items { get; set; } = new List<WatchlistItem>();
    }
}
```



```
}  
}
```

B68. TradingAppLibrary/Models/WatchlistItem.cs

Domæne-/EF Core-model, der repræsenterer data og relationer i TradingPros database.

```
namespace TradingAppLibrary.Models;  
  
public class WatchlistItem  
{  
    public int Id { get; set; }  
    public int WatchlistId { get; set; }  
    public Watchlist Watchlist { get; set; } = null!;  
    public int SymbolId { get; set; }  
    public Symbol Symbol { get; set; } = null!;  
    public DateTime AddedAt { get; set; } = DateTime.UtcNow;  
}
```

B69. TradingAppLibrary/Repositories/CandleRepository.cs

Databaseadgang for candles. Håndterer 1m-tabellen, TimescaleDB views for større timeframes, endTime-pagination og candle-opdateringer.

```
using Microsoft.EntityFrameworkCore;  
using TradingAppLibrary.Data;  
using TradingAppLibrary.Interfaces;  
using TradingAppLibrary.Models;  
  
namespace TradingAppLibrary.Repositories;  
  
public class CandleRepository : ICandleRepository  
{  
    private readonly AppDbContext _context;  
  
    public CandleRepository(AppDbContext context) => _context = context;  
  
    // Henter candles for det valgte symbol og interval.  
    public async Task<IEnumerable<Candle>> GetCandlesAsync(int symbolId, string interval, int limit = 1000, long? endTime = null)  
    {  
        var cleanInterval = interval.Trim().ToLowerInvariant();  
        var endDate = endTime.HasValue  
            ? DateTimeOffset.FromUnixTimeMilliseconds(endTime.Value).UtcDateTime  
            : (DateTime?)null;  
  
        if (cleanInterval == "1m")  
        {  
            var query = _context.Candles.AsNoTracking()  
                .Where(c => c.SymbolId == symbolId && c.Interval == "1m");  
  
            if (endDate.HasValue)  
                query = query.Where(c => c.OpenTime < endDate.Value);  
  
            return await query.OrderByDescending(c => c.OpenTime)  
                .Take(limit).OrderBy(c => c.OpenTime).ToListAsync();  
        }  
  
        var viewName = GetViewNameForInterval(cleanInterval);  
        if (string.IsNullOrEmpty(viewName))  
            throw new ArgumentException($"Desteklenmeyer zaman dilimi: {interval}");  
  
        string sqlQuery;  
        var parameters = new List<object> { symbolId };  
  
        if (endDate.HasValue)  
        {  
            parameters.Add(endDate.Value);  
            parameters.Add(limit);  
            sqlQuery = $"@  
SELECT  
    0 AS ""Id"",  
    ""SymbolId"",  
    '{cleanInterval}' AS ""Interval"",  
    ""OpenTime"",  
    ""Open"",  
    ""High"",  
    ""Low"",  
    ""Close"",  
    ""Volume""  
FROM public.{viewName}"
```

```

        WHERE ""SymbolId"" = {{0}} AND ""OpenTime"" < {{1}}
        ORDER BY ""OpenTime"" DESC
        LIMIT {{2}}";
    }
    else
    {
        parameters.Add(limit);
        sqlQuery = $"@
        SELECT
            0 AS ""Id"",
            ""SymbolId"",
            '{cleanInterval}' AS ""Interval"",
            ""OpenTime"",
            ""Open"",
            ""High"",
            ""Low"",
            ""Close"",
            ""Volume""
        FROM public.{viewName}
        WHERE ""SymbolId"" = {{0}}
        ORDER BY ""OpenTime"" DESC
        LIMIT {{1}}";
    }

    var result = await _context.Candles.FromSqlRaw(sqlQuery, parameters.ToArray())
        .AsNoTracking().ToListAsync();

    return result.OrderBy(c => c.OpenTime).ToList();
}

// Henter first candle.
public async Task<Candle?> GetFirstCandleAsync(int symbolId, string interval)
{
    var cleanInterval = interval.Trim().ToLowerInvariant();
    return await _context.Candles
        .AsNoTracking()
        .Where(c => c.SymbolId == symbolId && c.Interval == cleanInterval)
        .OrderBy(c => c.OpenTime)
        .FirstOrDefaultAsync();
}

// Finder det første manglende tidspunkt i den gemte 1m-historik.
public async Task<DateTime?> GetFirstMissingOpenTimeAsync(int symbolId, string interval)
{
    var cleanInterval = interval.Trim().ToLowerInvariant();
    if (cleanInterval != "1m")
        throw new ArgumentException("Gap detection currently supports only 1m candles.", nameof(interval));

    // Find det første interne hul mellem to gemte candles.
    // Den sidste candle ignoreres bevidst, fordi den ikke har en LEAD-række;
    // derfor bliver en åben/realtime afslutning aldrig fejlagtigt tolket som et historisk gap.
    const string sql = ""
        WITH ordered AS (
            SELECT
                "OpenTime",
                LEAD("OpenTime") OVER (ORDER BY "OpenTime") AS next_time
            FROM "Candles"
            WHERE "SymbolId" = @symbolId
            AND "Interval" = @interval
        )
        SELECT "OpenTime" + INTERVAL '1 minute'
        FROM ordered
        WHERE next_time IS NOT NULL
            AND next_time > "OpenTime" + INTERVAL '1 minute'
        ORDER BY "OpenTime"
        LIMIT 1;
        """;

    var connection = _context.Database.GetDbConnection();
    var shouldClose = connection.State != System.Data.ConnectionState.Open;

    if (shouldClose)
        await connection.OpenAsync();

    try
    {
        await using var command = connection.CreateCommand();
        command.CommandText = sql;

        var symbolParameter = command.CreateParameter();
        symbolParameter.ParameterName = "@symbolId";
        symbolParameter.Value = symbolId;
        command.Parameters.Add(symbolParameter);
    }

```

```
var intervalParameter = command.CreateParameter();
intervalParameter.ParameterName = "@interval";
intervalParameter.Value = cleanInterval;
command.Parameters.Add(intervalParameter);

var result = await command.ExecuteScalarAsync();
if (result is null || result is DBNull)
    return null;

var value = (DateTime)result;
return value.Kind == DateTimeKind.Unspecified
    ? DateTime.SpecifyKind(value, DateTimeKind.Utc)
    : value.ToUniversalTime();
}
finally
{
    if (shouldClose)
        await connection.CloseAsync();
}
}

// Gemmer en candle eller opdaterer den eksisterende candle med samme åbningstid.
public async Task AddOrUpdateCandleAsync(Candle candle)
{
    candle.Interval = "1m";
    var existing = await _context.Candles.FirstOrDefaultAsync(c =>
        c.SymbolId == candle.SymbolId && c.Interval == "1m" && c.OpenTime == candle.OpenTime);

    if (existing == null)
        await _context.Candles.AddAsync(candle);
    else
    {
        existing.Open = candle.Open;
        existing.High = candle.High;
        existing.Low = candle.Low;
        existing.Close = candle.Close;
        existing.Volume = candle.Volume;
    }

    await _context.SaveChangesAsync();
}

// Gemmer en samling candles og opdaterer eksisterende rækker uden at oprette dubletter.
public async Task AddOrUpdateRangeAsync(IEnumerable<Candle> candles)
{
    var list = candles.ToList();
    if (list.Count == 0) return;

    foreach (var candle in list)
        candle.Interval = "1m";

    var grouped = list.GroupBy(c => new { c.SymbolId, c.Interval });
    foreach (var group in grouped)
    {
        var minTime = group.Min(c => c.OpenTime);
        var maxTime = group.Max(c => c.OpenTime);

        var existing = await _context.Candles
            .Where(c => c.SymbolId == group.Key.SymbolId && c.Interval == group.Key.Interval &&
                c.OpenTime >= minTime && c.OpenTime <= maxTime)
            .ToDictionaryAsync(c => c.OpenTime);

        foreach (var candle in group)
        {
            if (existing.TryGetValue(candle.OpenTime, out var current))
            {
                current.Open = candle.Open;
                current.High = candle.High;
                current.Low = candle.Low;
                current.Close = candle.Close;
                current.Volume = candle.Volume;
            }
            else
            {
                await _context.Candles.AddAsync(candle);
            }
        }
    }

    await _context.SaveChangesAsync();
}
```

```

// Tilføjer range.
public async Task AddRangeAsync(IEnumerable<Candle> candles)
{
    await _context.Candles.AddRangeAsync(candles);
    await _context.SaveChangesAsync();
}

// Henter alerts by user id.
public async Task<List<Alert>> GetAlertsByUserIdAsync(string userId) =>
    await _context.Alerts.AsNoTracking()
        .Include(a => a.Symbol).ThenInclude(s => s.Exchange)
        .Where(a => a.UserId == userId).OrderByDescending(a => a.CreatedAt).ToListAsync();

// Henter active alerts by symbol.
public async Task<List<Alert>> GetActiveAlertsBySymbolAsync(int symbolId) =>
    await _context.Alerts.Include(a => a.Symbol).ThenInclude(s => s.Exchange)
        .Where(a => a.SymbolId == symbolId && a.IsActive && !a.IsTriggered).ToListAsync();

// Henter alert by id.
public async Task<Alert?> GetAlertByIdAsync(string id) =>
    await _context.Alerts.Include(a => a.Symbol).ThenInclude(s => s.Exchange)
        .FirstOrDefaultAsync(a => a.Id == id);

// Opretter alert.
public async Task<Alert> CreateAlertAsync(Alert alert)
{
    await _context.Alerts.AddAsync(alert);
    await _context.SaveChangesAsync();
    return alert;
}

// Sletter alert.
public async Task<bool> DeleteAlertAsync(string id, string userId)
{
    var alert = await _context.Alerts.FirstOrDefaultAsync(a => a.Id == id && a.UserId == userId);
    if (alert == null) return false;
    _context.Alerts.Remove(alert);
    await _context.SaveChangesAsync();
    return true;
}

// Opdaterer alert.
public async Task UpdateAlertAsync(Alert alert)
{
    _context.Alerts.Update(alert);
    await _context.SaveChangesAsync();
}

// Henter view name for interval.
private static string GetViewNameForInterval(string interval) => interval switch
{
    "3m" => "candles_3m",
    "5m" => "candles_5m",
    "15m" => "candles_15m",
    "30m" => "candles_30m",
    "1h" => "candles_1h",
    "2h" => "candles_2h",
    "4h" => "candles_4h",
    "1d" or "1day" => "candles_1d",
    "1w" or "1week" => "candles_1w",
    "1mon" or "1month" => "candles_1mon",
    "1y" or "1year" => "candles_1y",
    _ => string.Empty
};
}

```

B70. TradingAppLibrary/Repositories/SymbolRepository.cs

Repository-implementation med databaseoperationer for området. Holder databaseadgang adskilt fra controllers og forretningslogik.

```

using Microsoft.EntityFrameworkCore;
using TradingAppLibrary.Data;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Repositories;

public class SymbolRepository : ISymbolRepository
{

```

```

private readonly AppDbContext _context;

public SymbolRepository(AppDbContext context) => _context = context;

// Henter all.
public async Task<IEnumerable<Symbol>> GetAllAsync() =>
    await _context.Symbols.AsNoTracking().Include(s => s.Exchange)
        .OrderBy(s => s.Exchange.Code).ThenBy(s => s.Name).ToListAsync();

// Henter all active symbols.
public async Task<IEnumerable<Symbol>> GetAllActiveSymbolsAsync() =>
    await _context.Symbols.AsNoTracking().Include(s => s.Exchange)
        .Where(s => s.IsActive && s.Exchange.IsActive)
        .OrderBy(s => s.Exchange.Code).ThenBy(s => s.Name).ToListAsync();

// Henter all active symbols.
public async Task<IEnumerable<Symbol>> GetAllActiveSymbolsAsync(int exchangeId) =>
    await _context.Symbols.AsNoTracking().Include(s => s.Exchange)
        .Where(s => s.ExchangeId == exchangeId && s.IsActive && s.Exchange.IsActive)
        .OrderBy(s => s.Name).ToListAsync();

// Henter symbols by exchange.
public async Task<IEnumerable<Symbol>> GetSymbolsByExchangeAsync(int exchangeId) =>
    await _context.Symbols.Include(s => s.Exchange)
        .Where(s => s.ExchangeId == exchangeId).OrderBy(s => s.Name).ToListAsync();

// Søger efter den relevante operation.
public async Task<IEnumerable<Symbol>> SearchAsync(string query)
{
    var search = query.Trim().ToUpperInvariant();
    return await _context.Symbols.AsNoTracking().Include(s => s.Exchange)
        .Where(s => s.Name.ToUpper().Contains(search) || s.BaseAsset.ToUpper().Contains(search) || s.QuoteAsset.ToUpper().Contains(search))
        .OrderBy(s => s.Name).ToListAsync();
}

// Søger efter den relevante operation.
public async Task<IEnumerable<Symbol>> SearchAsync(int exchangeId, string query)
{
    var search = query.Trim().ToUpperInvariant();
    return await _context.Symbols.AsNoTracking().Include(s => s.Exchange)
        .Where(s => s.ExchangeId == exchangeId &&
            (s.Name.ToUpper().Contains(search) || s.BaseAsset.ToUpper().Contains(search) || s.QuoteAsset.ToUpper().Contains(search)))
        .OrderBy(s => s.Name).ToListAsync();
}

// Henter by id.
public async Task<Symbol?> GetByIdAsync(int id) =>
    await _context.Symbols.Include(s => s.Exchange).FirstOrDefaultAsync(s => s.Id == id);

// Henter by name.
public async Task<Symbol?> GetByNameAsync(int exchangeId, string name)
{
    var formatted = name.Trim().ToUpperInvariant();
    return await _context.Symbols.Include(s => s.Exchange)
        .FirstOrDefaultAsync(s => s.ExchangeId == exchangeId && s.Name.ToUpper() == formatted);
}

// Tilføjer den relevante operation.
public async Task AddAsync(Symbol symbol)
{
    Normalize(symbol);
    await _context.Symbols.AddAsync(symbol);
    await _context.SaveChangesAsync();
}

// Tilføjer range.
public async Task AddRangeAsync(IEnumerable<Symbol> symbols)
{
    var list = symbols.ToList();
    foreach (var symbol in list) Normalize(symbol);
    await _context.Symbols.AddRangeAsync(list);
    await _context.SaveChangesAsync();
}

// Gemmer changes.
public Task SaveChangesAsync() => _context.SaveChangesAsync();

// Normaliserer den relevante operation.
private static void Normalize(Symbol symbol)
{
    symbol.Name = symbol.Name.Trim().ToUpperInvariant();
}

```

```
symbol.BaseAsset = symbol.BaseAsset.Trim().ToUpperInvariant();  
symbol.QuoteAsset = symbol.QuoteAsset.Trim().ToUpperInvariant();  
}  
}
```

B71. TradingAppLibrary/Repositories/UserRepository.cs

Repository-implementation med databaseoperationer for området. Holder databaseadgang adskilt fra controllers og forretningslogik.

```
using System;  
using System.Collections.Generic;  
using System.Text;  
using TradingAppLibrary.Data;  
using TradingAppLibrary.Interfaces;  
using TradingAppLibrary.Models;  
using Microsoft.EntityFrameworkCore;  
  
namespace TradingAppLibrary.Repositories  
{  
    public class UserRepository: IUserRepository  
    {  
        private readonly AppDbContext _context;  
  
        public UserRepository(AppDbContext context)  
        {  
            _context = context;  
        }  
  
        // Henter by id.  
        public async Task<User?> GetByIdAsync(int id)  
        {  
            return await _context.Users.FindAsync(id);  
        }  
  
        // Henter by email.  
        public async Task<User?> GetByEmailAsync(string email)  
        {  
            if (string.IsNullOrEmpty(email)) return null;  
  
            string normalizedEmail = email.Trim().ToLower();  
  
            return await _context.Users  
                .IgnoreQueryFilters()  
                .FirstOrDefaultAsync(u => u.Email.ToLower() == normalizedEmail);  
        }  
  
        // Henter by username.  
        public async Task<User?> GetByUsernameAsync(string username)  
        {  
            var normalizedUsername = username.Trim().ToLowerInvariant();  
  
            return await _context.Users  
                .IgnoreQueryFilters()  
                .FirstOrDefaultAsync(u => u.Username.ToLower() == normalizedUsername);  
        }  
  
        // Henter by login identifier.  
        public async Task<User?> GetByLoginIdentifierAsync(string identifier)  
        {  
            if (string.IsNullOrEmpty(identifier)) return null;  
  
            var normalized = identifier.Trim().ToLowerInvariant();  
            return await _context.Users  
                .IgnoreQueryFilters()  
                .FirstOrDefaultAsync(u =>  
                    u.Email.ToLower() == normalized ||  
                    u.Username.ToLower() == normalized);  
        }  
  
        // Henter all users.  
        public async Task<IEnumerable<User>> GetAllUsersAsync()  
        {  
            return await _context.Users.ToListAsync();  
        }  
  
        // Tilføjer den relevante operation.  
        public async Task AddAsync(User user)  
        {  

```

```

        await _context.Users.AddAsync(user);
        await _context.SaveChangesAsync();
    }

    // Opdaterer den relevante operation.
    public async Task UpdateAsync(User user)
    {
        _context.Users.Update(user);
        await _context.SaveChangesAsync();
    }

    // Sletter den relevante operation.
    public async Task DeleteAsync(int id)
    {
        var user = await GetByldAsync(id);
        if (user != null)
        {
            user.IsDeleted = true;
            user.DeletionRequestedAt = DateTime.UtcNow;
            _context.Users.Update(user);
            await _context.SaveChangesAsync();
        }
    }

    // Behandler hard delete user.
    public async Task<bool> HardDeleteUserAsync(int userId)
    {
        var user = await _context.Users.FindAsync(userId);
        if (user == null)
            return false;

        // Da UserId er en string, sammenligner vi med userId.ToString().
        var stringUserId = userId.ToString();
        var preferences = await _context.UserPreferences
            .FirstOrDefaultAsync(p => p.UserId == stringUserId);

        if (preferences != null)
        {
            _context.UserPreferences.Remove(preferences);
        }

        // Fjern brugeren permanent fra databasen.
        _context.Users.Remove(user);
        await _context.SaveChangesAsync();
        return true;
    }

    // Henter preferences by user id.
    public async Task<UserPreferences?> GetPreferencesByUserIdAsync(string userId)
    {
        return await _context.UserPreferences.FirstOrDefaultAsync(p => p.UserId == userId);
    }

    // Gemmer preferences.
    public async Task SavePreferencesAsync(UserPreferences preferences)
    {
        var existing = await GetPreferencesByUserIdAsync(preferences.UserId);
        if (existing == null)
        {
            await _context.UserPreferences.AddAsync(preferences);
        }
        else
        {
            existing.SettingsJson = preferences.SettingsJson;
            existing.UpdatedAt = DateTime.UtcNow;
            _context.UserPreferences.Update(existing);
        }
        await _context.SaveChangesAsync();
    }
}

```

B72. TradingAppLibrary/Repositories/WatchlistRepository.cs

Repository-implementation med databaseoperationer for området. Holder databaseadgang adskilt fra controllers og forretningslogik.

```

using Microsoft.EntityFrameworkCore;
using TradingAppLibrary.Data;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Repositories;

```

```

public class WatchlistRepository : IWatchlistRepository
{
    private readonly AppDbContext _context;

    public WatchlistRepository(AppDbContext context) => _context = context;

    // Henter watchlists by user id.
    public async Task<IEnumerable<Watchlist>> GetWatchlistsByUserIdAsync(int userId) =>
        await _context.Watchlists.AsNoTracking()
            .Include(w => w.Items).ThenInclude(i => i.Symbol).ThenInclude(s => s.Exchange)
            .Where(w => w.UserId == userId).OrderBy(w => w.Name).ToListAsync();

    // Henter by id.
    public async Task<Watchlist?> GetByIdAsync(int id) =>
        await _context.Watchlists
            .Include(w => w.Items).ThenInclude(i => i.Symbol).ThenInclude(s => s.Exchange)
            .FirstOrDefaultAsync(w => w.Id == id);

    // Tilføjer den relevante operation.
    public async Task AddAsync(Watchlist watchlist)
    {
        await _context.Watchlists.AddAsync(watchlist);
        await _context.SaveChangesAsync();
    }

    // Tilføjer item.
    public async Task AddItemAsync(WatchlistItem item)
    {
        await _context.WatchlistItems.AddAsync(item);
        await _context.SaveChangesAsync();
    }

    // Fjerner item.
    public async Task RemoveItemAsync(int watchlistId, int symbolId)
    {
        var item = await _context.WatchlistItems
            .FirstOrDefaultAsync(wi => wi.WatchlistId == watchlistId && wi.SymbolId == symbolId);
        if (item == null) return;
        _context.WatchlistItems.Remove(item);
        await _context.SaveChangesAsync();
    }

    // Sletter den relevante operation.
    public async Task DeleteAsync(int id)
    {
        var watchlist = await _context.Watchlists.FirstOrDefaultAsync(w => w.Id == id);
        if (watchlist == null) return;
        _context.Watchlists.Remove(watchlist);
        await _context.SaveChangesAsync();
    }
}

```

B73. TradingAppLibrary/Runtime/HistoricalSyncQueue.cs

Runtime-state, queue eller stream-control, som hjælper background services med at koordinere market-data arbejde.

```

using System.Collections.Concurrent;
using System.Threading.Channels;
using TradingAppLibrary.Interfaces;

namespace TradingAppLibrary.Runtime;

public sealed class HistoricalSyncQueue : IHistoricalSyncQueue
{
    private readonly Channel<HistoricalSyncJob> _channel = Channel.CreateUnbounded<HistoricalSyncJob>(
        new UnboundedChannelOptions
        {
            SingleReader = true,
            SingleWriter = false,
            AllowSynchronousContinuations = false
        });

    private readonly ConcurrentDictionary<int, byte> _pendingOrRunning = new();

    // Behandler try enqueue.
    public bool TryEnqueue(int symbolId, string exchangeCode)
    {
        if (symbolId <= 0 || string.IsNullOrEmpty(exchangeCode))
            return false;
    }
}

```



```

    if (!_pendingOrRunning.TryAdd(symbolId, 0))
        return false;

    var job = new HistoricalSyncJob(
        symbolId,
        exchangeCode.Trim().ToUpperInvariant());

    if (_channel.Writer.TryWrite(job))
        return true;

    _pendingOrRunning.TryRemove(symbolId, out _);
    return false;
}

// Behandler dequeue.
public ValueTask<HistoricalSyncJob> DequeueAsync(Cancellation_token cancellation_token) =>
    _channel.Reader.ReadAsync(cancellation_token);

// Behandler complete.
public void Complete(int symbolId) =>
    _pendingOrRunning.TryRemove(symbolId, out _);

// Kontrollerer om pending or running.
public bool IsPendingOrRunning(int symbolId) =>
    _pendingOrRunning.ContainsKey(symbolId);
}

```

B74. TradingAppLibrary/Runtime/MarketDataRuntimeState.cs

Runtime-state, queue eller stream-control, som hjælper background services med at koordinere market-data arbejde.

```

using System.Collections.Concurrent;
using TradingAppLibrary.Interfaces;

namespace TradingAppLibrary.Runtime;

public sealed class MarketDataRuntimeState : IMarketDataRuntimeState
{
    private readonly ConcurrentDictionary<string, MutableExchangeRuntimeState> _states =
        new(StringComparer.Ordinal, ignoreCase);

    public DateTime StartedAtUtc { get; } = DateTime.UtcNow;

    // Indstiller historical sync running.
    public void SetHistoricalSyncRunning(string exchangeCode, bool running)
    {
        var state = GetOrCreate(exchangeCode);
        lock (state.SyncRoot)
        {
            if (running)
            {
                state.HistoricalSyncActivityCount++;
                if (state.HistoricalSyncActivityCount == 1)
                {
                    state.HistoricalSyncStartedAtUtc = DateTime.UtcNow;
                    state.HistoricalSyncCompletedAtUtc = null;
                }
            }
            else
            {
                state.HistoricalSyncActivityCount = Math.Max(0, state.HistoricalSyncActivityCount - 1);
                if (state.HistoricalSyncActivityCount == 0)
                    state.HistoricalSyncCompletedAtUtc = DateTime.UtcNow;
            }
        }
    }

    // Indstiller realtime connected.
    public void SetRealtimeConnected(string exchangeCode, bool connected)
    {
        var state = GetOrCreate(exchangeCode);
        lock (state.SyncRoot)
        {
            state.RealtimeConnected = connected;
            if (connected)
                state.RealtimeConnectedAtUtc = DateTime.UtcNow;
        }
    }

    // Behandler touch realtime message.
    public void TouchRealtimeMessage(string exchangeCode)
    {
    }
}

```

```
{
    var state = GetOrCreate(exchangeCode);
    lock (state.SyncRoot)
    {
        state.RealtimeConnected = true;
        state.RealtimeConnectedAtUtc ??= DateTime.UtcNow;
        state.LastRealtimeMessageAtUtc = DateTime.UtcNow;
    }
}

// Registrerer validation error.
public void RecordValidationError(string exchangeCode, long count = 1)
{
    if (count <= 0)
        return;

    var state = GetOrCreate(exchangeCode);
    Interlocked.Add(ref state.ValidationErrorMessageCount, count);
}

// Registrerer validation warning.
public void RecordValidationWarning(string exchangeCode, long count = 1)
{
    if (count <= 0)
        return;

    var state = GetOrCreate(exchangeCode);
    Interlocked.Add(ref state.ValidationWarningCount, count);
}

// Registrerer error.
public void RecordError(string exchangeCode, string error)
{
    var state = GetOrCreate(exchangeCode);
    lock (state.SyncRoot)
    {
        state.LastError = error;
        state.LastErrorAtUtc = DateTime.UtcNow;
    }
}

// Nulstiller error.
public void ClearError(string exchangeCode)
{
    var state = GetOrCreate(exchangeCode);
    lock (state.SyncRoot)
    {
        state.LastError = null;
        state.LastErrorAtUtc = null;
    }
}

// Henter snapshot.
public MarketDataRuntimeSnapshot GetSnapshot()
{
    var exchangeStates = _states
        .OrderBy(x => x.Key, StringComparer.OrdinalIgnoreCase)
        .ToDictionary(
            x => x.Key,
            x => ToSnapshot(x.Key, x.Value),
            StringComparer.OrdinalIgnoreCase);

    return new MarketDataRuntimeSnapshot(
        StartedAtUtc,
        exchangeStates);
}

// Henter or create.
private MutableExchangeRuntimeState GetOrCreate(string exchangeCode)
{
    var normalized = exchangeCode.Trim().ToUpperInvariant();
    return _states.GetOrCreate(normalized, _ => new MutableExchangeRuntimeState());
}

// Behandler to snapshot.
private static ExchangeRuntimeSnapshot ToSnapshot(
    string exchangeCode,
    MutableExchangeRuntimeState state)
{
    lock (state.SyncRoot)
    {
        return new ExchangeRuntimeSnapshot(

```

```

        exchangeCode,
        state.HistoricalSyncActivityCount > 0,
        state.HistoricalSyncStartedAtUtc,
        state.HistoricalSyncCompletedAtUtc,
        state.RealtimeConnected,
        state.RealtimeConnectedAtUtc,
        state.LastRealtimeMessageAtUtc,
        state.LastError,
        state.LastErrorAtUtc,
        Interlocked.Read(ref state.ValidationErrorCount),
        Interlocked.Read(ref state.ValidationWarningCount));
    }
}

private sealed class MutableExchangeRuntimeState
{
    public object SyncRoot { get; } = new();
    public int HistoricalSyncActivityCount { get; set; }
    public DateTime? HistoricalSyncStartedAtUtc { get; set; }
    public DateTime? HistoricalSyncCompletedAtUtc { get; set; }
    public bool RealtimeConnected { get; set; }
    public DateTime? RealtimeConnectedAtUtc { get; set; }
    public DateTime? LastRealtimeMessageAtUtc { get; set; }
    public string? LastError { get; set; }
    public DateTime? LastErrorAtUtc { get; set; }
    public long ValidationErrorCount;
    public long ValidationWarningCount;
}

// Behandler market data runtime snapshot.
public sealed record MarketDataRuntimeSnapshot(
    DateTime StartedAtUtc,
    IReadOnlyDictionary<string, ExchangeRuntimeSnapshot> Exchanges);

// Behandler exchange runtime snapshot.
public sealed record ExchangeRuntimeSnapshot(
    string ExchangeCode,
    bool HistoricalSyncRunning,
    DateTime? HistoricalSyncStartedAtUtc,
    DateTime? HistoricalSyncCompletedAtUtc,
    bool RealtimeConnected,
    DateTime? RealtimeConnectedAtUtc,
    DateTime? LastRealtimeMessageAtUtc,
    string? LastError,
    DateTime? LastErrorAtUtc,
    long ValidationErrorCount,
    long ValidationWarningCount);

```

B75. TradingAppLibrary/Runtime/MarketDataStreamControl.cs

Runtime-state, queue eller stream-control, som hjælper background services med at koordinere market-data arbejde.

```

using System.Collections.Concurrent;
using TradingAppLibrary.Interfaces;

namespace TradingAppLibrary.Runtime;

public sealed class MarketDataStreamControl : IMarketDataStreamControl
{
    private readonly ConcurrentDictionary<string, CancellationTokenSource> _streams =
        new(StringComparer.Ordinal, ignoreCase);

    // Opretter en ny bruger efter validering af de indsendte oplysninger.
    public void Register(string exchangeCode, CancellationTokenSource streamCancellation)
    {
        var code = Normalize(exchangeCode);
        _streams[code] = streamCancellation;
    }

    // Behandler unregister.
    public void Unregister(string exchangeCode, CancellationTokenSource streamCancellation)
    {
        var code = Normalize(exchangeCode);
        if (_streams.TryGetValue(code, out var current) && ReferenceEquals(current, streamCancellation))
            _streams.TryRemove(code, out _);
    }

    // Behandler restart.
    public void RequestRestart(string exchangeCode)
    {
        var code = Normalize(exchangeCode);
    }
}

```

```
if (!_streams.TryGetValue(code, out var cancellation))
    return;

try
{
    cancellation.Cancel();
}
catch (ObjectDisposedException)
{
    _streams.TryRemove(code, out _);
}
}

// Normaliserer den relevante operation.
private static string Normalize(string exchangeCode) =>
    exchangeCode.Trim().ToUpperInvariant();
}
```

B76. TradingAppLibrary/Services/AiAnalysisService.cs

Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.

```
using Microsoft.Extensions.Configuration;
using System.Globalization;
using System.Text;
using System.Text.Json;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingAppLibrary.Services;

public class AiAnalysisService : IAiAnalysisService
{
    private readonly ICandleRepository _candleRepository;
    private readonly ISymbolRepository _symbolRepository;
    private readonly HttpClient _httpClient;
    private readonly string _apiKey;

    public AiAnalysisService(
        ICandleRepository candleRepository,
        ISymbolRepository symbolRepository,
        HttpClient httpClient,
        IConfiguration configuration)
    {
        _candleRepository = candleRepository;
        _symbolRepository = symbolRepository;
        _httpClient = httpClient;
        _apiKey = configuration["Groq:ApiKey"] ?? throw new ArgumentNullException("Groq:ApiKey configuration is missing.");
    }

    // Behandler analyze market.
    public async Task<AiAnalysisResponseDto> AnalyzeMarketAsync(AiAnalysisRequestDto request)
    {
        var language = NormalizeLanguage(request.Language);
        var symbol = await _symbolRepository.GetByIdAsync(request.SymbolId);

        if (symbol == null)
        {
            return new AiAnalysisResponseDto(
                request.SymbolId,
                string.Empty,
                string.Empty,
                Localize(language,
                    tr: "Sembol bulunamadı.",
                    en: "Symbol could not be found.",
                    da: "Symbolet blev ikke fundet."),
                DateTime.UtcNow);
        }

        var candles = (await _candleRepository
            .GetCandlesAsync(request.SymbolId, request.Interval, 50))
            .ToList();

        if (candles.Count == 0)
        {
            return new AiAnalysisResponseDto(
                symbol.Id,
                symbol.Name,
                symbol.Exchange.Code,
                Localize(language,
                    tr: $"{symbol.Exchange.Code}/{symbol.Name} için analiz edilecek yeterli mum verisi bulunamadı.",
                    en: $"There is not enough candle data to analyze {symbol.Exchange.Code}/{symbol.Name}."),
                DateTime.UtcNow);
        }
    }
}
```

```
da: $"Der er ikke nok candle-data til at analysere {symbol.Exchange.Code}/{symbol.Name}.",
DateTime.UtcNow);
}

var candleData = string.Join("\n", candles.TakeLast(20).Select(c =>
    $"Time: {c.OpenTime:yyyy-MM-dd HH:mm} | Open: {c.Open.ToString(CultureInfo.InvariantCulture)} | High: {c.High.ToString(CultureInfo.InvariantCulture)} |
Low: {c.Low.ToString(CultureInfo.InvariantCulture)} | Close: {c.Close.ToString(CultureInfo.InvariantCulture)} | Volume:
{c.Volume.ToString(CultureInfo.InvariantCulture)}"));

var (systemPrompt, userPrompt) = BuildPrompt(
    language,
    symbol.Exchange.Code,
    symbol.Name,
    request.Interval,
    candleData);

var requestBody = new
{
    model = "llama-3.3-70b-versatile",
    messages = new[]
    {
        new { role = "system", content = systemPrompt },
        new { role = "user", content = userPrompt }
    },
    temperature = 0.2
};

using var httpRequest = new HttpRequestMessage(
    HttpMethod.Post,
    "https://api.groq.com/openai/v1/chat/completions");

httpRequest.Headers.Add("Authorization", $"Bearer {_apiKey}");
httpRequest.Content = new StringContent(
    JsonSerializer.Serialize(requestBody),
    Encoding.UTF8,
    "application/json");

try
{
    var response = await _httpClient.SendAsync(httpRequest);
    var jsonResponse = await response.Content.ReadAsStringAsync();

    if (!response.IsSuccessStatusCode)
    {
        return new AiAnalysisResponseDto(
            symbol.Id,
            symbol.Name,
            symbol.Exchange.Code,
            Localize(language,
                tr: $"Groq API hatası [{response.StatusCode}].",
                en: $"Groq API error [{response.StatusCode}].",
                da: $"Groq API-fejl [{response.StatusCode}].",
                DateTime.UtcNow);
    }

    using var doc = JsonDocument.Parse(jsonResponse);
    var aiText = doc.RootElement
        .GetProperty("choices")[0]
        .GetProperty("message")
        .GetProperty("content")
        .GetString();

    return new AiAnalysisResponseDto(
        symbol.Id,
        symbol.Name,
        symbol.Exchange.Code,
        aiText ?? Localize(language,
            tr: "Yapay zeka analiz sonucu üretilmedi.",
            en: "The AI could not produce an analysis result.",
            da: "AI'en kunne ikke generere et analyseresultat."),
        DateTime.UtcNow);
}
catch (Exception ex)
{
    return new AiAnalysisResponseDto(
        symbol.Id,
        symbol.Name,
        symbol.Exchange.Code,
        Localize(language,
            tr: $"İstek işlenirken hata oluştu: {ex.Message}",
            en: $"An error occurred while processing the request: {ex.Message}",
            da: $"Der opstod en fejl under behandlingen af anmodningen: {ex.Message}"),
        DateTime.UtcNow);
}
```

```

    }
}

// Normaliserer language.
private static string NormalizeLanguage(string? language)
{
    if (string.IsNullOrEmpty(language))
        return "en";

    var normalized = language.Trim().ToLowerInvariant();
    if (normalized.StartsWith("tr")) return "tr";
    if (normalized.StartsWith("da")) return "da";
    return "en";
}

// Behandler localize.
private static string Localize(string language, string tr, string en, string da) =>
    language switch
    {
        "tr" => tr,
        "da" => da,
        _ => en
    };

private static (string SystemPrompt, string UserPrompt) BuildPrompt(
    string language,
    string exchangeCode,
    string symbolName,
    string interval,
    string candleData)
{
    return language switch
    {
        "tr" => (
            "Sen uzman bir kripto ve finansal piyasalar teknik analistisin. Yalnız verilen piyasa verilerine dayan ve kesin kazanç vaadinde bulunma.",
            $"Aşağıda {exchangeCode}/{symbolName} sembolünün {interval} periyodundaki son mum verileri yer almaktadır: {candleData}"
        ),
        "da" => (
            "Du er ekspert i teknisk analyse af kryptovaluta- og finansmarkedet. Brug kun de angivne markedetsdata, og giv ingen garanti for afkast.",
            $"Nedenfor er de seneste candle-data for {exchangeCode}/{symbolName} på intervallet {interval}: {candleData}"
        ),
        _ => (
            "You are an expert technical analyst for cryptocurrency and financial markets. Base the analysis only on the supplied market data and do not guarantee returns.",
            $"Below are the latest candle data for {exchangeCode}/{symbolName} on the {interval} timeframe: {candleData}"
        )
    };
}

Based on these data, provide a short, clear, and easy-to-understand technical analysis:
- Overall trend (Bullish, Bearish, or Sideways)
- Key support and resistance levels
- Short-term outlook and Buy/Sell/Neutral assessment

Respond in English and use bullet points.")
}
}

```

B77. TradingAppLibrary/Services/AuthService.cs

Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.

```

using System.Security.Cryptography;
using System.Text;
using Microsoft.Extensions.Logging;
using Microsoft.Extensions.Options;

```

```
using TradingAppLibrary.Data;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Enums;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Services;

public class AuthService : IAuthService
{
    private readonly IUserRepository _userRepository;
    private readonly ITokenService _tokenService;
    private readonly IEmailService _emailService;
    private readonly EmailSettings _emailSettings;
    private readonly ILogger<AuthService> _logger;

    public AuthService(
        IUserRepository userRepository,
        ITokenService tokenService,
        IEmailService emailService,
        IOptions<EmailSettings> emailSettings,
        ILogger<AuthService> logger)
    {
        _userRepository = userRepository;
        _tokenService = tokenService;
        _emailService = emailService;
        _emailSettings = emailSettings.Value;
        _logger = logger;
    }

    // Validerer loginoplysninger og opretter et token for en gyldig bruger.
    public async Task<AuthResponseDto?> LoginAsync(LoginDto request)
    {
        var identifier = string.IsNullOrEmpty(request.Identifier)
            ? request.Email
            : request.Identifier;

        if (string.IsNullOrEmpty(identifier) || string.IsNullOrEmpty(request.Password))
            return null;

        var user = await _userRepository.GetByLoginIdentifierAsync(identifier);

        if (user == null || !PasswordHasher.VerifyPassword(request.Password, user.PasswordHash))
            return null;

        if (user.IsDeleted)
        {
            if (user.DeletionRequestedAt.HasValue)
            {
                var daysPassed = (DateTime.UtcNow - user.DeletionRequestedAt.Value).TotalDays;
                if (daysPassed > 30)
                    throw new InvalidOperationException("Bu hesap silinme süresini (30 gün) doldurduğu için kapatılmıştır.");
            }

            user.IsDeleted = false;
            user.DeletionRequestedAt = null;
            await _userRepository.UpdateAsync(user);
        }

        var userRole = user.Role ?? UserRole.User.ToString();
        var token = _tokenService.GenerateToken(user.Id, user.Username, userRole);
        return new AuthResponseDto(token, user.Username, user.Email, userRole, user.FirstName, user.LastName);
    }

    // Opretter en ny bruger efter validering af de indsendte oplysninger.
    public async Task<AuthResponseDto?> RegisterAsync(RegisterDto request)
    {
        if (string.IsNullOrEmpty(request.Password) || request.Password.Length < 8)
            throw new InvalidOperationException("Şifre en az 8 karakter olmalıdır.");

        var existingUserByEmail = await _userRepository.GetByEmailAsync(request.Email);
        if (existingUserByEmail != null)
            throw new InvalidOperationException("Bu e-posta adresi zaten kullanılmaktadır.");

        var existingUserByUsername = await _userRepository.GetByUsernameAsync(request.Username);
        if (existingUserByUsername != null)
            throw new InvalidOperationException("Bu kullanıcı adı zaten kullanılmaktadır.");

        var defaultRole = UserRole.User.ToString();
        var newUser = new User
        {

```

```
        FirstName = request.FirstName.Trim(),
        LastName = request.LastName.Trim(),
        Username = request.Username.Trim(),
        Email = request.Email.Trim().ToLowerInvariant(),
        PasswordHash = PasswordHasher.HashPassword(request.Password),
        Role = defaultRole,
        IsDeleted = false,
        CreatedAt = DateTime.UtcNow
    };

    await _userRepository.AddAsync(newUser);
    var token = _tokenService.GenerateToken(newUser.Id, newUser.Username, newUser.Role);
    return new AuthResponseDto(token, newUser.Username, newUser.Email, newUser.Role, newUser.FirstName, newUser.LastName);
}

// Opretter et password-reset token og sender reset-linket til brugeren.
public async Task RequestPasswordResetAsync(ForgotPasswordDto request, CancellationToken cancellationToken = default)
{
    if (string.IsNullOrEmpty(request.Identifier))
        return;

    var user = await _userRepository.GetByLoginIdentifierAsync(request.Identifier);
    if (user == null || user.IsDeleted)
        return;

    var tokenBytes = RandomNumberGenerator.GetBytes(32);
    var token = Base64UrlEncode(tokenBytes);
    user.PasswordResetTokenHash = HashResetToken(token);
    user.PasswordResetTokenExpiresAtUtc = DateTime.UtcNow.AddMinutes(Math.Max(5, _emailSettings.ResetTokenExpiryMinutes));
    await _userRepository.UpdateAsync(user);

    var baseUrl = _emailSettings.FrontendBaseUrl.TrimEnd('/');
    var resetUrl = $"{baseUrl}/?resetToken={Uri.EscapeDataString(token)}&userId={user.Id}";

    try
    {
        await _emailService.SendPasswordResetAsync(user.Email, user.FirstName, resetUrl, cancellationToken);
    }
    catch (Exception ex)
    {
        // Vi sender ikke en fejl til controlleren, så vi undgår account enumeration.
        _logger.LogError(ex, "Password reset email could not be sent for UserId={UserId}", user.Id);
    }
}

// Validerer reset-tokenet og gemmer brugerens nye password.
public async Task<bool> ResetPasswordAsync(ResetPasswordDto request, CancellationToken cancellationToken = default)
{
    cancellationToken.ThrowIfCancellationRequested();

    if (string.IsNullOrEmpty(request.Token) ||
        string.IsNullOrEmpty(request.NewPassword) ||
        request.NewPassword.Length < 8)
        return false;

    var user = await _userRepository.GetByIdAsync(request.UserId);
    if (user == null || user.IsDeleted)
        return false;

    var storedResetTokenHash = user.PasswordResetTokenHash;
    var resetTokenExpiresAtUtc = user.PasswordResetTokenExpiresAtUtc;

    if (string.IsNullOrEmpty(storedResetTokenHash) ||
        !resetTokenExpiresAtUtc.HasValue ||
        resetTokenExpiresAtUtc.Value < DateTime.UtcNow)
        return false;

    var incomingHash = HashResetToken(request.Token);
    if (!FixedTimeEquals(storedResetTokenHash, incomingHash))
        return false;

    user.PasswordHash = PasswordHasher.HashPassword(request.NewPassword);
    user.PasswordResetTokenHash = null;
    user.PasswordResetTokenExpiresAtUtc = null;
    await _userRepository.UpdateAsync(user);
    return true;
}

// Behandler account deletion.
public async Task<bool> RequestAccountDeletionAsync(int userId)
{
}
```



```

var user = await _userRepository.GetByIdAsync(userId);
if (user == null || user.IsDeleted) return false;

user.IsDeleted = true;
user.DeletionRequestedAt = DateTime.UtcNow;
await _userRepository.UpdateAsync(user);
return true;
}

// Henter all users.
public async Task<IEnumerable<UserManagementDto>> GetAllUsersAsync()
{
    var users = await _userRepository.GetAllUsersAsync();
    return users.Select(u => new UserManagementDto(
        u.Id, u.Username, u.Email, u.Role ?? UserRole.User.ToString(),
        u.FirstName, u.LastName, u.IsDeleted, u.DeletionRequestedAt));
}

// Logger brugeren ud den relevante operation.
public Task LogoutAsync() => Task.CompletedTask;

// Hasher reset token.
private static string HashResetToken(string token)
{
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes(token));
    return Convert.ToHexString(hash);
}

// Behandler fixed time equals.
private static bool FixedTimeEquals(string storedHex, string incomingHex)
{
    try
    {
        return CryptographicOperations.FixedTimeEquals(
            Convert.FromHexString(storedHex),
            Convert.FromHexString(incomingHex));
    }
    catch
    {
        return false;
    }
}

// Behandler base 64 url encode.
private static string Base64UrlEncode(byte[] bytes) =>
    Convert.ToBase64String(bytes).TrimEnd('=').Replace('+', '-').Replace('/', '_');
}

```

B78. TradingAppLibrary/Services/BinanceService.cs

Adapter til Binance REST- og WebSocket-API. Oversætter symbols og candles til TradingPros fælles DTO/modeller.

```

using System.Globalization;
using System.Net.Http.Json;
using System.Net.WebSockets;
using System.Text;
using System.Text.Json;
using TradingAppLibrary.Constants;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingAppLibrary.Services;

public class BinanceService : IExchangeService
{
    private readonly HttpClient _httpClient;
    public string ExchangeCode => ExchangeCodes.Binance;
    public DateTime HistoricalDataStartUtc => new(2017, 1, 1, 0, 0, 0, DateTimeKind.Utc);

    public BinanceService(HttpClient httpClient)
    {
        _httpClient = httpClient;
        _httpClient.BaseAddress ??= new Uri("https://api.binance.com");
    }

    // Henter symbols.
    public async Task<IReadOnlyList<ExchangeSymbolDto>> GetSymbolsAsync(Cancellation_token cancellation_token = default)
    {
        using var response = await _httpClient.GetAsync("/api/v3/exchangeInfo", cancellation_token);
        response.EnsureSuccessStatusCode();
        await using var stream = await response.Content.ReadAsStreamAsync(cancellation_token);
        using var document = await JsonDocument.ParseAsync(stream, cancellation_token: cancellation_token);
    }

```

```

if (!document.RootElement.TryGetProperty("symbols", out var symbolsElement))
    return Array.Empty<ExchangeSymbolDto>();

var symbols = new List<ExchangeSymbolDto>();
foreach (var item in symbolsElement.EnumerateArray())
{
    var name = item.GetProperty("symbol").GetString() ?? string.Empty;
    var baseAsset = item.GetProperty("baseAsset").GetString() ?? string.Empty;
    var quoteAsset = item.GetProperty("quoteAsset").GetString() ?? string.Empty;
    var status = item.GetProperty("status").GetString() ?? string.Empty;

    if (string.IsNullOrEmpty(name) || !quoteAsset.Equals("USDT", StringComparison.OrdinalIgnoreCase))
        continue;

    symbols.Add(new ExchangeSymbolDto(
        name.ToUpperInvariant(),
        baseAsset.ToUpperInvariant(),
        quoteAsset.ToUpperInvariant(),
        status.Equals("TRADING", StringComparison.OrdinalIgnoreCase)));
}

return symbols;
}

// Henter top popular symbols.
public async Task<IReadOnlyList<string>> GetTopPopularSymbolsAsync(int topCount = 50, CancellationToken cancellationToken = default)
{
    if (topCount <= 0) return Array.Empty<string>();
    var rawList = await _httpClient.GetFromJsonAsync<List<JsonElement>>("/api/v3/ticker/24hr", cancellationToken);
    if (rawList == null) return Array.Empty<string>();

    return rawList
        .Where(x => x.TryGetProperty("symbol", out var p) &&
            (p.GetString() ?? string.Empty).EndsWith("USDT", StringComparison.OrdinalIgnoreCase))
        .OrderByDescending(x => x.TryGetProperty("quoteVolume", out var p) &&
            decimal.TryParse(p.GetString(), NumberStyles.Any, CultureInfo.InvariantCulture, out var v) ? v : 0m)
        .Take(topCount)
        .Select(x => x.GetProperty("symbol").GetString()!.ToUpperInvariant())
        .ToList();
}

// Henter historical candles.
public async Task<IReadOnlyList<ExchangeCandleDto>> GetHistoricalCandlesAsync(
    string symbol,
    string interval,
    int limit = 500,
    long? startTime = null,
    CancellationToken cancellationToken = default)
{
    if (string.IsNullOrEmpty(symbol) || string.IsNullOrEmpty(interval))
        return Array.Empty<ExchangeCandleDto>();

    limit = Math.Clamp(limit, 1, 1000);
    var formattedSymbol = symbol.Trim().ToUpperInvariant();
    var formattedInterval = interval.Trim();
    var url = $"/api/v3/klines?symbol={Uri.EscapeDataString(formattedSymbol)}&interval={Uri.EscapeDataString(formattedInterval)}&limit={limit}";
    if (startTime.HasValue) url += "&startTime={startTime.Value}";

    var raw = await _httpClient.GetFromJsonAsync<List<List<JsonElement>>>(url, cancellationToken);
    if (raw == null || raw.Count == 0) return Array.Empty<ExchangeCandleDto>();

    var now = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    return raw.Where(x => x.Count >= 7).Select(item => new ExchangeCandleDto(
        item[0].GetInt64(),
        ParseDecimal(item[1]),
        ParseDecimal(item[2]),
        ParseDecimal(item[3]),
        ParseDecimal(item[4]),
        ParseDecimal(item[5]),
        formattedSymbol,
        formattedInterval,
        item[6].GetInt64() < now)).ToList();
}

// Behandler stream real time candles.
public async Task StreamRealTimeCandlesAsync(
    IReadOnlyCollection<string> symbols,
    string interval,
    Func<ExchangeCandleDto, Task> onCandleReceived,
    CancellationToken cancellationToken)
{

```

```

var streams = string.Join('/', symbols.Where(s => !string.IsNullOrEmpty(s))
    .Select(s => $"{s.Trim().ToLowerInvariant()}@kline_{interval}"));
if (string.IsNullOrEmpty(streams)) return;

var uri = new Uri($"wss://stream.binance.com:9443/stream?streams={streams}");

using var socket = new ClientWebSocket();
await socket.ConnectAsync(uri, cancellationTokens);

var buffer = new byte[16 * 1024];
var builder = new StringBuilder();

while (socket.State == WebSocketState.Open && !cancellationTokens.IsCancellationRequested)
{
    builder.Clear();
    WebSocketReceiveResult result;
    do
    {
        result = await socket.ReceiveAsync(new ArraySegment<byte>(buffer), cancellationTokens);
        if (result.MessageType == WebSocketMessageType.Close) break;
        builder.Append(Encoding.UTF8.GetString(buffer, 0, result.Count));
    } while (!result.EndOfMessage);

    if (result.MessageType == WebSocketMessageType.Close) break;
    if (result.MessageType != WebSocketMessageType.Text) continue;

    using var doc = JsonDocument.Parse(builder.ToString());
    var root = doc.RootElement;
    var data = root.TryGetProperty("data", out var dataProperty) ? dataProperty : root;
    if (!data.TryGetProperty("k", out var kline)) continue;

    var symbol = data.TryGetProperty("s", out var symbolProperty)
        ? symbolProperty.GetString() ?? string.Empty
        : string.Empty;
    if (string.IsNullOrEmpty(symbol)) continue;

    await onCandleReceived(new ExchangeCandleDto(
        kline.GetProperty("t").GetInt64(),
        ParseDecimal(kline.GetProperty("o")),
        ParseDecimal(kline.GetProperty("h")),
        ParseDecimal(kline.GetProperty("l")),
        ParseDecimal(kline.GetProperty("c")),
        ParseDecimal(kline.GetProperty("v")),
        symbol.ToUpperInvariant(),
        kline.GetProperty("i").GetString() ?? interval,
        kline.GetProperty("x").GetBoolean()));
    }

// Fortolker decimal.
private static decimal ParseDecimal(JsonElement element)
{
    if (element.ValueKind == JsonValueKind.Number) return element.GetDecimal();
    return decimal.TryParse(element.GetString(), NumberStyles.Any, CultureInfo.InvariantCulture, out var value) ? value : 0m;
}

```

B79. TradingAppLibrary/Services/CandleService.cs

Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.

```

using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Mappings;

namespace TradingAppLibrary.Services;

public class CandleService : ICandleService
{
    private readonly ICandleRepository _candleRepository;
    private readonly ISymbolRepository _symbolRepository;

    public CandleService(ICandleRepository candleRepository, ISymbolRepository symbolRepository)
    {
        _candleRepository = candleRepository;
        _symbolRepository = symbolRepository;
    }

    // Henter candles for det valgte symbol og interval.
    public async Task<IEnumerable<CandleResponseDto>> GetCandlesAsync(
        int symbolId,

```

```
string interval,
int limit = 1000,
long? endTime = null)
{
    if (symbolId <= 0 || string.IsNullOrEmpty(interval))
        return Enumerable.Empty<CandleResponseDto>();

    if (limit <= 0) limit = 1000;

    var symbol = await _symbolRepository.GetByIdAsync(symbolId);
    if (symbol == null) return Enumerable.Empty<CandleResponseDto>();

    var candles = await _candleRepository.GetCandlesAsync(symbolId, interval, limit, endTime);
    var normalizedInterval = interval.Trim().ToLowerInvariant();

    return candles
        .OrderBy(candle => candle.OpenTime)
        .Select(candle => candle.ToResponseDto(symbol, normalizedInterval))
        .ToList();
}

// Henter user alerts.
public async Task<List<AlertResponseDto>> GetUserAlertsAsync(string userId)
{
    if (string.IsNullOrEmpty(userId)) return new();
    return (await _candleRepository.GetAlertsByUserIdAsync(userId)).ToResponseDtos();
}

// Henter active alerts by symbol.
public async Task<List<AlertResponseDto>> GetActiveAlertsBySymbolAsync(int symbolId)
{
    if (symbolId <= 0) return new();
    return (await _candleRepository.GetActiveAlertsBySymbolAsync(symbolId)).ToResponseDtos();
}

// Henter alert by id.
public async Task<AlertResponseDto?> GetAlertByIdAsync(string alertId, string userId)
{
    if (string.IsNullOrEmpty(alertId) || string.IsNullOrEmpty(userId)) return null;

    var alert = await _candleRepository.GetAlertByIdAsync(alertId);
    return alert == null || alert.UserId != userId
        ? null
        : alert.ToResponseDto();
}

// Opretter alert.
public async Task<AlertResponseDto> CreateAlertAsync(string userId, CreateAlertDto dto)
{
    if (string.IsNullOrEmpty(userId))
        throw new ArgumentException("Kullanıcı ID boş olamaz.", nameof(userId));
    if (dto.SymbolId <= 0)
        throw new ArgumentException("Geçerli bir SymbolId gereklidir.", nameof(dto.SymbolId));
    if (dto.TargetPrice <= 0)
        throw new ArgumentException("Hedef fiyat 0'dan büyük olmalıdır.", nameof(dto.TargetPrice));

    var symbol = await _symbolRepository.GetByIdAsync(dto.SymbolId)
        ?? throw new ArgumentException("Belirtilen symbol bulunamadı.", nameof(dto.SymbolId));

    var condition = string.Equals(dto.Condition, "CROSSES_DOWN", StringComparison.OrdinalIgnoreCase)
        ? "CROSSES_DOWN"
        : "CROSSES_UP";

    var alert = dto.ToEntity(userId, symbol.Id, condition);
    var created = await _candleRepository.CreateAlertAsync(alert);
    created.Symbol = symbol;

    return created.ToResponseDto();
}

// Sletter alert.
public Task<bool> DeleteAlertAsync(string alertId, string userId) =>
    string.IsNullOrEmpty(alertId) || string.IsNullOrEmpty(userId)
        ? Task.FromResult(false)
        : _candleRepository.DeleteAlertAsync(alertId, userId);

// Skifter status for alert.
public async Task<bool> ToggleAlertAsync(string alertId, string userId)
{
    if (string.IsNullOrEmpty(alertId) || string.IsNullOrEmpty(userId)) return false;
```

```

var alert = await _candleRepository.GetAlertByIdAsync(alertId);
if (alert == null || alert.UserId != userId) return false;

alert.IsActive = !alert.IsActive;
if (alert.IsActive) alert.IsTriggered = false;

await _candleRepository.UpdateAlertAsync(alert);
return true;
}

// Kontrollerer and trigger alerts.
public async Task<List<AlertResponseDto>> CheckAndTriggerAlertsAsync(int symbolId, decimal currentPrice)
{
    if (symbolId <= 0 || currentPrice <= 0) return new();

    var activeAlerts = await _candleRepository.GetActiveAlertsBySymbolAsync(symbolId);
    var triggeredAlerts = new List<AlertResponseDto>();

    foreach (var alert in activeAlerts)
    {
        var isTriggered = alert.Condition switch
        {
            "CROSSES_UP" => currentPrice >= alert.TargetPrice,
            "CROSSES_DOWN" => currentPrice <= alert.TargetPrice,
            _ => false
        };

        if (!isTriggered) continue;

        alert.IsTriggered = true;
        alert.IsActive = false;
        await _candleRepository.UpdateAlertAsync(alert);
        triggeredAlerts.Add(alert.ToResponseDto());
    }

    return triggeredAlerts;
}

```

B80. TradingAppLibrary/Services/ChartDrawingService.cs

Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.

```

using System.Text.Json;
using Microsoft.EntityFrameworkCore;
using TradingAppLibrary.Data;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Services;

public class ChartDrawingService : IChartDrawingService
{
    private static readonly HashSet<string> AllowedTypes = new(StringComparer.OrdinalIgnoreCase)
    {
        "trendLine", "horizontalLine", "rectangle", "fibonacci", "text"
    };

    private readonly AppDbContext _dbContext;

    public ChartDrawingService(AppDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    // Henter den relevante operation.
    public async Task<IReadOnlyList<ChartDrawingResponseDto>> GetAsync(
        int userId,
        int symbolId,
        string interval,
        CancellationToken cancellationToken = default)
    {
        var normalizedInterval = NormalizeInterval(interval);

        var drawings = await _dbContext.ChartDrawings
            .AsNoTracking()
            .Include(d => d.Symbol)
            .ThenInclude(s => s.Exchange)

```

```
.Where(d => d.UserId == userId && d.SymbolId == symbolId && d.Interval == normalizedInterval)
.OrderBy(d => d.Id)
.ToListAsync(cancellationToken);

return drawings.Select(ToResponseDto).ToList();
}

// Opretter den relevante operation.
public async Task<ChartDrawingResponseDto> CreateAsync(
    int userId,
    CreateChartDrawingDto dto,
    CancellationToken cancellationToken = default)
{
    if (dto.SymbolId <= 0)
        throw new ArgumentException("Geçerli bir SymbolId zorunludur.");

    var interval = NormalizeInterval(dto.Interval);
    var drawingType = NormalizeDrawingType(dto.DrawingType);
    ValidateJson(dto.Data);

    var symbol = await _dbContext.Symbols
        .Include(s => s.Exchange)
        .FirstOrDefaultAsync(s => s.Id == dto.SymbolId, cancellationToken)
        ?? throw new InvalidOperationException("Sembol bulunamadı.");

    var entity = new ChartDrawing
    {
        UserId = userId,
        SymbolId = dto.SymbolId,
        Interval = interval,
        DrawingType = drawingType,
        DataJson = dto.Data.GetRawText(),
        IsVisible = dto.IsVisible,
        CreatedAt = DateTime.UtcNow,
        UpdatedAt = DateTime.UtcNow,
        Symbol = symbol
    };

    _dbContext.ChartDrawings.Add(entity);
    await _dbContext.SaveChangesAsync(cancellationToken);

    return ToResponseDto(entity);
}

// Opdaterer den relevante operation.
public async Task<ChartDrawingResponseDto?> UpdateAsync(
    int userId,
    long drawingId,
    UpdateChartDrawingDto dto,
    CancellationToken cancellationToken = default)
{
    ValidateJson(dto.Data);

    var entity = await _dbContext.ChartDrawings
        .Include(d => d.Symbol)
        .ThenInclude(s => s.Exchange)
        .FirstOrDefaultAsync(d => d.Id == drawingId && d.UserId == userId, cancellationToken);

    if (entity == null) return null;

    entity.DataJson = dto.Data.GetRawText();
    entity.IsVisible = dto.IsVisible;
    entity.UpdatedAt = DateTime.UtcNow;

    await _dbContext.SaveChangesAsync(cancellationToken);
    return ToResponseDto(entity);
}

// Sletter den relevante operation.
public async Task<bool> DeleteAsync(
    int userId,
    long drawingId,
    CancellationToken cancellationToken = default)
{
    var entity = await _dbContext.ChartDrawings
        .FirstOrDefaultAsync(d => d.Id == drawingId && d.UserId == userId, cancellationToken);

    if (entity == null) return false;

    _dbContext.ChartDrawings.Remove(entity);
    await _dbContext.SaveChangesAsync(cancellationToken);
}
```

```

        return true;
    }

    // Sletter all.
    public async Task<int> DeleteAllAsync(
        int userId,
        int symbolId,
        string interval,
        CancellationToken cancellationToken = default)
    {
        var normalizedInterval = NormalizeInterval(interval);

        return await _dbContext.ChartDrawings
            .Where(d => d.UserId == userId && d.SymbolId == symbolId && d.Interval == normalizedInterval)
            .ExecuteDeleteAsync(cancellationToken);
    }

    // Normaliserer interval.
    private static string NormalizeInterval(string interval)
    {
        if (string.IsNullOrEmpty(interval))
            throw new ArgumentException("Interval zorunludur.");

        var normalized = interval.Trim().ToLowerInvariant();
        if (normalized.Length > 10)
            throw new ArgumentException("Interval geçersiz.");

        return normalized;
    }

    // Normaliserer drawing type.
    private static string NormalizeDrawingType(string drawingType)
    {
        if (string.IsNullOrEmpty(drawingType) || !AllowedTypes.Contains(drawingType.Trim()))
            throw new ArgumentException("Desteklenmeyen çizim tipi.");

        return AllowedTypes.First(x => x.Equals(drawingType.Trim(), StringComparison.OrdinalIgnoreCase));
    }

    // Validerer json.
    private static void ValidateJson(JsonElement data)
    {
        if (data.ValueKind is JsonValueKind.Undefined or JsonValueKind.Null)
            throw new ArgumentException("Çizim verisi zorunludur.");

        if (data.GetRawText().Length > 32_000)
            throw new ArgumentException("Çizim verisi çok büyük.");
    }

    // Behandler to response dto.
    private static ChartDrawingResponseDto ToResponseDto(ChartDrawing drawing)
    {
        using var document = JsonDocument.Parse(drawing.DataJson);
        var data = document.RootElement.Clone();

        return new ChartDrawingResponseDto(
            drawing.Id,
            drawing.SymbolId,
            drawing.Symbol.Exchange?.Code ?? string.Empty,
            drawing.Symbol.Name,
            drawing.Interval,
            drawing.DrawingType,
            data,
            drawing.IsVisible,
            drawing.CreatedAt,
            drawing.UpdatedAt);
    }
}

```

B81. TradingAppLibrary/Services/ExchangeServiceFactory.cs

Finder den korrekte IExchangeService ud fra exchange-koden og holder exchange-valget ude af den øvrige forretningslogik.

```

using TradingAppLibrary.Interfaces;

namespace TradingAppLibrary.Services;

public class ExchangeServiceFactory : IExchangeServiceFactory
{

```

```

private readonly IReadOnlyDictionary<string, IExchangeService> _services;

public ExchangeServiceFactory(IEnumerable<IExchangeService> services)
{
    _services = services.ToDictionary(
        s => s.ExchangeCode.Trim().ToUpperInvariant(),
        s => s,
        StringComparer.OrdinalIgnoreCase);
}

// Henter exchange.
public IExchangeService GetExchange(string exchangeCode)
{
    if (string.IsNullOrEmpty(exchangeCode))
        throw new ArgumentException("Exchange kodu boş olamaz.", nameof(exchangeCode));

    var code = exchangeCode.Trim().ToUpperInvariant();
    return _services.TryGetValue(code, out var service)
        ? service
        : throw new NotSupportedException($"'{exchangeCode}' borsası için bir exchange service tanımlanmamış.");
}
}

```

B82. TradingAppLibrary/Services/MarketDataSyncService.cs

Central market-data service for symbol-sync, historical download, realtime behandling, validering, databasegemning og SignalR-publicering.

```

using Microsoft.AspNetCore.SignalR;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;
using Microsoft.Extensions.Logging.Abstractions;
using TradingAppLibrary.Data;
using TradingAppLibrary.Hubs;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Mappings;
using TradingAppLibrary.Models;
using TradingAppLibrary.Runtime;
using TradingAppLibrary.Validators;

namespace TradingAppLibrary.Services;

public class MarketDataSyncService : IMarketDataSyncService
{
    private readonly IExchangeServiceFactory _exchangeServiceFactory;
    private readonly ISymbolRepository _symbolRepository;
    private readonly ICandleRepository _candleRepository;
    private readonly ICandleValidator _candleValidator;
    private readonly AppDbContext _dbContext;
    private readonly IServiceScopeFactory _scopeFactory;
    private readonly IHubContext<MarketDataHub> _hubContext;
    private readonly ILogger<MarketDataSyncService> _logger;
    private readonly IMarketDataRuntimeState _runtimeState;

    // Bagudkompatibel constructor: eksisterende host-projekter behøver ikke at registrere
    // ICandleValidator med det samme. Hvis DI-registreringen findes, bruges den fulde constructor nedenfor.
    public MarketDataSyncService(
        IExchangeServiceFactory exchangeServiceFactory,
        ISymbolRepository symbolRepository,
        ICandleRepository candleRepository,
        AppDbContext dbContext,
        IServiceScopeFactory scopeFactory,
        IHubContext<MarketDataHub> hubContext)
        : this(
            exchangeServiceFactory,
            symbolRepository,
            candleRepository,
            new CandleValidator(),
            dbContext,
            scopeFactory,
            hubContext,
            NullLogger<MarketDataSyncService>.Instance,
            new MarketDataRuntimeState())
    {
    }

    public MarketDataSyncService(
        IExchangeServiceFactory exchangeServiceFactory,
        ISymbolRepository symbolRepository,
        ICandleRepository candleRepository,
        ICandleValidator candleValidator,

```



```

AppDbContext dbContext,
IServiceScopeFactory scopeFactory,
IHubContext<MarketDataHub> hubContext,
ILogger<MarketDataSyncService> logger,
IMarketDataRuntimeState runtimeState)
{
    _exchangeServiceFactory = exchangeServiceFactory;
    _symbolRepository = symbolRepository;
    _candleRepository = candleRepository;
    _candleValidator = candleValidator;
    _dbContext = dbContext;
    _scopeFactory = scopeFactory;
    _hubContext = hubContext;
    _logger = logger;
    _runtimeState = runtimeState;
}

// Synkroniserer symboler fra den valgte exchange med databasen.
public async Task SyncExchangeSymbolsAsync(
    string exchangeCode,
    CancellationToken cancellationToken = default)
{
    var normalizedCode = exchangeCode.Trim().ToUpperInvariant();
    var exchange = await _dbContext.Exchanges
        .FirstOrDefaultAsync(e => e.Code == normalizedCode, cancellationToken)
        ?? throw new InvalidOperationException($"Exchange bulunamadı: {normalizedCode}");

    var exchangeService = _exchangeServiceFactory.GetExchange(exchange.Code);
    var remoteSymbols = await exchangeService.GetSymbolsAsync(cancellationToken);
    var existingSymbols = (await _symbolRepository.GetSymbolsByExchangeAsync(exchange.Id))
        .ToDictionary(s => s.Name, StringComparer.OrdinalIgnoreCase);

    var newSymbols = new List<Symbol>();
    foreach (var remote in remoteSymbols)
    {
        if (existingSymbols.TryGetValue(remote.Name, out var existing))
        {
            existing.UpdateFromExchange(remote);
        }
        else
        {
            // Nye symboler er bevidst inaktive. IsActive styres af applikationen eller brugeren.
            newSymbols.Add(remote.ToEntity(exchange.Id));
        }
    }

    if (newSymbols.Count > 0)
        await _symbolRepository.AddRangeAsync(newSymbols);

    await _symbolRepository.SaveChangesAsync();
}

// Synkroniserer historical candles.
public async Task SyncHistoricalCandlesAsync(
    int symbolId,
    string interval = "1m",
    CancellationToken cancellationToken = default)
{
    if (!string.Equals(interval, "1m", StringComparison.OrdinalIgnoreCase))
    {
        throw new NotSupportedException(
            "Market data sync veritabanına yalnızca 1m candle kaydeder. " +
            "Üst timeframe verileri TimescaleDB aggregate viewlarından okunmalıdır.");
    }

    var symbol = await _symbolRepository.GetByIdAsync(symbolId)
        ?? throw new InvalidOperationException($"Symbol bulunamadı. Id: {symbolId}");

    if (!symbol.IsActive || !symbol.Exchange.IsActive)
        return;

    var exchangeService = _exchangeServiceFactory.GetExchange(symbol.Exchange.Code);
    var exchangeHistoryBaseline = new DateTimeOffset(exchangeService.HistoricalDataStartUtc)
        .ToUnixTimeMilliseconds();

    // HistoricalDataStartUtc gäller for hele exchange. Et symbol kan være blevet listet langt senere.
    // Spørg exchange efter symbollets første rigtige candle, så vi kan skelne en komplet
    // historik fra en database, der kun indeholder nyere realtime-candles.
    var firstRemotePage = await exchangeService.GetHistoricalCandlesAsync(
        symbol.Name,
        interval,
        1,

```

```
exchangeHistoryBaseline,
cancellationToken);

if (firstRemotePage.Count == 0)
{
    _logger.LogWarning(
        "No historical baseline candle returned. Exchange={Exchange}, Symbol={Symbol}",
        symbol.Exchange.Code,
        symbol.Name);
    return;
}

var remoteFirst = firstRemotePage.OrderBy(c => c.Time).First();
var remoteFirstValidation = _candleValidator.Validate(remoteFirst, symbol.Name, interval);
if (!remoteFirstValidation.IsValid)
{
    _runtimeState.RecordValidationError(symbol.Exchange.Code, remoteFirstValidation.Errors.Count);
    _logger.LogWarning(
        "Historical baseline candle is invalid. Exchange={Exchange}, Symbol={Symbol}, Errors={Errors}",
        symbol.Exchange.Code,
        symbol.Name,
        string.Join(" | ", remoteFirstValidation.Errors));
    return;
}

var actualHistoricalStart = remoteFirst.Time;

// Normal historical sync tilføjer kun data fremad:
// - Hvis databasen allerede har 1m-candles, fortsættes fra candlen efter MAX(OpenTime).
// - Hvis symbolet endnu ikke har gemte candles, startes der fra exchange's historiske udgangspunkt.
// Reparation af historiske gaps bør, hvis det bliver nødvendigt, håndteres af et separat vedligeholdelsesflow
// i stedet for at lade hver applikationsstart gennemgå gammel historik igen.
var startTime = await GetNextStartTimeAsync(symbol.Id, interval)
    ?? actualHistoricalStart;

var nowMs = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();

_logger.LogInformation(
    "Historical candle sync started. Exchange={Exchange}, Symbol={Symbol}, StartTime={StartTime}",
    symbol.Exchange.Code,
    symbol.Name,
    DateTimeOffset.FromUnixTimeMilliseconds(startTime));

while (startTime < nowMs && !cancellationToken.IsCancellationRequested)
{
    var remoteCandles = await exchangeService.GetHistoricalCandlesAsync(
        symbol.Name,
        interval,
        1000,
        startTime,
        cancellationToken);

    if (remoteCandles.Count == 0)
        break;

    var batchValidation = _candleValidator.ValidateBatch(
        remoteCandles,
        symbol.Name,
        interval);

    if (batchValidation.Errors.Count > 0)
    {
        _logger.LogWarning(
            "Historical candle batch contains {ErrorCount} validation errors. Exchange={Exchange}, Symbol={Symbol}. FirstErrors={Errors}",
            batchValidation.Errors.Count,
            symbol.Exchange.Code,
            symbol.Name,
            string.Join(" | ", batchValidation.Errors.Take(5)));
    }

    if (batchValidation.Warnings.Count > 0)
    {
        _runtimeState.RecordValidationWarning(symbol.Exchange.Code, batchValidation.Warnings.Count);
        _logger.LogWarning(
            "Historical candle batch contains {WarningCount} temporal warnings. Exchange={Exchange}, Symbol={Symbol}. FirstWarnings={Warnings}",
            batchValidation.Warnings.Count,
            symbol.Exchange.Code,
            symbol.Name,
            string.Join(" | ", batchValidation.Warnings.Take(5)));
    }

    var validClosedCandles = new List<Candle>(remoteCandles.Count);
```

```
foreach (var remoteCandle in remoteCandles)
{
    if (!remoteCandle.IsClosed)
        continue;

    var validation = _candleValidator.Validate(remoteCandle, symbol.Name, interval);
    if (!validation.IsValid)
    {
        _runtimeState.RecordValidationError(symbol.Exchange.Code, validation.Errors.Count);
        _logger.LogWarning(
            "Invalid historical candle ignored. Exchange={Exchange}, Symbol={Symbol}, Time={Time}, Errors={Errors}",
            symbol.Exchange.Code,
            symbol.Name,
            remoteCandle.Time,
            string.Join(" | ", validation.Errors));
        continue;
    }

    validClosedCandles.Add(remoteCandle.ToEntity(symbol.Id));
}

if (validClosedCandles.Count > 0)
    await _candleRepository.AddOrUpdateRangeAsync(validClosedCandles);

// Flyt cursoren ud fra det eksterne batch og ikke kun gyldige/lukkede rækker, da en ugyldig række ellers
// kan medføre, at den samme API-side bliver hentet igen og igen.
var last = remoteCandles.OrderBy(c => c.Time).Last();
var nextStart = GetNextOpenTime(last.Time, interval);
if (nextStart <= startTime)
{
    _logger.LogWarning(
        "Historical sync stopped because start time did not advance. Exchange={Exchange}, Symbol={Symbol}, StartTime={StartTime}, NextStart={NextStart}",
        symbol.Exchange.Code,
        symbol.Name,
        startTime,
        nextStart);
    break;
}

startTime = nextStart;

if (remoteCandles.Count < 1000)
    break;

await Task.Delay(50, cancellationToken);
}

_logger.LogInformation(
    "Historical candle sync finished. Exchange={Exchange}, Symbol={Symbol}",
    symbol.Exchange.Code,
    symbol.Name);
}

// Synkroniserer historiske candles for aktive symboler på den valgte exchange.
public async Task SyncExchangeHistoricalDataAsync(
    string exchangeCode,
    string interval = "1m",
    CancellationToken cancellationToken = default)
{
    var normalizedCode = exchangeCode.Trim().ToUpperInvariant();
    var exchange = await _dbContext.Exchanges.FirstOrDefaultAsync(
        e => e.Code == normalizedCode && e.IsActive,
        cancellationToken);

    if (exchange == null)
        return;

    var symbols = await _symbolRepository.GetAllActiveSymbolsAsync(exchange.Id);
    foreach (var symbol in symbols)
    {
        cancellationToken.ThrowIfCancellationRequested();
        await SyncHistoricalCandlesAsync(symbol.Id, interval, cancellationToken);
    }
}

// Starter realtime-streamen for aktive symboler på den valgte exchange.
public async Task StartRealtimeStreamAsync(
    string exchangeCode,
    string interval,
    CancellationToken cancellationToken)
{
    if (!string.Equals(interval, "1m", StringComparison.OrdinalIgnoreCase))
```

```
throw new NotSupportedException("Realtime persistence yalnızca 1m candle için desteklenir.");

var normalizedCode = exchangeCode.Trim().ToUpperInvariant();
var exchange = await _dbContext.Exchanges.FirstOrDefaultAsync(
    e => e.Code == normalizedCode && e.IsActive,
    cancellationTokens)
?? throw new InvalidOperationException($"Exchange aktif değil veya bulunamadı: {normalizedCode}");

var symbols = (await _symbolRepository.GetAllActiveSymbolsAsync(exchange.Id)).ToList();
if (symbols.Count == 0)
    return;

var symbolMap = symbols.ToDictionary(
    s => s.Name,
    s => new { s.Id, s.Name },
    StringComparer.OrdinalIgnoreCase);

var exchangeService = _exchangeServiceFactory.GetExchange(exchange.Code);

await exchangeService.StreamRealTimeCandlesAsync(
    symbols.Select(s => s.Name).ToArray(),
    interval,
    async remoteCandle =>
    {
        if (!symbolMap.TryGetValue(remoteCandle.Symbol, out var symbol))
            return;

        _runtimeState.TouchRealtimeMessage(exchange.Code);
        _runtimeState.ClearError(exchange.Code);

        var validation = _candleValidator.Validate(remoteCandle, symbol.Name, interval);
        if (!validation.IsValid)
        {
            _runtimeState.RecordValidationError(exchange.Code, validation.Errors.Count);
            _logger.LogWarning(
                "Invalid realtime candle ignored. Exchange={Exchange}, Symbol={Symbol}, Time={Time}, Errors={Errors}",
                exchange.Code,
                symbol.Name,
                remoteCandle.Time,
                string.Join(" | ", validation.Errors));
            return;
        }

        var responseDto = remoteCandle.ToResponseDto(
            symbol.Id,
            symbol.Name,
            exchange.Code);

        await _hubContext.Clients
            .Group(MarketDataHub.GetSymbolGroup(exchange.Code, symbol.Name))
            .SendAsync("ReceiveMarketData", responseDto, cancellationTokens);

        using var scope = _scopeFactory.CreateScope();
        var candleService = scope.ServiceProvider.GetRequiredService<ICandleService>();
        var triggeredAlerts = await candleService.CheckAndTriggerAlertsAsync(
            symbol.Id,
            remoteCandle.Close);

        foreach (var alert in triggeredAlerts)
        {
            await _hubContext.Clients
                .Group($"User_{alert.UserId}")
                .SendAsync("ReceiveAlertTriggered", alert, cancellationTokens);
        }

        if (!remoteCandle.IsClosed)
            return;

        var candleRepository = scope.ServiceProvider.GetRequiredService<ICandleRepository>();
        await candleRepository.AddOrUpdateCandleAsync(remoteCandle.ToEntity(symbol.Id));
    },
    cancellationTokens);
}

// Henter next start time.
private async Task<long?> GetNextStartTimeAsync(int symbolId, string interval)
{
    var last = (await _candleRepository.GetCandlesAsync(symbolId, interval, 1))
        .OrderByDescending(c => c.OpenTime)
        .FirstOrDefault();
}
```

```

    if (last == null)
        return null;

    var utc = last.OpenTime.Kind == DateTimeKind.Unspecified
        ? DateTime.SpecifyKind(last.OpenTime, DateTimeKind.Utc)
        : last.OpenTime.ToUniversalTime();

    return GetNextOpenTime(
        new DateTimeOffset(utc).ToUnixTimeMilliseconds(),
        interval);
}

// Henter next open time.
private static long GetNextOpenTime(long currentOpenTime, string interval)
{
    var milliseconds = interval.Trim().ToLowerInvariant() switch
    {
        "1m" => 60_000L,
        "3m" => 3 * 60_000L,
        "5m" => 5 * 60_000L,
        "15m" => 15 * 60_000L,
        "30m" => 30 * 60_000L,
        "1h" => 60 * 60_000L,
        "2h" => 2 * 60 * 60_000L,
        "4h" => 4 * 60 * 60_000L,
        "1d" => 24 * 60 * 60_000L,
        _ => throw new ArgumentException($"Desteklenmeyer interval: {interval}")
    };

    return currentOpenTime + milliseconds;
}
}

```

B83. TradingAppLibrary/Services/OkxService.cs

Adapter til OKX REST- og WebSocket-API. Håndterer OKX-specifikke endpoints og symbolformat og oversætter dem til fælles modeller.

```

using System.Collections.Concurrent;
using System.Globalization;
using System.Net.WebSockets;
using System.Text;
using System.Text.Json;
using Microsoft.Extensions.Configuration;
using TradingAppLibrary.Constants;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingAppLibrary.Services;

/// <summary>
/// OKX-adapter til offentlige SPOT-market data.
/// Autentificering er ikke nødvendig for instruments, tickers, historiske candles eller offentlige candle-streams.
/// </summary>
public sealed class OkxService : IExchangeService
{
    private readonly HttpClient _httpClient;
    private readonly string _businessWebSocketUrl;
    private readonly ConcurrentDictionary<string, long> _listingTimeCache = new(StringComparer.OrdinalIgnoreCase);

    public string ExchangeCode => ExchangeCodes.Okx;
    public DateTime HistoricalDataStartUtc => new(2017, 1, 1, 0, 0, 0, DateTimeKind.Utc);

    public OkxService(HttpClient httpClient, IConfiguration configuration)
    {
        _httpClient = httpClient;
        _httpClient.BaseAddress ??= new Uri("https://www.okx.com");
        _businessWebSocketUrl = configuration["Exchanges:OKX:BusinessWebSocketUrl"]
            ?? "wss://ws.okx.com:8443/ws/v5/business";
    }

    // Henter symbols.
    public async Task<IReadOnlyList<ExchangeSymbolDto>> GetSymbolsAsync(CancellationToken cancellationToken = default)
    {
        using var document = await GetDocumentAsync("/api/v5/public/instruments?instType=SPOT", cancellationToken);
        var data = GetDataArray(document.RootElement);
        var result = new List<ExchangeSymbolDto>();

        foreach (var item in data.EnumerateArray())
        {
            var instId = GetString(item, "instId");

```

```

        var baseAsset = GetString(item, "baseCcy");
        var quoteAsset = GetString(item, "quoteCcy");
        var state = GetString(item, "state");

        if (string.IsNullOrEmpty(instId) ||
            string.IsNullOrEmpty(baseAsset) ||
            !quoteAsset.Equals("USDT", StringComparison.OrdinalIgnoreCase))
        {
            continue;
        }

        if (item.TryGetProperty("listTime", out var listTimeElement) &&
            long.TryParse(listTimeElement.GetString(), NumberStyles.Integer, CultureInfo.InvariantCulture, out var listTime))
        {
            _listingTimeCache[instId] = listTime;
        }

        result.Add(new ExchangeSymbolDto(
            instId.ToUpperInvariant(),
            baseAsset.ToUpperInvariant(),
            quoteAsset.ToUpperInvariant(),
            state.Equals("live", StringComparison.OrdinalIgnoreCase)));
    }

    return result;
}

// Henter top popular symbols.
public async Task<IReadOnlyList<string>> GetTopPopularSymbolsAsync(
    int topCount = 50,
    CancellationToken cancellationToken = default)
{
    if (topCount <= 0) return Array.Empty<string>();

    using var document = await GetDocumentAsync("/api/v5/market/tickers?instType=SPOT", cancellationToken);
    var data = GetdataArray(document.RootElement);

    return data.EnumerateArray()
        .Select(item => new
        {
            Symbol = GetString(item, "instId"),
            QuoteVolume = ParseDecimal(item, "volCcy24h")
        })
        .Where(x => x.Symbol.EndsWith("-USDT", StringComparison.OrdinalIgnoreCase))
        .OrderByDescending(x => x.QuoteVolume)
        .Take(topCount)
        .Select(x => x.Symbol.ToUpperInvariant())
        .ToList();
}

// Henter historical candles.
public async Task<IReadOnlyList<ExchangeCandleDto>> GetHistoricalCandlesAsync(
    string symbol,
    string interval,
    int limit = 500,
    long? startTime = null,
    CancellationToken cancellationToken = default)
{
    if (string.IsNullOrEmpty(symbol) || string.IsNullOrEmpty(interval))
        return Array.Empty<ExchangeCandleDto>();

    var normalizedSymbol = symbol.Trim().ToUpperInvariant();
    var normalizedInterval = interval.Trim().ToLowerInvariant();
    var bar = ToOkxBar(normalizedInterval);
    var intervalMs = GetIntervalMilliseconds(normalizedInterval);
    var requestedLimit = Math.Clamp(limit, 1, 1000);

    // Den eksisterende market-data-synkronisering paginerer fremad fra startTime.
    // OKX bruger cursor-grænser i den modsatte retning, så adapteren anmoder om
    // afgrænsede vinduer og returnerer dem stigende for at efterligne Binares fremadgående kontrakt.
    var cursor = startTime ?? Math.Max(
        DateTimeOffset.UtcNow.ToUnixTimeMilliseconds() - (requestedLimit * intervalMs),
        new DateTimeOffset(HistoricalDataStartUtc).ToUnixTimeMilliseconds());

    var listingTime = await GetListingTimeAsync(normalizedSymbol, cancellationToken);
    if (listingTime.HasValue && cursor < listingTime.Value)
        cursor = listingTime.Value;

    var now = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    var candles = new SortedDictionary<long, ExchangeCandleDto>();
    var requestCount = 0;

```

```

while (candles.Count < requestedLimit && cursor < now && requestCount < 30)
{
    cancellationToken.ThrowIfCancellationRequested();

    var remaining = requestedLimit - candles.Count;
    var pageSize = Math.Min(100, remaining);
    var windowEnd = Math.Min(now + intervalMs, cursor + (pageSize * intervalMs));

    var url = $"api/v5/market/history-candles?instId={Uri.EscapeDataString(normalizedSymbol)}" +
        "&bar={Uri.EscapeDataString(bar)}&limit={pageSize}" +
        "&after={windowEnd}&before={Math.Max(0, cursor - 1)}";

    using var document = await GetDocumentAsync(url, cancellationToken);
    var data = GetdataArray(document.RootElement);
    requestCount++;

    var page = ParseCandles(data, normalizedSymbol, normalizedInterval)
        .Where(c => c.Time >= cursor && c.Time < windowEnd)
        .OrderBy(c => c.Time)
        .ToList();

    foreach (var candle in page)
        candles[candle.Time] = candle;

    // Flyt cursoren efter det afgrænsede vindue og ikke efter antallet af returnerede rækker. Det håndterer også
    // gyldige perioder uden handler uden at sidde fast på den samme cursor.
    cursor = windowEnd;

    if (requestCount % 10 == 0)
        await Task.Delay(120, cancellationToken);
}

return candles.Values.Take(requestedLimit).ToList();
}

// Behandler stream real time candles.
public async Task StreamRealTimeCandlesAsync(
    IReadOnlyCollection<string> symbols,
    string interval,
    Func<ExchangeCandleDto, Task> onCandleReceived,
    CancellationToken cancellationToken)
{
    var cleanSymbols = symbols
        .Where(s => !string.IsNullOrEmpty(s))
        .Select(s => s.Trim()).ToUpperInvariant()
        .Distinct(StringComparer.OrdinalIgnoreCase)
        .ToArray();

    if (cleanSymbols.Length == 0) return;

    using var socket = new ClientWebSocket();
    await socket.ConnectAsync(new Uri(_businessWebSocketUrl), cancellationToken);

    var channel = "candle" + ToOkxBar(interval.Trim()).ToLowerInvariant();

    foreach (var chunk in cleanSymbols.Chunk(100))
    {
        var subscribeMessage = JsonSerializer.Serialize(new
        {
            op = "subscribe",
            args = chunk.Select(symbol => new { channel, instId = symbol }).ToArray()
        });

        var bytes = Encoding.UTF8.GetBytes(subscribeMessage);
        await socket.SendAsync(bytes, WebSocketMessageType.Text, true, cancellationToken);
    }

    var buffer = new byte[32 * 1024];
    var builder = new StringBuilder();

    while (socket.State == WebSocketState.Open && !cancellationToken.IsCancellationRequested)
    {
        builder.Clear();
        WebSocketReceiveResult result;

        do
        {
            result = await socket.ReceiveAsync(new ArraySegment<byte>(buffer), cancellationToken);
            if (result.MessageType == WebSocketMessageType.Close) break;

```

```

        builder.Append(Encoding.UTF8.GetString(buffer, 0, result.Count));
    }
    while (!result.EndOfMessage);

    if (result.MessageType == WebSocketMessageType.Close) break;
    if (result.MessageType != WebSocketMessageType.Text) continue;

    var payload = builder.ToString();
    if (payload.Equals("pong", StringComparison.OrdinalIgnoreCase)) continue;

    using var document = JsonDocument.Parse(payload);
    var root = document.RootElement;
    if (!root.TryGetProperty("arg", out var arg) || !root.TryGetProperty("data", out var data))
        continue;

    var symbol = GetString(arg, "instId").ToUpperInvariant();
    if (string.IsNullOrEmpty(symbol)) continue;

    foreach (var candle in ParseCandles(data, symbol, interval.Trim().ToLowerInvariant()))
        await onCandleReceived(candle);
    }
}

// Henter listing time.
private async Task<long?> GetListingTimeAsync(string symbol, CancellationToken cancellationToken)
{
    if (_listingTimeCache.TryGetValue(symbol, out var cached))
        return cached;

    var url = $"/api/v5/public/instruments?instType=SPOT&instId={Uri.EscapeDataString(symbol)}";
    using var document = await GetDocumentAsync(url, cancellationToken);
    var data = GetDataArray(document.RootElement);

    foreach (var item in data.EnumerateArray())
    {
        if (!GetString(item, "instId").Equals(symbol, StringComparison.OrdinalIgnoreCase))
            continue;

        if (item.TryGetProperty("listTime", out var listTimeElement) &&
            long.TryParse(listTimeElement.GetString(), NumberStyles.Integer, CultureInfo.InvariantCulture, out var listTime))
        {
            _listingTimeCache[symbol] = listTime;
            return listTime;
        }
    }

    return null;
}

// Henter document.
private async Task<JsonDocument> GetDocumentAsync(string url, CancellationToken cancellationToken)
{
    using var response = await _httpClient.GetAsync(url, cancellationToken);
    response.EnsureSuccessStatusCode();

    await using var stream = await response.Content.ReadAsStreamAsync(cancellationToken);
    var document = await JsonDocument.ParseAsync(stream, cancellationToken: cancellationToken);

    var root = document.RootElement;
    var code = root.TryGetProperty("code", out var codeElement) ? codeElement.GetString() : "0";
    if (!string.Equals(code, "0", StringComparison.Ordinal))
    {
        var message = root.TryGetProperty("msg", out var msgElement) ? msgElement.GetString() : "Unknown OKX error";
        document.Dispose();
        throw new HttpRequestException($"OKX API error. Code={code}, Message={message}");
    }

    return document;
}

// Henter data array.
private static JsonElement GetDataArray(JsonElement root)
{
    if (!root.TryGetProperty("data", out var data) || data.ValueKind != JsonValueKind.Array)
        throw new InvalidOperationException("OKX response does not contain a valid data array.");
    return data;
}

// Fortolker candles.
private static IEnumerable<ExchangeCandleDto> ParseCandles(

```



```

    JsonElement data,
    string symbol,
    string interval)
{
    foreach (var row in data.EnumerateArray())
    {
        if (row.ValueKind != JsonValueKind.Array || row.GetArrayLength() < 9)
            continue;

        if (!long.TryParse(row[0].GetString(), NumberStyles.Integer, CultureInfo.InvariantCulture, out var time))
            continue;

        yield return new ExchangeCandleDto(
            time,
            ParseDecimal(row[1]),
            ParseDecimal(row[2]),
            ParseDecimal(row[3]),
            ParseDecimal(row[4]),
            ParseDecimal(row[5]),
            symbol,
            interval,
            string.Equals(row[8].GetString(), "1", StringComparison.Ordinal));
    }
}

// Behandler to okx bar.
private static string ToOkxBar(string interval) => interval switch
{
    "1m" => "1m",
    "3m" => "3m",
    "5m" => "5m",
    "15m" => "15m",
    "30m" => "30m",
    "1h" => "1H",
    "2h" => "2H",
    "4h" => "4H",
    "1d" => "1Dutc",
    "1w" => "1Wutc",
    "1mon" => "1Mutc",
    _ => throw new ArgumentException($"OKX için desteklenmeyen interval: {interval}")
};

// Henter interval milliseconds.
private static long GetIntervalMilliseconds(string interval) => interval switch
{
    "1m" => 60_000L,
    "3m" => 3 * 60_000L,
    "5m" => 5 * 60_000L,
    "15m" => 15 * 60_000L,
    "30m" => 30 * 60_000L,
    "1h" => 60 * 60_000L,
    "2h" => 2 * 60 * 60_000L,
    "4h" => 4 * 60 * 60_000L,
    "1d" => 24 * 60 * 60_000L,
    "1w" => 7 * 24 * 60 * 60_000L,
    "1mon" => 31 * 24 * 60 * 60_000L,
    _ => throw new ArgumentException($"OKX için desteklenmeyen interval: {interval}")
};

// Henter string.
private static string GetString(JsonElement element, string propertyName) =>
    element.TryGetProperty(propertyName, out var property) ? property.GetString() ?? string.Empty : string.Empty;

// Fortolker decimal.
private static decimal ParseDecimal(JsonElement element, string propertyName) =>
    element.TryGetProperty(propertyName, out var property) ? ParseDecimal(property) : 0m;

// Fortolker decimal.
private static decimal ParseDecimal(JsonElement element)
{
    if (element.ValueKind == JsonValueKind.Number)
        return element.GetDecimal();

    return decimal.TryParse(
        element.GetString(),
        NumberStyles.Any,
        CultureInfo.InvariantCulture,
        out var value)
        ? value
        : 0m;
}
}

```

B84. TradingAppLibrary/Services/SmtpEmailService.cs

Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.

```
using System.Net;
using System.Net.Mail;
using Microsoft.Extensions.Logging;
using Microsoft.Extensions.Options;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Services;

public class SmtpEmailService : IEmailService
{
    private readonly EmailSettings _settings;
    private readonly ILogger<SmtpEmailService> _logger;

    public SmtpEmailService(IOptions<EmailSettings> options, ILogger<SmtpEmailService> logger)
    {
        _settings = options.Value;
        _logger = logger;
    }

    // Sender password reset.
    public async Task SendPasswordResetAsync(string email, string firstName, string resetUrl, CancellationToken cancellationToken = default)
    {
        if (!_settings.Enabled)
        {
            // Udviklingshjælp: Hvis SMTP er deaktiveret, vises linket kun i backend-loggen.
            _logger.LogWarning("SMTP disabled. Password reset link for {Email}: {ResetUrl}", email, resetUrl);
            return;
        }

        using var message = new MailMessage
        {
            From = new MailAddress(_settings.FromEmail, _settings.FromName),
            Subject = "TradingPro - Şifre Sıfırlama",
            Body = $"Merhaba {firstName},\n\nŞifrenizi sıfırlamak için aşağıdaki bağlantıyı kullanın. Bu bağlantı kısa süre sonra geçersiz olacaktır.\n\n{resetUrl}\n\nBu isteği siz yapmadıysanız bu e-postayı yok sayabilirsiniz.",
            IsBodyHtml = false
        };
        message.To.Add(email);

        using var client = new SmtpClient(_settings.Host, _settings.Port)
        {
            EnableSsl = _settings.EnableSsl,
            Credentials = new NetworkCredential(_settings.Username, _settings.Password)
        };

        cancellationToken.ThrowIfCancellationRequested();
        await client.SendMailAsync(message, cancellationToken);
    }
}
```

B85. TradingAppLibrary/Services/SymbolService.cs

Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.

```
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Mappings;

namespace TradingAppLibrary.Services;

public class SymbolService : ISymbolService
{
    private readonly ISymbolRepository _symbolRepository;
    private readonly IHistoricalSyncQueue _historicalSyncQueue;
    private readonly IMarketDataStreamControl _streamControl;

    public SymbolService(
        ISymbolRepository symbolRepository,
        IHistoricalSyncQueue historicalSyncQueue,
        IMarketDataStreamControl streamControl)
    {
        _symbolRepository = symbolRepository;
        _historicalSyncQueue = historicalSyncQueue;
        _streamControl = streamControl;
    }
}
```

```
// Henter all symbols.
public async Task<IEnumerable<SymbolResponseDto>> GetAllSymbolsAsync() =>
    (await _symbolRepository.GetAllAsync()).ToResponseDtos();

// Henter symbols by exchange.
public async Task<IEnumerable<SymbolResponseDto>> GetSymbolsByExchangeAsync(int exchangeId)
{
    if (exchangeId <= 0) return Enumerable.Empty<SymbolResponseDto>();
    return (await _symbolRepository.GetSymbolsByExchangeAsync(exchangeId)).ToResponseDtos();
}

// Søger efter symbols.
public async Task<IEnumerable<SymbolResponseDto>> SearchSymbolsAsync(string query)
{
    if (string.IsNullOrEmpty(query)) return await GetAllSymbolsAsync();
    return (await _symbolRepository.SearchAsync(query.Trim())).ToResponseDtos();
}

// Søger efter symbols.
public async Task<IEnumerable<SymbolResponseDto>> SearchSymbolsAsync(int exchangeId, string query)
{
    if (exchangeId <= 0) return Enumerable.Empty<SymbolResponseDto>();
    if (string.IsNullOrEmpty(query)) return await GetSymbolsByExchangeAsync(exchangeId);
    return (await _symbolRepository.SearchAsync(exchangeId, query.Trim())).ToResponseDtos();
}

// Henter symbol by id.
public async Task<SymbolResponseDto?> GetSymbolByIdAsync(int symbolId)
{
    if (symbolId <= 0) return null;
    var symbol = await _symbolRepository.GetByIdAsync(symbolId);
    return symbol?.ToResponseDto();
}

// Henter active symbols.
public async Task<List<SymbolResponseDto>> GetActiveSymbolsAsync(int exchangeId)
{
    if (exchangeId <= 0) return new();
    return (await _symbolRepository.GetAllActiveSymbolsAsync(exchangeId)).ToResponseDtos().ToList();
}

// Opdaterer symbol status.
public async Task<SymbolStatusUpdateResultDto?> UpdateSymbolStatusAsync(
    int symbolId,
    bool isActive)
{
    if (symbolId <= 0)
        return null;

    var symbol = await _symbolRepository.GetByIdAsync(symbolId);
    if (symbol == null)
        return null;

    var exchangeCode = symbol.Exchange.Code.Trim().ToUpperInvariant();
    var changed = symbol.IsActive != isActive;
    var historicalSyncQueued = false;
    var realtimeRestartRequested = false;

    if (changed)
    {
        symbol.IsActive = isActive;
        await _symbolRepository.SaveChangesAsync();

        if (isActive)
        {
            // Aktivering sker bevidst i to faser:
            // 1) udfyld historiske 1m-data i baggrunden,
            // 2) genstart WebSocket efter backfill, så symbolet tilsluttes realtime.
            historicalSyncQueued = _historicalSyncQueue.TryEnqueue(
                symbol.Id,
                exchangeCode);
        }
        else
        {
            // Fjern straks et deaktiveret symbol fra det aktuelle WebSocket-abonnement.
            _streamControl.RequestRestart(exchangeCode);
            realtimeRestartRequested = true;
        }
    }
    else if (isActive && _historicalSyncQueue.IsPendingOrRunning(symbol.Id))
    {
        historicalSyncQueued = true;
    }
}
```

```
}

return new SymbolStatusUpdateResultDto(
    symbol.Id,
    exchangeCode,
    symbol.Name,
    symbol.IsActive,
    historicalSyncQueued,
    realtimeRestartRequested);
}
```

B86. TradingAppLibrary/Services/SystemHealthService.cs

Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.

```
using System.Data;
using Microsoft.EntityFrameworkCore;
using TradingAppLibrary.Data;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Runtime;

namespace TradingAppLibrary.Services;

public sealed class SystemHealthService : ISystemHealthService
{
    private static readonly TimeSpan RealtimeHealthyThreshold = TimeSpan.FromMinutes(3);

    private readonly AppDbContext _dbContext;
    private readonly IMarketDataRuntimeState _runtimeState;

    public SystemHealthService(
        AppDbContext dbContext,
        IMarketDataRuntimeState runtimeState)
    {
        _dbContext = dbContext;
        _runtimeState = runtimeState;
    }

    // Henter health.
    public async Task<SystemHealthDto> GetHealthAsync(
        CancellationToken cancellationToken = default)
    {
        var checkedAt = DateTime.UtcNow;
        var runtime = _runtimeState.GetSnapshot();

        var databaseHealthy = await CanConnectAsync(cancellationToken);
        var timescaleHealthy = false;
        var candlesHypertableHealthy = false;

        if (databaseHealthy)
        {
            timescaleHealthy = await IsTimescaleInstalledAsync(cancellationToken);
            if (timescaleHealthy)
                candlesHypertableHealthy = await IsCandlesHypertableAsync(cancellationToken);
        }

        var exchanges = databaseHealthy
            ? await _dbContext.Exchanges
                .AsNoTracking()
                .OrderBy(e => e.Code)
                .Select(e => new ExchangeHealthProjection(
                    e.Id,
                    e.Code,
                    e.Name,
                    e.IsActive,
                    e.Symbols.Count(s => s.IsActive),
                    e.Symbols
                        .SelectMany(s => s.Candles)
                        .Max(c => (DateTime?)c.OpenTime)))
                .ToListAsync(cancellationToken)
            : new List<ExchangeHealthProjection>();

        var exchangeHealth = exchanges
            .Select(exchange => BuildExchangeHealth(exchange, runtime, checkedAt))
            .ToList();

        var activeExchangeCount = exchangeHealth.Count(x => x.IsActive);
        var activeSymbolCount = exchangeHealth.Sum(x => x.ActiveSymbolCount);
        var lastCandle = exchangeHealth
```

```

        .Where(x => x.LastCandleTimeUtc.HasValue)
        .Select(x => x.LastCandleTimeUtc)
        .Max();

var validationErrors = runtime.Exchanges.Values.Sum(x => x.ValidationErrorCount);
var validationWarnings = runtime.Exchanges.Values.Sum(x => x.ValidationWarningCount);

var status = ResolveOverallStatus(
    databaseHealthy,
    timescaleHealthy,
    candlesHypertableHealthy,
    exchangeHealth);

return new SystemHealthDto
{
    Status = status,
    DatabaseHealthy = databaseHealthy,
    TimescaleHealthy = timescaleHealthy,
    CandlesHypertableHealthy = candlesHypertableHealthy,
    ActiveExchangeCount = activeExchangeCount,
    ActiveSymbolCount = activeSymbolCount,
    LastCandleTimeUtc = lastCandle,
    LastCandleAgeSeconds = GetAgeSeconds(lastCandle, checkedAt),
    ValidationErrorCount = validationErrors,
    ValidationWarningCount = validationWarnings,
    StartedAtUtc = runtime.StartedAtUtc,
    Uptime = checkedAt - runtime.StartedAtUtc,
    CheckedAtUtc = checkedAt,
    Exchanges = exchangeHealth
};
}

// Behandler can connect.
private async Task<bool> CanConnectAsync(Cancellation_token cancellation_token)
{
    try
    {
        return await _dbContext.Database.CanConnectAsync(cancellation_token);
    }
    catch
    {
        return false;
    }
}

// Kontrollerer om timescale installed.
private async Task<bool> IsTimescaleInstalledAsync(Cancellation_token cancellation_token)
{
    return await ExecuteBooleanScalarAsync(
        "SELECT EXISTS (SELECT 1 FROM pg_extension WHERE extname = 'timescaledb');",
        cancellation_token);
}

// Kontrollerer om candles hypertable.
private async Task<bool> IsCandlesHypertableAsync(Cancellation_token cancellation_token)
{
    return await ExecuteBooleanScalarAsync(
        "SELECT EXISTS (SELECT 1 FROM timescaledb_information.hypertables WHERE hypertable_schema = 'public' AND hypertable_name = 'Candles');",
        cancellation_token);
}

// Kører boolean scalar.
private async Task<bool> ExecuteBooleanScalarAsync(
    string sql,
    Cancellation_token cancellation_token)
{
    try
    {
        var connection = _dbContext.Database.GetDbConnection();
        var shouldClose = connection.State != ConnectionState.Open;

        if (shouldClose)
            await connection.OpenAsync(cancellation_token);

        try
        {
            await using var command = connection.CreateCommand();
            command.CommandText = sql;
            var result = await command.ExecuteScalarAsync(cancellation_token);
            return result is bool value && value;
        }
        finally
        {
            if (shouldClose)
                await connection.CloseAsync(cancellation_token);
        }
    }
    catch
    {
        return false;
    }
}

```

```

        if (shouldClose)
            await connection.CloseAsync();
    }
}
catch
{
    return false;
}
}

// Opbygger exchange health.
private static ExchangeHealthDto BuildExchangeHealth(
    ExchangeHealthProjection exchange,
    MarketDataRuntimeSnapshot runtime,
    DateTime checkedAt)
{
    runtime.Exchanges.TryGetValue(exchange.Code, out var state);

    var lastMessage = state?.LastRealtimeMessageAtUtc;
    var realtimeFresh = lastMessage.HasValue &&
        checkedAt - lastMessage.Value <= RealtimeHealthyThreshold;

    var realtimeConnected = state?.RealtimeConnected == true && realtimeFresh;

    var status = !exchange.IsActive
        ? "Disabled"
        : exchange.ActiveSymbolCount == 0
            ? "Idle"
            : state?.LastError is not null
                ? "Degraded"
                : realtimeConnected || state?.HistoricalSyncRunning == true
                    ? "Healthy"
                    : "Degraded";

    return new ExchangeHealthDto
    {
        ExchangeCode = exchange.Code,
        ExchangeName = exchange.Name,
        IsActive = exchange.IsActive,
        ActiveSymbolCount = exchange.ActiveSymbolCount,
        LastCandleTimeUtc = exchange.LastCandleTimeUtc,
        LastCandleAgeSeconds = GetAgeSeconds(exchange.LastCandleTimeUtc, checkedAt),
        HistoricalSyncRunning = state?.HistoricalSyncRunning ?? false,
        HistoricalSyncStartedAtUtc = state?.HistoricalSyncStartedAtUtc,
        HistoricalSyncCompletedAtUtc = state?.HistoricalSyncCompletedAtUtc,
        RealtimeConnected = realtimeConnected,
        RealtimeConnectedAtUtc = state?.RealtimeConnectedAtUtc,
        LastRealtimeMessageAtUtc = lastMessage,
        LastError = state?.LastError,
        LastErrorAtUtc = state?.LastErrorAtUtc,
        ValidationErrorCount = state?.ValidationErrorCount ?? 0,
        ValidationWarningCount = state?.ValidationWarningCount ?? 0,
        Status = status
    };
}

// Finder overall status.
private static string ResolveOverallStatus(
    bool databaseHealthy,
    bool timescaleHealthy,
    bool candlesHypertableHealthy,
    IReadOnlyCollection<ExchangeHealthDto> exchanges)
{
    if (!databaseHealthy || !timescaleHealthy || !candlesHypertableHealthy)
        return "Unhealthy";

    if (exchanges.Any(x => x.IsActive && x.Status == "Degraded"))
        return "Degraded";

    return "Healthy";
}

// Henter age seconds.
private static double? GetAgeSeconds(DateTime? value, DateTime nowUtc)
{
    if (!value.HasValue)
        return null;

    var utc = value.Value.Kind == DateTimeKind.Unspecified
        ? DateTime.SpecifyKind(value.Value, DateTimeKind.Utc)
        : value.Value.ToUniversalTime();

```

```
        return Math.Max(0, (nowUtc - utc).TotalSeconds);
    }

    // Behandler exchange health projection.
    private sealed record ExchangeHealthProjection(
        int Id,
        string Code,
        string Name,
        bool IsActive,
        int ActiveSymbolCount,
        DateTime? LastCandleTimeUtc);
}
```

B87. TradingAppLibrary/Services/TokenService.cs

Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.

```
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using System;
using System.Collections.Generic;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Services
{
    public class TokenService : ITokenService
    {
        private readonly JwtSettings _jwtSettings;

        public TokenService(IOptions<JwtSettings> jwtSettings)
        {
            _jwtSettings = jwtSettings.Value;
        }

        // Rolleparameteren kan sendes som enten string eller UserRole-enum.
        public string GenerateToken(int userId, string username, string role)
        {
            var tokenHandler = new JwtSecurityTokenHandler();
            var key = Encoding.UTF8.GetBytes(_jwtSettings.Secret);

            var claims = new List<Claim>
            {
                new(ClaimTypes.NameIdentifier, userId.ToString()),
                new(ClaimTypes.Name, username),
                new(ClaimTypes.Role, role)
            };

            var tokenDescriptor = new SecurityTokenDescriptor
            {
                Subject = new ClaimsIdentity(claims),
                Expires = DateTime.UtcNow.AddMinutes(_jwtSettings.ExpiryMinutes),
                Issuer = _jwtSettings.Issuer,
                Audience = _jwtSettings.Audience,
                SigningCredentials = new SigningCredentials(
                    new SymmetricSecurityKey(key),
                    SecurityAlgorithms.HmacSha256Signature)
            };

            var token = tokenHandler.CreateToken(tokenDescriptor);
            return tokenHandler.WriteToken(token);
        }
    }
}
```

B88. TradingAppLibrary/Services/UserService.cs

Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.

```
using Org.BouncyCastle.Crypto.Generators;
using System;
using System.Collections.Generic;
using System.Text;
using System.Text.Json;
using TradingAppLibrary.Data;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Models;
```

```
namespace TradingAppLibrary.Services
{
    public class UserService: IUserService
    {
        private readonly IUserRepository _userRepository;

        public UserService(IUserRepository userRepository)
        {
            _userRepository = userRepository;
        }

        // Henter user by id.
        public async Task<User?> GetUserByIdAsync(int id)
        {
            return await _userRepository.GetByIdAsync(id);
        }

        // Henter user by email.
        public async Task<User?> GetUserByEmailAsync(string email)
        {
            return await _userRepository.GetByEmailAsync(email);
        }

        // Henter user by username.
        public async Task<User?> GetUserByUsernameAsync(string username)
        {
            return await _userRepository.GetByUsernameAsync(username);
        }

        // Henter all users.
        public async Task<IEnumerable<User>> GetAllUsersAsync()
        {
            return await _userRepository.GetAllUsersAsync();
        }

        // Opdaterer user.
        public async Task UpdateUserAsync(User user)
        {
            await _userRepository.UpdateAsync(user);
        }

        // Sletter user.
        public async Task DeleteUserAsync(int id)
        {
            await _userRepository.DeleteAsync(id);
        }

        // Behandler hard delete user.
        public async Task<bool> HardDeleteUserAsync(int userId)
        {
            var user = await _userRepository.GetByIdAsync(userId);
            if (user == null)
                return false;

            // ⚠ Sikkerhedsbeskyttelse: SuperAdmin-kontoen kan ikke slettes permanent.
            if (user.Role?.ToString().Equals("SuperAdmin", StringComparison.OrdinalIgnoreCase) == true)
            {
                throw new InvalidOperationException("SuperAdmin hesabı kalıcı olarak silinemez.");
            }

            return await _userRepository.HardDeleteUserAsync(userId);
        }

        // Henter user preferences.
        public async Task<UserPreferencesDto?> GetUserPreferencesAsync(string userId)
        {
            var entity = await _userRepository.GetPreferencesByUserIdAsync(userId);
            if (entity == null || string.IsNullOrEmpty(entity.SettingsJson))
            {
                return new UserPreferencesDto(); // Standardindstillinger med tomme/default-værdier.
            }

            try
            {
                return JsonSerializer.Deserialize<UserPreferencesDto>(entity.SettingsJson, new JsonSerializerOptions
                {
                    PropertyNameCaseInsensitive = true
                }) ?? new UserPreferencesDto();
            }
            catch
            {
                return new UserPreferencesDto();
            }
        }
    }
}
```



```

    }

    // Gemmer user preferences.
    public async Task SaveUserPreferencesAsync(string userId, UserPreferencesDto dto)
    {
        var json = JsonSerializer.Serialize(dto, new JsonSerializerOptions
        {
            WriteIndented = false
        });

        var preferences = new UserPreferences
        {
            UserId = userId,
            SettingsJson = json,
            UpdatedAt = DateTime.UtcNow
        };

        await _userRepository.SavePreferencesAsync(preferences);
    }

    /// Validerer den nuværende adgangskode med PasswordHasher og gemmer den nye adgangskode som hash, hvis valideringen lykkes.
    public async Task<bool> ChangePasswordAsync(int userId, string currentPassword, string newPassword)
    {
        var user = await _userRepository.GetByIdAsync(userId);
        if (user == null || user.IsDeleted || string.IsNullOrEmpty(user.PasswordHash))
        {
            return false;
        }

        // Kontroller den nuværende adgangskode med PasswordHasher-klassen.
        bool isCurrentValid = PasswordHasher.VerifyPassword(currentPassword, user.PasswordHash);
        if (!isCurrentValid)
        {
            return false;
        }

        // Hash den nye adgangskode med PBKDF2, og gem den på brugeren.
        user.PasswordHash = PasswordHasher.HashPassword(newPassword);
        await _userRepository.UpdateAsync(user);

        return true;
    }
}

```

B89. TradingAppLibrary/Services/WatchlistService.cs

Serviceklasse med forretningslogik, koordinering eller ekstern integration for dette område.

```

using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Mappings;
using TradingAppLibrary.Models;

namespace TradingAppLibrary.Services;

public class WatchlistService : IWatchlistService
{
    private readonly IWatchlistRepository _watchlistRepository;
    private readonly ISymbolRepository _symbolRepository;

    public WatchlistService(IWatchlistRepository watchlistRepository, ISymbolRepository symbolRepository)
    {
        _watchlistRepository = watchlistRepository;
        _symbolRepository = symbolRepository;
    }

    // Henter user watchlists.
    public async Task<IEnumerable<WatchlistResponseDto>> GetUserWatchlistsAsync(int userId)
    {
        if (userId <= 0) return Enumerable.Empty<WatchlistResponseDto>();
        return (await _watchlistRepository.GetWatchlistsByUserIdAsync(userId)).ToResponseDtos();
    }

    // Opretter watchlist.
    public async Task<WatchlistResponseDto> CreateWatchlistAsync(int userId, string name)
    {
        if (userId <= 0)
            throw new ArgumentException("Geçerli bir UserId gereklidir.", nameof(userId));

        var watchlist = new Watchlist

```

```

{
    UserId = userId,
    Name = string.IsNullOrEmpty(name) ? "Favorilerim" : name.Trim()
};

await _watchlistRepository.AddAsync(watchlist);
return watchlist.ToResponseDto();
}

// Tilføjer symbol to watchlist.
public async Task<bool> AddSymbolToWatchlistAsync(AddWatchlistItemDto dto)
{
    if (dto.WatchlistId <= 0 || dto.SymbolId <= 0) return false;

    var watchlist = await _watchlistRepository.GetByldAsync(dto.WatchlistId);
    if (watchlist == null) return false;
    if (await _symbolRepository.GetByldAsync(dto.SymbolId) == null) return false;
    if (watchlist.Items.Any(item => item.SymbolId == dto.SymbolId)) return true;

    await _watchlistRepository.AddItemAsync(dto.ToEntity());
    return true;
}

// Fjerner symbol from watchlist.
public async Task<bool> RemoveSymbolFromWatchlistAsync(int watchlistId, int symbolId)
{
    if (watchlistId <= 0 || symbolId <= 0) return false;

    var watchlist = await _watchlistRepository.GetByldAsync(watchlistId);
    if (watchlist == null || !watchlist.Items.Any(item => item.SymbolId == symbolId)) return false;

    await _watchlistRepository.RemoveItemAsync(watchlistId, symbolId);
    return true;
}
}

```

B90. TradingAppLibrary/Validation/CandleValidationResult.cs

Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.

```

namespace TradingAppLibrary.Validation;

// Behandler candle validation result.
public sealed record CandleValidationResult(bool IsValid, IReadOnlyList<string> Errors)
{
    // Behandler success.
    public static CandleValidationResult Success() => new(true, Array.Empty<string>());

    // Behandler failure.
    public static CandleValidationResult Failure(params string[] errors) =>
        new(false, errors.Where(x => !string.IsNullOrEmpty(x)).ToArray());
}

// Behandler candle batch validation result.
public sealed record CandleBatchValidationResult(
    bool IsValid,
    IReadOnlyList<string> Errors,
    IReadOnlyList<string> Warnings)
{
    // Behandler success.
    public static CandleBatchValidationResult Success(IReadOnlyList<string>? warnings = null) =>
        new(true, Array.Empty<string>(), warnings ?? Array.Empty<string>());
}

```

B91. TradingAppLibrary/Validators/CandleValidator.cs

Valideringsklasse, som kontrollerer market-data før de accepteres og gemmes.

```

using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;
using TradingAppLibrary.Validation;

namespace TradingAppLibrary.Validators;

public sealed class CandleValidator : ICandleValidator
{
    private static readonly TimeSpan FutureTolerance = TimeSpan.FromMinutes(2);

    // Validerer den relevante operation.
    public CandleValidationResult Validate(

```

```

    ExchangeCandleDto candle,
    string expectedSymbol,
    string expectedInterval)
{
    var errors = new List<string>();

    var normalizedSymbol = expectedSymbol.Trim().ToUpperInvariant();
    var normalizedInterval = expectedInterval.Trim().ToLowerInvariant();

    if (candle.Time <= 0)
        errors.Add("Candle timestamp must be greater than zero.");
    else
    {
        try
        {
            var candleTime = DateTimeOffset.FromUnixTimeMilliseconds(candle.Time);
            if (candleTime > DateTimeOffset.UtcNow.Add(FutureTolerance))
                errors.Add("Candle timestamp is too far in the future.");
        }
        catch (ArgumentOutOfRangeException)
        {
            errors.Add("Candle timestamp is outside the supported Unix timestamp range.");
        }
    }

    if (candle.Open <= 0) errors.Add("Open must be greater than zero.");
    if (candle.High <= 0) errors.Add("High must be greater than zero.");
    if (candle.Low <= 0) errors.Add("Low must be greater than zero.");
    if (candle.Close <= 0) errors.Add("Close must be greater than zero.");
    if (candle.Volume < 0) errors.Add("Volume cannot be negative.");

    if (candle.High < candle.Low)
        errors.Add("High cannot be lower than Low.");

    if (candle.High < candle.Open || candle.High < candle.Close)
        errors.Add("High cannot be lower than Open or Close.");

    if (candle.Low > candle.Open || candle.Low > candle.Close)
        errors.Add("Low cannot be higher than Open or Close.");

    if (!string.Equals(candle.Symbol?.Trim(), normalizedSymbol, StringComparison.OrdinalIgnoreCase))
        errors.Add($"Unexpected symbol. Expected={normalizedSymbol}, Actual={candle.Symbol}.");

    if (!string.Equals(candle.Interval?.Trim(), normalizedInterval, StringComparison.OrdinalIgnoreCase))
        errors.Add($"Unexpected interval. Expected={normalizedInterval}, Actual={candle.Interval}.");

    return errors.Count == 0
        ? CandleValidationResult.Success()
        : new CandleValidationResult(false, errors);
}

// Validerer batch.
public CandleBatchValidationResult ValidateBatch(
    IReadOnlyList<ExchangeCandleDto> candles,
    string expectedSymbol,
    string expectedInterval)
{
    if (candles.Count == 0)
        return CandleBatchValidationResult.Success();

    var errors = new List<string>();
    var warnings = new List<string>();
    var expectedStep = GetIntervalMilliseconds(expectedInterval);

    for (var i = 0; i < candles.Count; i++)
    {
        var result = Validate(candles[i], expectedSymbol, expectedInterval);
        if (!result.IsValid)
        {
            foreach (var error in result.Errors)
                errors.Add($"Index={i}, Time={candles[i].Time}: {error}");
        }
    }

    for (var i = 1; i < candles.Count; i++)
    {
        if (candles[i].Time < candles[i - 1].Time)
        {
            errors.Add(
                $"Candle batch is not ordered at indexes {i - 1}/{i}: {candles[i - 1].Time} -> {candles[i].Time}.");
        }
    }
}

```

```

    }

    var ordered = candles.OrderBy(c => c.Time).ToList();
    var duplicateTimes = ordered
        .GroupBy(c => c.Time)
        .Where(g => g.Count() > 1)
        .Select(g => g.Key)
        .ToList();

    foreach (var duplicateTime in duplicateTimes)
        errors.Add($"Duplicate candle timestamp detected: {duplicateTime}.");

    if (expectedStep.HasValue)
    {
        for (var i = 1; i < ordered.Count; i++)
        {
            var difference = ordered[i].Time - ordered[i - 1].Time;
            if (difference > expectedStep.Value)
            {
                var missingCount = difference / expectedStep.Value - 1;
                warnings.Add(
                    $"Gap detected between {ordered[i - 1].Time} and {ordered[i].Time}. Missing candle count: {missingCount}.");
            }
            else if (difference > 0 && difference < expectedStep.Value)
            {
                errors.Add(
                    $"Unexpected candle spacing between {ordered[i - 1].Time} and {ordered[i].Time}: {difference} ms.");
            }
        }
    }

    return new CandleBatchValidationResult(errors.Count == 0, errors, warnings);
}

// Henter interval milliseconds.
private static long? GetIntervalMilliseconds(string interval) =>
    interval.Trim().ToLowerInvariant() switch
    {
        "1m" => 60_000L,
        "3m" => 3 * 60_000L,
        "5m" => 5 * 60_000L,
        "15m" => 15 * 60_000L,
        "30m" => 30 * 60_000L,
        "1h" => 60 * 60_000L,
        "2h" => 2 * 60 * 60_000L,
        "4h" => 4 * 60 * 60_000L,
        "1d" => 24 * 60 * 60_000L,
        _ => null
    };
}

```

5. Komplet frontend-kode

F1. src/App.tsx

Frontendens overordnede React-komponent. Samler navigation, valgt exchange/symbol, paneler og centrale states.

```

import React, { useState, useEffect, useCallback, useMemo, useRef } from 'react';
import { useTranslation } from 'react-i18next';
import { WifiOff } from 'lucide-react';
import { api } from './Services/api';
import { authService } from './Services/authService';
import { watchlistService } from './Services/watchlistService';
import { symbolApi } from './Services/symbolApi';
import { preferenceService } from './Services/preferenceService';
import { alertService } from './Services/alertService';
import { signalrService } from './Services/signalrService';
import type { WatchlistResponseDto } from './Types/watchlist';
import type { SymbolResponseDto } from './Types/symbol';
import type { ChartType } from './Types/candle';
import type { Alert, CreateAlertRequest } from './Types/alert';
import type { User } from './Types/auth';
import { DEFAULT_INDICATORS, type IndicatorConfig } from './Types/indicator';
import { DEFAULT_CHART_SETTINGS, type ChartVisualSettings } from './Types/chartSettings';
import { AuthPage } from './Pages/AuthPage';
import { AdminPanel } from './Pages/AdminPanel';
import { ProtectedRoute } from './Components/Common/ProtectedRoute';
import { HeaderNavbar } from './Components/HeaderNavbar';
import { CandlestickChart } from './Components/Chart/CandlestickChart';
import { ChartSettingsModal } from './Components/ChartSettingsModal';
import { CreateAlertModal } from './Components/CreateAlertModal';
import { useTheme } from './Context/ThemeContext';

```

```

// LocalStorage-nøgler
const STORAGE_KEYS = {
  SYMBOL: 'app_last_symbol',
  INTERVAL: 'app_last_interval',
  CHART_TYPE: 'app_chart_type',
  LAYOUT_MODE: 'app_layout_mode',
  INDICATORS: 'app_indicators',
  CHART_SETTINGS: 'app_chart_settings',
};

// Hjælpfunktion, der sikkert kombinerer gemte indikatorer med DEFAULT_INDICATORS.
const mergeIndicators = (savedList: any[] | null | undefined): IndicatorConfig[] => {
  if (!Array.isArray(savedList) || savedList.length === 0) {
    return DEFAULT_INDICATORS;
  }

  return DEFAULT_INDICATORS.map((defaultInd) => {
    const saved = savedList.find(
      (s) =>
        (s.id || s.Id) === defaultInd.id ||
        (s.name || s.Name) === defaultInd.name
    );

    if (saved) {
      const enabled = saved.enabled ?? saved.Enabled;
      return {
        ...defaultInd,
        enabled: typeof enabled === 'boolean' ? enabled : defaultInd.enabled,
        color: saved.color || saved.Color || defaultInd.color,
        period: saved.period || saved.Period || defaultInd.period,
        fastPeriod: saved.fastPeriod || saved.FastPeriod || defaultInd.fastPeriod,
        slowPeriod: saved.slowPeriod || saved.SlowPeriod || defaultInd.slowPeriod,
        signalPeriod: saved.signalPeriod || saved.SignalPeriod || defaultInd.signalPeriod,
        stdDev: saved.stdDev || saved.StdDev || defaultInd.stdDev,
      };
    }
    return defaultInd;
  });
};

// Hjælpfunktion, der normaliserer indstillinger fra backend eller storage.
const normalizeVisualSettings = (rawSettings: any): ChartVisualSettings => {
  if (!rawSettings) return DEFAULT_CHART_SETTINGS;

  return {
    ...DEFAULT_CHART_SETTINGS,
    upColor: rawSettings.upColor || rawSettings.UpColor || DEFAULT_CHART_SETTINGS.upColor,
    downColor: rawSettings.downColor || rawSettings.DownColor || DEFAULT_CHART_SETTINGS.downColor,
    showBorders: rawSettings.showBorders ?? rawSettings.ShowBorders ?? DEFAULT_CHART_SETTINGS.showBorders,
    showWicks: rawSettings.showWicks ?? rawSettings.ShowWicks ?? DEFAULT_CHART_SETTINGS.showWicks,
    showGrid: rawSettings.showGrid ?? rawSettings.ShowGrid ?? DEFAULT_CHART_SETTINGS.showGrid,
    gridColorDark: rawSettings.gridColorDark || rawSettings.GridColorDark || DEFAULT_CHART_SETTINGS.gridColorDark,
    gridColorLight: rawSettings.gridColorLight || rawSettings.GridColorLight || DEFAULT_CHART_SETTINGS.gridColorLight,
    showPriceLine: rawSettings.showPriceLine ?? rawSettings.ShowPriceLine ?? DEFAULT_CHART_SETTINGS.showPriceLine,
    soundEnabled: rawSettings.soundEnabled ?? rawSettings.SoundEnabled ?? DEFAULT_CHART_SETTINGS.soundEnabled,
  };
};

export const App: React.FC = () => {
  const { t } = useTranslation();
  const { isDark, setTheme } = useTheme();
  const [isAuthenticated, setIsAuthenticated] = useState<boolean>(() => authService.isAuthenticated());
  const currentUser = useMemo(() => authService.getCurrentUser(), [isAuthenticated]);

  // 🚫 State for mistet backend-forbindelse.
  const [isBackendDown, setIsBackendDown] = useState<boolean>(false);

  // 🚫 AKTIV VISNING: Admin/SuperAdmin starter som standard i admin-visningen, normale brugere i terminalen.
  const [currentView, setCurrentView] = useState<'terminal' | 'admin'>(() => {
    const user = authService.getCurrentUser();
    const role = (user?.role || '').toLowerCase();
    return role === 'admin' || role === 'superadmin' ? 'admin' : 'terminal';
  });

  // 1. Startværdier (LocalStorage eller standardværdier)
  const [currentSymbol, setCurrentSymbol] = useState<string>(() => {
    return localStorage.getItem(STORAGE_KEYS.SYMBOL) || 'BTCUSD';
  });

  const [currentSymbolId, setCurrentSymbolId] = useState<number>(() => {
    const saved = Number(localStorage.getItem('tradingapp_symbol_id'));

```

```

    return Number.isFinite(saved) && saved > 0 ? saved : 0;
  });
  const [currentExchangeCode, setCurrentExchangeCode] = useState<string>(() => {
    return localStorage.getItem('tradingapp_exchange_code') || '';
  });

  const [currentInterval, setCurrentInterval] = useState<string>(() => {
    return localStorage.getItem(STORAGE_KEYS.INTERVAL) || '15m';
  });

  const [chartType, setChartType] = useState<ChartType>(() => {
    return (localStorage.getItem(STORAGE_KEYS.CHART_TYPE) as ChartType) || 'candles';
  });

  const [layoutMode, setLayoutMode] = useState<'1x1' | '1x2' | '2x2'>(() => {
    return (localStorage.getItem(STORAGE_KEYS.LAYOUT_MODE) as '1x1' | '1x2' | '2x2') || '1x1';
  });

  const [indicators, setIndicators] = useState<IndicatorConfig[]>(() => {
    const saved = localStorage.getItem(STORAGE_KEYS.INDICATORS);
    if (saved) {
      try {
        return mergeIndicators(JSON.parse(saved));
      } catch {
        return DEFAULT_INDICATORS;
      }
    }
    return DEFAULT_INDICATORS;
  });

  const [chartSettings, setChartSettings] = useState<ChartVisualSettings>(() => {
    const saved = localStorage.getItem(STORAGE_KEYS.CHART_SETTINGS);
    if (saved) {
      try {
        return normalizeVisualSettings(JSON.parse(saved));
      } catch {
        return DEFAULT_CHART_SETTINGS;
      }
    }
    return DEFAULT_CHART_SETTINGS;
  });

  // Tilstand for modaler, lister og alarmer
  const [activeSymbols, setActiveSymbols] = useState<SymbolResponseDto[]>([]);
  const [watchlists, setWatchlists] = useState<WatchlistResponseDto[]>([]);
  const [alerts, setAlerts] = useState<Alert[]>([]);
  const [isSettingsOpen, setIsSettingsOpen] = useState<boolean>(false);
  const [isAlertModalOpen, setIsAlertModalOpen] = useState<boolean>(false);

  const isInitialFetchDone = useRef<boolean>(false);
  const saveTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);

  // 🕒 Overvåg backend-forbindelsesfejl og health check.
  useEffect(() => {
    // Håndterer forbindelsen til realtime-data via SignalR.
    const handleConnectionError = () => setIsBackendDown(true);
    // Håndterer forbindelsen til realtime-data via SignalR.
    const handleConnectionRestored = () => setIsBackendDown(false);

    window.addEventListener('backend-connection-error', handleConnectionError);
    window.addEventListener('backend-connection-restored', handleConnectionRestored);

    // Kontroller hvert 5. sekund, om serveren er tilgængelig igen (health check).
    const healthInterval = setInterval(async () => {
      if (isBackendDown) {
        try {
          await api.get('/exchanges'); // Ping backend via et offentligt endpoint.
          setIsBackendDown(false);
        } catch (err: any) {
          if (err.response) {
            // Hvis serveren svarer, er den tilgængelig, også selv om svaret f.eks. er 401.
            setIsBackendDown(false);
          }
        }
      }
    }, 5000);

    return () => {
      window.removeEventListener('backend-connection-error', handleConnectionError);
      window.removeEventListener('backend-connection-restored', handleConnectionRestored);
      clearInterval(healthInterval);
    };
  });

```

```
}, [isBackendDown]);

// 2. Hent data og indstillinger fra backend, når siden åbnes.
useEffect(() => {
  if (!isAuthenticated) return;

  let isMounted = true;

  Promise.all([
    symbolApi.searchSymbols().then((all) => all.filter((s) => s.isActive)).catch(() => []),
    watchlistService.getMyWatchlists().catch(() => []),
    preferenceService.getPreferences().catch(() => null),
    alertService.getMyAlerts().catch(() => []),
  ]).then(([syms, lists, dbPrefs, userAlerts]: [any, any, any, any]) => {
    if (!isMounted) return;

    if (Array.isArray(syms)) {
      setActiveSymbols(syms);
      const preferredName = (dbPrefs?.lastSymbol || dbPrefs?.LastSymbol || currentSymbol || '').toUpperCase();
      const selected = syms.find((s: SymbolResponseDto) => s.name.toUpperCase() === preferredName) || syms[0];
      if (selected) {
        setCurrentSymbol(selected.name);
        setCurrentSymbolId(selected.id);
        setCurrentExchangeCode(selected.exchangeCode);
        localStorage.setItem(STORAGE_KEYS.SYMBOL, selected.name);
        localStorage.setItem('tradingapp_symbol_id', String(selected.id));
        localStorage.setItem('tradingapp_exchange_code', selected.exchangeCode);
      }
    }
    if (Array.isArray(lists)) setWatchlists(lists);
    if (Array.isArray(userAlerts)) setAlerts(userAlerts);

    if (dbPrefs) {
      // TEMA
      const themeVal = dbPrefs.theme || dbPrefs.Theme;
      if (themeVal && (themeVal === 'dark' || themeVal === 'light')) {
        if (typeof setTheme === 'function') {
          setTheme(themeVal, false);
        }
      }

      // SYMBOL
      const symbolVal = dbPrefs.lastSymbol || dbPrefs.LastSymbol;
      if (symbolVal) {
        setCurrentSymbol(symbolVal);
        localStorage.setItem(STORAGE_KEYS.SYMBOL, symbolVal);
      }

      // INTERVAL
      const intervalVal = dbPrefs.lastInterval || dbPrefs.LastInterval;
      if (intervalVal) {
        setCurrentInterval(intervalVal);
        localStorage.setItem(STORAGE_KEYS.INTERVAL, intervalVal);
      }

      // CHARTTYPE
      const chartTypeVal = dbPrefs.chartType || dbPrefs.ChartType;
      if (chartTypeVal) {
        setChartType(chartTypeVal as ChartType);
        localStorage.setItem(STORAGE_KEYS.CHART_TYPE, chartTypeVal);
      }

      // LAYOUT
      const layoutVal = dbPrefs.layoutMode || dbPrefs.LayoutMode;
      if (layoutVal) {
        setLayoutMode(layoutVal as '1x1' | '1x2' | '2x2');
        localStorage.setItem(STORAGE_KEYS.LAYOUT_MODE, layoutVal);
      }

      // INDIKATORER
      const rawIndicators = dbPrefs.indicators || dbPrefs.Indicators;
      if (rawIndicators && Array.isArray(rawIndicators)) {
        const merged = mergeIndicators(rawIndicators);
        setIndicators(merged);
        localStorage.setItem(STORAGE_KEYS.INDICATORS, JSON.stringify(merged));
      }

      // VISUELLE INDSTILLINGER
      const rawVisual = dbPrefs.visualSettings || dbPrefs.VisualSettings || dbPrefs.chartSettings;
      if (rawVisual) {
        const normalized = normalizeVisualSettings(rawVisual);
        setChartSettings(normalized);
      }
    }
  });
}
```

```

        localStorage.setItem(STORAGE_KEYS.CHART_SETTINGS, JSON.stringify(normalized));
      } else if (dbPrefs.candleColors) {
        setChartSettings((prev) => {
          const merged = { ...prev, ...dbPrefs.candleColors };
          localStorage.setItem(STORAGE_KEYS.CHART_SETTINGS, JSON.stringify(merged));
          return merged;
        });
      }
    }
  }

  isInitialFetchDone.current = true;
});

return () => {
  isMounted = false;
};
}, [isAuthenticated, setTheme]);

// 3. Realtidslistener til SignalR til udløste alarmer.
useEffect(() => {
  if (!isAuthenticated) return;

  // Behandler den relevante brugerhandling eller event.
  const handleAlertTriggered = (triggeredAlert: Alert) => {
    if (chartSettings.soundEnabled) {
      try {
        const audio = new Audio('/sounds/alert.mp3');
        audio.volume = 0.8;
        audio.play().catch(() => {});
      } catch {}
    }
  }

  setAlerts((prev) =>
    prev.map((a) =>
      a.id === triggeredAlert.id ? { ...a, isTriggered: true, isActive: false } : a
    )
  );
});

signalRService.subscribeToAlertTriggered(handleAlertTriggered);

return () => {
  signalRService.unsubscribeFromAlertTriggered(handleAlertTriggered);
};
}, [isAuthenticated, chartSettings.soundEnabled]);

// 4. Gem brugerindstillinger eller tema i databasen ved ændringer (debounced 500 ms).
useEffect(() => {
  if (!isAuthenticated || !isInitialFetchDone.current) return;

  if (saveTimeoutRef.current) {
    clearTimeout(saveTimeoutRef.current);
  }

  saveTimeoutRef.current = setTimeout(() => {
    preferenceService
      .savePreferences({
        Theme: isDark ? 'dark' : 'light',
        theme: isDark ? 'dark' : 'light',
        LastSymbol: currentSymbol,
        lastSymbol: currentSymbol,
        LastInterval: currentInterval,
        lastInterval: currentInterval,
        ChartType: chartType,
        chartType: chartType as any,
        LayoutMode: layoutMode,
        layoutMode: layoutMode,
        Indicators: indicators.map((ind) => ({
          Id: ind.id,
          Type: ind.type,
          Name: ind.name,
          Enabled: ind.enabled,
          Color: ind.color,
          Period: ind.period ?? null,
          FastPeriod: ind.fastPeriod ?? null,
          SlowPeriod: ind.slowPeriod ?? null,
          SignalPeriod: ind.signalPeriod ?? null,
          StdDev: ind.stdDev ?? null,
        })),
        VisualSettings: {
          UpColor: chartSettings.upColor,
          DownColor: chartSettings.downColor,

```



```

        ShowBorders: chartSettings.showBorders,
        ShowWicks: chartSettings.showWicks,
        ShowGrid: chartSettings.showGrid,
        GridColorDark: chartSettings.gridColorDark,
        GridColorLight: chartSettings.gridColorLight,
        ShowPriceLine: chartSettings.showPriceLine,
        SoundEnabled: chartSettings.soundEnabled ?? true,
    },
  } as any)
  .catch((err) => {
    console.warn('Tercihler veritabanına kaydedilemedi:', err);
  });
}, 500);

return () => {
  if (saveTimeoutRef.current) {
    clearTimeout(saveTimeoutRef.current);
  }
};
}, [isAuthenticated, isDark, currentSymbol, currentInterval, chartType, layoutMode, indicators, chartSettings]);

// 5. Alarmfonksiyonlar
const handleCreateAlert = async (data: CreateAlertRequest) => {
  const created = await alertService.createAlert(data);
  if (created) {
    setAlerts((prev) => [created, ...prev]);
  }
};

// Behandler den relevante brugerhandling eller event.
const handleToggleAlert = async (alertId: string) => {
  await alertService.toggleAlert(alertId);
  setAlerts((prev) =>
    prev.map((a) => (a.id === alertId ? { ...a, isActive: !a.isActive } : a))
  );
};

// Fjerner det valgte element.
const handleDeleteAlert = async (alertId: string) => {
  await alertService.deleteAlert(alertId);
  setAlerts((prev) => prev.filter((a) => a.id !== alertId));
};

// Kontroller rolle og naviger efter vellykket login.
const handleLoginSuccess = useCallback((user?: User) => {
  setIsAuthenticated(true);
  const loggedUser = user || authService.getCurrentUser();
  const role = (loggedUser?.role || "").toLowerCase();

  if (role === 'admin' || role === 'superadmin') {
    setCurrentView('admin');
  } else {
    setCurrentView('terminal');
  }
}, []);

// Behandler den relevante brugerhandling eller event.
const handleLogout = useCallback(() => {
  authService.logout();
  setIsAuthenticated(false);
  setWatchlists([]);
  setAlerts([]);
  isInitialFetchDone.current = false;
  setCurrentView('terminal');
}, []);

// Skift aktivt symbol og interval.
const handleActiveStateChange = useCallback((symbol: string, interval: string, symbolInfo?: SymbolResponseDto) => {
  const resolved = symbolInfo || activeSymbols.find((s) => s.name.toUpperCase() === symbol.toUpperCase());
  if (resolved) {
    setCurrentSymbolId(resolved.id);
    setCurrentExchangeCode(resolved.exchangeCode);
    localStorage.setItem('tradingapp_symbol_id', String(resolved.id));
    localStorage.setItem('tradingapp_exchange_code', resolved.exchangeCode);
  }
  setCurrentSymbol((prev) => {
    if (prev !== symbol) {
      localStorage.setItem(STORAGE_KEYS.SYMBOL, symbol);
      return symbol;
    }
    return prev;
  });
});

```



```

{currentView === 'admin' ? (
  <AdminPanel
    onSwitchToTerminal={handleSwitchToTerminal}
    onLogout={handleLogout}
  />
): (
  /* 2. STANDART GRAFIK & TERMINAL GÖRÜNÜMÜ */
  <div className="flex flex-col h-screen overflow-hidden">
    { /* ØVERSTE KONTROLLINJE */ }
    <HeaderNavbar
      currentUser={currentUser}
      onLogout={handleLogout}
      watchlists={watchlists}
      currentSymbol={currentSymbol}
      currentSymbolId={currentSymbolId}
      currentExchangeCode={currentExchangeCode}
      currentInterval={currentInterval}
      chartType={chartType}
      onSelectChartType={handleSelectChartType}
      layoutMode={layoutMode}
      activeSymbols={activeSymbols}
      indicators={indicators}
      onToggleIndicator={handleToggleIndicator}
      onSelectSymbol={(sym) => handleActiveStateChange(sym.name, currentInterval, sym)}
      onSelectInterval={(inv) => handleActiveStateChange(currentSymbol, inv)}
      onChangeLayout={handleChangeLayout}
      onOpenSettings={() => setIsSettingsOpen(true)}
      onOpenAlertModal={() => setIsAlertModalOpen(true)}
    />

    { /* CHARTOMRÅDE */ }
    <main className="flex-1 p-1 overflow-hidden">
      <CandlestickChart
        layoutMode={layoutMode}
        activeSymbol={currentSymbol}
        activeSymbolId={currentSymbolId}
        activeExchangeCode={currentExchangeCode}
        activeInterval={currentInterval}
        chartType={chartType}
        indicators={indicators}
        chartSettings={chartSettings}
        alerts={alerts}
        onToggleAlert={handleToggleAlert}
        onDeleteAlert={handleDeleteAlert}
        onOpenCreateAlert={() => setIsAlertModalOpen(true)}
        onActiveStateChange={handleActiveStateChange}
      />
    </main>
  </div>
)}

{ /* MODAL TIL CHARTINDSTILLINGER */ }
<ChartSettingsModal
  isOpen={isSettingsOpen}
  onClose={() => setIsSettingsOpen(false)}
  settings={chartSettings}
  onSave={handleSaveChartSettings}
  isDark={isDark}
/>

{ /* MODAL TIL OPRETTELSE AF ALARM */ }
<CreateAlertModal
  isOpen={isAlertModalOpen}
  onClose={() => setIsAlertModalOpen(false)}
  symbolId={currentSymbolId}
  symbol={currentSymbol}
  onSave={handleCreateAlert}
  isDark={isDark}
/>
</ProtectedRoute>
</div>
);
};

export default App;

```

F2. src/Components/AiAnalysisPanel.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```

import React from 'react';
import { useTranslation } from 'react-i18next';
import { useTheme } from '../Context/ThemeContext';
import type { AiAnalysisResponseDto } from '../Types/ai';

```

```

interface AIAnalysisPanelProps {
  isOpen: boolean;
  onClose: () => void;
  symbol: string;
  interval: string;
  analysis: AIAnalysisResponseDto | null;
  loading: boolean;
  onRefresh: () => void;
}

export const AIAnalysisPanel: React.FC<AIAnalysisPanelProps> = ({
  isOpen,
  onClose,
  symbol,
  interval,
  analysis,
  loading,
  onRefresh,
}) => {
  const { t, i18n } = useTranslation();
  const { isDark } = useTheme();

  // Render ikke noget, hvis panelet er lukket.
  if (!isOpen) return null;

  // Datoformatkode ud fra det valgte sprog ('tr-TR', 'en-US', 'da-DK').
  const localeCode = i18n.language.startsWith('da')
    ? 'da-DK'
    : i18n.language.startsWith('en')
    ? 'en-US'
    : 'tr-TR';

  return (
    <aside
      className={`w-80 sm:w-96 h-full p-5 flex flex-col justify-between z-50 shadow-2xl shrink-0 absolute right-0 top-0 bottom-0 border-l transition-colors
        duration-300 ${isDark ? 'bg-[#111827] border-gray-800 text-gray-200' : 'bg-white border-gray-300 text-gray-800'}
      `}
    >
      <div className="flex flex-col h-full min-h-0">
        { /* Sidehoved / luk-knap */ }
        <div
          className={`flex items-center justify-between pb-4 border-b shrink-0 ${isDark ? 'border-gray-800' : 'border-gray-200'}
        `}
        >
          <div className="flex items-center space-x-2.5">
            <span className="text-xl">✕</span>
            <div>
              <h3 className={`text-sm font-bold ${isDark ? 'text-white' : 'text-gray-900'}>
                {t('aiPanel.title', 'Piyasa Analizi')}
              </h3>
              <p className="text-xs text-gray-400">
                {symbol} • {interval} {t('aiPanel.timeframe', 'Periyodu')}
              </p>
            </div>
          </div>
          <button
            type="button"
            onClick={onClose}
            className={`p-1 rounded-lg transition-colors ${isDark
              ? 'text-gray-400 hover:text-white hover:bg-gray-800'
              : 'text-gray-600 hover:text-gray-900 hover:bg-gray-100'}
          `}
            title={t('aiPanel.close', 'Paneli Kapat')}
          >
            ✕
          </button>
        </div>

        { /* Indholdsområde (scrollbart) */ }
        <div className="flex-1 overflow-y-auto mt-4 pr-1 space-y-4 min-h-0">
          {loading ? (
            <div className="flex flex-col items-center justify-center py-20 space-y-3">
              <div className="w-8 h-8 border-2 border-purple-500 border-t-transparent rounded-full animate-spin" />
              <span className="text-xs text-gray-400 font-medium text-center">
                {t('aiPanel.loading', 'Yapay zekâ teknik analizi hazırlanıyor...')}
              </span>
            </div>
          ) : analysis ? (
            <>
              { /* Informationskort */ }
            </div>
            <div
              className={`flex items-center justify-between px-3.5 py-2.5 rounded-lg border text-xs ${isDark ? 'bg-[#1f2937]/50 border-gray-800' : 'bg-gray-
50 border-gray-200'}

```

```

    >
    <span className="font-bold text-purple-400">{analysis.symbol}</span>
    <span className="text-gray-400 text-[11px]">
      {analysis.generatedAt &&
        new Date(analysis.generatedAt).toLocaleTimeString(localeCode, {
          hour: '2-digit',
          minute: '2-digit',
          second: '2-digit',
        })}
    </span>
  </div>

  {/ "Vurderingstekst" */}
  <div>
    <h4 className={`text-xs font-semibold mb-2 ${isDark ? 'text-gray-300' : 'text-gray-700'}`}>
      {t('aiPanel.technicalEvaluation', 'Teknik Değerlendirme')}
    </h4>
    <div
      className={`text-xs leading-relaxed p-4 rounded-xl border whitespace-pre-line shadow-inner font-sans ${isDark
        ? 'bg-[#1f2937]/30 border-gray-800/80 text-gray-300'
        : 'bg-gray-50 border-gray-200 text-gray-700'}
      `}
    >
      {analysis.analysisText}
    </div>
  </div>
</>
): (
  <div className="flex flex-col items-center justify-center py-16 text-center space-y-2">
    <span className="text-2xl text-gray-400">❌</span>
    <p className="text-xs text-gray-400">
      {t('aiPanel.noData', 'Analiz verisi yüklenemedi.')}
    </p>
  </div>
)
</div>

{/ "Nederste del: opdater-knap" */}
<div
  className={`pt-4 border-t shrink-0 ${isDark ? 'border-gray-800' : 'border-gray-200'}
  `}
>
  <button
    type="button"
    onClick={onRefresh}
    disabled={loading}
    className="w-full py-2.5 bg-purple-600/20 hover:bg-purple-600/30 active:scale-[0.98] text-purple-400 text-xs font-semibold rounded-lg transition-all
border border-purple-500/30 flex items-center justify-center space-x-2 disabled:opacity-50 disabled:cursor-not-allowed"
  >
    <span className={loading ? 'animate-spin' : ''}>🔄</span>
    <span>{t('aiPanel.refresh', 'Analizi Yenile')}</span>
  </button>
</div>
</div>
</aside>
);
};

```

F3. src/Components/Chart/CandlestickChart.tsx

Den centrale chart-komponent. Viser OHLC-data og håndterer historical loading, live-opdateringer og chart-interaktion.

```

import React, { useEffect, useState, useCallback } from 'react';
import { useTranslation } from 'react-i18next';
import { symbolApi } from '../Services/symbolApi';
import { aiApi } from '../Services/aiApi';
import { watchlistService } from '../Services/watchlistService';
import type { WatchlistResponseDto } from '../Types/watchlist';
import type { AiAnalysisResponseDto } from '../Types/ai';
import type { ChartType } from '../Types/candle';
import type { IndicatorConfig } from '../Types/indicator';
import type { ChartVisualSettings } from '../Types/chartSettings';
import type { Alert } from '../Types/alert';
import type { SymbolResponseDto } from '../Types/symbol';
import { AiAnalysisPanel } from '../AiAnalysisPanel';
import { Watchlist } from '../Watchlist';
import { ChartPane } from '../ChartPane';
import { useTheme } from '../Context/ThemeContext';
import { Bell, Trash2, Power } from 'lucide-react';

interface CandlestickChartProps {
  defaultSymbol?: string;
  defaultInterval?: string;
}

```

```

layoutMode: '1x1' | '1x2' | '2x2';
chartType?: ChartType;
activeSymbol: string;
activeSymbolId: number;
activeExchangeCode: string;
activeInterval: string;
indicators: IndicatorConfig[];
chartSettings: ChartVisualSettings;
alerts?: Alert[];
onToggleAlert?: (alertId: string) => Promise<void>;
onDeleteAlert?: (alertId: string) => Promise<void>;
onOpenCreateAlert?: () => void;
onActiveStateChange: (symbol: string, interval: string, symbolInfo?: SymbolResponseDto) => void;
}

export interface PaneState {
  symbolId: number;
  symbol: string;
  exchangeCode: string;
  interval: string;
}

export const CandlestickChart: React.FC<CandlestickChartProps> = ({
  defaultSymbol = 'BTCUSDT',
  defaultInterval = '15m',
  layoutMode,
  chartType = 'candles',
  activeSymbol,
  activeSymbolId,
  activeExchangeCode,
  activeInterval,
  indicators = [],
  chartSettings,
  alerts = [],
  onToggleAlert,
  onDeleteAlert,
  onOpenCreateAlert,
  onActiveStateChange,
}) => {
  const { t, i18n } = useTranslation();
  const { isDark } = useTheme();

  const [activeSymbols, setActiveSymbols] = useState<SymbolResponseDto[]>([]);
  const [selectedPanelId, setSelectedPanelId] = useState<number>{1};

  const [panes, setPanes] = useState<Record<number, PaneState>>({
    1: { symbolId: activeSymbolId, symbol: activeSymbol || defaultSymbol, exchangeCode: activeExchangeCode, interval: activeInterval || defaultInterval },
    2: { symbolId: activeSymbolId, symbol: activeSymbol || defaultSymbol, exchangeCode: activeExchangeCode, interval: '1h' },
    3: { symbolId: activeSymbolId, symbol: activeSymbol || defaultSymbol, exchangeCode: activeExchangeCode, interval: '4h' },
    4: { symbolId: activeSymbolId, symbol: activeSymbol || defaultSymbol, exchangeCode: activeExchangeCode, interval: '1d' },
  });

  const [watchlists, setWatchlists] = useState<WatchlistResponseDto[]>([]);
  const [activePanel, setActivePanel] = useState<'watchlist' | 'ai' | 'alerts' | null>('watchlist');

  const [aiAnalysis, setAiAnalysis] = useState<AiAnalysisResponseDto | null>(null);
  const [aiLoading, setAiLoading] = useState<boolean>(false);

  useEffect(() => {
    let isMounted = true;

    Promise.all([
      symbolApi.searchSymbols().then((all) => all.filter((s) => s.isActive)).catch(() => []),
      watchlistService.getMyWatchlists().catch(() => []),
    ]).then(([symbols, lists]) => {
      if (isMounted) {
        if (symbols) setActiveSymbols(symbols);
        if (lists) setWatchlists(lists);
      }
    });

    return () => {
      isMounted = false;
    };
  }, []);

  useEffect(() => {
    if (activeSymbol || activeInterval) {
      setPanes((prev) => ({
        ...prev,
        [selectedPanelId]: {
          symbolId: activeSymbolId || prev[selectedPanelId].symbolId,

```

```

        symbol: activeSymbol || prev[selectedPanelId].symbol,
        exchangeCode: activeExchangeCode || prev[selectedPanelId].exchangeCode,
        interval: activeInterval || prev[selectedPanelId].interval,
    },
    }));
}
}, [activeSymbol, activeSymbolId, activeExchangeCode, activeInterval, selectedPanelId]);

// Behandler den relevante brugerhandling eller event.
const handleSelectPane = (panelId: number) => {
    setSelectedPanelId(panelId);
    const targetPane = panes[panelId];
    if (targetPane) {
        onActiveStateChange(targetPane.symbol, targetPane.interval, activeSymbols.find((s) => s.id === targetPane.symbolId));
    }
};

const getPaneConfig = (panelId: number): PaneState => {
    return panes[panelId] || { symbolId: activeSymbolId, symbol: defaultSymbol, exchangeCode: activeExchangeCode, interval: defaultInterval };
};

const currentActivePane = getPaneConfig(selectedPanelId);

// Henter de nødvendige data til denne funktion.
const fetchAiAnalysis = useCallback(async (symbolId: number, intv: string) => {
    if (!symbolId || symbolId <= 0) {
        setAiAnalysis(null);
        return;
    }

    setAiLoading(true);
    try {
        const data = await aiApi.getDetailedAnalysis(
            symbolId,
            intv,
            i18n.resolvedLanguage || i18n.language,
        );
        setAiAnalysis(data);
    } catch {
        setAiAnalysis(null);
    } finally {
        setAiLoading(false);
    }
}, [i18n.resolvedLanguage, i18n.language]);

const visiblePanels =
    layoutMode === '1x1' ? [1] : layoutMode === '1x2' ? [1, 2] : [1, 2, 3, 4];

return (
    <div
        className={`flex w-full h-full rounded-xl overflow-hidden border relative select-none transition-colors duration-300 ${isDark ? 'bg-[#0b0e14] text-gray-200 border-gray-800' : 'bg-gray-100 text-gray-800 border-gray-300'}
    >
        <div className="flex-1 p-1.5 w-full h-full min-w-0 overflow-hidden">
            <div
                className={`w-full h-full gap-1.5 grid ${layoutMode === '1x1'
                    ? 'grid-cols-1 grid-rows-1'
                    : layoutMode === '1x2'
                    ? 'grid-cols-2 grid-rows-1'
                    : 'grid-cols-2 grid-rows-2'
                }
            >
                {visiblePanels.map((id) => {
                    const paneConfig = getPaneConfig(id);
                    return (
                        <ChartPane
                            key={id}
                            symbolId={paneConfig.symbolId}
                            exchangeCode={paneConfig.exchangeCode}
                            symbol={paneConfig.symbol}
                            interval={paneConfig.interval}
                            chartType={paneConfig.chartType}
                            isDark={isDark}
                            isSelected={selectedPanelId === id}
                            indicators={indicators}
                            chartSettings={chartSettings}
                            alerts={alerts}
                            onSelectPane={() => handleSelectPane(id)}
                        />
                    );
                })}
            </div>
        </div>
    >

```

```

</div>

{/* HØJRE PANEL 1: Watchlist */}
{activePanel === 'watchlist' && (
  <div className="h-full relative flex shrink-0 shadow-2xl z-20">
    <Watchlist
      watchlists={watchlists}
      selectedSymbolId={currentActivePane.symbolId}
      onSelectSymbol={({sym: SymbolResponseDto}) => {
        setPanels((prev) => ({
          ...prev,
          [selectedPanelId]: { ...prev[selectedPanelId], symbolId: sym.id, symbol: sym.name, exchangeCode: sym.exchangeCode },
        }));
        onActiveStateChange(sym.name, currentActivePane.interval, sym);
      }}
      onCreateWatchlist={async (name: string) => {
        const created = await watchlistService.createWatchlist(name);
        if (created) setWatchlists((prev) => [...prev, created]);
      }}
      onDeleteWatchlist={async (watchlistId: number) => {
        await watchlistService.deleteWatchlist(watchlistId);
        setWatchlists((prev) => prev.filter((list) => list.id !== watchlistId));
      }}
      onAddSymbol={async (wld: number, sym: SymbolResponseDto) => {
        await watchlistService.addSymbolToWatchlist({ watchlistId: wld, symbolId: sym.id });
        const updated = await watchlistService.getMyWatchlists();
        if (updated) setWatchlists(updated);
      }}
      onRemoveSymbol={async (wld: number, symbolId: number) => {
        await watchlistService.removeSymbolFromWatchlist(wld, symbolId);
        const updated = await watchlistService.getMyWatchlists();
        if (updated) setWatchlists(updated);
      }}
      activeSymbols={activeSymbols}
    />
  </div>
)}

{/* HØJRE PANEL 2: AI-analyse */}
<AIAnalysisPanel
  isOpen={activePanel === 'ai'}
  onClose={() => setActivePanel(null)}
  symbol={currentActivePane.symbol}
  interval={currentActivePane.interval}
  analysis={aiAnalysis}
  loading={aiLoading}
  onRefresh={() => fetchAiAnalysis(currentActivePane.symbolId, currentActivePane.interval)}
/>

{/* HØJRE PANEL 3: Alarmliste */}
{activePanel === 'alerts' && (
  <div
    className={`w-72 h-full border-l flex flex-col shrink-0 z-20 shadow-2xl ${isDark ? 'bg-[#131722] border-gray-800 text-gray-200' : 'bg-white border-gray-200 text-gray-800'}
  >
    <div className="p-3 border-b flex items-center justify-between border-gray-700/50">
      <div className="flex items-center space-x-2">
        <Bell className="w-4 h-4 text-amber-400" />
        <h3 className="text-xs font-bold uppercase tracking-wider">
          {t('alerts.title', 'Fiyat Alarmları')}
        </h3>
      </div>
    </div>
    {onOpenCreateAlert && (
      <button
        type="button"
        onClick={onOpenCreateAlert}
        className="px-2 py-0.5 rounded text-[11px] font-bold bg-amber-500/20 text-amber-400 hover:bg-amber-500/30 border border-amber-500/30
        transition-colors"
      >
        {t('alerts.newAlert', '+ Yeni')}
      </button>
    )}
  </div>

  <div className="flex-1 overflow-y-auto divide-y divide-gray-700/20 p-2 space-y-1.5">
    {alerts.length === 0 ? (
      <div className="flex flex-col items-center justify-center h-48 text-center p-4">
        <Bell className="w-8 h-8 text-gray-500 mb-2 opacity-40" />
        <p className="text-xs text-gray-400">
          {t('alerts.noAlerts', 'Henüz kurulu alarminız yok.')}
        </p>
      </div>
    )}
  </div>
)}

```



```

): (
  alerts.map((alert) => (
    <div
      key={alert.id}
      className={`p-2.5 rounded-lg border text-xs flex flex-col space-y-1.5 transition-all ${alert.isTriggered
        ? 'bg-red-500/10 border-red-500/30 opacity-75'
        : alert.isActive
          ? isDark
            ? 'bg-[#1e222d] border-gray-700/60'
            : 'bg-gray-50 border-gray-200'
          : isDark
            ? 'bg-[#131722] border-gray-800 opacity-60'
            : 'bg-gray-100 border-gray-200 opacity-60'
        }}
    >
      <div className="flex items-center justify-between">
        <span className="font-bold text-xs text-blue-400">{alert.symbol}</span>
        <span
          className={`text-[10px] font-bold px-1.5 py-0.5 rounded ${alert.isTriggered
            ? 'bg-red-500/20 text-red-400'
            : alert.isActive
              ? 'bg-green-500/20 text-green-400'
              : 'bg-gray-500/20 text-gray-400'
            }}
        >
          {alert.isTriggered
            ? t('alerts.triggered', 'TETİKLENDİ')
            : alert.isActive
              ? t('alerts.active', 'AKTİF')
              : t('alerts.passive', 'PASİF')}
        </span>
      </div>

      <div className="flex items-center justify-between text-[11px] font-mono">
        <span className="text-gray-400">
          {alert.condition === 'CROSSES_UP'
            ? t('alerts.crossesUp', '▲ Yukarı Kesince')
            : t('alerts.crossesDown', '▼ Aşağı Kesince')}
        </span>
        <span className="font-bold text-white font-mono">
          ${Number(alert.targetPrice).toFixed(2)}
        </span>
      </div>

      {alert.note && (
        <p className="text-[10px] text-gray-400 italic truncate">{alert.note}</p>
      )}

      <div className="flex items-center justify-end space-x-2 pt-1 border-t border-gray-700/20">
        {!alert.isTriggered && onToggleAlert && (
          <button
            type="button"
            onClick={() => onToggleAlert(alert.id)}
            className={`p-1 rounded hover:bg-gray-700/40 transition-colors ${alert.isActive ? 'text-green-400' : 'text-gray-400'
              }}
            title={alert.isActive ? t('alerts.passive', 'Pasife Al') : t('alerts.active', 'Aktif Et')}
          >
            <Power className="w-3.5 h-3.5" />
          </button>
        )}

        {onDeleteAlert && (
          <button
            type="button"
            onClick={() => onDeleteAlert(alert.id)}
            className="p-1 rounded hover:bg-red-500/20 text-gray-400 hover:text-red-400 transition-colors"
            title={t('watchlistPanel.removeFromList', 'Alarmı Sil')}
          >
            <Trash2 className="w-3.5 h-3.5" />
          </button>
        )}
      </div>
    </div>
  ))
</div>
</div>
)}

{ /* HÖJRE VÆRKTØJSLINJE (TOOLBAR) */ }
<div
  className={`w-11 border-l flex flex-col items-center py-3 space-y-2 shrink-0 z-30 shadow-lg transition-colors duration-300 ${isDark ? 'bg-[#0b0e14]
border-gray-800' : 'bg-gray-100 border-gray-300'
  }}

```

```

>
  { /* 1. Watchlist-knap */ }
  <button
    type="button"
    onClick={() => setActivePanel(activePanel === 'watchlist' ? null : 'watchlist')}
    className={`w-8 h-8 rounded-lg flex items-center justify-center transition-all ${activePanel === 'watchlist'
      ? 'bg-blue-600 text-white shadow-md'
      : isDark
        ? 'text-gray-400 hover:bg-gray-800 hover:text-white'
        : 'text-gray-600 hover:bg-gray-200 hover:text-gray-900'}
    `}
    title={t('header.watchlist', 'İzleme Listesi')}
  >
    <svg className="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M5 5a2 2 0 012 2v16l-7-3.5L5 21V5z" />
    </svg>
  </button>

  { /* 2. AI-knap */ }
  <button
    type="button"
    onClick={() => {
      const willOpen = activePanel !== 'ai';
      setActivePanel(willOpen ? 'ai' : null);
      if (willOpen) fetchAiAnalysis(currentActivePane.symbolId, currentActivePane.interval);
    }}
    className={`w-8 h-8 rounded-lg flex items-center justify-center transition-all ${activePanel === 'ai'
      ? 'bg-purple-600 text-white shadow-md'
      : isDark
        ? 'text-gray-400 hover:bg-gray-800 hover:text-white'
        : 'text-gray-600 hover:bg-gray-200 hover:text-gray-900'}
    `}
    title={t('header.aiAssistant', 'Yapay Zeka Asistanı')}
  >
    <span className="text-xs font-bold">🤖</span>
  </button>

  { /* 3. Knap til alarmpanel */ }
  <button
    type="button"
    onClick={() => setActivePanel(activePanel === 'alerts' ? null : 'alerts')}
    className={`relative w-8 h-8 rounded-lg flex items-center justify-center transition-all ${activePanel === 'alerts'
      ? 'bg-amber-600 text-white shadow-md'
      : isDark
        ? 'text-gray-400 hover:bg-gray-800 hover:text-white'
        : 'text-gray-600 hover:bg-gray-200 hover:text-gray-900'}
    `}
    title={t('header.alerts', 'Alarmlar')}
  >
    <Bell className="w-4 h-4" />
    {alerts.filter((a) => a.isActive && !a.isTriggered).length > 0 && (
      <span className="absolute -top-1 -right-1 bg-amber-500 text-black text-[8px] font-black h-3.5 min-w-[14px] px-0.5 rounded-full flex items-center justify-center shadow">
        {alerts.filter((a) => a.isActive && !a.isTriggered).length}
      </span>
    )}
  </button>
</div>
</div>
);
};

```

F4. src/Components/Chart/DrawingLayer.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```

import React, { useCallback, useEffect, useMemo, useState } from 'react';
import type { IChartApi, Time } from 'lightweight-charts';
import {
  Eraser,
  Minus,
  MousePointer2,
  Ruler,
  Square,
  Trash2,
  TrendingUp,
  Type,
} from 'lucide-react';
import { useTranslation } from 'react-i18next';
import { chartDrawingService } from '../../Services/chartDrawingService';
import type {
  ChartDrawingDto,
  DrawingData,
  DrawingPoint,

```

```
DrawingTool,
PersistedDrawingType,
} from '../Types/chartDrawing';

interface DrawingLayerProps {
  symbolId: number;
  interval: string;
  isDark: boolean;
  isSelected: boolean;
  chart: IChartApi | null;
  series: any;
  renderVersion: number;
}

const FIB_LEVELS = [0, 0.236, 0.382, 0.5, 0.618, 0.786, 1];
const DEFAULT_COLOR = '#2962ff';

const isTwoPointData = (data: DrawingData): data is { points: [DrawingPoint, DrawingPoint]; style?: any } =>
  'points' in data;

export const DrawingLayer: React.FC<DrawingLayerProps> = ({
  symbolId,
  interval,
  isDark,
  isSelected,
  chart,
  series,
  renderVersion,
}) => {
  const { t } = useTranslation();
  const [tool, setTool] = useState<DrawingTool>('cursor');
  const [drawings, setDrawings] = useState<ChartDrawingDto[]>([]);
  const [draftStart, setDraftStart] = useState<DrawingPoint | null>(null);
  const [draftCurrent, setDraftCurrent] = useState<DrawingPoint | null>(null);
  const [busy, setBusy] = useState(false);

  const canDraw = symbolId > 0 && !!interval && !!chart && !!series;

  // Opretter eller tilføjer det valgte element.
  const loadDrawings = useCallback(async () => {
    if (!symbolId || !interval) {
      setDrawings([]);
      return;
    }

    try {
      setDrawings(await chartDrawingService.getDrawings(symbolId, interval));
    } catch (error) {
      console.error("[DrawingLayer] Çizimler yüklenemedi:", error);
    }
  }, [symbolId, interval]);

  useEffect(() => {
    setTool('cursor');
    setDraftStart(null);
    setDraftCurrent(null);
    void loadDrawings();
  }, [loadDrawings]);

  const pointFromMouse = (event: React.MouseEvent<SVGSVGElement>): DrawingPoint | null => {
    if (!chart || !series) return null;
    const rect = event.currentTarget.getBoundingClientRect();
    const x = event.clientX - rect.left;
    const y = event.clientY - rect.top;
    const time = chart.timeScale().coordinateToTime(x);
    const price = series.coordinateToPrice(y);
    if (time == null || price == null || !Number.isFinite(Number(price))) return null;
    return { time: Number(time), price: Number(price) };
  };

  const toCoordinates = (point: DrawingPoint): { x: number; y: number } | null => {
    if (!chart || !series) return null;
    const x = chart.timeScale().timeToCoordinate(point.time as Time);
    const y = series.priceToCoordinate(point.price);
    if (x == null || y == null) return null;
    return { x, y };
  };

  // Opretter eller tilføjer det valgte element.
  const createDrawing = async (drawingType: PersistedDrawingType, data: DrawingData) => {
    if (!symbolId || busy) return;
    setBusy(true);
```

```
try {
  const created = await chartDrawingService.createDrawing({
    symbolId,
    interval,
    drawingType,
    data,
    isVisible: true,
  });
  setDrawings((prev) => [...prev, created]);
} catch (error) {
  console.error('[DrawingLayer] Çizim kaydedilemedi:', error);
} finally {
  setBusy(false);
}
};

// Behandler den relevante brugerhandling eller event.
const handleCanvasClick = async (event: React.MouseEvent<SVGSVGElement>) => {
  if (!canDraw || tool === 'cursor' || tool === 'eraser') return;
  event.stopPropagation();
  const point = pointFromMouse(event);
  if (!point) return;

  if (tool === 'horizontalLine') {
    await createDrawing('horizontalLine', { price: point.price, style: { color: DEFAULT_COLOR, lineWidth: 2 } });
    return;
  }

  if (tool === 'text') {
    const text = window.prompt(t('drawings.textPrompt'))?.trim();
    if (!text) return;
    await createDrawing('text', { point, text, style: { color: DEFAULT_COLOR, lineWidth: 2 } });
    return;
  }

  if (!draftStart) {
    setDraftStart(point);
    setDraftCurrent(point);
    return;
  }

  const drawingType = tool as PersistedDrawingType;
  await createDrawing(drawingType, {
    points: [draftStart, point],
    style: { color: DEFAULT_COLOR, lineWidth: 2, fillOpacity: 0.08 },
  });
  setDraftStart(null);
  setDraftCurrent(null);
};

// Behandler den relevante brugerhandling eller event.
const handleMouseMove = (event: React.MouseEvent<SVGSVGElement>) => {
  if (!draftStart || tool === 'cursor' || tool === 'eraser') return;
  const point = pointFromMouse(event);
  if (point) setDraftCurrent(point);
};

// Fjerner det valgte element.
const deleteDrawing = async (drawingId: number) => {
  try {
    await chartDrawingService.deleteDrawing(drawingId);
    setDrawings((prev) => prev.filter((d) => d.id !== drawingId));
  } catch (error) {
    console.error('[DrawingLayer] Çizim silinemedi:', error);
  }
};

// Håndterer clear all.
const clearAll = async () => {
  if (!symbolId || !drawings.length) return;
  if (!window.confirm(t('drawings.clearConfirm'))) return;
  try {
    await chartDrawingService.clearDrawings(symbolId, interval);
    setDrawings([]);
  } catch (error) {
    console.error('[DrawingLayer] Çizimler temizlenemedi:', error);
  }
};

const visibleDrawings = useMemo(() => drawings.filter((d) => d.isVisible), [drawings]);

// Håndterer render drawing.
```

```

const renderDrawing = (drawing: ChartDrawingDto, draft = false) => {
  const data = drawing.data;
  const color = ('style' in data && data.style?.color) || DEFAULT_COLOR;
  const strokeWidth = ('style' in data && data.style?.lineWidth) || 2;
  const common = {
    stroke: color,
    strokeWidth,
    vectorEffect: 'non-scaling-stroke' as const,
    opacity: draft ? 0.65 : 1,
  };
  const eraseHandler = tool === 'eraser' && !draft
    ? (e: React.MouseEvent) => { e.stopPropagation(); void deleteDrawing(drawing.id); }
    : undefined;

  if (drawing.drawingType === 'horizontalLine' && 'price' in data) {
    if (!series) return null;
    const y = series.priceToCoordinate(data.price);
    if (y == null) return null;
    return <line key={drawing.id} x1="0" y1={y} x2="100%" y2={y} {...common} onClick={eraseHandler} className={tool === 'eraser' ? 'cursor-crosshair' : ''} />;
  }

  if (drawing.drawingType === 'text' && 'point' in data) {
    const p = toCoordinates(data.point);
    if (!p) return null;
    return (
      <g key={drawing.id} onClick={eraseHandler} className={tool === 'eraser' ? 'cursor-crosshair' : ''}>
        <circle cx={p.x} cy={p.y} r={3} fill={color} />
        <text x={p.x + 7} y={p.y - 7} fill={color} fontSize="12" fontWeight="600">{data.text}</text>
      </g>
    );
  }

  if (!isTwoPointData(data)) return null;
  const p1 = toCoordinates(data.points[0]);
  const p2 = toCoordinates(data.points[1]);
  if (!p1 || !p2) return null;

  if (drawing.drawingType === 'rectangle') {
    const x = Math.min(p1.x, p2.x);
    const y = Math.min(p1.y, p2.y);
    const width = Math.abs(p2.x - p1.x);
    const height = Math.abs(p2.y - p1.y);
    return (
      <rect key={drawing.id} x={x} y={y} width={width} height={height}
        fill={color} fillOpacity={data.style?.fillOpacity ?? 0.08} {...common}
        onClick={eraseHandler} className={tool === 'eraser' ? 'cursor-crosshair' : ''} />
    );
  }

  if (drawing.drawingType === 'fibonacci') {
    const low = data.points[0].price;
    const high = data.points[1].price;
    return (
      <g key={drawing.id} onClick={eraseHandler} className={tool === 'eraser' ? 'cursor-crosshair' : ''}>
        {FIB_LEVELS.map((level) => {
          const price = low + (high - low) * level;
          const y = series?.priceToCoordinate(price);
          if (y == null) return null;
          return (
            <g key={level}>
              <line x1={p1.x} y1={y} x2={p2.x} y2={y} {...common} opacity={draft ? 0.5 : 0.85} />
              <text x={Math.min(p1.x, p2.x) + 4} y={y - 3} fill={color} fontSize="10">{(level * 100).toFixed(1)}%</text>
            </g>
          );
        })}
      </g>
    );
  }

  return <line key={drawing.id} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y} {...common} onClick={eraseHandler} className={tool === 'eraser' ? 'cursor-crosshair' : ''} />;
};

const draftDrawing: ChartDrawingDto | null = draftStart && draftCurrent && ![cursor, 'eraser', 'horizontalLine', 'text'].includes(tool)
  ? {
    id: -1,
    symbolId,
    exchangeCode: "",
    symbolName: "",
    interval,
    drawingType: tool as PersistedDrawingType,
    data: { points: [draftStart, draftCurrent], style: { color: DEFAULT_COLOR, lineWidth: 2, fillOpacity: 0.08 } },
    isVisible: true,
    createdAt: "",
  }
  : null;

```

```

    updatedAt: "",
  }
  : null;

const tools: Array<{ id: DrawingTool; icon: React.ReactNode; label: string }> = [
  { id: 'cursor', icon: <MousePointer2 size={17} />, label: t('drawings.cursor') },
  { id: 'trendLine', icon: <TrendingUp size={17} />, label: t('drawings.trendLine') },
  { id: 'horizontalLine', icon: <Minus size={17} />, label: t('drawings.horizontalLine') },
  { id: 'rectangle', icon: <Square size={17} />, label: t('drawings.rectangle') },
  { id: 'fibonacci', icon: <Ruler size={17} />, label: t('drawings.fibonacci') },
  { id: 'text', icon: <Type size={17} />, label: t('drawings.text') },
  { id: 'eraser', icon: <Eraser size={17} />, label: t('drawings.eraser') },
];

// renderVersion udløser bevidst genberegning af koordinater efter zoom, scroll eller resize af chartet.
void renderVersion;

return (
  <>
    {isSelected && (
      <div className={ `absolute left-2 top-2 z-30 flex flex-col gap-1 rounded-lg border p-1 shadow-xl backdrop-blur ${isDark ? 'bg-[#131722]/95 border-[#2a2e39]' : 'bg-white/95 border-gray-200'}` }>
        {tools.map((item) => (
          <button
            key={item.id}
            type="button"
            title={item.label}
            onClick={(e) => {
              e.stopPropagation();
              setTool(item.id);
              setDraftStart(null);
              setDraftCurrent(null);
            }}
            className={ `w-8 h-8 rounded-md flex items-center justify-center transition ${tool === item.id ? 'bg-[#2962ff] text-white' : isDark ? 'text-gray-300 hover:bg-white/10' : 'text-gray-700 hover:bg-gray-100'}` }>
              <div className={ `h-px my-0.5 ${isDark ? 'bg-[#2a2e39]' : 'bg-gray-200'}` } />
              <button
                type="button"
                title={t('drawings.clearAll')}
                disabled={!drawings.length}
                onClick={(e) => { e.stopPropagation(); void clearAll(); }}
                className={ `w-8 h-8 rounded-md flex items-center justify-center transition disabled:opacity-30 ${isDark ? 'text-red-300 hover:bg-red-500/10' : 'text-red-600 hover:bg-red-50'}` }>
                  <Trash2 size={17} />
                </button>
              </div>
            </button>
          )
        )}
      <div className={ `w-8 h-8 rounded-md flex items-center justify-center transition disabled:opacity-30 ${isDark ? 'text-red-300 hover:bg-red-500/10' : 'text-red-600 hover:bg-red-50'}` }>
        <Trash2 size={17} />
      </div>
    )}
  </div>
  <svg
    className={ `absolute inset-0 z-20 w-full h-full ${tool === 'cursor' ? 'pointer-events-none' : 'pointer-events-auto'} ${tool !== 'cursor' ? 'cursor-crosshair' : 'cursor'}` }
    onClick={handleCanvasClick}
    onMouseMove={handleMouseMove}
    onDoubleClick={(e) => { e.preventDefault(); setDraftStart(null); setDraftCurrent(null); }}
    >
    {visibleDrawings.map((drawing) => renderDrawing(drawing))}
    {draftDrawing && renderDrawing(draftDrawing, true)}
  </svg>
</>
);
};

```

F5. src/Components/ChartPane.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```

import React, { useEffect, useRef, useState } from 'react';
import {
  createChart,
  CandlestickSeries,
  BarSeries,
  LineSeries,
  AreaSeries,
} from 'lightweight-charts';
import type {
  IChartApi,
  ISeriesApi,
  CandlestickData,
  LineData,

```

```

    Time,
    LogicalRange,
    IPriceLine,
    MouseEventParams,
  } from 'lightweight-charts';
import { TrendingUp } from 'lucide-react';
import { DrawingLayer } from './Chart/DrawingLayer';
import { useTranslation } from 'react-i18next';
import { signalrService } from '../Services/signalrService';
import { candleApi } from '../Services/candleApi';
import type { CandleData, ChartType } from '../Types/candle';
import type { IndicatorConfig } from '../Types/indicator';
import type { ChartVisualSettings } from '../Types/chartSettings';
import type { Alert } from '../Types/alert';
import {
  calculateSMA,
  calculateEMA,
  calculateBollingerBands,
  calculateRSI,
} from '../Utils/indicatorCalculators';

interface ChartPaneProps {
  symbolId: number;
  exchangeCode: string;
  symbol: string;
  interval: string;
  chartType?: ChartType;
  isDark: boolean;
  isSelected: boolean;
  indicators?: IndicatorConfig[];
  chartSettings?: ChartVisualSettings;
  alerts?: Alert[];
  onSelectPane: () => void;
  onPriceClick?: (price: number) => void;
}

type ChartSeriesApi =
  | ISeriesApi<'Candlestick'>
  | ISeriesApi<'Bar'>
  | ISeriesApi<'Line'>
  | ISeriesApi<'Area'>;

const getIntervalTime = (timestampInSeconds: number, interval: string): number => {
  const date = new Date(timestampInSeconds * 1000);
  if (interval.endsWith('m')) {
    const minutes = parseInt(interval, 10) || 1;
    date.setUTCMinutes(Math.floor(date.getUTCMinutes() / minutes) * minutes, 0, 0);
  } else if (interval.endsWith('h')) {
    const hours = parseInt(interval, 10) || 1;
    date.setUTCHours(Math.floor(date.getUTCHours() / hours) * hours, 0, 0);
  } else if (interval.endsWith('d')) {
    date.setUTCDate(0, 0, 0);
  } else if (interval.endsWith('w')) {
    // Timescale time_bucket('1 week', ...) bruger ISO-grænser med mandag som ugens start.
    const day = date.getUTCDate();
    const daysFromMonday = (day + 6) % 7;
    date.setUTCDate(date.getUTCDate() - daysFromMonday);
    date.setUTCHours(0, 0, 0, 0);
  } else if (interval.endsWith('mon')) {
    date.setUTCDate(1);
    date.setUTCHours(0, 0, 0, 0);
  } else if (interval.endsWith('y')) {
    date.setUTCDate(0, 1);
    date.setUTCHours(0, 0, 0, 0);
  }
  return Math.floor(date.getTime() / 1000);
};

const getIntervalDurationSeconds = (interval: string): number => {
  const normalized = interval.trim().toLowerCase();
  const value = parseInt(normalized, 10) || 1;

  if (normalized.endsWith('m') && !normalized.endsWith('mon')) return value * 60;
  if (normalized.endsWith('h')) return value * 60 * 60;
  if (normalized.endsWith('d')) return value * 24 * 60 * 60;
  if (normalized.endsWith('w')) return value * 7 * 24 * 60 * 60;
  if (normalized.endsWith('mon')) return value * 31 * 24 * 60 * 60;
  if (normalized.endsWith('y')) return value * 366 * 24 * 60 * 60;

  return 60;
};

const getLatestContiguousSegment = (
  candles: CandlestickData<Time>[],

```

```
interval: string
): CandlestickData<Time>[] => {
  if (candles.length <= 1) return candles;

  const expectedStep = getIntervalDurationSeconds(interval);
  // Nogle få manglende candles eller korte exchange-afbrydelser tolereres; gaps over flere år behandles separat.
  const maxAllowedGap = expectedStep * 3;
  let startIndex = candles.length - 1;

  for (let i = candles.length - 1; i > 0; i--) {
    const current = Number(candles[i].time);
    const previous = Number(candles[i - 1].time);
    const gap = current - previous;

    if (!Number.isFinite(gap) || gap <= 0 || gap > maxAllowedGap) {
      break;
    }

    startIndex = i - 1;
  }

  return candles.slice(startIndex);
};

const calculateHeikinAshi = (candles: CandlestickData<Time>[]): CandlestickData<Time>[] => {
  if (candles.length === 0) return [];
  const haCandles: CandlestickData<Time>[] = [];

  for (let i = 0; i < candles.length; i++) {
    const current = candles[i];
    const haClose = (current.open + current.high + current.low + current.close) / 4;
    let haOpen = current.open;

    if (i > 0) {
      const prevHa = haCandles[i - 1];
      haOpen = (prevHa.open + prevHa.close) / 2;
    }

    const haHigh = Math.max(current.high, haOpen, haClose);
    const haLow = Math.min(current.low, haOpen, haClose);

    haCandles.push({
      time: current.time,
      open: haOpen,
      high: haHigh,
      low: haLow,
      close: haClose,
    });
  }

  return haCandles;
};

export const ChartPane: React.FC<ChartPaneProps> = ({
  symbolId,
  exchangeCode,
  symbol,
  interval,
  chartType = 'candles',
  isDark,
  isSelected,
  indicators = [],
  chartSettings = {
    upColor: '#089981',
    downColor: '#f23645',
    showBorders: false,
    showWicks: true,
    showGrid: true,
    gridColorDark: '#1e222d',
    gridColorLight: '#f0f3fa',
    showPriceLine: true,
    soundEnabled: true,
  },
  alerts = [],
  onSelectPane,
  onPriceClick,
}) => {
  const { t } = useTranslation();
  const chartContainerRef = useRef<HTMLDivElement>(null);
  const chartRef = useRef<IChartApi | null>(null);
  const seriesRef = useRef<ChartSeriesApi | null>(null);
  const indicatorSeriesRef = useRef<Map<string, ISeriesApi<'Line'>[]>>(new Map());
```



```

const alertLinesRef = useRef<IPriceLine[]>([]);

const [loading, setLoading] = useState<boolean>(true);
const [lastCandle, setLastCandle] = useState<CandleData | null>(null);
const [drawingRenderVersion, setDrawingRenderVersion] = useState(0);
const [drawingReady, setDrawingReady] = useState(false);
const [historyPending, setHistoryPending] = useState(false);

const candlesRef = useRef<CandlestickData<Time>[]>([]);
const isLoadingMoreRef = useRef<boolean>(false);
const hasMoreDataRef = useRef<boolean>(true);
const lazyLoadCooldownUntilRef = useRef<number>(0);
const lazyRetryTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
const isNearLeftEdgeRef = useRef<boolean>(false);

const isDisposedRef = useRef<boolean>(false);
const rafIdRef = useRef<number | null>(null);

// Opdaterer de aktuelle data i brugergrænsefladen.
const updateIndicatorsData = (candles: CandlestickData<Time>[]) => {
  if (isDisposedRef.current || !chartRef.current || candles.length === 0) return;

  indicators.forEach((ind) => {
    if (!ind.enabled) return;
    const lines = indicatorSeriesRef.current.get(ind.id);
    if (!lines || lines.length === 0) return;

    try {
      if (ind.type === 'SMA') {
        const smaData = calculateSMA(candles, ind.period || 20);
        lines[0]?.setData(smaData);
      } else if (ind.type === 'EMA') {
        const emaData = calculateEMA(candles, ind.period || 50);
        lines[0]?.setData(emaData);
      } else if (ind.type === 'BB') {
        const bb = calculateBollingerBands(candles, ind.period || 20, ind.stdDev || 2);
        lines[0]?.setData(bb.upper);
        lines[1]?.setData(bb.middle);
        lines[2]?.setData(bb.lower);
      } else if (ind.type === 'RSI') {
        const rsiData = calculateRSI(candles, ind.period || 14);
        lines[0]?.setData(rsiData);
      }
    } catch (e) {
      console.warn(`[ChartPane] ${ind.name} indikator güncelleme hatası:`, e);
    }
  });
};

// 📌 Tegn prisalarmer som stiplede linjer på chartet.
useEffect(() => {
  const series = seriesRef.current;
  if (!series || isDisposedRef.current) return;

  // 1. Fjern tidligere alarmlinjer.
  alertLinesRef.current.forEach((line) => {
    try {
      series.removePriceLine(line);
    } catch {}
  });
  alertLinesRef.current = [];

  // 2. Filtrér aktive og ikke-udløste alarmer for dette symbol.
  const activeAlerts = alerts.filter(
    (a) =>
      (a.symbolId === symbolId || a.symbol.toUpperCase() === symbol.toUpperCase()) &&
      a.isActive &&
      !a.isTriggered
  );

  // 3. Opret en linje for hver alarm.
  activeAlerts.forEach((alert) => {
    try {
      const isUp = alert.condition === 'CROSSES_UP';
      const alertLabel = t('chartPane.alertLabel', 'ALARM');
      const line = series.createPriceLine({
        price: Number(alert.targetPrice),
        color: isUp ? '#089981' : '#23645',
        lineWidth: 1,
        lineStyle: 2,
        axisLabelVisible: true,
        title: `📌 ${alertLabel} ${Number(alert.targetPrice).toFixed(2)}`,
      });
    } catch {}
  });
});

```

```
    });
    alertLinesRef.current.push(line);
  } catch (err) {
    console.warn('[ChartPane] Alarm çizgisi çizilemedi:', err);
  }
});
}, [alerts, symbol, symbolId, t]);

useEffect(() => {
  if (!chartContainerRef.current) return;

  isDisposedRef.current = false;
  chartContainerRef.current.innerHTML = '';
  hasMoreDataRef.current = true;
  isLoadingMoreRef.current = false;
  lazyLoadCooldownUntilRef.current = 0;
  isNearLeftEdgeRef.current = false;
  if (lazyRetryTimerRef.current) {
    clearTimeout(lazyRetryTimerRef.current);
    lazyRetryTimerRef.current = null;
  }
  candlesRef.current = [];
  setHistoryPending(false);
  indicatorSeriesRef.current.clear();
  alertLinesRef.current = [];
  setLastCandle(null);

  const width = chartContainerRef.current.clientWidth || 600;
  const height = chartContainerRef.current.clientHeight || 400;

  const gridColor = chartSettings.showGrid
    ? isDark
      ? chartSettings.gridColorDark
      : chartSettings.gridColorLight
    : 'transparent';

  const chart = createChart(chartContainerRef.current, {
    width,
    height,
    layout: {
      background: { color: isDark ? '#131722' : '#ffffff' },
      textColor: isDark ? '#787b86' : '#131722',
    },
    grid: {
      vertLines: { color: gridColor },
      horzLines: { color: gridColor },
    },
    crosshair: { mode: 1 },
    timeScale: {
      borderColor: isDark ? '#2a2e39' : '#e0e3eb',
      timeVisible: true,
      secondsVisible: false,
      rightOffset: 12,
      barSpacing: 8,
    },
    rightPriceScale: {
      borderColor: isDark ? '#2a2e39' : '#e0e3eb',
      autoScale: true,
    },
  });
});

chartRef.current = chart;
setDrawingReady(true);
setDrawingRenderVersion((v) => v + 1);

if (onPriceClick) {
  chart.subscribeClick((param: MouseEventParams<Time>) => {
    if (!param.point || !seriesRef.current) return;
    const price = seriesRef.current.coordinateToPrice(param.point.y);
    if (price !== null && typeof price === 'number') {
      onPriceClick(price);
    }
  });
}

let series: ChartSeriesApi;
if (chartType === 'bars') {
  series = chart.addSeries(BarSeries, {
    upColor: chartSettings.upColor,
    downColor: chartSettings.downColor,
    priceLineVisible: chartSettings.showPriceLine,
  });
} else if (chartType === 'line') {
```

```

    series = chart.addSeries(LineSeries, {
      color: '#2962ff',
      lineWidth: 2,
      priceLineVisible: chartSettings.showPriceLine,
    });
  } else if (chartType === 'area') {
    series = chart.addSeries(AreaSeries, {
      topColor: 'rgba(41, 98, 255, 0.4)',
      bottomColor: 'rgba(41, 98, 255, 0.02)',
      lineColor: '#2962ff',
      lineWidth: 2,
      priceLineVisible: chartSettings.showPriceLine,
    });
  } else {
    series = chart.addSeries(CandlestickSeries, {
      upColor: chartSettings.upColor,
      downColor: chartSettings.downColor,
      borderVisible: chartSettings.showBorders,
      borderColor: chartSettings.upColor,
      wickVisible: chartSettings.showWicks,
      wickUpColor: chartSettings.upColor,
      wickDownColor: chartSettings.downColor,
      priceLineVisible: chartSettings.showPriceLine,
    });
  }
  seriesRef.current = series;

  indicators.forEach((ind) => {
    if (!ind.enabled) return;

    if (ind.type === 'SMA' || ind.type === 'EMA') {
      const line = chart.addSeries(LineSeries, {
        color: ind.color,
        lineWidth: 1.5 as any,
        priceLineVisible: false,
        lastValueVisible: true,
      });
      indicatorSeriesRef.current.set(ind.id, [line]);
    } else if (ind.type === 'BB') {
      const upper = chart.addSeries(LineSeries, {
        color: ind.color,
        lineWidth: 1 as any,
        priceLineVisible: false,
      });
      const middle = chart.addSeries(LineSeries, {
        color: ind.color,
        lineWidth: 1 as any,
        lineStyle: 2,
        priceLineVisible: false,
      });
      const lower = chart.addSeries(LineSeries, {
        color: ind.color,
        lineWidth: 1 as any,
        priceLineVisible: false,
      });
      indicatorSeriesRef.current.set(ind.id, [upper, middle, lower]);
    } else if (ind.type === 'RSI') {
      const rsiLine = chart.addSeries(LineSeries, {
        color: ind.color,
        lineWidth: 1.5 as any,
        priceScaleId: 'rsi',
      });
      chart.priceScale('rsi').applyOptions({
        scaleMargins: { top: 0.75, bottom: 0.05 },
      });
      indicatorSeriesRef.current.set(ind.id, [rsiLine]);
    }
  });
});

// Håndterer focus latest bars.
const focusLatestBars = (candles: CandlestickData<Time>[]) => {
  if (!chartRef.current || candles.length === 0 || isDisposedRef.current) return;

  // Alle data fra databasen bliver i serien; kun det første viewport fokuserer på den nyeste sammenhængende blok.
  // Selv om 2017 og 2026 findes i samme datasæt, påvirker de gamle priser derfor ikke
  // den aktuelle prisskala ved første åbning. Brugeren kan nå de gamle databasedata ved at scrolle mod venstre.
  const latestSegment = getLatestContiguousSegment(candles, interval);
  const latestCount = Math.max(1, latestSegment.length);
  const latestStartIndex = Math.max(0, candles.length - latestCount);
  const rightPaddingBars = 18;
  // Medtag ikke den gamle blok lige til venstre for et stort gap i det første viewport.
  const from = Math.max(-10, latestStartIndex - 0.5);
  const to = candles.length - 1 + rightPaddingBars;

```

```
try {
  chartRef.current.timeScale().setVisibleLogicalRange({ from, to });
} catch (err) {
  console.warn('[ChartPane] Son bar görünümü ayarlanamadı:', err);
}
};

// Håndterer apply data to series.
const applyDataToSeries = (candles: CandlestickData<Time>[]) => {
  if (isDisposedRef.current || !seriesRef.current) return;

  try {
    if (chartType === 'line' || chartType === 'area') {
      const lineData: LineData<Time>[] = candles.map((c) => ({
        time: c.time,
        value: c.close,
      }));
      (seriesRef.current as ISeriesApi<'Line' | 'Area'>).setData(lineData);
    } else if (chartType === 'heikin-ashi') {
      const haData = calculateHeikinAshi(candles);
      (seriesRef.current as ISeriesApi<'Candlestick'>).setData(haData);
    } else if (chartType === 'bars') {
      (seriesRef.current as ISeriesApi<'Bar'>).setData(candles);
    } else {
      (seriesRef.current as ISeriesApi<'Candlestick'>).setData(candles);
    }

    updateIndicatorsData(candles);
  } catch (err) {
    console.warn('[ChartPane] Veri atama hatası:', err);
  }
};

// Henter candles til det valgte symbol og timeframe.
const fetchCandles = async (endTime?: number) => {
  if (isDisposedRef.current || isLoadingMoreRef.current) return;
  if (endTime && !hasMoreDataRef.current) return;
  if (endTime && Date.now() < lazyLoadCooldownUntilRef.current) return;

  try {
    if (!endTime) setLoading(true);
    else isLoadingMoreRef.current = true;

    const pageSize = 500;
    const data = await candleApi.getHistoricalCandles(symbolId, interval, pageSize, endTime);

    if (isDisposedRef.current) return;

    if (!data || data.length === 0) {
      // Et tomt resultat betyder kun, når der faktisk anmodes om en ældre side,
      // at der ikke findes ældre poster i databasen.
      if (endTime) hasMoreDataRef.current = false;
      return;
    }

    const formatted: CandlestickData<Time>[] = data
      .map((c) => {
        const rawTime = Number(c.time);
        const timeInSec = rawTime > 10000000000 ? Math.floor(rawTime / 1000) : rawTime;
        return {
          time: timeInSec as Time,
          open: Number(c.open),
          high: Number(c.high),
          low: Number(c.low),
          close: Number(c.close),
        };
      })
      .filter((c) =>
        Number.isFinite(Number(c.time)) &&
        Number.isFinite(c.open) &&
        Number.isFinite(c.high) &&
        Number.isFinite(c.low) &&
        Number.isFinite(c.close)
      )
      .sort((a, b) => Number(a.time) - Number(b.time));

    if (isDisposedRef.current || formatted.length === 0) return;

    if (endTime) {
      const current = candlesRef.current;
      const oldLength = current.length;
      const previousRange = chartRef.current?.timeScale().getVisibleLogicalRange() ?? null;
```

```

const merged = [...formatted, ...current];
const byTime = new Map<number, CandlestickData<Time>>();
for (const candle of merged) byTime.set(Number(candle.time), candle);
const unique = Array.from(byTime.values()).sort(
  (a, b) => Number(a.time) - Number(b.time)
);

// Vis det, der findes i databasen: tilføj gamle candles i starten af serien, også hvis der er et gap.
// Det første viewport ændres ikke; brugeren ser disse candles, når der scrolles mod venstre.
const prependedCount = Math.max(0, unique.length - oldLength);

if (prependingCount > 0) {
  candlesRef.current = unique;
  applyDataToSeries(unique);
  lazyLoadCooldownUntilRef.current = 0;

  // Bevar brugerens synlige område efter prepend, så chartet ikke hopper.
  if (previousRange && chartRef.current) {
    try {
      chartRef.current.timeScale().setVisibleLogicalRange({
        from: previousRange.from + prependedCount,
        to: previousRange.to + prependedCount,
      });
    } catch {}
  }
}

// At en side indeholder færre end 500 rækker betyder ikke nødvendigvis, at der ikke findes ældre data.
// Det gælder især, hvis der er et stort gap mellem realtime-blokken og historical-blokken;
// den første ældre side kan være lille. Den reelle slutning accepteres først, når næste forespørgsel
// returnerer tomt.
hasMoreDataRef.current = prependingCount > 0;
setHistoryPending(false);
} else {
  // DB-first: Den første request kan nogle gange kun returnere 2-3 aktuelle candles, der er oprettet via realtime.
  // Det betyder ikke, at databasen ikke indeholder ældre data.
  // Hent også en side fra historical-blokken, og tilføj den til det samme datasæt.
  let initialCandles = formatted;
  const latestSegment = getLatestContiguousSegment(formatted, interval);

  if (latestSegment.length < 100 && formatted.length > 0) {
    try {
      const oldestTimeMs = Number(formatted[0].time) * 1000;
      const olderData = await candleApi.getHistoricalCandles(
        symbolId,
        interval,
        pageSize,
        oldestTimeMs
      );

      const olderFormularatted: CandlestickData<Time>[] = (olderData ?? []).
        .map((c) => {
          const rawTime = Number(c.time);
          const timeInSec = rawTime > 10000000000
            ? Math.floor(rawTime / 1000)
            : rawTime;
          return {
            time: timeInSec as Time,
            open: Number(c.open),
            high: Number(c.high),
            low: Number(c.low),
            close: Number(c.close),
          };
        })
        .filter((c) =>
          Number.isFinite(Number(c.time)) &&
          Number.isFinite(c.open) &&
          Number.isFinite(c.high) &&
          Number.isFinite(c.low) &&
          Number.isFinite(c.close)
        );

      if (olderFormularatted.length > 0) {
        const byTime = new Map<number, CandlestickData<Time>>();
        for (const candle of [...olderFormularatted, ...formatted]) {
          byTime.set(Number(candle.time), candle);
        }
        initialCandles = Array.from(byTime.values()).sort(
          (a, b) => Number(a.time) - Number(b.time)
        );
      }
    } catch (olderError) {

```

```

        console.warn('[ChartPane] İlk historical sayfa yüklenemedi:', olderError);
    }
}

candlesRef.current = initialCandles;
applyDataToSeries(initialCandles);

// Hvis den aktuelle sammenhængende blok kun består af få candles, men databasen
// har en fyldt historical-blok, vises den fyldte historical-blok i stedet for et tomt chart med tre candles.
// De aktuelle candles bliver i højre side af datasættet og
// kan nås ved at scrolle mod højre.
const currentLatestSegment = getLatestContiguousSegment(initialCandles, interval);
if (currentLatestSegment.length >= 20 || initialCandles.length < 20) {
    focusLatestBars(initialCandles);
} else if (chartRef.current) {
    const historicalOnly = initialCandles.slice(0, initialCandles.length - currentLatestSegment.length);
    const historicalSegment = getLatestContiguousSegment(historicalOnly, interval);
    if (historicalSegment.length > 0) {
        const segmentStart = historicalOnly.length - historicalSegment.length;
        const visibleBars = Math.min(120, historicalSegment.length);
        const from = Math.max(segmentStart, historicalOnly.length - visibleBars);
        const to = historicalOnly.length - 1 + 8;
        try {
            chartRef.current.timeScale().setVisibleLogicalRange({ from, to });
        } catch {
            focusLatestBars(initialCandles);
        }
    } else {
        focusLatestBars(initialCandles);
    }
}

setHistoryPending(false);
// En lille første side betyder ikke, at pagination er færdig.
hasMoreDataRef.current = initialCandles.length > 0;

if (formatted.length > 0) {
    const latest = formatted[formatted.length - 1];
    setLastCandle({
        time: Number(latest.time),
        open: latest.open,
        high: latest.high,
        low: latest.low,
        close: latest.close,
    });
}
} catch (err) {
    if (!isDisposedRef.current) {
        console.error('[ChartPane] Veri çekme hatası:', err);
    }
} finally {
    if (!isDisposedRef.current) {
        setLoading(false);
        isLoadingMoreRef.current = false;
    }
}
};

chart.timeScale().subscribeVisibleLogicalRangeChange((logicalRange: LogicalRange | null) => {
    if (isDisposedRef.current || !logicalRange) return;
    setDrawingRenderVersion((v) => v + 1);

    const nearLeftEdge = logicalRange.from < 20;
    isNearLeftEdgeRef.current = nearLeftEdge;

    if (
        nearLeftEdge &&
        !isLoadingMoreRef.current &&
        hasMoreDataRef.current &&
        Date.now() >= lazyLoadCooldownUntilRef.current
    ) {
        const oldestCandle = candlesRef.current[0];
        if (oldestCandle) {
            void fetchCandles(Number(oldestCandle.time) * 1000);
        }
    }
});

// Opdaterer de aktuelle data i brugergrænsefladen.
const handleCandleUpdate = (newCandle: CandleData) => {
    if (isDisposedRef.current || !seriesRef.current || !chartRef.current) return;
    if (newCandle.symbolId && newCandle.symbolId !== symbolId) return;

```

```

if (newCandle.exchange && newCandle.exchange.toUpperCase() !== exchangeCode.toUpperCase()) return;

if (rafIdRef.current) {
  cancelAnimationFrame(rafIdRef.current);
}

rafIdRef.current = requestAnimationFrame(() => {
  if (isDisposedRef.current || !seriesRef.current) return;

  const rawTime = Number(newCandle.time);
  const rawSec = rawTime > 10000000000 ? Math.floor(rawTime / 1000) : rawTime;
  const candleTimeSec = getIntervalTime(rawSec, interval);
  const candleTime = candleTimeSec as Time;
  const lastCandleInRef = candlesRef.current[candlesRef.current.length - 1];

  if (lastCandleInRef && candleTimeSec < Number(lastCandleInRef.time)) {
    return;
  }

  let high = Number(newCandle.high);
  let low = Number(newCandle.low);
  let open = Number(newCandle.open);
  const close = Number(newCandle.close);

  if (isNaN(high) || isNaN(low) || isNaN(open) || isNaN(close)) return;

  if (lastCandleInRef && Number(lastCandleInRef.time) === candleTimeSec) {
    high = Math.max(Number(lastCandleInRef.high), high);
    low = Math.min(Number(lastCandleInRef.low), low);
    open = Number(lastCandleInRef.open);
  }

  const updated: CandlestickData<Time> = {
    time: candleTime,
    open,
    high,
    low,
    close,
  };

  // Slet ikke databasedata, selv om der er et stort gap mellem realtime- og historical-blokken.
  // Den nye candle tilføjes normalt i slutningen af serien, mens viewport bliver på den aktuelle side.

  try {
    if (chartType === 'line' || chartType === 'area') {
      (seriesRef.current as ISeriesApi<'Line' | 'Area'>).update({
        time: candleTime,
        value: close,
      });
    } else if (chartType === 'heikin-ashi') {
      const temp = [...candlesRef.current];
      if (lastCandleInRef && Number(lastCandleInRef.time) === candleTimeSec) {
        temp[temp.length - 1] = updated;
      } else {
        temp.push(updated);
      }
      const ha = calculateHeikinAshi(temp);
      if (ha.length > 0) {
        (seriesRef.current as ISeriesApi<'Candlestick'>).update(ha[ha.length - 1]);
      }
    } else if (chartType === 'bars') {
      (seriesRef.current as ISeriesApi<'Bar'>).update(updated);
    } else {
      (seriesRef.current as ISeriesApi<'Candlestick'>).update(updated);
    }
  }

  if (lastCandleInRef && Number(lastCandleInRef.time) === candleTimeSec) {
    candlesRef.current[candlesRef.current.length - 1] = updated;
  } else {
    candlesRef.current.push(updated);
  }

  updateIndicatorsData(candlesRef.current);

  setLastCandle({
    time: candleTimeSec,
    open,
    high,
    low,
    close,
  });
});

```

```

    } catch (err) {
      console.warn('[ChartPane] Canlı veri güncelleme atlandı:', err);
    }
  });
};

const hasResolvedSymbol = symbolId > 0 && exchangeCode.trim().length > 0 && symbol.trim().length > 0;

// Händerer init.
const init = async () => {
  // Ved første render har App muligvis endnu ikke fundet SymbolId/ExchangeCode.
  // Abonnér aldrig på gamle SignalR-grupper uden exchange, f.eks. :BTCUSDT.
  if (!hasResolvedSymbol) return;

  await fetchCandles();
  if (isDisposedRef.current) return;
  try {
    await signalRService.startConnection();
    if (isDisposedRef.current) return;
    await signalRService.subscribeToSymbolGroup(exchangeCode, symbol);
    signalRService.subscribeToCandleUpdates(handleCandleUpdate);
  } catch (e) {
    if (isDisposedRef.current) {
      console.error('[ChartPane] SignalR başlatma hatası:', e);
    }
  }
};

init();

const resizeObserver = new ResizeObserver((entries) => {
  if (isDisposedRef.current || !entries || entries.length === 0) return;
  const { width: w, height: h } = entries[0].contentRect;
  if (chartRef.current && w > 0 && h > 0) {
    chartRef.current.applyOptions({ width: w, height: h });
    setDrawingRenderVersion((v) => v + 1);
  }
});

resizeObserver.observe(chartContainerRef.current);

return () => {
  isDisposedRef.current = true;

  if (rafIdRef.current) {
    cancelAnimationFrame(rafIdRef.current);
    rafIdRef.current = null;
  }
  if (lazyRetryTimerRef.current) {
    clearTimeout(lazyRetryTimerRef.current);
    lazyRetryTimerRef.current = null;
  }

  resizeObserver.disconnect();
  if (hasResolvedSymbol) {
    void signalRService.unsubscribeFromSymbolGroup(exchangeCode, symbol);
  }
  signalRService.unsubscribeFromCandleUpdates(handleCandleUpdate);

  indicatorSeriesRef.current.clear();
  alertLinesRef.current = [];
  setDrawingReady(false);
  seriesRef.current = null;
  if (chartRef.current) {
    chartRef.current.remove();
    chartRef.current = null;
  }
};
}, [symbolId, exchangeCode, symbol, interval, chartType, isDark, indicators, chartSettings, onPriceClick]);

const isPriceUp = lastCandle ? lastCandle.close >= lastCandle.open : true;
const priceColor = isPriceUp ? 'text-[#089981]' : 'text-[#f23645]';

return (
  <div
    onClick={onSelectPane}
    className={
      `flex flex-col w-full h-full relative overflow-hidden cursor-pointer transition-all border ${isSelected}
        ? 'border-[#2962ff] ring-1 ring-[#2962ff]/40 shadow-md'
        : isDark
        ? 'border-[#2a2e39] bg-[#131722]'
        : 'border-gray-200 bg-white'
      `
    }
  >

```



```

>
  { /* Legend og OHLC */
  <div
    className={`px-3 py-1.5 flex items-center justify-between text-xs font-medium z-10 select-none border-b ${isDark ? 'border-#2a2e39/60 bg-
[#131722]/80' : 'border-gray-100 bg-white/80'} backdrop-blur-sm`}
  >
    <div className="flex items-center space-x-2">
      <div className="w-5 h-5 rounded-full bg-#00897b text-white flex items-center justify-center text-[10px] font-bold shadow-sm">
        {symbol.slice(0, 3)}
      </div>
      <span className="font-semibold text-xs tracking-tight text-gray-300">
        {symbol} · {interval} · <span className="text-gray-400">{chartType.toUpperCase()}</span>
      </span>

      <div className="hidden md:flex items-center space-x-1.5 ml-2">
        {indicators
          .filter((i) => i.enabled)
          .map((ind) => (
            <span
              key={ind.id}
              className="px-1.5 py-0.5 rounded text-[10px] font-bold flex items-center space-x-1"
              style={{ backgroundColor: `${ind.color}15`, color: ind.color }}
            >
              <span>{ind.name}</span>
            </span>
          ))}
      </div>
    </div>

    {lastCandle && (
      <div className="hidden sm:flex items-center space-x-2.5 font-mono text-[11px]">
        <span className="text-gray-400">
          {t('chartPane.open', 'O')} <span className={priceColor}>{lastCandle.open.toFixed(2)}</span>
        </span>
        <span className="text-gray-400">
          {t('chartPane.high', 'H')} <span className={priceColor}>{lastCandle.high.toFixed(2)}</span>
        </span>
        <span className="text-gray-400">
          {t('chartPane.low', 'L')} <span className={priceColor}>{lastCandle.low.toFixed(2)}</span>
        </span>
        <span className="text-gray-400">
          {t('chartPane.close', 'C')} <span className={priceColor}>{lastCandle.close.toFixed(2)}</span>
        </span>
      </div>
    )}
  </div>

  { /* Chartets tegneområde */
  <div className="flex-1 relative w-full h-full min-h-[200px] overflow-hidden">
    {loading && (
      <div
        className={`absolute inset-0 z-10 flex items-center justify-center backdrop-blur-xs ${isDark ? 'bg-#131722/70' : 'bg-white/70'}
        }
      >
        <div className="w-6 h-6 border-2 border-#2962ff border-t-transparent rounded-full animate-spin" />
      </div>
    )}

    {historyPending && !loading && (
      <div
        className={`absolute left-3 top-3 z-10 flex items-center gap-2 rounded-md border px-2.5 py-1.5 text-[11px] shadow-sm backdrop-blur-sm pointer-
events-none select-none ${
          isDark
            ? 'border-#2a2e39 bg-#131722/85 text-gray-300'
            : 'border-gray-200 bg-white/90 text-gray-600'
        }
        }
      >
        <span className="h-1.5 w-1.5 rounded-full bg-#2962ff animate-pulse" />
        <span>{t('chartPane.historySyncing', 'Geçmiş veriler senkronize ediliyor')}</span>
      </div>
    )}

    <div className="absolute left-3 bottom-8 z-10 flex items-center space-x-1.5 opacity-30 hover:opacity-80 transition-opacity pointer-events-none select-
none">
      <div className="p-1 bg-#2962ff rounded-md shadow-sm">
        <TrendingUp className="h-3 w-3 text-white" />
      </div>
      <span className="font-extrabold tracking-wider text-xs text-white">
        TRADING<span className="text-#2962ff">PRO</span>
      </span>
    </div>

    <div ref={chartContainerRef} className="w-full h-full absolute inset-0 [&_a]!hidden" />
  </div>

```

```

        {drawingReady && (
          <DrawingLayer
            symbolId={symbolId}
            interval={interval}
            isDark={isDark}
            isSelected={isSelected}
            chart={chartRef.current}
            series={seriesRef.current}
            renderVersion={drawingRenderVersion}
          />
        )}
      </div>
    </div>
  );
};

```

F6. src/Components/ChartSettingsModal.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```

import React, { useState, useEffect } from 'react';
import { X, RotateCcw, CandlestickChart as CandleIcon, Grid, Sliders, Bell } from 'lucide-react';
import { useTranslation } from 'react-i18next';
import { DEFAULT_CHART_SETTINGS, type ChartVisualSettings } from '../Types/chartSettings';

interface ChartSettingsModalProps {
  isOpen: boolean;
  onClose: () => void;
  settings: ChartVisualSettings;
  onSave: (newSettings: ChartVisualSettings) => void;
  isDark: boolean;
}

type TabType = 'symbol' | 'grid' | 'priceLine' | 'sound';

export const ChartSettingsModal: React.FC<ChartSettingsModalProps> = ({
  isOpen,
  onClose,
  settings,
  onSave,
  isDark,
}) => {
  const { t } = useTranslation();
  const [activeTab, setActiveTab] = useState<TabType>('symbol');
  const [formData, setFormularData] = useState<ChartVisualSettings>(() => ({
    ...DEFAULT_CHART_SETTINGS,
    ...settings,
  }));

  useEffect(() => {
    if (settings) {
      setFormularData({
        ...DEFAULT_CHART_SETTINGS,
        ...settings,
      });
    }
  }, [settings, isOpen]);

  if (!isOpen) return null;

  const handleChange = <K extends keyof ChartVisualSettings>({
    key: K,
    value: ChartVisualSettings[K]
  }) => {
    setFormularData((prev) => ({
      ...prev,
      [key]: value,
    }));
  };

  // Behandler den relevante brugerhandling eller event.
  const handleReset = () => {
    setFormularData({ ...DEFAULT_CHART_SETTINGS });
  };

  // Behandler den relevante brugerhandling eller event.
  const handleSubmit = (e: React.FormularEvent) => {
    e.preventDefault();
    onSave(formData);
    onClose();
  };
};

```

```

return (
  <div
    role="dialog"
    aria-modal="true"
    onClick={onClose}
    className="fixed inset-0 z-50 flex items-center justify-center bg-black/60 backdrop-blur-xs p-4 animate-in fade-in duration-150"
  >
    <div
      onClick={(e) => e.stopPropagation()}
      className={
        `w-full max-w-2xl h-[460px] rounded-xl shadow-2xl border flex flex-col overflow-hidden transition-colors ${isDark
          ? 'bg-[#131722] border-[#2a2e39] text-gray-200'
          : 'bg-white border-gray-200 text-gray-800'
        }`
      }
    >
      {/* Sidehoved */}
      <div
        className={`flex items-center justify-between px-5 py-3.5 border-b ${isDark ? 'border-[#2a2e39] bg-[#1e222d]' : 'border-gray-100 bg-gray-50'
        }`}
      >
        <div className="flex items-center space-x-2 font-bold text-sm">
          <Sliders className="w-4 h-4 text-[#2962ff]" />
          <span>{t('chartSettings.title', 'Grafik & Terminal Indstillinger')}</span>
        </div>
        <button
          type="button"
          onClick={onClose}
          className="p-1 rounded-md hover:bg-gray-500/10 text-gray-400 hover:text-gray-200 transition-colors"
          title={t('chartSettings.cancel', 'Kapat')}
        >
          <X className="w-4 h-4" />
        </button>
      </div>

      {/* Indhold (venstre faner + højre indhold) */}
      <div className="flex flex-1 overflow-hidden">
        {/* Venstre fanemenu */}
        <div
          className={`w-48 p-2 space-y-1 border-r flex flex-col ${isDark ? 'border-[#2a2e39] bg-[#131722]' : 'border-gray-100 bg-gray-50/50'
          }`}
        >
          <button
            type="button"
            onClick={() => setActiveTab('symbol')}
            className={
              `flex items-center space-x-2.5 w-full px-3 py-2 rounded-lg text-xs font-semibold transition-all ${activeTab === 'symbol'
                ? 'bg-[#2962ff] text-white shadow-sm'
                : 'text-gray-400 hover:bg-gray-500/10 hover:text-gray-200'
              }`
            }
          >
            <CandleIcon className="w-4 h-4" />
            <span>{t('chartSettings.tabSymbol', 'Sembol / Mum')}</span>
          </button>

          <button
            type="button"
            onClick={() => setActiveTab('grid')}
            className={
              `flex items-center space-x-2.5 w-full px-3 py-2 rounded-lg text-xs font-semibold transition-all ${activeTab === 'grid'
                ? 'bg-[#2962ff] text-white shadow-sm'
                : 'text-gray-400 hover:bg-gray-500/10 hover:text-gray-200'
              }`
            }
          >
            <Grid className="w-4 h-4" />
            <span>{t('chartSettings.tabGrid', 'Tuval / Izgara')}</span>
          </button>

          <button
            type="button"
            onClick={() => setActiveTab('priceLine')}
            className={
              `flex items-center space-x-2.5 w-full px-3 py-2 rounded-lg text-xs font-semibold transition-all ${activeTab === 'priceLine'
                ? 'bg-[#2962ff] text-white shadow-sm'
                : 'text-gray-400 hover:bg-gray-500/10 hover:text-gray-200'
              }`
            }
          >
            <Sliders className="w-4 h-4" />
            <span>{t('chartSettings.tabPriceLine', 'Fiyat Çizgileri')}</span>
          </button>

          <button
            type="button"
            onClick={() => setActiveTab('sound')}
            className={
              `flex items-center space-x-2.5 w-full px-3 py-2 rounded-lg text-xs font-semibold transition-all ${activeTab === 'sound'
                ? 'bg-[#2962ff] text-white shadow-sm'
                : 'text-gray-400 hover:bg-gray-500/10 hover:text-gray-200'
              }`
            }
          >

```

```

>
<Bell className="w-4 h-4" />
<span>{t('chartSettings.tabSound', 'Alarmlar & Ses')}</span>
</button>
</div>

{/* Indhold i højre panel */}
<div className="flex-1 p-5 overflow-y-auto">
  {activeTab === 'symbol' && (
    <div className="space-y-5">
      <h4 className="text-xs font-bold uppercase tracking-wider text-gray-400">
        {t('chartSettings.bodyColor', 'Mum Gövdesi ve Renkleri')}
      </h4>

      <div className="flex items-center justify-between py-2 border-b border-gray-500/10">
        <span className="text-xs font-medium">{t('chartSettings.bodyColor', 'Gövde Rengi (Boğa / Ayı))}</span>
        <div className="flex items-center space-x-4">
          <label className="flex items-center space-x-1.5 cursor-pointer">
            <input
              type="color"
              value={formData.upColor}
              onChange={(e) => handleChange('upColor', e.target.value)}
              className="w-6 h-6 rounded cursor-pointer border-none bg-transparent"
            />
            <span className="text-xs text-[#089981] font-semibold">{t('chartSettings.bullColor', 'Boğa')}</span>
          </label>
          <label className="flex items-center space-x-1.5 cursor-pointer">
            <input
              type="color"
              value={formData.downColor}
              onChange={(e) => handleChange('downColor', e.target.value)}
              className="w-6 h-6 rounded cursor-pointer border-none bg-transparent"
            />
            <span className="text-xs text-[#f23645] font-semibold">{t('chartSettings.bearColor', 'Ayı')}</span>
          </label>
        </div>
      </div>

      <label className="flex items-center space-x-2.5 cursor-pointer py-1">
        <input
          type="checkbox"
          checked={formData.showBorders}
          onChange={(e) => handleChange('showBorders', e.target.checked)}
          className="w-4 h-4 rounded text-[#2962ff] focus:ring-0 cursor-pointer"
        />
        <span className="text-xs font-medium">{t('chartSettings.showBorders', 'Kenarlıkları Göster (Borders))}</span>
      </label>

      <label className="flex items-center space-x-2.5 cursor-pointer py-1">
        <input
          type="checkbox"
          checked={formData.showWicks}
          onChange={(e) => handleChange('showWicks', e.target.checked)}
          className="w-4 h-4 rounded text-[#2962ff] focus:ring-0 cursor-pointer"
        />
        <span className="text-xs font-medium">{t('chartSettings.showWicks', 'Fitilleri Göster (Wicks))}</span>
      </label>
    </div>
  )}

  {activeTab === 'grid' && (
    <div className="space-y-5">
      <h4 className="text-xs font-bold uppercase tracking-wider text-gray-400">
        {t('chartSettings.tabGrid', 'Izgara ve Çizgiler')}
      </h4>

      <label className="flex items-center space-x-2.5 cursor-pointer py-1">
        <input
          type="checkbox"
          checked={formData.showGrid}
          onChange={(e) => handleChange('showGrid', e.target.checked)}
          className="w-4 h-4 rounded text-[#2962ff] focus:ring-0 cursor-pointer"
        />
        <span className="text-xs font-medium">{t('chartSettings.showGrid', 'Izgara Çizgilerini Göster')}</span>
      </label>

      <div className="flex items-center justify-between py-2 border-b border-gray-500/10">
        <span className="text-xs font-medium">{t('chartSettings.gridDark', 'Izgara Rengi (Koyu Tema))}</span>
        <input
          type="color"
          value={formData.gridColorDark}
          onChange={(e) => handleChange('gridColorDark', e.target.value)}
          className="w-6 h-6 rounded cursor-pointer border-none bg-transparent"
        />
      </div>
    </div>
  )}

```

```

</div>

<div className="flex items-center justify-between py-2 border-b border-gray-500/10">
  <span className="text-xs font-medium">{t('chartSettings.gridLight', 'Izgara Rengi (Açık Tema))}</span>
  <input
    type="color"
    value={formData.gridColorLight}
    onChange={(e) => handleChange('gridColorLight', e.target.value)}
    className="w-6 h-6 rounded cursor-pointer border-none bg-transparent"
  />
</div>
</div>
))

{activeTab === 'priceLine' && (
  <div className="space-y-5">
    <h4 className="text-xs font-bold uppercase tracking-wider text-gray-400">
      {t('chartSettings.tabPriceLine', 'Prislinje Indstillinger')}
    </h4>

    <label className="flex items-center space-x-2.5 cursor-pointer py-1">
      <input
        type="checkbox"
        checked={formData.showPriceLine}
        onChange={(e) => handleChange('showPriceLine', e.target.checked)}
        className="w-4 h-4 rounded text-[#2962ff] focus:ring-0 cursor-pointer"
      />
      <span className="text-xs font-medium">{t('chartSettings.showPriceLine', 'Son Prislinjeni Göster')}</span>
    </label>
  </div>
))

{activeTab === 'sound' && (
  <div className="space-y-5">
    <h4 className="text-xs font-bold uppercase tracking-wider text-gray-400">
      {t('chartSettings.tabSound', 'Ses ve Bildirimler')}
    </h4>

    <label className="flex items-center space-x-2.5 cursor-pointer py-1">
      <input
        type="checkbox"
        checked={formData.soundEnabled ?? true}
        onChange={(e) => handleChange('soundEnabled', e.target.checked)}
        className="w-4 h-4 rounded text-[#2962ff] focus:ring-0 cursor-pointer"
      />
      <span className="text-xs font-medium">{t('chartSettings.soundAlert', 'Alarm Tetiklendiğinde Ses Çal')}</span>
    </label>
  </div>
))
</div>
</div>

{ /* Sidefod */ }
<div
  className={`px-5 py-3 border-t flex items-center justify-between ${isDark ? 'border-[#2a2e39] bg-[#1e222d]' : 'border-gray-100 bg-gray-50'}
  `}
>
  <button
    type="button"
    onClick={handleReset}
    className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg border border-gray-500/20 hover:bg-gray-500/10 text-xs font-medium text-gray-400 hover:text-gray-200 transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" />
    <span>{t('chartSettings.reset', 'Sıfırla')}</span>
  </button>

  <div className="flex space-x-2">
    <button
      type="button"
      onClick={onClose}
      className="px-4 py-1.5 rounded-lg border border-gray-500/20 hover:bg-gray-500/10 text-xs font-medium transition-colors"
    >
      {t('chartSettings.cancel', 'İptal')}
    </button>
    <button
      type="button"
      onClick={handleSubmit}
      className="px-4 py-1.5 rounded-lg bg-[#2962ff] hover:bg-blue-600 active:bg-blue-700 text-white text-xs font-semibold shadow-sm transition-colors"
    >
      {t('chartSettings.saveApply', 'Kaydet & Uygula')}
    </button>
  </div>
</div>

```

```
        </div>
      </div>
    </div>
  );
};
```

F7. src/Components/Common/ProtectedRoute.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```
import React from 'react';
import { authService } from '../Services/authService';

interface ProtectedRouteProps {
  children: React.ReactNode;
  fallback: React.ReactNode;
}

export const ProtectedRoute: React.FC<ProtectedRouteProps> = ({ children, fallback }) => {
  const isAuthenticated = authService.isAuthenticated();

  // Vis fallback-komponenten (AuthPage), hvis brugeren ikke er logget ind.
  if (!isAuthenticated) {
    return <>{fallback}</>;
  }

  // Vis det beskyttede indhold (Dashboard), hvis brugeren er logget ind.
  return <>{children}</>;
};
```

F8. src/Components/CreateAlertModal.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```
import React, { useState, useEffect } from 'react';
import { X, Bell } from 'lucide-react';
import { useTranslation } from 'react-i18next';
import type { AlertCondition, CreateAlertRequest } from '../Types/alert';

interface CreateAlertModalProps {
  symbolId: number;
  symbol: string;
  currentPrice?: number;
  isOpen: boolean;
  onClose: () => void;
  onSave: (data: CreateAlertRequest) => Promise<void>;
  isDark?: boolean;
}

export const CreateAlertModal: React.FC<CreateAlertModalProps> = ({
  symbolId,
  symbol,
  currentPrice = 0,
  isOpen,
  onClose,
  onSave,
  isDark = true,
}) => {
  const { t } = useTranslation();
  const [targetPrice, setTargetPrice] = useState<string>("");
  const [condition, setCondition] = useState<AlertCondition>('CROSSES_UP');
  const [note, setNote] = useState<string>("");
  const [isSubmitting, setIsSubmitting] = useState<boolean>(false);

  useEffect(() => {
    if (isOpen) {
      setTargetPrice(currentPrice > 0 ? currentPrice.toString() : "");
      setNote("");
      setCondition("CROSSES_UP");
    }
  }, [isOpen, currentPrice, symbol]);

  if (!isOpen) return null;

  // Behandler den relevante brugerhandling eller event.
  const handleSubmit = async (e: React.FormularEvent) => {
    e.preventDefault();
    const numericPrice = parseFloat(targetPrice);
    if (isNaN(numericPrice) || numericPrice <= 0) return;

    setIsSubmitting(true);
```

TradingPro | Ozan Korkmaz | Side 127 af 205

```

        : 'bg-gray-50 border-gray-300 text-gray-900 focus:border-blue-500'
      }}
    >
    <option value="CROSSES_UP">
      {t('alerts.crossesUp', '▲ Yukarı Kestiğinde (Fiyat >= Hedef)')}
    </option>
    <option value="CROSSES_DOWN">
      {t('alerts.crossesDown', '▼ Aşağı Kestiğinde (Fiyat <= Hedef)')}
    </option>
  </select>
</div>

<div>
  <label className="block text-xs font-semibold uppercase tracking-wider text-gray-400 mb-1">
    {t('alerts.note', 'Not (Opsiyonel)')}
  </label>
  <input
    type="text"
    placeholder={t('alerts.notePlaceholder', 'Örn: Destek kırılırsa sat')}
    value={note}
    onChange={(e) => setNote(e.target.value)}
    maxLength={255}
    className={
      `w-full text-xs p-2.5 rounded-lg border outline-none transition-colors ${isDark
        ? 'bg-[#131722] border-gray-700 text-white focus:border-blue-500'
        : 'bg-gray-50 border-gray-300 text-gray-900 focus:border-blue-500'
      }`
    }
  />
</div>

/* Footer-knapper */
<div
  className={
    `flex justify-end space-x-2.5 pt-3 border-t ${isDark ? 'border-[#2a2e39]' : 'border-gray-100'
    }`
  }
>
  <button
    type="button"
    onClick={onClose}
    className="px-4 py-2 rounded-lg border border-gray-500/20 hover:bg-gray-500/10 text-xs font-medium transition-colors"
  >
    {t('chartSettings.cancel', 'İptal')}
  </button>
  <button
    type="submit"
    disabled={isSubmitting}
    className="px-4 py-2 rounded-lg bg-blue-600 hover:bg-blue-500 active:bg-blue-700 text-white font-semibold text-xs shadow-sm disabled:opacity-50 transition-colors"
  >
    {isSubmitting
      ? '...'
      : t('alerts.startAlert', 'Alarmı Başlat')}
  </button>
</div>
</form>
</div>
</div>
);
};

```

F9. src/Components/HeaderNavbar.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```

import React, { useState, useRef, useEffect, useMemo } from 'react';
import {
  User,
  LogOut,
  Plus,
  BarChart2,
  Check,
  ChevronDown,
  Settings,
  Bell,
  Globe,
  Home,
  HelpCircle,
  Send,
  CheckCircle2,
  X,
  TrendingUp
} from 'lucide-react';
import { useTranslation } from 'react-i18next';
import { useTheme } from '../Context/ThemeContext';
import { preferenceService } from '../Services/preferenceService';
import { type UserManagementDto } from '../Services/userService';
import { ProfileModal, type UserProfileDto } from '../ProfileModal'; // Modalen er importeret.

```



```

import type { WatchlistResponseDto } from '../Types/watchlist';
import type { ChartType } from '../Types/candle';
import type { IndicatorConfig } from '../Types/indicator';
import type { Alert } from '../Types/alert';
import type { SymbolResponseDto } from '../Types/symbol';

interface HeaderNavbarProps {
  currentUser: UserProfileDto | null;
  onLogout: () => void;
  watchlists: WatchlistResponseDto[];
  currentSymbol: string;
  currentSymbolId: number;
  currentExchangeCode: string;
  currentInterval: string;
  chartType: ChartType;
  layoutMode: '1x1' | '1x2' | '2x2';
  activeSymbols: SymbolResponseDto[];
  indicators: IndicatorConfig[];
  currentPrice?: number;
  alerts?: Alert[];
  onToggleIndicator: (indicatorId: string) => void;
  onSelectSymbol: (symbol: SymbolResponseDto) => void;
  onSelectInterval: (interval: string) => void;
  onSelectChartType: (type: ChartType) => void;
  onChangeLayout: (mode: '1x1' | '1x2' | '2x2') => void;
  onOpenSettings: () => void;
  onOpenAlertModal: () => void;
  onProfileUpdated?: (updated: UserManagementDto) => void;
}

const TIMEFRAMES = ['1m', '3m', '5m', '15m', '30m', '1h', '2h', '4h', '1d', '1w', '1mon', '1y'];

export const HeaderNavbar: React.FC<HeaderNavbarProps> = ({
  currentUser,
  onLogout,
  watchlists,
  currentSymbol,
  currentSymbolId,
  currentExchangeCode,
  currentInterval,
  chartType,
  layoutMode,
  activeSymbols,
  indicators,
  currentPrice,
  alerts = [],
  onToggleIndicator,
  onSelectSymbol,
  onSelectInterval,
  onSelectChartType,
  onChangeLayout,
  onOpenSettings,
  onOpenAlertModal,
  onProfileUpdated,
}) => {
  const { t, i18n } = useTranslation();
  const { isDark, toggleTheme } = useTheme();

  const [isHamburgerOpen, setIsHamburgerOpen] = useState(false);
  const [isSearchOpen, setIsSearchOpen] = useState(false);
  const [isTfOpen, setIsTfOpen] = useState(false);
  const [isChartTypeOpen, setIsChartTypeOpen] = useState(false);
  const [isIndicatorsOpen, setIsIndicatorsOpen] = useState(false);
  const [isLayoutOpen, setIsLayoutOpen] = useState(false);
  const [isLangOpen, setIsLangOpen] = useState(false);
  const [searchTerm, setSearchTerm] = useState("");

  // Modaler
  const [isProfileModalOpen, setIsProfileModalOpen] = useState(false);
  const [isHelpModalOpen, setIsHelpModalOpen] = useState(false);
  const [helpTab, setHelpTab] = useState<'faq' | 'contact'>('faq');
  const [helpSent, setHelpSent] = useState(false);
  const [contactFormular, setContactFormular] = useState({ name: "", email: "", subject: "", message: "" });

  const searchRef = useRef<HTMLDivElement>(null);
  const hamburgerRef = useRef<HTMLDivElement>(null);
  const tfRef = useRef<HTMLDivElement>(null);
  const chartTypeRef = useRef<HTMLDivElement>(null);
  const indicatorsRef = useRef<HTMLDivElement>(null);
  const layoutRef = useRef<HTMLDivElement>(null);
  const langRef = useRef<HTMLDivElement>(null);

  const CHART_TYPES: { id: ChartType; label: string; icon: string }[] = useMemo(
    () => [

```

```

    { id: 'candles', label: t('header.chartTypes.candles', 'Mumlar (Candles)'), icon: '🕯' },
    { id: 'bars', label: t('header.chartTypes.bars', 'Çubuklar (Bars)'), icon: '📊' },
    { id: 'line', label: t('header.chartTypes.line', 'Çizgi (Line)'), icon: '📈' },
    { id: 'area', label: t('header.chartTypes.area', 'Alan (Area)'), icon: '📉' },
    { id: 'heikin-ashi', label: t('header.chartTypes.heikinAshi', 'Heikin Ashi'), icon: '📊' },
  ],
  [t]
);

const filteredSymbols = useMemo(() => {
  if (!searchTerm.trim()) return activeSymbols;
  const q = searchTerm.toLowerCase();
  return activeSymbols.filter((s) =>
    s.name.toLowerCase().includes(q) ||
    s.exchangeCode.toLowerCase().includes(q) ||
    s.baseAsset.toLowerCase().includes(q) ||
    s.quoteAsset.toLowerCase().includes(q)
  );
}, [searchTerm, activeSymbols]);

const activeIndicatorsCount = indicators.filter((i) => i.enabled).length;

const activeAlertsCount = useMemo(() => {
  return alerts.filter(
    (a) => (a.symbolId === currentSymbolId || a.symbol.toUpperCase() === currentSymbol.toUpperCase()) && a.isActive && !a.isTriggered
  ).length;
}, [alerts, currentSymbol, currentSymbolId]);

useEffect(() => {
  // Behandler den relevante brugerhandling eller event.
  const handleClickOutside = (e: MouseEvent) => {
    const target = e.target as Node;
    if (searchRef.current && !searchRef.current.contains(target)) setTsSearchOpen(false);
    if (hamburgerRef.current && !hamburgerRef.current.contains(target)) setTsHamburgerOpen(false);
    if (tfRef.current && !tfRef.current.contains(target)) setTsTfOpen(false);
    if (chartTypeRef.current && !chartTypeRef.current.contains(target)) setTsChartTypeOpen(false);
    if (indicatorsRef.current && !indicatorsRef.current.contains(target)) setTsIndicatorsOpen(false);
    if (layoutRef.current && !layoutRef.current.contains(target)) setTsLayoutOpen(false);
    if (langRef.current && !langRef.current.contains(target)) setTsLangOpen(false);
  };
  document.addEventListener('mousedown', handleClickOutside);
  return () => document.removeEventListener('mousedown', handleClickOutside);
}, []);

// Behandler den relevante brugerhandling eller event.
const handleLanguageChange = (lang: 'tr' | 'en' | 'da') => {
  i18n.changeLanguage(lang);
  localStorage.setItem('tradingpro_lang', lang);
  setTsLangOpen(false);
  preferenceService.savePreferences({ Language: lang } as any).catch(() => {});
};

// Behandler den relevante brugerhandling eller event.
const handleContactSubmit = (e: React.FormularEvent) => {
  e.preventDefault();
  setHelpSent(true);
  setTimeout(() => {
    setHelpSent(false);
    setContactFormular({ name: "", email: "", subject: "", message: "" });
    setTsHelpModalOpen(false);
  }, 2000);
};

const totalWatchlistCount = watchlists.reduce((acc, w) => acc + (w.items?.length || 0), 0);

const currentLangLabel = i18n.language.startsWith('da')
  ? 'DK DA'
  : i18n.language.startsWith('en')
    ? 'GB EN'
    : 'TR TR';

return (
  <>
    <header
      className={`h-11 px-3 border-b flex items-center justify-between shrink-0 select-none z-40 transition-colors duration-200 ${isDark ? 'bg-[#131722] border-[#2a2e39] text-[#d1d4dc]': 'bg-white border-gray-200 text-[#131722]`}
    >
      <div className="flex items-center space-x-1 sm:space-x-2">
        <div className="relative" ref={hamburgerRef}>

```

```

700'
    <button
      type="button"
      onClick={() => setIsHamburgerOpen(!isHamburgerOpen)}
      className={`relative p-1.5 rounded hover:bg-gray-500/10 transition-colors flex items-center justify-center ${isDark ? 'text-gray-300' : 'text-gray-
    }}
    title={t('header.mainMenu', 'Ana Menü')}
  >
    <svg className="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2.2" d="M4 6h16M4 12h16M4 18h16" />
    </svg>
    {totalWatchlistCount > 0 && (
      <span className="absolute -top-1 -right-1 bg-[#f23645] text-white text-[9px] font-black h-4 min-w-[16px] px-1 rounded-full flex items-center
justify-center shadow">
        {totalWatchlistCount}
      </span>
    )}
  </button>

  {isHamburgerOpen && (
    <div
      className={`absolute top-full left-0 mt-1.5 w-64 rounded-xl shadow-2xl z-50 py-2 divide-y text-xs backdrop-blur-xl ${isDark
        ? 'bg-[#1e222d] border border-[#2a2e39] divide-[#2a2e39] text-gray-200'
        : 'bg-white border border-gray-200 divide-gray-100 text-gray-800'}
    >
      <div className="px-4 py-2 flex items-center justify-between">
        <div className="flex items-center space-x-2">
          <TrendingUp className="h-4 w-4 text-blue-500" />
          <span className="font-bold tracking-wide">TradingPro Terminal</span>
        </div>
        <span className="text-[10px] text-blue-500 font-mono font-bold bg-blue-500/10 px-1.5 py-0.5 rounded">v2.0</span>
      </div>

      <div className="p-1 space-y-0.5">
        <button
          type="button"
          onClick={() => setIsHamburgerOpen(false)}
          className="w-full flex items-center space-x-2.5 px-3 py-2 rounded-lg hover:bg-blue-500/10 text-blue-400 font-semibold transition-colors"
        >
          <Home className="w-4 h-4" />
          <span>{t('menu.home', 'Ana Sayfa / Terminal')}</span>
        </button>

        <button
          type="button"
          onClick={() => {
            setIsHamburgerOpen(false);
            setIsProfileModalOpen(true);
          }}
          className="w-full flex items-center space-x-2.5 px-3 py-2 rounded-lg hover:bg-gray-500/10 transition-colors font-medium"
        >
          <User className="w-4 h-4 text-emerald-400" />
          <span>{t('menu.profile', 'Profil & Hesap')}</span>
        </button>

        <button
          type="button"
          onClick={() => {
            setIsHamburgerOpen(false);
            setIsHelpModalOpen(true);
          }}
          className="w-full flex items-center space-x-2.5 px-3 py-2 rounded-lg hover:bg-gray-500/10 transition-colors font-medium"
        >
          <HelpCircle className="w-4 h-4 text-purple-400" />
          <span>{t('menu.helpCenter', 'Yardım Merkezi & İletişim')}</span>
        </button>
      </div>

      <div className="p-1">
        <button
          type="button"
          onClick={toggleTheme}
          className="w-full flex items-center justify-between px-3 py-2 rounded-lg hover:bg-gray-500/10 transition-colors font-medium"
        >
          <span>{isDark ? t('header.darkTheme', '🌙 Koyu Tema') : t('header.lightTheme', '☀️ Açık Tema')}</span>
          <span
            className={`w-7 h-3.5 flex items-center rounded-full p-0.5 ${isDark ? 'bg-blue-600 justify-end' : 'bg-gray-400 justify-start'}
          >
            <span className="w-2.5 h-2.5 bg-white rounded-full shadow" />
          </span>
        </button>
      </div>

```

```

    </div>
  })
</div>

{/* Symbolsøgning */}
<div className="relative" ref={searchRef}>
  <button
    type="button"
    onClick={() => setIsSearchOpen(true)}
    className={`flex items-center space-x-2 px-3 py-1 rounded-full font-bold text-xs transition-all border ${isDark
      ? 'bg-[#2a2e39]/60 hover:bg-[#2a2e39] border-transparent text-[#2962ff]'
      : 'bg-[#e0e3eb]/70 hover:bg-[#e0e3eb] border-transparent text-[#2962ff]'
    }`}
  >
    <span className="tracking-wide text-xs">{currentSymbol}</span><span className="text-[9px] text-gray-500">{currentExchangeCode}</span>
    <svg className="w-3.5 h-3.5 text-gray-400" fill="none" stroke="currentColor" viewBox="0 0 24 24">
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0 114 0z" />
    </svg>
  </button>

  {isSearchOpen && (
    <div
      className={`absolute top-full left-0 mt-2 w-64 rounded-xl shadow-2xl z-50 p-2 border ${isDark ? 'bg-[#1e222d] border-[#2a2e39] text-gray-200'
        : 'bg-white border-gray-200 text-gray-800'
      }`}
    >
      <input
        type="text"
        autoFocus
        placeholder={t('header.searchPlaceholder', 'Sembol ara...')}
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
        className={`w-full text-xs rounded-lg px-2.5 py-1.5 focus:outline-none focus:border-blue-500 border mb-1.5 font-semibold ${isDark ? 'bg-[#131722] border-gray-700 text-white' : 'bg-gray-100 border-gray-300 text-gray-900'
        }`}
      />
      <div className="max-h-48 overflow-y-auto divide-y divide-gray-500/10">
        {filteredSymbols.map((sym) => (
          <button
            key={` ${sym.exchangeCode}:${sym.id}`}
            type="button"
            onClick={() => {
              onSelectSymbol(sym);
              setIsSearchOpen(false);
              setSearchTerm("");
            }}
            className={`w-full text-left px-2.5 py-1.5 text-xs font-semibold rounded transition-colors flex items-center justify-between
              ${currentSymbolId === sym.id ? 'text-[#2962ff] bg-blue-500/10' : 'hover:bg-gray-500/10'
            }`}
          >
            <span className="flex flex-col"><span>{sym.name}</span><span className="text-[9px] font-medium text-gray-500">{sym.exchangeCode} · {sym.baseAsset}/{sym.quoteAsset}</span></span>
            {currentSymbolId === sym.id && <Check className="w-3 h-3 text-[#2962ff]" />}
          </button>
        ))}
      </div>
    </div>
  )}

  <button
    type="button"
    onClick={() => setIsSearchOpen(true)}
    className="p-1 rounded-full hover:bg-gray-500/10 text-gray-400 hover:text-gray-200 transition-colors"
    title={t('header.addSymbol', 'Sembol Ekle')}
  >
    <Plus className="w-4 h-4" />
  </button>

  <div className={`h-4 w-[1px] mx-1 ${isDark ? 'bg-[#2a2e39]' : 'bg-gray-200'}`} />

  {/* Tidsinterval */}
  <div className="relative" ref={tfRef}>
    <button
      type="button"
      onClick={() => setIsTfOpen(!isTfOpen)}
      className="px-2 py-1 rounded text-xs font-semibold text-[#2962ff] hover:bg-gray-500/10 transition-colors flex items-center space-x-1"
    >
      <span>{currentInterval}</span>
      <ChevronDown className="w-3 h-3 text-gray-400" />
    </button>

    {isTfOpen && (
      <div

```

```

        className={ `absolute top-full left-0 mt-1 w-24 rounded-lg shadow-xl z-50 py-1 border ${isDark ? 'bg-[#1e222d] border-[#2a2e39]' : 'bg-white border-gray-200'}` }
      >
        {TIMEFRAMES.map((tf) => (
          <button
            key={tf}
            type="button"
            onClick={() => {
              onSelectInterval(tf);
              setIsTfOpen(false);
            }}
            className={ `w-full text-left px-3 py-1 text-xs font-semibold transition-colors ${currentInterval === tf ? 'text-[#2962ff] bg-blue-500/10' : 'hover:bg-gray-500/10'}` }
          >
            {tf}
          </button>
        ))}
      </div>

    <div className={ `h-4 w-[1px] mx-1 ${isDark ? 'bg-[#2a2e39]' : 'bg-gray-200'}` } />

    { /* Charttype */ }
    <div className="relative" ref={chartTypeRef}>
      <button
        type="button"
        onClick={() => setIsChartTypeOpen(!isChartTypeOpen)}
        className="p-1.5 rounded hover:bg-gray-500/10 text-gray-300 hover:text-white transition-colors flex items-center space-x-1 text-xs font-semibold"
      >
        <span>{CHART_TYPES.find((t) => t.id === chartType)?.icon || '📊'}</span>
        <span className="text-[10px] text-gray-400">▼</span>
      </button>

      {isChartTypeOpen && (
        <div
          className={ `absolute top-full left-0 mt-1 w-48 rounded-xl shadow-2xl z-50 py-1.5 border divide-y divide-gray-500/10 ${isDark ? 'bg-[#1e222d] border-[#2a2e39] text-gray-200' : 'bg-white border-gray-200 text-gray-800'}` }
        >
          {CHART_TYPES.map((type) => (
            <button
              key={type.id}
              type="button"
              onClick={() => {
                onSelectChartType(type.id);
                setIsChartTypeOpen(false);
              }}
              className={ `w-full flex items-center space-x-2.5 px-3 py-2 text-xs font-semibold transition-colors ${chartType === type.id ? 'text-[#2962ff] bg-blue-500/10' : 'hover:bg-gray-500/10'}` }
            >
              <span className="text-sm">{type.icon}</span>
              <span>{type.label}</span>
            </button>
          ))}
        </div>
      )}

    <div className={ `h-4 w-[1px] mx-1 ${isDark ? 'bg-[#2a2e39]' : 'bg-gray-200'}` } />

    { /* Indikatorer */ }
    <div className="relative" ref={indicatorsRef}>
      <button
        type="button"
        onClick={() => setIsIndicatorsOpen(!isIndicatorsOpen)}
        className={ `flex items-center space-x-1.5 px-2 py-1 rounded text-xs font-semibold transition-colors ${activeIndicatorsCount > 0 ? 'bg-blue-500/10 text-[#2962ff]' : isDark ? 'text-gray-300 hover:bg-gray-500/10' : 'text-gray-700 hover:bg-gray-100'}` }
      >
        <BarChart2 className="w-4 h-4 text-[#2962ff]" />
        <span className="hidden sm:inline">{t('header.indicators', 'Indikatorer')}</span>
        {activeIndicatorsCount > 0 && (
          <span className="bg-[#2962ff] text-white text-[10px] font-bold px-1.5 rounded-full">
            {activeIndicatorsCount}
          </span>
        )}
      >
    </div>
  )}

```

```

        <ChevronDown className="w-3 h-3 opacity-60" />
      </button>

      {isIndicatorsOpen && (
        <div
          className={`absolute top-full left-0 mt-1.5 w-60 rounded-xl shadow-2xl z-50 p-1.5 border divide-y divide-gray-500/10 ${isDark ? 'bg-[#1e222d] border-[#2a2e39] text-gray-200' : 'bg-white border-gray-200 text-gray-800'}`}
        >
          <div className="px-2.5 py-1 text-[11px] font-bold text-gray-400 uppercase tracking-wider">
            {t('header.technicalIndicators', 'Teknik Göstergeler')}
          </div>
          <div className="py-1 space-y-0.5">
            {indicators.map((ind) => (
              <button
                key={ind.id}
                type="button"
                onClick={() => onToggleIndicator(ind.id)}
                className={`w-full flex items-center justify-between px-2.5 py-2 text-xs font-semibold rounded-lg transition-colors ${ind.enabled
                  ? isDark
                    ? 'bg-blue-600/15 text-[#2962ff]'
                    : 'bg-blue-50 text-[#2962ff]'
                  : isDark
                    ? 'hover:bg-gray-500/10 text-gray-300'
                    : 'hover:bg-gray-100 text-gray-700'}`}
              >
                <div className="flex items-center space-x-2.5">
                  <span
                    className="w-2.5 h-2.5 rounded-full shrink-0"
                    style={{ backgroundColor: ind.color }}
                  />
                  <span>{ind.name}</span>
                </div>

                <div
                  className={`w-4 h-4 rounded flex items-center justify-center border transition-all ${ind.enabled
                    ? 'bg-[#2962ff] border-[#2962ff] text-white'
                    : isDark
                      ? 'border-gray-600 bg-gray-800'
                      : 'border-gray-300 bg-white'}`}
                >
                  {ind.enabled && <Check className="w-3 h-3 stroke-[3]" />}
                </div>
              </button>
            ))}
          </div>
        </div>
      )}

      <div className={`h-4 w-[1px] mx-1 ${isDark ? 'bg-[#2a2e39]' : 'bg-gray-200'}`} />

      {/* Layout */}
      <div className="relative" ref={layoutRef}>
        <button
          type="button"
          onClick={() => setIsLayoutOpen(!isLayoutOpen)}
          className="flex items-center space-x-1 px-2 py-1 rounded hover:bg-gray-500/10 text-xs font-semibold text-gray-400"
        >
          <span>☐ {layoutMode}</span>
        </button>

        {isLayoutOpen && (
          <div
            className={`absolute top-full left-0 mt-1 w-32 rounded-lg shadow-xl z-50 py-1 border ${isDark ? 'bg-[#1e222d] border-[#2a2e39]' : 'bg-white border-gray-200'}`}
          >
            <div>
              {[['1x1', '1x2', '2x2'] as const].map((mode) => (
                <button
                  key={mode}
                  type="button"
                  onClick={() => {
                    onChangeLayout(mode);
                    setIsLayoutOpen(false);
                  }}
                  className={`w-full text-left px-3 py-1.5 text-xs font-semibold ${layoutMode === mode ? 'text-[#2962ff] bg-blue-500/10' : 'hover:bg-gray-500/10'}`}
                >
                  {mode === '1x1'
                    ? t('header.layout.single', '1x1 (Tek)')}

```

```

        : mode === '1x2'
        ? t('header.layout.sideBySide', '1x2 (Yan Yana)')
        : t('header.layout.grid', '2x2 (Grid)'))
    </button>
  )))
</div>
})
</div>

{/* Prisalarm */}
<div className={h-4 w-[1px] mx-1 ${isDark ? 'bg-[#2a2e39]' : 'bg-gray-200'}} />

<button
  type="button"
  onClick={onOpenAlertModal}
  className={relative flex items-center space-x-1 px-2.5 py-1 rounded text-xs font-semibold transition-all ${activeAlertsCount > 0
    ? 'bg-amber-500/15 text-amber-400 border border-amber-500/30 hover:bg-amber-500/25'
    : isDark
      ? 'text-gray-300 hover:bg-gray-500/10 hover:text-amber-400'
      : 'text-gray-700 hover:bg-gray-100 hover:text-amber-600'
    }}
  title={t('header.createAlertTooltip', 'Prisalarm Kur')}
>
  <Bell className="w-3.5 h-3.5" />
  <span className="hidden lg:inline">{t('header.createAlert', 'Alarm')}</span>
  {activeAlertsCount > 0 && (
    <span className="bg-amber-500 text-black text-[9px] font-black px-1.5 py-0.2 rounded-full">
      {activeAlertsCount}
    </span>
  )}
</button>
</div>

{/* HØJRE SIDE */}
<div className="flex items-center space-x-1.5 sm:space-x-2">
  {currentPrice !== undefined && currentPrice > 0 && (
    <div className="hidden lg:flex items-center space-x-1.5 mr-1 text-xs font-mono">
      <span className="text-gray-400">{t('alerts.currentPrice', 'Son')}</span>
      <span className="font-bold text-emerald-400">${currentPrice.toLocaleString()}</span>
    </div>
  )}

  {/* Sprogvälger */}
  <div className="relative" ref={langRef}>
    <button
      type="button"
      onClick={() => setIsLangOpen(!isLangOpen)}
      className={flex items-center space-x-1 px-2 py-1 rounded text-xs font-semibold transition-colors border ${isDark
        ? 'border-[#2a2e39] bg-[#1e222d] text-gray-300 hover:bg-gray-500/10'
        : 'border-gray-200 bg-gray-50 text-gray-700 hover:bg-gray-100'
      }}
      title={t('header.changeLanguage', 'Dil Değiştir')}
    >
      <Globe className="w-3.5 h-3.5 text-blue-400" />
      <span>{currentLangLabel}</span>
      <ChevronDown className="w-3 h-3 opacity-60" />
    </button>

    {isLangOpen && (
      <div
        className={absolute top-full right-0 mt-1 w-28 rounded-lg shadow-xl z-50 py-1 border divide-y divide-gray-500/10 ${isDark ? 'bg-[#1e222d] border-[#2a2e39] text-gray-200' : 'bg-white border-gray-200 text-gray-800'}}
      >
        <button
          type="button"
          onClick={() => handleLanguageChange('tr')}
          className={w-full text-left px-3 py-1.5 text-xs font-semibold flex items-center justify-between ${i18n.language.startsWith('tr') ? 'text-[#2962ff] bg-blue-500/10' : 'hover:bg-gray-500/10'}}
        >
          <span>TR Türkçe</span>
          {i18n.language.startsWith('tr') && <Check className="w-3 h-3 text-[#2962ff]" />}
        </button>

        <button
          type="button"
          onClick={() => handleLanguageChange('en')}
          className={w-full text-left px-3 py-1.5 text-xs font-semibold flex items-center justify-between ${i18n.language.startsWith('en') ? 'text-[#2962ff] bg-blue-500/10' : 'hover:bg-gray-500/10'}}
        >
          <span>GB English</span>
          {i18n.language.startsWith('en') && <Check className="w-3 h-3 text-[#2962ff]" />}
        </button>
      >
    </div>
  )}
</div>

```

```

        </button>

        <button
          type="button"
          onClick={() => handleLanguageChange('da')}
          className={`w-full text-left px-3 py-1.5 text-xs font-semibold flex items-center justify-between ${i18n.language.startsWith('da') ? 'text-
[#2962ff] bg-blue-500/10' : 'hover:bg-gray-500/10'
        }`}
        >
          <span>Dansk</span>
          {i18n.language.startsWith('da') && <Check className="w-3 h-3 text-[#2962ff]" />}
        </button>
      </div>
    ))
  </div>

  {/* Indstillinger */}
  <button
    type="button"
    onClick={onOpenSettings}
    className="p-1.5 rounded-lg hover:bg-gray-500/10 text-gray-400 hover:text-white transition-colors"
    title={t('header.settings', 'Grafik & Arayüz Indstillinger')}
  >
    <Settings className="h-4 w-4" />
  </button>

  {/* Profilknop */}
  <div
    onClick={() => setIsProfileModalOpen(true)}
    className={`hidden md:flex items-center space-x-1.5 px-2 py-0.5 rounded border text-xs cursor-pointer hover:border-blue-500/50 transition-colors
    ${isDark ? 'bg-[#1e222d] border-[#2a2e39]' : 'bg-gray-50 border-gray-200'
  }`}
  >
    <User className="h-3 w-3 text-blue-400" />
    <span className="font-medium">
      {currentUser?.firstName || t('header.user', 'Kullanıcı')} {currentUser?.lastName || ""}
    </span>
  </div>

  {/* Log ud */}
  <button
    onClick={onLogout}
    className="p-1.5 rounded-lg hover:bg-red-500/10 text-gray-400 hover:text-red-400 transition-colors"
    title={t('header.logout', 'Log ud')}
  >
    <Logout className="h-4 w-4" />
  </button>
</div>
</header>

{/* 🗂 MODAL TIL BRUGERPROFIL */}
<ProfileModal
  isOpen={isProfileModalOpen}
  onClose={() => setIsProfileModalOpen(false)}
  isDark={isDark}
  currentUser={currentUser}
  onLogout={onLogout}
  onProfileUpdated={onProfileUpdated}
/>

{/* 🏠 MODAL TIL HJÆLPECENTER */}
{isHelpModalOpen && (
  <div
    role="dialog"
    aria-modal="true"
    onClick={() => setIsHelpModalOpen(false)}
    className="fixed inset-0 z-50 flex items-center justify-center bg-black/60 backdrop-blur-xs p-4 animate-in fade-in duration-150"
  >
    <div
      onClick={(e) => e.stopPropagation()}
      className={`w-full max-w-lg rounded-xl shadow-2xl border flex flex-col overflow-hidden transition-colors ${isDark ? 'bg-[#131722] border-[#2a2e39]
      text-gray-200' : 'bg-white border-gray-200 text-gray-800'
    }`}
    >
      <div className={`flex items-center justify-between px-5 py-3.5 border-b ${isDark ? 'border-[#2a2e39] bg-[#1e222d]' : 'border-gray-100 bg-gray-50'
      }`}>
        <div className="flex items-center space-x-2 font-bold text-sm">
          <HelpCircle className="w-4 h-4 text-purple-500" />
          <span>{t('helpCenter.title', 'Yardım Merkezi & Destek')}</span>
        </div>
        <button
          type="button"
          onClick={() => setIsHelpModalOpen(false)}

```



```

        className="p-1 rounded-md hover:bg-gray-500/10 text-gray-400 hover:text-gray-200"
      >
        <X className="w-4 h-4" />
      </button>
    </div>

    <div className={`flex border-b text-xs font-semibold ${isDark ? 'border-{#2a2e39}' : 'border-gray-100'}`}>
      <button
        type="button"
        onClick={() => setHelpTab('faq')}
        className={`flex-1 py-2.5 text-center transition-colors border-b-2 ${helpTab === 'faq'
          ? 'border-purple-500 text-purple-400 bg-purple-500/5'
          : 'border-transparent text-gray-400 hover:text-gray-200'
        }`}
      >
        {t('helpCenter.faqTab', 'Sıkça Sorulan Sorular')}
      </button>
      <button
        type="button"
        onClick={() => setHelpTab('contact')}
        className={`flex-1 py-2.5 text-center transition-colors border-b-2 ${helpTab === 'contact'
          ? 'border-purple-500 text-purple-400 bg-purple-500/5'
          : 'border-transparent text-gray-400 hover:text-gray-200'
        }`}
      >
        {t('helpCenter.contactTab', 'Bize Ulaşın')}
      </button>
    </div>

    <div className="p-5 max-h-[380px] overflow-y-auto">
      {helpTab === 'faq' ? (
        <div className="space-y-4 text-xs">
          <div className={`p-3 rounded-lg border ${isDark ? 'bg-{#1e222d}/60 border-gray-800' : 'bg-gray-50 border-gray-200'}`}>
            <h5 className="font-bold text-blue-400 mb-1">? {t('helpCenter.faq1Q', 'SignalR canlı fiyatlar neden donuyor?')}</h5>
            <p className="text-gray-400 leading-relaxed">{t('helpCenter.faq1A', 'İnternet bağlantınızı kontrol edin. Kopma durumunda sistem
otomatik olarak 5 saniye içinde yeniden bağlanır.')}</p>
          </div>

          <div className={`p-3 rounded-lg border ${isDark ? 'bg-{#1e222d}/60 border-gray-800' : 'bg-gray-50 border-gray-200'}`}>
            <h5 className="font-bold text-amber-400 mb-1">? {t('helpCenter.faq2Q', 'Fiyat alarmları nasıl çalışır?')}</h5>
            <p className="text-gray-400 leading-relaxed">{t('helpCenter.faq2A', 'Kurduğunuz hedef fiyat piyasada gerçekleştiğinde sesli ve görsel
bildirim alırsınız.')}</p>
          </div>
        </div>
      ) : helpSent ? (
        <div className="flex flex-col items-center justify-center py-10 space-y-3 text-center">
          <CheckCircle2 className="w-10 h-10 text-emerald-500 animate-in zoom-in" />
          <p className="text-xs font-semibold text-emerald-400">{t('helpCenter.successMessage', 'Mesajınız iletildi.')}</p>
        </div>
      ) : (
        <form onSubmit={handleContactSubmit} className="space-y-3 text-xs">
          <div className="grid grid-cols-2 gap-3">
            <div>
              <label className="block text-gray-400 mb-1">{t('helpCenter.contactName', 'Adınız')}</label>
              <input
                type="text"
                required
                value={contactFormular.name}
                onChange={(e) => setContactFormular({ ...contactFormular, name: e.target.value })}
                className={`w-full p-2 rounded-lg border outline-none ${isDark ? 'bg-{#1e222d} border-gray-700 text-white' : 'bg-gray-50 border-
gray-300 text-gray-900'
              }`}
            />
            </div>
            <div>
              <label className="block text-gray-400 mb-1">{t('helpCenter.contactEmail', 'E-Posta')}</label>
              <input
                type="email"
                required
                value={contactFormular.email}
                onChange={(e) => setContactFormular({ ...contactFormular, email: e.target.value })}
                className={`w-full p-2 rounded-lg border outline-none ${isDark ? 'bg-{#1e222d} border-gray-700 text-white' : 'bg-gray-50 border-
gray-300 text-gray-900'
              }`}
            />
            </div>
          </div>
          <div>
            <label className="block text-gray-400 mb-1">{t('helpCenter.subject', 'Konu')}</label>
            <input
              type="text"
              required
              value={contactFormular.subject}
            />
          </div>
        </form>
      )
    </div>

```

```

        onChange={(e) => setContactFormular({ ...contactFormular, subject: e.target.value })}
        className={`w-full p-2 rounded-lg border outline-none ${isDark ? 'bg-[#1e222d] border-gray-700 text-white' : 'bg-gray-50 border-gray-
300 text-gray-900'}
      }}
    />
  </div>

  <div>
    <label className="block text-gray-400 mb-1">{t('helpCenter.message', 'Mesajınız')}</label>
    <textarea
      required
      rows={3}
      value={contactFormular.message}
      onChange={(e) => setContactFormular({ ...contactFormular, message: e.target.value })}
      className={`w-full p-2 rounded-lg border outline-none resize-none ${isDark ? 'bg-[#1e222d] border-gray-700 text-white' : 'bg-gray-50
border-gray-300 text-gray-900'}
    }}
    />
  </div>

  <button
    type="submit"
    className="w-full py-2 bg-purple-600 hover:bg-purple-500 text-white font-bold rounded-lg flex items-center justify-center space-x-1.5
transition-colors"
  >
    <Send className="w-3.5 h-3.5" />
    <span>{t('helpCenter.send', 'Mesajı Gönder')}</span>
  </button>
</form>
  </div>
</div>
</div>
  </>
);
};

```

F10. src/Components/HelpCenterModal.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```

import React, { useState } from 'react';
import { X, HelpCircle, Send, CheckCircle2 } from 'lucide-react';
import { useTranslation } from 'react-i18next';

interface HelpCenterModalProps {
  isOpen: boolean;
  onClose: () => void;
  isDark: boolean;
}

export const HelpCenterModal: React.FC<HelpCenterModalProps> = ({
  isOpen,
  onClose,
  isDark,
}) => {
  const { t } = useTranslation();
  const [activeTab, setActiveTab] = useState<'faq' | 'contact'>('faq');
  const [sent, setSent] = useState(false);
  const [formData, setFormularData] = useState({
    name: "",
    email: "",
    subject: "",
    message: "",
  });

  if (!isOpen) return null;

  // Behandler den relevante brugerhandling eller event.
  const handleSubmit = (e: React.FormularEvent) => {
    e.preventDefault();
    setSent(true);
    setTimeout(() => {
      setSent(false);
      setFormularData({ name: "", email: "", subject: "", message: "" });
      onClose();
    }, 2000);
  };

  return (
    <div
      role="dialog"

```

text-gray-900'

```

    </div>
    <div>
      <label className="block text-gray-400 mb-1">{t('helpCenter.contactEmail')}</label>
      <input
        type="email"
        required
        value={formData.email}
        onChange={(e) => setFormularData({ ...formData, email: e.target.value })}
        className={`w-full p-2 rounded-lg border outline-none ${isDark ? 'bg-[#1e222d] border-gray-700 text-white' : 'bg-gray-50 border-gray-300`}
      />
    </div>
  </div>

  <div>
    <label className="block text-gray-400 mb-1">{t('helpCenter.subject')}</label>
    <input
      type="text"
      required
      value={formData.subject}
      onChange={(e) => setFormularData({ ...formData, subject: e.target.value })}
      className={`w-full p-2 rounded-lg border outline-none ${isDark ? 'bg-[#1e222d] border-gray-700 text-white' : 'bg-gray-50 border-gray-300`}
    />
  </div>

  <div>
    <label className="block text-gray-400 mb-1">{t('helpCenter.message')}</label>
    <textarea
      required
      rows={3}
      value={formData.message}
      onChange={(e) => setFormularData({ ...formData, message: e.target.value })}
      className={`w-full p-2 rounded-lg border outline-none resize-none ${isDark ? 'bg-[#1e222d] border-gray-700 text-white' : 'bg-gray-50 border-gray-300`}
    />
  </div>

  <button
    type="submit"
    className="w-full py-2 bg-blue-600 hover:bg-blue-500 text-white font-bold rounded-lg flex items-center justify-center space-x-1.5 transition-colors"
  >
    <Send className="w-3.5 h-3.5" />
    <span>{t('helpCenter.send')}</span>
  </button>
</form>
  </div>
</div>
);
};

```

F11. src/Components/Indicators/IndicatorsModal.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```

import React, { useEffect } from 'react';
import { X, Check } from 'lucide-react';
import type { IndicatorConfig } from '../Types/indicator';

interface IndicatorsModalProps {
  isOpen: boolean;
  onClose: () => void;
  indicators: IndicatorConfig[];
  onToggleIndicator: (id: string) => void;
  isDark: boolean;
}

export const IndicatorsModal: React.FC<IndicatorsModalProps> = ({
  isOpen,
  onClose,
  indicators,
  onToggleIndicator,
  isDark,
}) => {
  // Listener til lukning med ESC-tasten.
  useEffect(() => {
    // Behandler den relevante brugerhandling eller event.
    const handleKeyDown = (e: KeyboardEvent) => {

```

```

    if (e.key === 'Escape' && isOpen) {
      onClose();
    }
  };

  window.addEventListener('keydown', handleKeyDown);
  return () => window.removeEventListener('keydown', handleKeyDown);
}, [isOpen, onClose]);

if (!isOpen) return null;

return (
  <div
    role="dialog"
    aria-modal="true"
    aria-labelledby="modal-title"
    onClick={onClose}
    className="fixed inset-0 z-50 flex items-center justify-center bg-black/60 backdrop-blur-xs p-4 animate-in fade-in duration-150"
  >
    <div
      onClick={(e) => e.stopPropagation()}
      className={
        `w-full max-w-md rounded-xl shadow-2xl border flex flex-col overflow-hidden transition-colors ${isDark
          ? 'bg-[#1e222d] border-[#2a2e39] text-gray-200'
          : 'bg-white border-gray-200 text-gray-800'
        }`
      }
    >
      { /* Modal-header */ }
      <div
        className={
          `flex items-center justify-between px-4 py-3 border-b ${isDark ? 'border-[#2a2e39]' : 'border-gray-100'
          }`
        }
      >
        <h3 id="modal-title" className="font-bold text-sm">
          Teknik Indikatorer
        </h3>
        <button
          type="button"
          onClick={onClose}
          aria-label="Kapat"
          className="p-1 rounded-md hover:bg-gray-500/10 text-gray-400 hover:text-gray-200 transition-colors"
        >
          <X className="w-4 h-4" />
        </button>
      </div>

      { /* Indikatorliste */ }
      <div className="p-3 divide-y divide-gray-500/10 max-h-80 overflow-y-auto space-y-0.5">
        {indicators.length === 0 ? (
          <div className="py-8 text-center text-xs text-gray-400">
            Kullanılabilir indikatör bulunamadı.
          </div>
        ) : (
          indicators.map((ind) => (
            <div
              key={ind.id}
              role="checkbox"
              aria-checked={ind.enabled}
              tabIndex={0}
              onClick={() => onToggleIndicator(ind.id)}
              onKeyDown={(e) => {
                if (e.key === 'Enter' || e.key === '') {
                  e.preventDefault();
                  onToggleIndicator(ind.id);
                }
              }}
              className={
                `flex items-center justify-between p-2.5 rounded-lg cursor-pointer select-none transition-colors outline-none focus-visible:ring-2
                focus-visible:ring-[#2962ff] ${ind.enabled
                  ? isDark
                    ? 'bg-blue-600/15'
                    : 'bg-blue-50'
                  : 'hover:bg-gray-500/10'
                }`
              }
            >
              <div className="flex items-center space-x-3">
                <span
                  className="w-3 h-3 rounded-full shrink-0 shadow-xs"
                  style={{ backgroundColor: ind.color }}
                />
                <div>
                  <div className="text-xs font-semibold">{ind.name}</div>
                  <div className="text-[10px] text-gray-400">
                    {ind.type} {ind.period ? ` (Periyot: ${ind.period})` : ''}
                  </div>
                </div>
              </div>
            </div>
          ))
        )}
      </div>

```

```

        <div
          className={`w-5 h-5 rounded flex items-center justify-center border transition-all ${ind.enabled
            ? 'bg-[#2962ff] border-[#2962ff] text-white'
            : isDark
              ? 'border-gray-700 bg-gray-800'
              : 'border-gray-300 bg-white'
            }`}
        >
          {ind.enabled && <Check className="w-3.5 h-3.5 stroke-[3]" />}
        </div>
      </div>
    ))
  })
</div>

{/* Modal-footer */}
<div
  className={`px-4 py-2.5 border-t flex justify-end ${isDark
    ? 'border-[#2a2e39] bg-[#131722]'
    : 'border-gray-100 bg-gray-50'
  }`}
>
  <button
    type="button"
    onClick={onClose}
    className="px-4 py-1.5 rounded-lg bg-[#2962ff] hover:bg-blue-600 active:bg-blue-700 text-white text-xs font-semibold transition-colors"
  >
    Tamam
  </button>
</div>
</div>
</div>
);
};

```

F12. src/Components/LanguageSwitcher.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```

import React from 'react';
import { useTranslation } from 'react-i18next';
import { preferenceService } from '../Services/preferenceService';

export const LanguageSwitcher: React.FC = () => {
  const { i18n } = useTranslation();

  // Behandler den relevante brugerhandling eller event.
  const handleLanguageChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
    const lang = e.target.value;
    i18n.changeLanguage(lang);
    localStorage.setItem('tradingpro_lang', lang);

    // Gem indstillingen i databasen.
    preferenceService.savePreferences({ Language: lang } as any).catch(() => {});
  };

  return (
    <div className="relative inline-block">
      <select
        value={i18n.language.startsWith('da') ? 'da' : i18n.language.startsWith('en') ? 'en' : 'tr'}
        onChange={handleLanguageChange}
        className="bg-transparent border border-gray-500/30 hover:border-gray-500/60 rounded-md px-2 py-1 text-xs font-semibold text-gray-300 outline-none cursor-pointer"
      >
        <option value="tr" className="bg-[#1e222d] text-white">TR TR</option>
        <option value="en" className="bg-[#1e222d] text-white">GB EN</option>
        <option value="da" className="bg-[#1e222d] text-white">DK DA</option>
      </select>
    </div>
  );
};

```

F13. src/Components/ProfileModal.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```

import React, { useState, useEffect } from 'react';
import {
  User,
  X,
  Lock,

```

```
CheckCircle,
AlertCircle,
KeyRound,
Save,
Loader2,
Trash2,
AlertTriangle
} from 'lucide-react';
import { useTranslation } from 'react-i18next';
import { userService, type UserManagementDto } from '../Services/userService';

export interface UserProfileDto {
  id?: string | number;
  email?: string;
  username?: string;
  firstName?: string;
  lastName?: string;
  role?: string;
}

interface ProfileModalProps {
  isOpen: boolean;
  onClose: () => void;
  isDark: boolean;
  currentUser: UserProfileDto | null;
  onLogout: () => void;
  onProfileUpdated?: (updated: UserManagementDto) => void;
}

export const ProfileModal: React.FC<ProfileModalProps> = ({
  isOpen,
  onClose,
  isDark,
  currentUser,
  onLogout,
  onProfileUpdated,
}) => {
  const { t } = useTranslation();
  const [profileTab, setProfileTab] = useState<'info' | 'password'>('info');

  // Tilstand for profilformular og data
  const [profileData, setProfileData] = useState<UserManagementDto | null>(<null>);
  const [fetchingProfile, setFetchingProfile] = useState(false);
  const [profileFormular, setProfileFormular] = useState({
    firstName: currentUser?.firstName || "",
    lastName: currentUser?.lastName || "",
    email: currentUser?.email || "",
  });

  const [profileLoading, setProfileLoading] = useState(false);
  const [profileSuccess, setProfileSuccess] = useState<string | null>(<null>);
  const [profileError, setProfileError] = useState<string | null>(<null>);

  // Tilstand for adgangskodeformular
  const [currentPassword, setCurrentPassword] = useState("");
  const [newPassword, setNewPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [pwdLoading, setPwdLoading] = useState(false);
  const [pwdSuccess, setPwdSuccess] = useState<string | null>(<null>);
  const [pwdError, setPwdError] = useState<string | null>(<null>);

  // Tilstand for kontosletning
  const [isDeleteConfirmOpen, setIsDeleteConfirmOpen] = useState(false);
  const [deleteLoading, setDeleteLoading] = useState(false);

  useEffect(() => {
    if (isOpen) {
      setFetchingProfile(true);
      setProfileError(null);
      setProfileSuccess(null);
      setPwdError(null);
      setPwdSuccess(null);
      setIsDeleteConfirmOpen(false);

      userService
        .getMyProfile()
        .then((data) => {
          setProfileData(data);
          setProfileFormular({
            firstName: data.firstName || "",
            lastName: data.lastName || "",
            email: data.email || "",
          });
        });
    }
  });
}
```

```
.catch(() => {
  if (currentUser) {
    setProfileFormular({
      firstName: currentUser.firstName || "",
      lastName: currentUser.lastName || "",
      email: currentUser.email || "",
    });
  }
})
    .finally(() => setFetchingProfile(false));
  }
}, [isOpen, currentUser]);

if (!isOpen) return null;

// Opdaterer de aktuelle data i brugergrænsefladen.
const handleProfileUpdateSubmit = async (e: React.FormularEvent) => {
  e.preventDefault();
  setProfileError(null);
  setProfileSuccess(null);
  setProfileLoading(true);

  try {
    const updated = await userService.updateMyProfile(profileFormular);
    setProfileData(updated);
    setProfileSuccess(t('profile.profileUpdated', 'Profil bilgileriniz başarıyla güncellendi.'));
    if (onProfileUpdated) {
      onProfileUpdated(updated);
    }
  } catch (err: any) {
    setProfileError(err.response?.data?.message || 'Profil güncellenirken bir hata oluştu.');
```

```
  } finally {
    setProfileLoading(false);
  }
};

// Behandler den relevante brugerhandling eller event.
const handlePasswordChangeSubmit = async (e: React.FormularEvent) => {
  e.preventDefault();
  setPwdError(null);
  setPwdSuccess(null);

  if (newPassword !== confirmPassword) {
    setPwdError(t('profile.passwordMismatch', 'Yeni şifreler eşleşmiyor!'));
    return;
  }

  if (newPassword.length < 6) {
    setPwdError('Yeni şifre en az 6 karakter olmalıdır.');
```

```
    return;
  }

  setPwdLoading(true);
  try {
    await userService.changePassword({ currentPassword, newPassword });
    setPwdSuccess(t('profile.passwordChanged', 'Şifreniz başarıyla değiştirildi.');
```

```
    setCurrentPassword("");
    setNewPassword("");
    setConfirmPassword("");
  } catch (err: any) {
    setPwdError(err.response?.data?.message || 'Mevcut şifreniz hatalı veya geçersiz.');
```

```
  } finally {
    setPwdLoading(false);
  }
};

// Fjerner det valgte element.
const handleDeleteAccount = async () => {
  setDeleteLoading(true);
  try {
    await userService.deleteMyAccount();
    setIsDeleteConfirmOpen(false);
    onClose();
    onLogout();
  } catch (err: any) {
    alert(err.response?.data?.message || 'Hesap silinirken bir hata oluştu.');
```

```
  } finally {
    setDeleteLoading(false);
  }
};

const displayUsername = profileData?.username || currentUser?.username || 'trader';
```



```

return (
  <div
    role="dialog"
    aria-modal="true"
    onClick={onClose}
    className="fixed inset-0 z-50 flex items-center justify-center bg-black/60 backdrop-blur-xs p-4 animate-in fade-in duration-150 select-none"
  >
    <div
      onClick={(e) => e.stopPropagation()}
      className={ `w-full max-w-md rounded-xl shadow-2xl border flex flex-col overflow-hidden transition-colors ${isDark ? 'bg-[#131722] border-[#2a2e39] text-gray-200' : 'bg-white border-gray-200 text-gray-800'}` }
    >
      <div className={ `flex items-center justify-between px-5 py-3.5 border-b ${isDark ? 'border-[#2a2e39] bg-[#1e222d]' : 'border-gray-100 bg-gray-50'}` }>
        <div className="flex items-center space-x-2 font-bold text-sm">
          <User className="w-4 h-4 text-blue-500" />
          <span>{t('profile.title', 'Kullanıcı Profili & Güvenlik')}</span>
        </div>
        <button
          type="button"
          onClick={onClose}
          className="p-1 rounded-md hover:bg-gray-500/10 text-gray-400 hover:text-gray-200"
        >
          <X className="w-4 h-4" />
        </button>
      </div>

      { /* Faner */ }
      <div className={ `flex border-b text-xs font-semibold ${isDark ? 'border-[#2a2e39]' : 'border-gray-100'}` }>
        <button
          type="button"
          onClick={() => { setProfileTab('info'); setProfileError(null); setProfileSuccess(null); }}
          className={ `flex-1 py-2.5 text-center transition-colors border-b-2 flex items-center justify-center space-x-1.5 ${profileTab === 'info' ? 'border-blue-500 text-blue-500 bg-blue-500/5' : 'border-transparent text-gray-400 hover:text-gray-200'}` }
        >
          <User className="w-3.5 h-3.5" />
          <span>{t('profile.personalInfo', 'Kişisel Bilgiler')}</span>
        </button>
        <button
          type="button"
          onClick={() => { setProfileTab('password'); setPwdError(null); setPwdSuccess(null); }}
          className={ `flex-1 py-2.5 text-center transition-colors border-b-2 flex items-center justify-center space-x-1.5 ${profileTab === 'password' ? 'border-blue-500 text-blue-500 bg-blue-500/5' : 'border-transparent text-gray-400 hover:text-gray-200'}` }
        >
          <KeyRound className="w-3.5 h-3.5" />
          <span>{t('profile.security', 'Şifre & Güvenlik')}</span>
        </button>
      </div>

      { /* Indhold */ }
      <div className="p-5 max-h-[460px] overflow-y-auto">
        {fetchingProfile ? (
          <div className="py-12 flex flex-col items-center justify-center space-y-2 text-gray-400">
            <Loader2 className="w-6 h-6 animate-spin text-blue-500" />
            <span className="text-xs">{t('profile.loading')}</span>
          </div>
        ) : profileTab === 'info' ? (
          <div className="space-y-5">
            <form onSubmit={handleProfileUpdateSubmit} className="space-y-4 text-xs">
              { /* Titeltkort */ }
              <div className="flex items-center space-x-3.5 pb-3 border-b border-gray-500/10">
                <div className="w-12 h-12 rounded-full bg-blue-600/20 text-blue-400 font-extrabold text-lg flex items-center justify-center border border-blue-500/30">
                  {((profileFormular.firstName || currentUser?.firstName || displayUsername)[0]??.toUpperCase())}
                </div>
                <div>
                  <h3 className="font-bold text-sm">
                    {profileFormular.firstName || currentUser?.firstName || 'User'} {profileFormular.lastName || currentUser?.lastName || ''}
                  </h3>
                  <p className="text-gray-400 font-mono text-[11px]">@{displayUsername}</p>
                  <div className="inline-flex items-center space-x-1 mt-1 px-2 py-0.5 rounded-full bg-emerald-500/15 text-emerald-400 text-[10px] font-semibold">
                    <CheckCircle className="w-2.5 h-2.5" />
                    <span>{profileData?.role || currentUser?.role || 'TRADER'}</span>
                  </div>
                </div>
              </div>
            </form>
            {profileError && (

```

```

<div className="p-2.5 bg-red-500/10 border border-red-500/30 text-red-400 rounded-lg flex items-center space-x-2">
  <AlertCircle className="w-4 h-4 shrink-0" />
  <span>{profileError}</span>
</div>
)}

{profileSuccess && (
  <div className="p-2.5 bg-emerald-500/10 border border-emerald-500/30 text-emerald-400 rounded-lg flex items-center space-x-2">
    <CheckCircle className="w-4 h-4 shrink-0" />
    <span>{profileSuccess}</span>
  </div>
)}

{/* Formularfelter */}
<div className="space-y-3">
  <div>
    <div className="flex items-center justify-between mb-1">
      <label className="text-gray-400 font-medium">{t('auth.username', 'Kullanıcı Adı')}</label>
      <span className="text-[10px] text-gray-500 flex items-center space-x-1">
        <Lock className="w-2.5 h-2.5" />
        <span>{t('profile.immutable')}</span>
      </span>
    </div>
    <input
      type="text"
      disabled
      value={`@${displayUsername}`}
      className={`w-full p-2.5 rounded-lg border outline-none cursor-not-allowed font-medium ${isDark
        ? 'bg-[#151921] border-gray-800 text-gray-400'
        : 'bg-gray-100 border-gray-300 text-gray-500'
      }`}
    />
  </div>

  <div className="grid grid-cols-2 gap-3">
    <div>
      <label className="block text-gray-400 font-medium mb-1">{t('profile.firstName', 'Ad')}</label>
      <input
        type="text"
        required
        value={profileFormular.firstName}
        onChange={(e) => setProfileFormular({ ...profileFormular, firstName: e.target.value })}
        className={`w-full p-2.5 rounded-lg border outline-none focus:border-blue-500 ${isDark ? 'bg-[#1e222d] border-gray-700 text-white' :
          'bg-gray-50 border-gray-300 text-gray-900'
        }`}
      />
    </div>
    <div>
      <label className="block text-gray-400 font-medium mb-1">{t('profile.lastName', 'Soyad')}</label>
      <input
        type="text"
        required
        value={profileFormular.lastName}
        onChange={(e) => setProfileFormular({ ...profileFormular, lastName: e.target.value })}
        className={`w-full p-2.5 rounded-lg border outline-none focus:border-blue-500 ${isDark ? 'bg-[#1e222d] border-gray-700 text-white' :
          'bg-gray-50 border-gray-300 text-gray-900'
        }`}
      />
    </div>
  </div>

  <div>
    <label className="block text-gray-400 font-medium mb-1">{t('profile.email', 'E-Posta')}</label>
    <input
      type="email"
      required
      value={profileFormular.email}
      onChange={(e) => setProfileFormular({ ...profileFormular, email: e.target.value })}
      className={`w-full p-2.5 rounded-lg border outline-none focus:border-blue-500 ${isDark ? 'bg-[#1e222d] border-gray-700 text-white' :
        'bg-gray-50 border-gray-300 text-gray-900'
      }`}
    />
  </div>

  <button
    type="submit"
    disabled={profileLoading}
    className="w-full py-2.5 bg-blue-600 hover:bg-blue-500 active:bg-blue-700 text-white font-bold rounded-lg transition-colors
disabled:opacity-50 flex items-center justify-center space-x-2 shadow-sm"
  >
    <Save className="w-3.5 h-3.5" />
    <span>{profileLoading ? t('profile.updating', 'Güncelleniyor...') : t('profile.save', 'Bilgileri Güncelle')}</span>
  </button>

```

```

</form>

{/* Kritik område / slet konto */}
<div className={`pt-4 border-t ${isDark ? 'border-red-500/20' : 'border-red-200'}}>
  <div className="flex items-center space-x-1.5 text-red-400 font-bold text-xs mb-1.5">
    <AlertTriangle className="w-3.5 h-3.5" />
    <span>{t('profile.dangerZone', 'Tehlikeli Bölge')}</span>
  </div>
  <p className="text-[11px] text-gray-400 mb-3 leading-relaxed">
    {t('profile.deleteAccountDesc', 'Hesabınızı sildiğinizde oturumunuz hemen kapatılır. 30 gün içinde tekrar giriş yaparsanız hesabınız otomatik
    olarak kurtarılır.')}
  </p>

  {isDeleteConfirmOpen ? (
    <button
      type="button"
      onClick={() => setIsDeleteConfirmOpen(true)}
      className="w-full py-2 bg-red-500/10 hover:bg-red-500/20 text-red-400 border border-red-500/30 font-semibold rounded-lg transition-
      colors flex items-center justify-center space-x-1.5 text-xs"
    >
      <Trash2 className="w-3.5 h-3.5" />
      <span>{t('profile.deleteAccount', 'Hesabımı Sil')}</span>
    </button>
  ) : (
    <div className={`p-3 rounded-lg border space-y-2.5 ${isDark ? 'bg-red-500/10 border-red-500/40 text-red-300' : 'bg-red-50 border-red-200
    text-red-700'}}>
      <p className="text-xs font-semibold">{t('profile.deleteConfirmTitle', 'Hesabınızı silmek istediğinize emin misiniz?')}</p>
      <p className="text-[11px] text-gray-400">{t('profile.deleteConfirmNotice', 'Hesabınız askıya alınacaktır. 30 gün boyunca giriş
      yapmazsanız verileriniz kalıcı olarak silinir.')}</p>
      <div className="flex items-center space-x-2 pt-1">
        <button
          type="button"
          disabled={deleteLoading}
          onClick={handleDeleteAccount}
          className="flex-1 py-1.5 bg-red-600 hover:bg-red-700 text-white font-bold rounded text-xs transition-colors disabled:opacity-50"
        >
          {deleteLoading ? t('profile.deleting', 'İşleniyor...') : t('profile.confirmDeleteBtn', 'Evet, Hesabımı Kapat')}
        </button>
        <button
          type="button"
          disabled={deleteLoading}
          onClick={() => setIsDeleteConfirmOpen(false)}
          className="flex-1 py-1.5 bg-gray-500/20 hover:bg-gray-500/30 text-gray-300 font-medium rounded text-xs transition-colors"
        >
          {t('profile.cancel', 'Vazgeç')}
        </button>
      </div>
    </div>
  )}
</div>
</div>
): (
  <form onSubmit={handlePasswordChangeSubmit} className="space-y-3.5 text-xs">
    {pwdError && (
      <div className="p-2.5 bg-red-500/10 border border-red-500/30 text-red-400 rounded-lg flex items-center space-x-2">
        <AlertCircle className="w-4 h-4 shrink-0" />
        <span>{pwdError}</span>
      </div>
    )}

    {pwdSuccess && (
      <div className="p-2.5 bg-emerald-500/10 border border-emerald-500/30 text-emerald-400 rounded-lg flex items-center space-x-2">
        <CheckCircle className="w-4 h-4 shrink-0" />
        <span>{pwdSuccess}</span>
      </div>
    )}

    <div>
      <label className="block text-gray-400 font-medium mb-1">
        {t('profile.currentPassword', 'Mevcut Şifre')}
      </label>
      <div className="relative">
        <Lock className="w-3.5 h-3.5 absolute left-3 top-3 text-gray-500" />
        <input
          type="password"
          required
          value={currentPassword}
          onChange={(e) => setCurrentPassword(e.target.value)}
          placeholder="*****"
          className={`w-full pl-8 pr-3 py-2 rounded-lg border outline-none focus:border-blue-500 ${isDark ? 'bg-[#1e222d] border-gray-700 text-
          white' : 'bg-gray-50 border-gray-300 text-gray-900'}
        >
      </div>
    </div>
  </div>

```

```

<div>
  <label className="block text-gray-400 font-medium mb-1">
    {t('profile.newPassword', 'Yeni Şifre')}
  </label>
  <div className="relative">
    <Lock className="w-3.5 h-3.5 absolute left-3 top-3 text-gray-500" />
    <input
      type="password"
      required
      value={newPassword}
      onChange={(e) => setNewPassword(e.target.value)}
      placeholder="*****"
      className={`w-full pl-8 pr-3 py-2 rounded-lg border outline-none focus:border-blue-500 ${isDark ? 'bg-[#1e222d] border-gray-700 text-white' : 'bg-gray-50 border-gray-300 text-gray-900'}`}
    />
  </div>
</div>

<div>
  <label className="block text-gray-400 font-medium mb-1">
    {t('profile.confirmPassword', 'Yeni Şifre (Tekrar)')}
  </label>
  <div className="relative">
    <Lock className="w-3.5 h-3.5 absolute left-3 top-3 text-gray-500" />
    <input
      type="password"
      required
      value={confirmPassword}
      onChange={(e) => setConfirmPassword(e.target.value)}
      placeholder="*****"
      className={`w-full pl-8 pr-3 py-2 rounded-lg border outline-none focus:border-blue-500 ${isDark ? 'bg-[#1e222d] border-gray-700 text-white' : 'bg-gray-50 border-gray-300 text-gray-900'}`}
    />
  </div>
</div>

<button
  type="submit"
  disabled={pwdLoading}
  className="w-full py-2.5 bg-blue-600 hover:bg-blue-500 active:bg-blue-700 text-white font-bold rounded-lg transition-colors disabled:opacity-50 mt-2 shadow-sm"
  >
  {pwdLoading ? t('profile.updating', 'Güncelleniyor...') : t('profile.changePassword', 'Şifreyi Güncelle')}
</button>
</form>
</div>
</div>
</div>
);
};

```

F14. src/Components/TriggeredAlertToast.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```

import React from 'react';
import { useTranslation } from 'react-i18next';
import type { Alert } from '../Types/alert';

interface Props {
  alert: Alert | null;
  onClose: () => void;
}

export const TriggeredAlertToast: React.FC<Props> = ({ alert, onClose }) => {
  const { t } = useTranslation();

  if (!alert) return null;

  return (
    <div
      role="alert"
      className="fixed bottom-6 right-6 z-50 flex items-center gap-4 rounded-xl bg-red-950/90 border border-red-500 p-4 shadow-2xl backdrop-blur-md text-white animate-bounce"
    >
      <div className="flex h-10 w-10 shrink-0 items-center justify-center rounded-full bg-red-600 font-bold text-xl">
        🚨
      </div>
      <div>

```

```

    <h4 className="font-bold text-sm tracking-wide text-red-300">
      {t('alerts.triggeredTitle', 'FİYAT ALARMI ÇALDI!')}
    </h4>
    <p className="text-xs text-gray-200">
      <strong className="text-white">{alert.symbol}</strong>{ ' ' }
      {t('alerts.targetReached', 'hedef fiyatı ulaştı')}:{ ' ' }
      <span className="font-semibold text-yellow-400 font-mono">
        ${Number(alert.targetPrice).toLocaleString()}
      </span>
    </p>
    {alert.note && (
      <p className="text-[11px] text-gray-400 italic mt-0.5">{alert.note}</p>
    )}
  </div>
  <button
    type="button"
    onClick={onClose}
    aria-label={t('chartSettings.cancel', 'Kapat')}
    className="text-gray-400 hover:text-white pl-2 text-lg transition-colors"
  >
    ✕
  </button>
</div>
);
};

```

F15. src/Components/Watchlist.tsx

React-komponent for en afgrænset del af brugergrænsefladen.

```

import React, { useEffect, useMemo, useState } from 'react';
import { ListPlus, Plus, Trash2, X } from 'lucide-react';
import { useTranslation } from 'react-i18next';
import { useTheme } from '../Context/ThemeContext';
import { candleApi } from '../Services/candleApi';
import { signalrService } from '../Services/signalrService';
import type { CandleData } from '../Types/candle';
import type { WatchlistResponseDto } from '../Types/watchlist';
import type { SymbolResponseDto } from '../Types/symbol';

interface WatchlistProps {
  watchlists: WatchlistResponseDto[];
  selectedSymbolId?: number | null;
  onSelectSymbol: (symbol: SymbolResponseDto) => void;
  onCreateWatchlist: (name: string) => Promise<void> | void;
  onDeleteWatchlist: (watchlistId: number) => Promise<void> | void;
  onAddSymbol: (watchlistId: number, symbol: SymbolResponseDto) => Promise<void> | void;
  onRemoveSymbol: (watchlistId: number, symbolId: number) => Promise<void> | void;
  activeSymbols: SymbolResponseDto[];
}

interface LiveQuote {
  price: number;
  changePercent: number;
  updatedAt: number;
}

// Håndterer format price.
const formatPrice = (value?: number) => {
  if (value == null || !Number.isFinite(value)) return '—';
  if (value >= 1000) return value.toLocaleString(undefined, { minimumFractionDigits: 2, maximumFractionDigits: 2 });
  if (value >= 1) return value.toLocaleString(undefined, { minimumFractionDigits: 2, maximumFractionDigits: 6 });
  return value.toLocaleString(undefined, { minimumFractionDigits: 4, maximumFractionDigits: 8 });
};

export const Watchlist: React.FC<WatchlistProps> = ({
  watchlists = [],
  selectedSymbolId,
  onSelectSymbol,
  onCreateWatchlist,
  onDeleteWatchlist,
  onAddSymbol,
  onRemoveSymbol,
  activeSymbols = [],
}) => {
  const { t } = useTranslation();
  const { isDark } = useTheme();
  const [newListName, setNewListName] = useState('');
  const [activeListId, setActiveListId] = useState<number | null>(null);
  const [isCreatingList, setIsCreatingList] = useState(false);
  const [isAddingSymbol, setIsAddingSymbol] = useState(false);
  const [selectedSymbolToAdd, setSelectedSymbolToAdd] = useState<number | ">(");
  const [liveQuotes, setLiveQuotes] = useState<Record<number, LiveQuote>>({});
  const [busy, setBusy] = useState(false);

```

```
useEffect(() => {
  if (watchlists.length === 0) {
    setActiveListId(null);
    return;
  }

  if (activeListId == null || !watchlists.some((w) => w.id === activeListId)) {
    setActiveListId(watchlists[0].id);
  }
}, [watchlists, activeListId]);

const currentList = watchlists.find((w) => w.id === activeListId) || watchlists[0];
const symbolMap = useMemo(() => new Map(activeSymbols.map((s) => [s.id, s])), [activeSymbols]);

const availableSymbols = useMemo(() => {
  const currentIds = new Set((currentList?.items || []).map((item) => item.symbolId));
  return activeSymbols.filter((symbol) => !currentIds.has(symbol.id));
}, [activeSymbols, currentList]);

// Symbolerne i den aktuelle watchlist abonneres uafhængigt af det synlige chart.
// SignalR-gruppeabonnementet bruger reference counting, så det samme symbol kan deles
// med et chart-panel uden ved en fejl at afmelde den anden forbruger.
useEffect(() => {
  const items = currentList?.items || [];
  if (items.length === 0) return;

  let disposed = false;
  const subscriptions = items
    .map((item) => symbolMap.get(item.symbolId))
    .filter((symbol): symbol is SymbolResponseDto => Boolean(symbol));

  // Håndterer seed prices.
  const seedPrices = async () => {
    const snapshots = await Promise.all(
      subscriptions.map(async (symbol) => {
        try {
          const candles = await candleApi.getHistoricalCandles(symbol.id, '1m', 2);
          const last = candles[candles.length - 1];
          if (!last) return null;
          const reference = Number(last.open) || Number(last.close);
          const price = Number(last.close);
          const changePercent = reference ? ((price - reference) / reference) * 100 : 0;
          return { symbolId: symbol.id, price, changePercent };
        } catch {
          return null;
        }
      })
    );
  };

  if (disposed) return;
  setLiveQuotes((prev) => {
    const next = { ...prev };
    snapshots.forEach((snapshot) => {
      if (!snapshot) return;
      next[snapshot.symbolId] = {
        price: snapshot.price,
        changePercent: snapshot.changePercent,
        updatedAt: Date.now(),
      };
    });
    return next;
  });
};

// Behandler den relevante brugerhandling eller event.
const handleCandle = (candle: CandleData) => {
  const symbolId = Number(candle.symbolId || 0);
  if (!symbolId || !items.some((item) => item.symbolId === symbolId)) return;

  const price = Number(candle.close);
  const open = Number(candle.open);
  if (!Number.isFinite(price)) return;

  setLiveQuotes((prev) => ({
    ...prev,
    [symbolId]: {
      price,
      changePercent: open ? ((price - open) / open) * 100 : (prev[symbolId]?.changePercent ?? 0),
      updatedAt: Date.now(),
    },
  })));
});
```

```

};

// Håndterer forbindelsen til realtime-data via SignalR.
const connect = async () => {
  await signalRService.startConnection();
  if (disposed) return;
  await Promise.all(
    subscriptions.map((symbol) => signalRService.subscribeToSymbolGroup(symbol.exchangeCode, symbol.name))
  );
  if (!disposed) signalRService.subscribeToCandleUpdates(handleCandle);
};

void seedPrices();
void connect();

return () => {
  disposed = true;
  signalRService.unsubscribeFromCandleUpdates(handleCandle);
  subscriptions.forEach((symbol) => {
    void signalRService.unsubscribeFromSymbolGroup(symbol.exchangeCode, symbol.name);
  });
};
}, [currentList?.id, currentList?.items, symbolMap]);

// Håndterer submit list.
const submitList = async (event: React.FormularEvent) => {
  event.preventDefault();
  const name = newListName.trim();
  if (!name || busy) return;
  setBusy(true);
  try {
    await onCreateWatchlist(name);
    setNewListName("");
    setIsCreatingList(false);
  } finally {
    setBusy(false);
  }
};

// Håndterer submit symbol.
const submitSymbol = async () => {
  if (!currentList || selectedSymbolToAdd === "" || busy) return;
  const symbol = symbolMap.get(Number(selectedSymbolToAdd));
  if (!symbol) return;
  setBusy(true);
  try {
    await onAddSymbol(currentList.id, symbol);
    setSelectedSymbolToAdd("");
    setIsAddingSymbol(false);
  } finally {
    setBusy(false);
  }
};

// Fjerner det valgte element.
const deleteCurrentList = async () => {
  if (!currentList || busy) return;
  const confirmed = window.confirm(
    t('watchlistPanel.deleteConfirm', { name: currentList.name })
  );
  if (!confirmed) return;
  setBusy(true);
  try {
    await onDeleteWatchlist(currentList.id);
  } finally {
    setBusy(false);
  }
};

const panel = isDark
  ? 'bg-#111827 border-gray-800 text-gray-200'
  : 'bg-white border-gray-200 text-gray-800';

return (
  <aside className={`w-80 border-l flex flex-col h-full shrink-0 ${panel}`}>
    <div className={`p-4 border-b ${isDark ? 'border-gray-800' : 'border-gray-200'}`>
      <div className="flex items-center justify-between gap-2">
        <div className="min-w-0">
          <div className="text-xs font-black uppercase tracking-[0.18em] text-gray-400">
            {t('watchlistPanel.title')}
          </div>
          <div className="text-[10px] mt-1 text-gray-500">{t('watchlistPanel.exchangeAwareHint')}</div>
        </div>
      </div>
    </div>
  </aside>

```

```

<div className="flex items-center gap-1">
  <button
    type="button"
    onClick={() => setIsCreatingList((v) => !v)}
    className="p-2 rounded-xl bg-indigo-500/10 text-indigo-400 hover:bg-indigo-500/20 border border-indigo-500/20"
    title={t('watchlistPanel.createButton')}
  >
    <ListPlus className="w-4 h-4" />
  </button>
  {currentList && (
    <button
      type="button"
      onClick={() => setIsAddingSymbol((v) => !v)}
      className="p-2 rounded-xl bg-blue-500/10 text-blue-400 hover:bg-blue-500/20 border border-blue-500/20"
      title={t('watchlistPanel.addSymbol')}
    >
      {isAddingSymbol ? <X className="w-4 h-4" /> : <Plus className="w-4 h-4" />}
    </button>
  )}
  {currentList && (
    <button
      type="button"
      disabled={busy}
      onClick={deleteCurrentList}
      className="p-2 rounded-xl bg-rose-500/10 text-rose-400 hover:bg-rose-500/20 border border-rose-500/20 disabled:opacity-50"
      title={t('watchlistPanel.deleteList')}
    >
      <Trash2 className="w-4 h-4" />
    </button>
  )}
</div>
</div>

{isCreatingList && (
  <form onSubmit={submitList} className="mt-4 flex gap-2">
    <input
      autoFocus
      value={newListName}
      onChange={(e) => setNewListName(e.target.value)}
      placeholder={t('watchlistPanel.placeholderQuick')}
      className={min-w-0 flex-1 rounded-xl border px-3 py-2 text-xs outline-none ${isDark ? 'bg-[#0b1220] border-gray-700' : 'bg-gray-50 border-gray-200'}}
    />
    <button disabled={busy || !newListName.trim()} type="submit" className="px-3 rounded-xl bg-indigo-600 text-white text-xs font-bold disabled:opacity-50">
      {t('watchlistPanel.create')}
    </button>
  </form>
)}

<div className="flex gap-2 mt-4 overflow-x-auto pb-1">
  {watchlists.map((list) => (
    <button
      key={list.id}
      type="button"
      onClick={() => setActiveListId(list.id)}
      className={px-3 py-1.5 rounded-lg text-[11px] font-bold whitespace-nowrap border ${currentList?.id === list.id ? 'bg-blue-600/15 border-blue-500/30 text-blue-400' : isDark ? 'border-gray-700 text-gray-400 hover:text-white' : 'border-gray-200 text-gray-500 hover:text-gray-900'}}
    >
      {list.name}
    </button>
  )}
</div>
</div>

{isAddingSymbol && currentList && (
  <div className={p-3 border-b flex gap-2 ${isDark ? 'border-gray-800 bg-[#0b1220]' : 'border-gray-200 bg-gray-50'}}>
    <select
      value={selectedSymbolToAdd}
      onChange={(e) => setSelectedSymbolToAdd(e.target.value ? Number(e.target.value) : '')}
      className={min-w-0 flex-1 rounded-lg border px-2 py-2 text-xs outline-none ${isDark ? 'bg-[#111827] border-gray-700' : 'bg-white border-gray-300'}}
    >
      <option value="">{t('watchlistPanel.selectSymbol')}</option>
      {availableSymbols.map((symbol) => (
        <option key={symbol.id} value={symbol.id}>
          {symbol.exchangeCode} · {symbol.name}
        </option>
      )}
    </select>
    <button disabled={busy || selectedSymbolToAdd === ''} type="button" onClick={submitSymbol} className="px-3 rounded-lg bg-emerald-600 text-

```



```

white text-xs font-bold disabled:opacity-50">
    {t('watchlistPanel.add')}}
  </button>
</div>
})

<div className="flex-1 overflow-y-auto p-2">
  {currentList ? (
    <div className="h-full grid place-items-center text-center text-xs text-gray-500 px-6">
      <div>
        <p>{t('watchlistPanel.emptyIntro')}</p>
        <button type="button" onClick={() => setIsCreatingList(true)} className="mt-3 px-3 py-2 rounded-lg bg-indigo-600 text-white font-bold">
          {t('watchlistPanel.createButton')}
        </button>
      </div>
    </div>
  ) : (currentList.items || []).length === 0 ? (
    <div className="h-full grid place-items-center text-center text-xs text-gray-500 px-6">
      <div>
        <p>{t('watchlistPanel.noSymbols')}</p>
        <p className="mt-1 text-[10px]">{t('watchlistPanel.addFromAbove')}</p>
      </div>
    </div>
  ) : (
    <div className="space-y-1">
      {currentList.items.map((item) => {
        const symbol = symbolMap.get(item.symbolId);
        const selected = selectedSymbolId === item.symbolId;
        const quote = liveQuotes[item.symbolId];
        const positive = (quote?.changePercent ?? 0) >= 0;

        return (
          <div
            key={item.id}
            className={`group flex items-center rounded-xl border transition-colors ${selected
              ? 'border-blue-500/30 bg-blue-500/10'
              : isDark
                ? 'border-transparent hover:bg-gray-800/60'
                : 'border-transparent hover:bg-gray-100'
            }`}
          >
            <button
              type="button"
              disabled={!symbol}
              onClick={() => symbol && onSelectSymbol(symbol)}
              className="flex-1 min-w-0 text-left px-3 py-2.5"
            >
              <div className="flex items-start justify-between gap-3">
                <div className="min-w-0">
                  <div className="flex items-center gap-1.5">
                    <span className="text-xs font-black tracking-wide truncate">{item.symbol}</span>
                    <span className="text-[9px] px-1.5 py-0.5 rounded bg-gray-500/10 text-gray-500">{item.exchange}</span>
                  </div>
                  <div className="text-[10px] text-gray-500 mt-1">{t('watchlistPanel.live')}</div>
                </div>
                <div className="text-right shrink-0">
                  <div className="text-xs font-bold tabular-nums">{formatPrice(quote?.price)}</div>
                  <div className={`text-[10px] font-bold tabular-nums ${quote ? (positive ? 'text-emerald-500' : 'text-rose-500') : 'text-gray-500'}`}>
                    {quote ? `${positive ? '+' : ''}${quote.changePercent.toFixed(2)}%` : t('watchlistPanel.waiting')}
                  </div>
                </div>
              </div>
            </button>
            <button
              type="button"
              disabled={busy}
              onClick={() => onRemoveSymbol(currentList.id, item.symbolId)}
              className="p-2 mr-1 opacity-0 group-hover:opacity-100 text-gray-500 hover:text-rose-400 disabled:opacity-30"
              title={t('watchlistPanel.removeFromList')}
            >
              <Trash2 className="w-3.5 h-3.5" />
            </button>
          </div>
        );
      })}
    </div>
  )
}</div>
</aside>
);
};

```

F16. src/Context/ThemeContext.tsx

Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.

```
import React, { createContext, useContext, useEffect, useState, useCallback } from 'react';
import { preferenceService } from '../Services/preferenceService';
import { authService } from '../Services/authService';

export type Theme = 'dark' | 'light';

interface ThemeContextType {
  theme: Theme;
  setTheme: (theme: Theme, saveToDb?: boolean) => void;
  toggleTheme: () => void;
  isDark: boolean;
}

const ThemeContext = createContext<ThemeContextType | undefined>(undefined);

export const ThemeProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  // Hent temaet fra localStorage ved første åbning; brug ellers dark som standard.
  const [theme, setThemeState] = useState<Theme>(() => {
    const savedTheme = localStorage.getItem('tradingpro_theme') as Theme;
    return savedTheme === 'light' || savedTheme === 'dark' ? savedTheme : 'dark';
  });

  // Synkronisering af DOM og LocalStorage
  useEffect(() => {
    localStorage.setItem('tradingpro_theme', theme);
    const root = document.documentElement;
    if (theme === 'dark') {
      root.classList.add('dark');
    } else {
      root.classList.remove('dark');
    }
  }, [theme]);

  // Angiv temaet direkte, når data kommer fra databasen eller brugeren vælger et tema.
  const setTheme = useCallback((newTheme: Theme, saveToDb = false) => {
    setThemeState(newTheme);
    localStorage.setItem('tradingpro_theme', newTheme);

    if (saveToDb && authService.isAuthenticated()) {
      preferenceService
        .savePreferences({
          Theme: newTheme,
          theme: newTheme,
        } as any)
        .catch(() => {});
    }
  }, []);

  // Skift tema, når knappen trykkes.
  const toggleTheme = useCallback(() => {
    setThemeState((prev) => {
      const next = prev === 'dark' ? 'light' : 'dark';
      localStorage.setItem('tradingpro_theme', next);

      // Gem straks i databasen, når brugeren trykker på knappen.
      if (authService.isAuthenticated()) {
        preferenceService
          .savePreferences({
            Theme: next,
            theme: next,
          } as any)
          .catch(() => {});
      }
      return next;
    });
  }, []);

  return (
    <ThemeContext.Provider
      value={{
        theme,
        setTheme,
        toggleTheme,
        isDark: theme === 'dark',
      }}
    >
      {children}
    </ThemeContext.Provider>
  );
};
```

```

    );
};

export const useTheme = () => {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error('useTheme must be used within a ThemeProvider');
  }
  return context;
};

```

F17. src/Pages/AdminPanel.tsx

Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.

```

import React, { useEffect, useState, useMemo, useCallback } from 'react';
import { useTranslation } from 'react-i18next';
import {
  Users,
  ShieldCheck,
  ArrowLeft,
  Search,
  UserCheck,
  CheckCircle2,
  AlertCircle,
  Loader2,
  Coins,
  Plus,
  RefreshCw,
  LogOut,
  Trash2,
  X,
  Mail,
  User as UserIcon,
  Lock,
  UserMinus,
  UserPlus,
  Activity,
  KeyRound,
  Save,
  Globe,
  Check
} from 'lucide-react';
import { api } from '../Services/api';
import { userService, type UserManagementDto } from '../Services/userService';
import { authService } from '../Services/authService';
import { symbolApi } from '../Services/symbolApi';
import { systemHealthService } from '../Services/systemHealthService';
import { exchangeApi } from '../Services/exchangeApi';
import type { ExchangeResponseDto, SymbolResponseDto } from '../Types/symbol';
import type { SystemHealthDto } from '../Types/systemHealth';

interface AdminPanelProps {
  onSwitchToTerminal: () => void;
  onLogout: () => void;
}

export const AdminPanel: React.FC<AdminPanelProps> = ({ onSwitchToTerminal, onLogout }) => {
  const { t, i18n } = useTranslation();
  const [isLangMenuOpen, setIsLangMenuOpen] = useState(false);
  const [activeTab, setActiveTab] = useState<'health' | 'users' | 'admins' | 'symbols' | 'exchanges' | 'profile'>('health');

  // Tilstand for brugerlister
  const [users, setUsers] = useState<UserManagementDto[]>([]);
  const [loadingUsers, setLoadingUsers] = useState(true);
  const [searchQuery, setSearchQuery] = useState('');

  // Tilstand for symbolliste
  const [symbols, setSymbols] = useState<SymbolResponseDto[]>([]);
  const [exchanges, setExchanges] = useState<ExchangeResponseDto[]>([]);
  const [selectedExchangeId, setSelectedExchangeId] = useState<number | 'all'>('all');
  const [syncingExchangeCode, setSyncingExchangeCode] = useState<string | null>(null);
  const [loadingSymbols, setLoadingSymbols] = useState(true);
  const [symbolSearchQuery, setSymbolSearchQuery] = useState('');

  // Tilstand for exchange-administration
  const [exchangeFormular, setExchangeFormular] = useState({ code: '', name: '', isActive: true });
  const [editingExchange, setEditingExchange] = useState<ExchangeResponseDto | null>(null);
  const [savingExchange, setSavingExchange] = useState(false);

  const [health, setHealth] = useState<SystemHealthDto | null>(null);
  const [loadingHealth, setLoadingHealth] = useState(true);

```

```
const [message, setMessage] = useState<{ text: string; type: 'success' | 'error' } | null>(null);

// Tilstand for modal til ny admin
const [isAddAdminOpen, setIsAddAdminOpen] = useState(false);
const [adminModalError, setAdminModalError] = useState<string | null>(null);
const [adminFormular, setAdminFormular] = useState({
  firstName: "",
  lastName: "",
  username: "",
  email: "",
  password: ""
});
const [creatingAdmin, setCreatingAdmin] = useState(false);

// Reference til aktiv bruger
const currentUser = useMemo(() => authService.getCurrentUser(), []);
const isSuperAdmin = (currentUser?.role || "").toLowerCase() === 'superadmin';

// Tilstand for opdatering af profil og adgangskode
const [currentRole, setCurrentRole] = useState<string>(currentUser?.role || 'Admin');
const [profileFormular, setProfileFormular] = useState({
  firstName: currentUser?.firstName || "",
  lastName: currentUser?.lastName || "",
  username: currentUser?.username || "",
  email: currentUser?.email || ""
});
const [savingProfile, setSavingProfile] = useState(false);

const [passwordFormular, setPasswordFormular] = useState({
  currentPassword: "",
  newPassword: "",
  confirmPassword: ""
});
const [savingPassword, setSavingPassword] = useState(false);

// Skift sprog
const handleLanguageChange = (lang: 'tr' | 'en' | 'da') => {
  i18n.changeLanguage(lang);
  localStorage.setItem('tradingpro_lang', lang);
  setIsLangMenuOpen(false);
};

const currentLangLabel = i18n.language.startsWith('da')
  ? 'DK DA'
  : i18n.language.startsWith('en')
    ? 'GB EN'
    : 'TR TR';

// 1. Hent profiloplysninger
const fetchMyProfile = useCallback(async () => {
  try {
    const myProfile = await userService.getMyProfile();
    if (myProfile) {
      setProfileFormular({
        firstName: myProfile.firstName || "",
        lastName: myProfile.lastName || "",
        username: myProfile.username || "",
        email: myProfile.email || ""
      });
      if (myProfile.role) {
        setCurrentRole(myProfile.role);
      }
    }
  } catch {
    // Ignorer fejlen uden visning.
  }
}, []);

// 2. Hent brugere
const fetchUsers = useCallback(async () => {
  setLoadingUsers(true);
  try {
    const data = await userService.getAllUsers();
    setUsers(Array.isArray(data) ? data : []);
  } catch (err: any) {
    const errMsg = err.response?.data?.message || t('admin.fetchUsersError');
    setMessage({ text: errMsg, type: 'error' });
  } finally {
    setLoadingUsers(false);
  }
}, [t]);
```

```
// Henter de nødvendige data til denne funktion.
const fetchHealth = useCallback(async () => {
  setLoadingHealth(true);
  try {
    setHealth(await systemHealthService.getHealth());
  } catch (err: any) {
    setMessage({ text: err.response?.data?.message || t('admin.healthFetchError'), type: 'error' });
  } finally {
    setLoadingHealth(false);
  }
}, []);

// 3. Hent symboler
const fetchSymbols = useCallback(async () => {
  setLoadingSymbols(true);
  try {
    const [data, exchangeData] = await Promise.all([
      symbolApi.searchSymbols(),
      exchangeApi.getAdminExchanges(),
    ]);
    setSymbols(Array.isArray(data) ? data : []);
    setExchanges(Array.isArray(exchangeData) ? exchangeData : []);
  } catch {
    setMessage({ text: t('admin.fetchSymbolsError'), type: 'error' });
  } finally {
    setLoadingSymbols(false);
  }
}, [t]);

// Håndterer refresh all.
const refreshAll = useCallback(() => {
  fetchUsers();
  fetchSymbols();
  fetchMyProfile();
  fetchHealth();
}, [fetchUsers, fetchSymbols, fetchMyProfile, fetchHealth]);

useEffect(() => {
  refreshAll();
}, [refreshAll]);

// Rollefortolker
const getUserRoleString = (role: any): string => {
  if (role === 0 || role === '0' || String(role).toLowerCase() === 'user') return 'user';
  if (role === 1 || role === '1' || String(role).toLowerCase() === 'admin') return 'admin';
  if (role === 2 || role === '2' || String(role).toLowerCase() === 'superadmin') return 'superadmin';
  return String(role || '').toLowerCase();
};

// Kontroller, at den aktuelle bruger skjules fra listen.
const isSelfUser = useCallback((user: UserManagementDto) => {
  if (!currentUser && !profileFormular.username && !profileFormular.email) return false;

  const currentId = (currentUser as any)?.id;
  const currentUsername = (profileFormular.username || currentUser?.username || '').trim().toLowerCase();
  const currentEmail = (profileFormular.email || currentUser?.email || '').trim().toLowerCase();

  const targetId = user.id;
  const targetUsername = (user.username || '').trim().toLowerCase();
  const targetEmail = (user.email || '').trim().toLowerCase();

  if (currentId && targetId && currentId === targetId) return true;
  if (currentUsername && targetUsername && currentUsername === targetUsername) return true;
  if (currentEmail && targetEmail && currentEmail === targetEmail) return true;

  return false;
}, [currentUser, profileFormular.username, profileFormular.email]);

// Standardbrugere (ekskl. den aktuelle bruger)
const standardUsers = useMemo(() => {
  return (users || []).filter(u => !isSelfUser(u) && getUserRoleString(u.role) === 'user');
}, [users, isSelfUser]);

// Administratorer (ekskl. den aktuelle Admin/SuperAdmin)
const adminUsers = useMemo(() => {
  return (users || []).filter(u => !isSelfUser(u) && getUserRoleString(u.role) === 'admin');
}, [users, isSelfUser]);

// Skift rolle
const handleRoleChange = async (user: UserManagementDto, newRole: 'User' | 'Admin') => {
```

```
try {
  await userService.assignRole({ ...user, role: newRole });
  setMessage({
    text: t('admin.roleUpdated', { username: user.username, role: newRole }),
    type: 'success'
  });
  fetchUsers();
} catch (err: any) {
  setMessage({ text: err.response?.data?.message || t('admin.roleUpdateError'), type: 'error' });
}
};

// Permanent sletning (hard delete)
const handleHardDeleteUser = async (userId: number, username: string) => {
  const confirmMsg = t('admin.deleteConfirm', { username });
  if (!window.confirm(confirmMsg)) return;

  try {
    const res = await userService.hardDeleteUser(userId);
    setMessage({ text: res.message || t('admin.userDeleted', { username }), type: 'success' });
    fetchUsers();
  } catch (err: any) {
    setMessage({ text: err.response?.data?.message || t('admin.userDeleteError'), type: 'error' });
  }
};

// Opret ny admin
const handleCreateAdminSubmit = async (e: React.FormularEvent) => {
  e.preventDefault();
  setAdminModalError(null);
  setCreatingAdmin(true);
  try {
    const currentToken = localStorage.getItem('token');
    const currentUserData = localStorage.getItem('user');

    await api.post('/auth/register', {
      firstName: adminFormular.firstName.trim(),
      lastName: adminFormular.lastName.trim(),
      username: adminFormular.username.trim(),
      email: adminFormular.email.trim(),
      password: adminFormular.password
    });

    if (currentToken) localStorage.setItem('token', currentToken);
    if (currentUserData) localStorage.setItem('user', currentUserData);

    const allUsers = await userService.getAllUsers();
    const created = allUsers.find(
      (u) => (u.email || "").toLowerCase() === adminFormular.email.trim().toLowerCase()
    );

    if (created) {
      await userService.assignRole({ ...created, role: 'Admin' });
    }

    setMessage({ text: t('admin.adminCreatedSuccess', { username: adminFormular.username }), type: 'success' });
    setIsAddAdminOpen(false);
    setAdminFormular({ firstName: "", lastName: "", username: "", email: "", password: "" });
    await fetchUsers();
    setActiveTab('admins');
  } catch (err: any) {
    console.error('Admin oluşturma hatası:', err);
    const backendMsg = err.response?.data?.message;
    const validationErrors = err.response?.data?.errors;

    if (backendMsg) {
      setAdminModalError(backendMsg);
    } else if (validationErrors && typeof validationErrors === 'object') {
      const firstError = Object.values(validationErrors).flat()[0] as string;
      setAdminModalError(firstError || t('admin.checkFormularError'));
    } else {
      setAdminModalError(t('admin.adminCreateError'));
    }
  } finally {
    setCreatingAdmin(false);
  }
};

// Opret exchange
const handleCreateExchange = async (e: React.FormularEvent) => {
  e.preventDefault();
  const code = exchangeFormular.code.trim().toUpperCase();
```

```
const name = exchangeFormular.name.trim();
if (!code || !name) return;

setSavingExchange(true);
try {
  await exchangeApi.createExchange({ code, name, isActive: exchangeFormular.isActive });
  setExchangeFormular({ code: '', name: '', isActive: true });
  setMessage({ text: t('admin.exchangeCreated', { exchange: code }), type: 'success' });
  await fetchSymbols();
} catch (err: any) {
  setMessage({ text: err.response?.data?.message || t('admin.exchangeSaveError'), type: 'error' });
} finally {
  setSavingExchange(false);
}
};

// Opdater exchange
const handleUpdateExchange = async (e: React.FormularEvent) => {
  e.preventDefault();
  if (!editingExchange) return;

  setSavingExchange(true);
  try {
    const updated = await exchangeApi.updateExchange(editingExchange.id, {
      code: editingExchange.code.trim().toUpperCase(),
      name: editingExchange.name.trim(),
      isActive: editingExchange.isActive,
    });
    setEditingExchange(null);
    setMessage({ text: t('admin.exchangeUpdated', { exchange: updated.code }), type: 'success' });
    await fetchSymbols();
  } catch (err: any) {
    setMessage({ text: err.response?.data?.message || t('admin.exchangeSaveError'), type: 'error' });
  } finally {
    setSavingExchange(false);
  }
};

// Slet exchange - backend forhindrer af sikkerhedshensyn sletning af exchanges med historik og kræver i stedet deaktivering.
const handleDeleteExchange = async (exchange: ExchangeResponseDto) => {
  if (!window.confirm(t('admin.exchangeDeleteConfirm', { exchange: exchange.name }))) return;

  try {
    await exchangeApi.deleteExchange(exchange.id);
    setMessage({ text: t('admin.exchangeDeleted', { exchange: exchange.code }), type: 'success' });
    await fetchSymbols();
  } catch (err: any) {
    setMessage({ text: err.response?.data?.message || t('admin.exchangeDeleteError'), type: 'error' });
  }
};

// Opdater profiloplysninger
const handleProfileSubmit = async (e: React.FormularEvent) => {
  e.preventDefault();
  setSavingProfile(true);
  try {
    const payload: Partial<UserManagementDto> = {
      ...profileFormular,
      role: currentRole || currentUser?.role || (isSuperAdmin ? 'SuperAdmin' : 'Admin')
    };

    const updated = await userService.updateMyProfile(payload);
    setMessage({ text: t('admin.profileUpdated'), type: 'success' });

    const updatedUser = { ...currentUser, ...updated };
    localStorage.setItem('user', JSON.stringify(updatedUser));
    fetchUsers();
  } catch (err: any) {
    const backendErrors = err.response?.data?.errors;
    let errorText = err.response?.data?.message;
    if (backendErrors && typeof backendErrors === 'object') {
      errorText = Object.values(backendErrors).flat().join(' ');
    }
    setMessage({ text: errorText || t('admin.profileUpdateError'), type: 'error' });
  } finally {
    setSavingProfile(false);
  }
};

// Opdater adgangskode
const handlePasswordSubmit = async (e: React.FormularEvent) => {
  e.preventDefault();
  if (passwordFormular.newPassword !== passwordFormular.confirmPassword) {
```

```

    setMessage({ text: t('admin.passwordMismatch'), type: 'error' });
    return;
  }
  setSavingPassword(true);
  try {
    const res = await userService.changePassword({
      currentPassword: passwordFormular.currentPassword,
      newPassword: passwordFormular.newPassword
    });
    setMessage({ text: res.message || t('admin.passwordChanged'), type: 'success' });
    setPasswordFormular({ currentPassword: "", newPassword: "", confirmPassword: "" });
  } catch (err: any) {
    setMessage({ text: err.response?.data?.message || t('admin.passwordChangeError'), type: 'error' });
  } finally {
    setSavingPassword(false);
  }
};

// Skift symbolstatus
const handleToggleSymbol = async (sym: SymbolResponseDto) => {
  const currentStatus = Boolean(sym.isActive);
  const targetStatus = !currentStatus;
  const symName = (sym.name || "").trim();

  try {
    setSymbols(prev =>
      prev.map(s => s.id === sym.id ? { ...s, isActive: targetStatus } : s)
    );
    const result = await symbolApi.toggleSymbolStatus(sym.id, targetStatus);
    setMessage({
      text: result.historicalSyncQueued
        ? t('admin.symbolHistoricalSyncQueued', { symbol: symName })
        : t(targetStatus ? 'admin.symbolActivated' : 'admin.symbolDeactivated', {
            symbol: symName
          }),
      type: 'success'
    });
  } catch {
    setSymbols(prev =>
      prev.map(s => s.id === sym.id ? { ...s, isActive: currentStatus } : s)
    );
    setMessage({ text: t('admin.symbolUpdateError', { symbol: symName }), type: 'error' });
  }
};

// Behandler den relevante brugerhandling eller event.
const handleSyncExchange = async (exchangeCode: string) => {
  setSyncingExchangeCode(exchangeCode);
  try {
    const result = await symbolApi.syncExchange(exchangeCode);
    setMessage({ text: result.message || t('admin.syncSuccess', { exchange: exchangeCode }), type: 'success' });
    await fetchSymbols();
  } catch (err: any) {
    setMessage({ text: err.response?.data?.message || t('admin.syncError', { exchange: exchangeCode }), type: 'error' });
  } finally {
    setSyncingExchangeCode(null);
  }
};

// Søgefiltre
const qSearch = (searchQuery || "").trim().toLowerCase();

const filteredStandardUsers = standardUsers.filter(u => {
  const username = (u.username || "").toLowerCase();
  const email = (u.email || "").toLowerCase();
  const fullName = `${u.firstName || ""} ${u.lastName || ""}`.toLowerCase();
  return username.includes(qSearch) || email.includes(qSearch) || fullName.includes(qSearch);
});

const filteredAdminUsers = adminUsers.filter(u => {
  const username = (u.username || "").toLowerCase();
  const email = (u.email || "").toLowerCase();
  const fullName = `${u.firstName || ""} ${u.lastName || ""}`.toLowerCase();
  return username.includes(qSearch) || email.includes(qSearch) || fullName.includes(qSearch);
});

const qSym = (symbolSearchQuery || "").trim().toLowerCase();
const filteredSymbols = (symbols || []).filter(s => {
  if (selectedExchangeId !== 'all' && s.exchangeId !== selectedExchangeId) return false;
  const name = (s.name || "").toLowerCase();
  const base = (s.baseAsset || "").toLowerCase();
  const quote = (s.quoteAsset || "").toLowerCase();
  const exchange = (s.exchangeCode || "").toLowerCase();
  return name.includes(qSym) || base.includes(qSym) || quote.includes(qSym) || exchange.includes(qSym);
});

```



```

});

return (
  <div className="min-h-screen bg-[#0b0e14] text-gray-200 p-4 sm:p-8 font-sans select-none">
    <div className="max-w-7xl mx-auto space-y-6">

      {/ * Øverste header */}
      <div className="flex flex-col md:flex-row md:items-center justify-between gap-4 pb-6 border-b border-[#21262d]">
        <div className="flex items-center space-x-3.5">
          <div className="p-3 bg-purple-600/15 text-purple-400 rounded-2xl border border-purple-500/25 shadow-xl">
            <ShieldCheck className="w-8 h-8" />
          </div>
          <div>
            <div className="flex items-center space-x-2.5">
              <h1 className="text-xl font-extrabold text-white tracking-tight">{t('admin.panelTitle')}</h1>
              <span className={
                {
                  px: 2.5, py: 0.5, rounded: 'full', text: '[10px]', font: 'black', tracking: 'wider',
                  {isSuperAdmin ? 'bg-purple-500/20 text-purple-400 border border-purple-500/30' : 'bg-blue-500/20 text-blue-400 border border-blue-500/30'}
                }
              }>
                {isSuperAdmin ? 'SUPERADMIN' : 'ADMIN'}
              </span>
            </div>
            <p className="text-xs text-gray-400 mt-0.5">{t('admin.panelSubtitle')}</p>
          </div>
        </div>

        <div className="flex flex-wrap items-center gap-3">
          {/ * Sprogvalg */}
          <div className="relative">
            <button
              type="button"
              onClick={() => setLangMenuOpen(!isLangMenuOpen)}
              className="flex items-center space-x-1.5 px-3 py-2 rounded-xl border border-[#30363d] bg-[#161b22] hover:bg-[#21262d] text-xs font-
semibold text-gray-200 transition-colors shadow-sm"
            >
              <Globe className="w-3.5 h-3.5 text-blue-400" />
              <span>{currentLangLabel}</span>
            </button>

            {isLangMenuOpen && (
              <div className="absolute top-full right-0 mt-1.5 w-32 rounded-xl bg-[#161b22] border border-[#30363d] shadow-2xl p-1.5 space-y-0.5 z-50
text-xs">
                <button
                  type="button"
                  onClick={() => handleLanguageChange('tr')}
                  className={
                    {
                      'w-full flex items-center justify-between px-2.5 py-1.5 rounded-lg font-medium transition-colors',
                      {i18n.language.startsWith('tr') ? 'bg-blue-600/20 text-blue-400' : 'text-gray-300 hover:bg-[#21262d]'}
                    }
                  }
                >
                  <span>tr Türkçe</span>
                  {i18n.language.startsWith('tr') && <Check className="w-3 h-3" />}
                </button>
                <button
                  type="button"
                  onClick={() => handleLanguageChange('en')}
                  className={
                    {
                      'w-full flex items-center justify-between px-2.5 py-1.5 rounded-lg font-medium transition-colors',
                      {i18n.language.startsWith('en') ? 'bg-blue-600/20 text-blue-400' : 'text-gray-300 hover:bg-[#21262d]'}
                    }
                  }
                >
                  <span>en English</span>
                  {i18n.language.startsWith('en') && <Check className="w-3 h-3" />}
                </button>
                <button
                  type="button"
                  onClick={() => handleLanguageChange('da')}
                  className={
                    {
                      'w-full flex items-center justify-between px-2.5 py-1.5 rounded-lg font-medium transition-colors',
                      {i18n.language.startsWith('da') ? 'bg-blue-600/20 text-blue-400' : 'text-gray-300 hover:bg-[#21262d]'}
                    }
                  }
                >
                  <span>dk Dansk</span>
                  {i18n.language.startsWith('da') && <Check className="w-3 h-3" />}
                </button>
              </div>
            )}
          </div>

          {/ * Badge for den aktuelle bruger */}
          <div
            onClick={() => setActiveTab('profile')}

```

```
transition-all shadow-sm"
  title={t('admin.editProfile')})
  >
  <div className="w-8 h-8 rounded-lg bg-purple-600/20 border border-purple-500/30 flex items-center justify-center text-purple-300 font-bold text-
xs">
  {({profileFormular.firstName?.[0] || profileFormular.username?.[0] || 'A').toUpperCase()}}
  </div>
  <div className="text-left leading-tight">
    <div className="text-xs font-bold text-white flex items-center space-x-1">
      <span>{profileFormular.firstName ? `${profileFormular.firstName} ${profileFormular.lastName}` : profileFormular.username}</span>
    </div>
    <div className="text-[10px] text-gray-400 font-mono">@{profileFormular.username}</div>
  </div>
  </div>

  {/ * Opdater knap */}
  <button
    type="button"
    onClick={refreshAll}
    className="p-2.5 rounded-xl bg-[#161b22] hover:bg-[#21262d] border border-[#30363d] text-gray-300 transition-colors shadow-sm"
    title={t('admin.refresh')}
  >
    <RefreshCw className={ `w-4 h-4 ${loadingUsers || loadingSymbols ? 'animate-spin text-purple-400' : ''} ` } />
  </button>

  {/ * Gå til terminal */}
  <button
    type="button"
    onClick={onSwitchToTerminal}
    className="flex items-center space-x-2 px-4 py-2.5 bg-blue-600 hover:bg-blue-500 active:bg-blue-700 text-white rounded-xl text-xs font-bold
transition-all shadow-lg shadow-blue-600/25"
  >
    <ArrowLeft className="w-4 h-4" />
    <span>{t('admin.goToTerminal')}</span>
  </button>

  {/ * Log ud */}
  <button
    type="button"
    onClick={onLogout}
    className="p-2.5 rounded-xl bg-red-500/10 hover:bg-red-500/20 text-red-400 border border-red-500/25 transition-colors"
    title={t('admin.logout')}
  >
    <LogOut className="w-4 h-4" />
  </button>
</div>
</div>

{/ * KPI-statistik kort */}
<div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
  <div className="bg-[#161b22] border border-[#30363d] rounded-2xl p-4 flex items-center justify-between shadow-lg">
    <div className="space-y-1">
      <span className="text-[11px] font-bold text-gray-400 uppercase tracking-wider">{t('admin.registeredUsers')}</span>
      <div className="text-2xl font-black text-white">{standardUsers.length}</div>
    </div>
    <div className="p-3 bg-blue-500/10 text-blue-400 rounded-xl border border-blue-500/20">
      <Users className="w-5 h-5" />
    </div>
  </div>
  </div>

  <div className="bg-[#161b22] border border-[#30363d] rounded-2xl p-4 flex items-center justify-between shadow-lg">
    <div className="space-y-1">
      <span className="text-[11px] font-bold text-gray-400 uppercase tracking-wider">{t('admin.otherAdmins')}</span>
      <div className="text-2xl font-black text-purple-400">{adminUsers.length}</div>
    </div>
    <div className="p-3 bg-purple-500/10 text-purple-400 rounded-xl border border-purple-500/20">
      <ShieldCheck className="w-5 h-5" />
    </div>
  </div>
  </div>

  <div className="bg-[#161b22] border border-[#30363d] rounded-2xl p-4 flex items-center justify-between shadow-lg">
    <div className="space-y-1">
      <span className="text-[11px] font-bold text-gray-400 uppercase tracking-wider">{t('admin.activeMarkets')}</span>
      <div className="text-2xl font-black text-emerald-400">
        {symbols.filter(s => Boolean(s.isActive)).length} <span className="text-xs text-gray-500 font-normal"> / {symbols.length}</span>
      </div>
    </div>
    <div className="p-3 bg-emerald-500/10 text-emerald-400 rounded-xl border border-emerald-500/20">
      <Activity className="w-5 h-5" />
    </div>
  </div>
</div>
</div>
```

```

    {/ * Faner */}
    <div className="flex border-b border-[#21262d] text-xs font-bold gap-8 overflow-x-auto">
      <button
        type="button"
        onClick={() => setActiveTab('health')}
        className={`pb-3.5 border-b-2 transition-colors flex items-center space-x-2 ${activeTab === 'health' ? 'border-cyan-500 text-cyan-400' : 'border-transparent text-gray-400 hover:text-gray-200'}`}
      >
        <Activity className="w-4 h-4" />
        <span>{t('admin.tabHealth')}</span>
      </button>

      <button
        type="button"
        onClick={() => { setActiveTab('users'); setSearchQuery(""); }}
        className={`pb-3.5 border-b-2 transition-colors flex items-center space-x-2 ${activeTab === 'users' ? 'border-blue-500 text-blue-400' : 'border-transparent text-gray-400 hover:text-gray-200'}`}
      >
        <Users className="w-4 h-4" />
        <span>{t('admin.tabUsers')} {standardUsers.length}</span>
      </button>

      <button
        type="button"
        onClick={() => { setActiveTab('admins'); setSearchQuery(""); }}
        className={`pb-3.5 border-b-2 transition-colors flex items-center space-x-2 ${activeTab === 'admins' ? 'border-purple-500 text-purple-400' : 'border-transparent text-gray-400 hover:text-gray-200'}`}
      >
        <ShieldCheck className="w-4 h-4" />
        <span>{t('admin.tabAdmins')} {adminUsers.length}</span>
      </button>

      <button
        type="button"
        onClick={() => setActiveTab('symbols')}
        className={`pb-3.5 border-b-2 transition-colors flex items-center space-x-2 ${activeTab === 'symbols' ? 'border-emerald-500 text-emerald-400' : 'border-transparent text-gray-400 hover:text-gray-200'}`}
      >
        <Coins className="w-4 h-4" />
        <span>{t('admin.tabSymbols')} {symbols.length}</span>
      </button>

      <button
        type="button"
        onClick={() => setActiveTab('exchanges')}
        className={`pb-3.5 border-b-2 transition-colors flex items-center space-x-2 ${activeTab === 'exchanges' ? 'border-sky-500 text-sky-400' : 'border-transparent text-gray-400 hover:text-gray-200'}`}
      >
        <Globe className="w-4 h-4" />
        <span>{t('admin.tabExchanges')} {exchanges.length}</span>
      </button>

      <button
        type="button"
        onClick={() => setActiveTab('profile')}
        className={`pb-3.5 border-b-2 transition-colors flex items-center space-x-2 ${activeTab === 'profile' ? 'border-amber-500 text-amber-400' : 'border-transparent text-gray-400 hover:text-gray-200'}`}
      >
        <KeyRound className="w-4 h-4" />
        <span>{t('admin.tabProfile')}</span>
      </button>
    </div>

    {/ * Notifikationsbesked */}
    {message && (
      <div className={`p-3.5 rounded-xl flex items-center justify-between text-xs animate-in fade-in duration-200 ${message.type === 'success' ? 'bg-emerald-500/10 border border-emerald-500/30 text-emerald-400' : 'bg-red-500/10 border border-red-500/30 text-red-400'}`}
      >
        <div className="flex items-center space-x-2.5">
          {message.type === 'success' ? <CheckCircle2 className="w-4 h-4 shrink-0" /> : <AlertCircle2 className="w-4 h-4 shrink-0" />}
          <span className="font-medium">{message.text}</span>
        </div>
        <button type="button" onClick={() => setMessage(null)} className="text-gray-400 hover:text-white">
          <X className="w-4 h-4" />
        </button>
      </div>
    )}

    {activeTab === 'health' && (
      <div className="space-y-5">

```

```
<div className="flex items-center justify-between gap-3">
  <div>
    <h2 className="text-base font-black text-white">{t('admin.healthTitle')}</h2>
    <p className="text-xs text-gray-500 mt-1">{t('admin.healthSubtitle')}</p>
  </div>
  <button type="button" onClick={fetchHealth} disabled={loadingHealth} className="flex items-center gap-2 px-3 py-2 rounded-xl bg-cyan-500/10 text-cyan-300 border border-cyan-500/20 hover:bg-cyan-500/20 disabled:opacity-50 text-xs font-bold">
    <RefreshCw className={w-3.5 h-3.5 $loadingHealth ? 'animate-spin' : ''} />
    {t('admin.refresh')}
  </button>
</div>

{loadingHealth && !health ? (
  <div className="py-24 grid place-items-center bg-[#161b22] border border-[#30363d] rounded-2xl"><Loader2 className="w-8 h-8 animate-spin text-cyan-400" /></div>
) : health ? (
  <div>
    <div className="grid grid-cols-2 lg:grid-cols-5 gap-3">
      {
        [t('admin.healthOverall'), health.status.toLowerCase() === 'healthy' ? t('admin.healthHealthy') : health.status.toLowerCase() === 'degraded' ? t('admin.healthDegraded') : t('admin.healthUnhealthy'), health.status.toLowerCase() === 'healthy'],
        [t('admin.healthDatabase'), health.databaseHealthy ? t('admin.healthOnline') : t('admin.healthOffline'), health.databaseHealthy],
        [t('admin.healthTimescale'), health.timescaleHealthy ? t('admin.healthOnline') : t('admin.healthOffline'), health.timescaleHealthy],
        [t('admin.healthHypertable'), health.candlesHypertableHealthy ? t('admin.healthReady') : t('admin.healthProblem'), health.candlesHypertableHealthy],
        [t('admin.healthActiveSymbols'), t('admin.healthSymbolCount', { count: health.activeSymbolCount }), health.activeSymbolCount > 0],
      ].map(([,label, value, ok]) => (
        <div key={String(label)} className="bg-[#161b22] border border-[#30363d] rounded-2xl p-4">
          <div className="flex items-center justify-between gap-2">
            <span className="text-[10px] uppercase tracking-wider font-bold text-gray-500">{String(label)}</span>
            {ok ? <CheckCircle2 className="w-4 h-4 text-emerald-400" /> : <AlertCircle className="w-4 h-4 text-rose-400" />}
          </div>
          <div className="mt-2 text-lg font-black $ok ? 'text-emerald-300' : 'text-rose-300'">{String(value)}</div>
        </div>
      ))
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-3 gap-3">
      <div className="lg:col-span-2 bg-[#161b22] border border-[#30363d] rounded-2xl overflow-hidden">
        <div className="px-4 py-3 border-b border-[#30363d] text-xs font-black text-white">{t('admin.healthExchangeRuntime')}</div>
        <div className="divide-y divide-[#30363d]">
          {health.exchanges.map((exchange) => (
            <div key={exchange.exchangeCode} className="p-4 grid grid-cols-2 md:grid-cols-6 gap-3 items-center text-xs">
              <div className="md:col-span-2">
                <div className="font-black text-white">{exchange.exchangeName}</div>
                <div className="text-[10px] text-gray-500 font-mono">{exchange.exchangeCode}</div>
              </div>
              <div><div className="text-[9px] uppercase text-gray-500">{t('admin.healthRealtime')}</div><div className={exchange.realtimeConnected ? 'text-emerald-400 font-bold' : 'text-rose-400 font-bold'}>{exchange.realtimeConnected ? t('admin.healthConnected') : t('admin.healthDisconnected')}</div></div>
              <div><div className="text-[9px] uppercase text-gray-500">{t('admin.healthHistorical')}</div><div className={exchange.historicalSyncRunning ? 'text-amber-300 font-bold' : 'text-gray-300 font-bold'}>{exchange.historicalSyncRunning ? t('admin.healthSyncing') : t('admin.healthIdle')}</div></div>
              <div><div className="text-[9px] uppercase text-gray-500">{t('admin.healthSymbols')}</div><div className="text-white font-bold">{exchange.activeSymbolCount}</div></div>
              <div><div className="text-[9px] uppercase text-gray-500">{t('admin.healthStatus')}</div><div className="font-bold text-cyan-300">{exchange.status?.toLowerCase() === 'healthy' ? t('admin.healthHealthy') : exchange.status?.toLowerCase() === 'degraded' ? t('admin.healthDegraded') : exchange.status?.toLowerCase() === 'unhealthy' ? t('admin.healthUnhealthy') : exchange.status}</div></div>
              {exchange.lastError && <div className="col-span-2 md:col-span-6 text-[10px] text-rose-300 bg-rose-500/10 rounded-lg px-3 py-2">{exchange.lastError}</div>}
            </div>
          ))
        </div>
      </div>

      <div className="bg-[#161b22] border border-[#30363d] rounded-2xl p-4 space-y-4">
        <div><div className="text-[10px] uppercase text-gray-500 font-bold">{t('admin.healthLastCandle')}</div><div className="text-sm font-mono text-white mt-1">{health.lastCandleTimeUtc ? new Date(health.lastCandleTimeUtc).toLocaleString(18n.language) : '—'}</div></div>
        <div><div className="text-[10px] uppercase text-gray-500 font-bold">{t('admin.healthCandleAge')}</div><div className="text-xl font-black text-cyan-300 mt-1">{health.lastCandleAgeSeconds != null ? t('admin.healthSeconds', { count: Math.round(health.lastCandleAgeSeconds) }) : '—'}</div></div>
        <div><div className="grid grid-cols-2 gap-2"><div className="rounded-xl bg-rose-500/10 p-3"><div className="text-[9px] uppercase text-rose-300">{t('admin.healthValidationError')}</div><div className="text-xl font-black text-rose-300">{health.validationErrorCount}</div></div><div className="rounded-xl bg-amber-500/10 p-3"><div className="text-[9px] uppercase text-amber-300">{t('admin.healthWarning')}</div><div className="text-xl font-black text-amber-300">{health.validationWarningCount}</div></div></div>
        <div><div className="text-[10px] uppercase text-gray-500 font-bold">{t('admin.healthUptime')}</div><div className="text-sm font-mono text-gray-300 mt-1">{health.uptime}</div></div>
      </div>
    </div>
  </div>
) : null}
</div>
)}

{ /* 1. Standardbrugere */
{activeTab === 'users' && (
```

```

<div className="space-y-4">
  <div className="relative max-w-md">
    <Search className="w-4 h-4 absolute left-3.5 top-3 text-gray-400" />
    <input
      type="text"
      placeholder={t('admin.searchUsersPlaceholder')}
      value={searchQuery}
      onChange={(e) => setSearchQuery(e.target.value)}
      className="w-full pl-10 pr-4 py-2.5 bg-[#161b22] border border-[#30363d] rounded-xl text-xs outline-none focus:border-blue-500 text-white
transition-colors"
    />
  </div>

  <div className="bg-[#161b22] border border-[#30363d] rounded-2xl overflow-hidden shadow-2xl">
    {loadingUsers ? (
      <div className="py-20 flex flex-col items-center justify-center space-y-2 text-gray-400">
        <Loader2 className="w-8 h-8 animate-spin text-blue-500" />
        <span className="text-xs">{t('admin.loadingUsers')}</span>
      </div>
    ) : (
      <div className="overflow-x-auto">
        <table className="w-full text-left text-xs">
          <thead className="bg-[#0d1117] text-gray-400 uppercase text-[10px] tracking-wider border-b border-[#30363d]">
            <tr>
              <th className="px-6 py-4">{t('admin.tableUser')}</th>
              <th className="px-6 py-4">{t('admin.tableEmail')}</th>
              <th className="px-6 py-4">{t('admin.tableStatus')}</th>
              <th className="px-6 py-4 text-right">{t('admin.tableActions')}</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-[#30363d]">
            {filteredStandardUsers.length === 0 ? (
              <tr>
                <td colSpan={4} className="px-6 py-8 text-center text-gray-400">
                  {t('admin.noUsersFound')}
                </td>
              </tr>
            ) : (
              filteredStandardUsers.map((user) => (
                <tr key={user.id} className="hover:bg-gray-800/20 transition-colors">
                  <td className="px-6 py-4">
                    <div className="font-semibold text-white">
                      {user.firstName} {user.lastName}
                    </div>
                    <div className="text-gray-400 font-mono text-[11px]">@{user.username}</div>
                  </td>

                  <td className="px-6 py-4 text-gray-300 font-mono">{user.email}</td>

                  <td className="px-6 py-4">
                    <span className="inline-flex items-center space-x-1 text-emerald-400 text-[11px]">
                      <UserCheck className="w-3.5 h-3.5" />
                      <span>{t('admin.activeAccount')}</span>
                    </span>
                  </td>

                  <td className="px-6 py-4 text-right">
                    <div className="flex items-center justify-end space-x-2">
                      {isSuperAdmin && (
                        <button
                          type="button"
                          onClick={() => handleRoleChange(user, 'Admin')}
                          className="flex items-center space-x-1 px-3 py-1.5 rounded-lg bg-purple-600/15 hover:bg-purple-600/25 text-purple-
300 border border-purple-500/30 text-xs font-semibold transition-colors"
                          title={t('admin.makeAdmin')}
                        >
                          <UserPlus className="w-3.5 h-3.5" />
                          <span>{t('admin.makeAdminBtn')}</span>
                        </button>
                      )}

                      <button
                        type="button"
                        onClick={() => handleHardDeleteUser(user.id, user.username)}
                        className="p-1.5 rounded-lg hover:bg-red-500/20 text-gray-400 hover:text-red-400 transition-colors border border-
transparent hover:border-red-500/30"
                        title={t('admin.deleteUserTooltip')}
                      >
                        <Trash2 className="w-4 h-4" />
                      </button>
                    </div>
                  </td>
                </tr>
              )}
            )}
          </tbody>
        </table>
      </div>
    )}
  </div>
)

```

```

    })
  </tbody>
</table>
</div>
})
</div>
</div>
})

{/* 2. Administratorer */}
{activeTab === 'admins' && (
  <div className="space-y-4">
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3">
      <div className="relative flex-1 max-w-md">
        <Search className="w-4 h-4 absolute left-3.5 top-3 text-gray-400" />
        <input
          type="text"
          placeholder={t('admin.searchAdminsPlaceholder')}
          value={searchQuery}
          onChange={(e) => setSearchQuery(e.target.value)}
          className="w-full pl-10 pr-4 py-2.5 bg-[#161b22] border border-[#30363d] rounded-xl text-xs outline-none focus:border-purple-500 text-
white transition-colors"
        />
      </div>

      {isSuperAdmin && (
        <button
          type="button"
          onClick={() => { setIsAddAdminOpen(true); setAdminModalError(null); }}
          className="flex items-center space-x-1.5 px-4 py-2.5 bg-purple-600 hover:bg-purple-500 active:bg-purple-700 text-white rounded-xl text-xs
font-bold transition-all shadow-lg shadow-purple-600/25 self-start sm:self-auto"
          >
          <Plus className="w-4 h-4" />
          <span>{t('admin.addAdminBtn')}</span>
        </button>
      )}
    </div>

    <div className="bg-[#161b22] border border-[#30363d] rounded-2xl overflow-hidden shadow-2xl">
      {loadingUsers ? (
        <div className="py-20 flex flex-col items-center justify-center space-y-2 text-gray-400">
          <Loader2 className="w-8 h-8 animate-spin text-purple-500" />
          <span className="text-xs">{t('admin.loadingAdmins')}</span>
        </div>
      ) : (
        <div className="overflow-x-auto">
          <table className="w-full text-left text-xs">
            <thead className="bg-[#0d1117] text-gray-400 uppercase text-[10px] tracking-wider border-b border-[#30363d]">
              <tr>
                <th className="px-6 py-4">{t('admin.tableAdmin')}</th>
                <th className="px-6 py-4">{t('admin.tableEmail')}</th>
                <th className="px-6 py-4">{t('admin.tableRole')}</th>
                {isSuperAdmin && <th className="px-6 py-4 text-right">{t('admin.tableActions')}</th>}
              </tr>
            </thead>
            <tbody className="divide-y divide-[#30363d]">
              {filteredAdminUsers.length === 0 ? (
                <tr>
                  <td colspan={4} className="px-6 py-8 text-center text-gray-400">
                    {t('admin.noOtherAdmins')}
                  </td>
                </tr>
              ) : (
                filteredAdminUsers.map((user) => (
                  <tr key={user.id} className="hover:bg-gray-800/20 transition-colors">
                    <td className="px-6 py-4">
                      <div className="font-semibold text-white flex items-center space-x-2">
                        <span>{user.firstName} {user.lastName}</span>
                      </div>
                      <div className="text-gray-400 font-mono text-[11px]">@{user.username}</div>
                    </td>

                    <td className="px-6 py-4 text-gray-300 font-mono">{user.email}</td>

                    <td className="px-6 py-4">
                      <span className="px-2.5 py-0.5 rounded-full text-[10px] font-bold bg-purple-500/20 text-purple-300 border border-purple-
500/30">
                        {t('admin.roleAdminBadge', 'Admin (Panel Yetkilisi)')}
                      </span>
                    </td>

                    {isSuperAdmin && (
                      <td className="px-6 py-4 text-right">
                        <div className="flex items-center justify-end space-x-2">

```

```

        type="button"
        onClick={() => handleRoleChange(user, 'User')}
        className="flex items-center space-x-1 px-3 py-1.5 rounded-lg bg-gray-500/10 hover:bg-gray-500/20 text-gray-300 border border-gray-500/30 text-xs font-semibold transition-colors"
        title={t('admin.revokeRoleTitle')}
      >
        <UserMinus className="w-3.5 h-3.5" />
        <span>{t('admin.revokeRoleBtn')}

```



```

{ /* 4. Exchange-administration */ }
{ activeTab === 'exchanges' && (
  <div className="space-y-5">
    <form onSubmit={handleCreateExchange} className="bg-#161b22 border border-#30363d rounded-2xl p-5 shadow-xl">
      <div className="flex items-center justify-between gap-3 mb-4">
        <div>
          <h2 className="text-sm font-black text-white">{t('admin.exchangeManagementTitle')}</h2>
          <p className="text-[11px] text-gray-400 mt-1">{t('admin.exchangeManagementHint')}</p>
        </div>
        <div className="text-[10px] font-bold px-2.5 py-1 rounded-full bg-sky-500/10 text-sky-300 border border-sky-500/20">
          BINANCE · OKX
        </div>
      </div>
      <div className="grid grid-cols-1 md:grid-cols-[180px_1fr_auto_auto] gap-3 items-end">
        <label className="space-y-1.5">
          <span className="text-[10px] uppercase tracking-wider text-gray-500 font-bold">{t('admin.exchangeCode')}</span>
          <input value={exchangeFormular.code} onChange={(e) => setExchangeFormular(v => ({ ...v, code: e.target.value.toUpperCase() })))
placeholder="OKX" maxLength={20} className="w-full px-3 py-2.5 bg-#0d1117 border border-#30363d rounded-xl text-xs text-white outline-none focus:border-sky-500 font-mono" />
        </label>
        <label className="space-y-1.5">
          <span className="text-[10px] uppercase tracking-wider text-gray-500 font-bold">{t('admin.exchangeName')}</span>
          <input value={exchangeFormular.name} onChange={(e) => setExchangeFormular(v => ({ ...v, name: e.target.value }))) placeholder="OKX"
maxLength={50} className="w-full px-3 py-2.5 bg-#0d1117 border border-#30363d rounded-xl text-xs text-white outline-none focus:border-sky-500" />
        </label>
        <label className="h-[42px] flex items-center gap-2 px-3 rounded-xl bg-#0d1117 border border-#30363d text-xs text-gray-300 cursor-

```



```
pointer">
    <input type="checkbox" checked={exchangeFormular.isActive} onChange={(e) => setExchangeFormular(v => ({ ...v, isActive:
e.target.checked }))) />
    {t('admin.active')}
  </label>
  <button type="submit" disabled={savingExchange || !exchangeFormular.code.trim() || !exchangeFormular.name.trim()} className="h-[42px]
flex items-center justify-center gap-1.5 px-4 rounded-xl bg-sky-600 hover:bg-sky-500 disabled:opacity-50 text-white text-xs font-black">
    {savingExchange ? <Loader2 className="w-4 h-4 animate-spin" /> : <Plus className="w-4 h-4" />}
    {t('admin.addExchange')}
  </button>
</div>
</form>

<div className="bg-[#161b22] border border-[#30363d] rounded-2xl overflow-hidden shadow-xl">
  <div className="overflow-x-auto">
    <table className="w-full text-left text-xs">
      <thead className="bg-[#0d1117] text-gray-400 uppercase text-[10px] tracking-wider border-b border-[#30363d]">
        <tr>
          <th className="px-6 py-4">{t('admin.exchangeCode')}</th>
          <th className="px-6 py-4">{t('admin.exchangeName')}</th>
          <th className="px-6 py-4">{t('admin.tableStatus')}</th>
          <th className="px-6 py-4 text-right">{t('admin.tableActions')}</th>
        </tr>
      </thead>
      <tbody className="divide-y divide-[#30363d]">
        {exchanges.map(exchange => (
          <tr key={exchange.id} className="hover:bg-gray-800/20">
            <td className="px-6 py-4 font-mono font-bold text-sky-300">{exchange.code}</td>
            <td className="px-6 py-4 text-white font-semibold">{exchange.name}</td>
            <td className="px-6 py-4">
              <span className="px-2.5 py-0.5 rounded-full text-[10px] font-bold ${exchange.isActive ? 'bg-emerald-500/15 text-emerald-400
border border-emerald-500/25' : 'bg-red-500/15 text-red-400 border border-red-500/25'}">
                {exchange.isActive ? t('admin.active') : t('admin.inactive')}
              </span>
            </td>
            <td className="px-6 py-4">
              <div className="flex justify-end gap-2">
                <button type="button" onClick={() => setEditingExchange({ ...exchange })} className="px-3 py-1.5 rounded-lg border border-
sky-500/25 bg-sky-500/10 text-sky-300 hover:bg-sky-500/20 font-bold">{t('admin.edit')}</button>
                <button type="button" onClick={() => handleDeleteExchange(exchange)} className="p-1.5 rounded-lg border border-red-
500/25 bg-red-500/10 text-red-400 hover:bg-red-500/20" title={t('admin.deleteExchange')}><Trash2 className="w-4 h-4" /></button>
              </div>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
</div>

{editingExchange && (
  <div className="fixed inset-0 z-[100] bg-black/70 backdrop-blur-sm grid place-items-center p-4">
    <form onSubmit={handleUpdateExchange} className="w-full max-w-md bg-[#161b22] border border-[#30363d] rounded-2xl p-5 shadow-2xl
space-y-4">
      <div className="flex items-center justify-between">
        <h3 className="text-sm font-black text-white">{t('admin.editExchange')}</h3>
        <button type="button" onClick={() => setEditingExchange(null)} className="p-1.5 text-gray-400 hover:text-white"><X className="w-4 h-
4" /></button>
      </div>
      <input value={editingExchange.code} onChange={(e) => setEditingExchange(v => v ? ({ ...v, code: e.target.value.toUpperCase() }) : v)}
maxLength={20} className="w-full px-3 py-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-xs text-white font-mono" />
      <input value={editingExchange.name} onChange={(e) => setEditingExchange(v => v ? ({ ...v, name: e.target.value }) : v)} maxLength={50}
className="w-full px-3 py-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-xs text-white" />
      <label className="flex items-center gap-2 text-xs text-gray-300"><input type="checkbox" checked={editingExchange.isActive}
onChange={(e) => setEditingExchange(v => v ? ({ ...v, isActive: e.target.checked }) : v)} />{t('admin.active')}</label>
      <div className="flex justify-end gap-2 pt-2">
        <button type="button" onClick={() => setEditingExchange(null)} className="px-4 py-2 rounded-xl border border-[#30363d] text-gray-300
text-xs font-bold">{t('admin.cancel')}</button>
        <button type="submit" disabled={savingExchange} className="px-4 py-2 rounded-xl bg-sky-600 text-white text-xs font-black flex items-
center gap-1.5 disabled:opacity-50">{savingExchange ? <Loader2 className="w-4 h-4 animate-spin" /> : <Save className="w-4 h-4"
/>}{t('admin.saveExchange')}</button>
      </div>
    </form>
  </div>
)}

{t('5. Fane til profil og sikkerhed')}
{activeTab === 'profile' && (
  <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
    {t('Formular til profiloplysninger')}
    <div className="bg-[#161b22] border border-[#30363d] rounded-2xl p-6 shadow-xl space-y-4">
      <div className="flex items-center space-x-2 pb-3 border-b border-[#30363d]">
        <UserIcon className="w-5 h-5 text-amber-400" />

```

```
<h3 className="font-bold text-sm text-white">{t('admin.profileTitle')}</h3>
</div>

<form onSubmit={handleProfileSubmit} className="space-y-4 text-xs">
  <div className="grid grid-cols-2 gap-3">
    <div>
      <label className="block text-gray-400 font-medium mb-1">{t('auth.firstName')}</label>
      <input
        type="text"
        required
        value={profileFormular.firstName}
        onChange={(e) => setProfileFormular({ ...profileFormular, firstName: e.target.value })}
        className="w-full p-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-white outline-none focus:border-amber-500"
      />
    </div>
    <div>
      <label className="block text-gray-400 font-medium mb-1">{t('auth.lastName')}</label>
      <input
        type="text"
        required
        value={profileFormular.lastName}
        onChange={(e) => setProfileFormular({ ...profileFormular, lastName: e.target.value })}
        className="w-full p-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-white outline-none focus:border-amber-500"
      />
    </div>
  </div>
  <div>
    <label className="block text-gray-400 font-medium mb-1">{t('auth.username')}</label>
    <input
      type="text"
      required
      value={profileFormular.username}
      onChange={(e) => setProfileFormular({ ...profileFormular, username: e.target.value })}
      className="w-full p-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-white outline-none focus:border-amber-500"
    />
  </div>
  <div>
    <label className="block text-gray-400 font-medium mb-1">{t('auth.email')}</label>
    <input
      type="email"
      required
      value={profileFormular.email}
      onChange={(e) => setProfileFormular({ ...profileFormular, email: e.target.value })}
      className="w-full p-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-white outline-none focus:border-amber-500"
    />
  </div>
  <button
    type="submit"
    disabled={savingProfile}
    className="w-full py-2.5 bg-amber-600 hover:bg-amber-500 active:bg-amber-700 text-white font-bold rounded-xl transition-colors flex
items-center justify-center space-x-2 shadow-lg shadow-amber-600/20 disabled:opacity-50"
  >
    <Save className="w-4 h-4" />
    <span>{savingProfile ? t('admin.saving') : t('admin.updateProfileBtn')}</span>
  </button>
</form>
</div>

{ /* Formular til ændring af adgangskode */ }
<div className="bg-[#161b22] border border-[#30363d] rounded-2xl p-6 shadow-xl space-y-4">
  <div className="flex items-center space-x-2 pb-3 border-b border-[#30363d]">
    <Lock className="w-5 h-5 text-pink-400" />
    <h3 className="font-bold text-sm text-white">{t('admin.securityTitle')}</h3>
  </div>

  <form onSubmit={handlePasswordSubmit} className="space-y-4 text-xs">
    <div>
      <label className="block text-gray-400 font-medium mb-1">{t('admin.currentPassword')}</label>
      <input
        type="password"
        required
        placeholder={t('admin.passwordPlaceholder', '*****')}
        value={passwordFormular.currentPassword}
        onChange={(e) => setPasswordFormular({ ...passwordFormular, currentPassword: e.target.value })}
        className="w-full p-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-white outline-none focus:border-pink-500"
      />
    </div>
    <div>
      <label className="block text-gray-400 font-medium mb-1">{t('admin.newPassword')}</label>
      <input
```

```

        type="password"
        required
        placeholder={t('admin.min6chars')}
        value={passwordFormular.newPassword}
        onChange={(e) => setPasswordFormular({ ...passwordFormular, newPassword: e.target.value })}
        className="w-full p-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-white outline-none focus:border-pink-500"
      />
    </div>

    <div>
      <label className="block text-gray-400 font-medium mb-1">{t('admin.confirmNewPassword')}</label>
      <input
        type="password"
        required
        placeholder={t('admin.min6chars')}
        value={passwordFormular.confirmPassword}
        onChange={(e) => setPasswordFormular({ ...passwordFormular, confirmPassword: e.target.value })}
        className="w-full p-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-white outline-none focus:border-pink-500"
      />
    </div>

    <button
      type="submit"
      disabled={savingPassword}
      className="w-full py-2.5 bg-pink-600 hover:bg-pink-500 active:bg-pink-700 text-white font-bold rounded-xl transition-colors flex items-center justify-center space-x-2 shadow-lg shadow-pink-600/20 disabled:opacity-50"
    >
      <KeyRound className="w-4 h-4" />
      <span>{savingPassword ? t('admin.updating') : t('admin.changePasswordBtn')}</span>
    </button>
  </form>
</div>
</div>
</div>
</div>

{ /* Modal til oprettelse af ny admin */ }
{isAddAdminOpen && (
  <div
    role="dialog"
    aria-modal="true"
    onClick={() => { setisAddAdminOpen(false); setAdminModalError(null); }}
    className="fixed inset-0 z-50 flex items-center justify-center bg-black/75 backdrop-blur-xs p-4 animate-in fade-in duration-150"
  >
    <div
      onClick={(e) => e.stopPropagation()}
      className="w-full max-w-md bg-[#161b22] border border-[#30363d] rounded-2xl p-6 shadow-2xl space-y-4"
    >
      <div className="flex items-center justify-between pb-3 border-b border-[#30363d]">
        <div className="flex items-center space-x-2">
          <ShieldCheck className="w-5 h-5 text-purple-400" />
          <h3 className="font-bold text-sm text-white">{t('admin.modalNewAdminTitle')}</h3>
        </div>
        <button
          type="button"
          onClick={() => { setisAddAdminOpen(false); setAdminModalError(null); }}
          className="text-gray-400 hover:text-white p-1"
        >
          <X className="w-4 h-4" />
        </button>
      </div>

      <div>
        <div className="p-3 bg-red-500/15 border border-red-500/30 rounded-xl text-red-400 flex items-start space-x-2 animate-in fade-in text-xs">
          <AlertCircle className="w-4 h-4 shrink-0 mt-0.5" />
          <span className="font-medium leading-relaxed">{adminModalError}</span>
        </div>
      </div>
    </div>

    <form onSubmit={handleCreateAdminSubmit} className="space-y-3.5 text-xs">
      <div className="grid grid-cols-2 gap-3">
        <div>
          <label className="block text-gray-400 font-medium mb-1">{t('auth.firstName')}</label>
          <input
            type="text"
            required
            value={adminFormular.firstName}
            onChange={(e) => setAdminFormular({ ...adminFormular, firstName: e.target.value })}
            className="w-full p-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-white outline-none focus:border-purple-500"
          />
        </div>
      </div>
    </form>
  </div>
) }

```

```

<label className="block text-gray-400 font-medium mb-1">{t('auth.lastName')}</label>
<input
  type="text"
  required
  value={adminFormular.lastName}
  onChange={(e) => setAdminFormular({ ...adminFormular, lastName: e.target.value })}
  className="w-full p-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-white outline-none focus:border-purple-500"
/>
</div>
</div>

<div>
<label className="block text-gray-400 font-medium mb-1">{t('auth.username')}</label>
<div className="relative">
  <UserIcon className="w-3.5 h-3.5 absolute left-3 top-3 text-gray-500" />
  <input
    type="text"
    required
    value={adminFormular.username}
    onChange={(e) => setAdminFormular({ ...adminFormular, username: e.target.value })}
    className="w-full pl-9 pr-3 py-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-white outline-none focus:border-purple-500"
  />
</div>
</div>

<div>
<label className="block text-gray-400 font-medium mb-1">{t('auth.email')}</label>
<div className="relative">
  <Mail className="w-3.5 h-3.5 absolute left-3 top-3 text-gray-500" />
  <input
    type="email"
    required
    value={adminFormular.email}
    onChange={(e) => setAdminFormular({ ...adminFormular, email: e.target.value })}
    className="w-full pl-9 pr-3 py-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-white outline-none focus:border-purple-500"
  />
</div>
</div>

<div>
<label className="block text-gray-400 font-medium mb-1">{t('auth.password')}</label>
<div className="relative">
  <Lock className="w-3.5 h-3.5 absolute left-3 top-3 text-gray-500" />
  <input
    type="password"
    required
    placeholder={t('admin.passwordPlaceholder', '*****')}
    value={adminFormular.password}
    onChange={(e) => setAdminFormular({ ...adminFormular, password: e.target.value })}
    className="w-full pl-9 pr-3 py-2.5 bg-[#0d1117] border border-[#30363d] rounded-xl text-white outline-none focus:border-purple-500"
  />
</div>
</div>

<div className="pt-2 flex items-center space-x-2">
  <button
    type="submit"
    disabled={creatingAdmin}
    className="flex-1 py-2.5 bg-purple-600 hover:bg-purple-500 active:bg-purple-700 text-white font-bold rounded-xl transition-colors
disabled:opacity-50 flex items-center justify-center space-x-1.5 shadow-lg shadow-purple-600/25"
  >
    {creatingAdmin ? (
      <>
        <Loader2 className="w-3.5 h-3.5 animate-spin" />
        <span>{t('admin.creating')}</span>
      </>
    ) : (
      <span>{t('admin.createAdminSubmitBtn')}</span>
    )}
  </button>
  <button
    type="button"
    onClick={() => { setIsAddAdminOpen(false); setAdminModalError(null); }}
    className="flex-1 py-2.5 bg-[#0d1117] hover:bg-[#21262d] text-gray-300 font-medium rounded-xl border border-[#30363d] transition-colors"
  >
    {t('admin.cancel')}
  </button>
</div>
</form>
</div>
</div>
)}
</div>
);
};

```

F18. src/Pages/AuthPage.tsx

Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.

```
import React, { useMemo, useState } from 'react';
import { useTranslation } from 'react-i18next';
import {
  AlertCircle,
  ArrowLeft,
  ArrowRight,
  Check,
  CheckCircle2,
  Globe,
  Lock,
  Mail,
  ShieldCheck,
  TrendingUp,
  User,
} from 'lucide-react';
import { authService } from '../Services/authService';
import type { User as AuthUser } from '../Types/auth';

interface AuthPageProps {
  onLoginSuccess: (user: AuthUser) => void;
}

type AuthMode = 'login' | 'register' | 'forgot' | 'reset';

// Henter de nødvendige data til denne funktion.
const getApiErrorMessage = (err: any, fallback: string, connectionError: string) => {
  if (err?.response?.data?.message) return err.response.data.message as string;

  const validationErrors = err?.response?.data?.errors;
  if (validationErrors && typeof validationErrors === 'object') {
    const firstError = Object.values(validationErrors).flat()[0];
    if (typeof firstError === 'string') return firstError;
  }

  if (err?.request) return connectionError;
  return fallback;
};

export const AuthPage: React.FC<AuthPageProps> = ({ onLoginSuccess }) => {
  const { t, i18n } = useTranslation();

  const resetQuery = useMemo(() => {
    const params = new URLSearchParams(window.location.search);
    const token = params.get('resetToken') ?? '';
    const userId = Number(params.get('userId') ?? 0);
    return { token, userId };
  }, []);

  const [mode, setMode] = useState<AuthMode>({
    resetQuery.token && resetQuery.userId > 0 ? 'reset' : 'login'
  });
  const [error, setError] = useState<string | null>(null);
  const [success, setSuccess] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);
  const [isLangMenuOpen, setIsLangMenuOpen] = useState(false);
  const [formData, setFormularData] = useState({
    firstName: '',
    lastName: '',
    username: '',
    email: '',
    identifier: '',
    password: '',
    newPassword: '',
    confirmPassword: '',
  });

  // Håndterer set auth mode.
  const setAuthMode = (newMode: AuthMode) => {
    setMode(newMode);
    setError(null);
    setSuccess(null);
  };

  // Behandler den relevante brugerhandling eller event.
  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setFormularData((prev) => ({ ...prev, [e.target.name]: e.target.value }));
  };
};
```

```
// Behandler den relevante brugerhandling eller event.
const handleLanguageChange = (lang: 'tr' | 'en' | 'da') => {
  i18n.changeLanguage(lang);
  localStorage.setItem('tradingpro_lang', lang);
  setIsLangMenuOpen(false);
};

const currentLangLabel = i18n.language.startsWith('da')
  ? 'DK DA'
  : i18n.language.startsWith('en')
    ? 'GB EN'
    : 'TR TR';

// Behandler den relevante brugerhandling eller event.
const handleSubmit = async (e: React.FormularEvent) => {
  e.preventDefault();
  setError(null);
  setSuccess(null);
  setLoading(true);

  try {
    if (mode === 'forgot') {
      const result = await authService.forgotPassword({
        identifier: formData.identifier.trim(),
      });
      setSuccess(result.message || t('auth.forgotSentFallback'));
      return;
    }

    if (mode === 'reset') {
      if (formData.newPassword.length < 8) {
        setError(t('auth.resetMinLength'));
        return;
      }
      if (formData.newPassword !== formData.confirmPassword) {
        setError(t('auth.resetMismatch'));
        return;
      }

      const result = await authService.resetPassword({
        userId: resetQuery.userId,
        token: resetQuery.token,
        newPassword: formData.newPassword,
      });
      setSuccess(result.message || t('auth.resetSuccessFallback'));
      window.history.replaceState({}, document.title, window.location.pathname);
      setTimeout(() => setAuthMode('login'), 1000);
      return;
    }

    if (mode === 'register') {
      await authService.register({
        firstName: formData.firstName.trim(),
        lastName: formData.lastName.trim(),
        username: formData.username.trim(),
        email: formData.email.trim(),
        password: formData.password,
      });

      const loginIdentifier = formData.email.trim() || formData.username.trim();
      setSuccess(t('auth.registerSuccess'));
      setFormularData((prev) => ({
        ...prev,
        identifier: loginIdentifier,
        password: "",
        newPassword: "",
        confirmPassword: "",
      }));

      window.setTimeout(() => {
        setMode('login');
        setError(null);
        // Behold succesbeskeden synlig efter skift til loginformularen.
      }, 1400);
      return;
    }

    const authResponse = await authService.login({
      identifier: formData.identifier.trim(),
      password: formData.password,
    });
  }
};
```

```

    onLoginSuccess({
      username: authResponse.username,
      email: authResponse.email,
      role: authResponse.role,
      firstName: authResponse.firstName,
      lastName: authResponse.lastName,
    });
  } catch (err: any) {
    console.error('Kimlik doğrulama hatası:', err);
    setError(getApiErrorMessage(err, t('auth.operationFailed'), t('auth.connectionError')));
  } finally {
    setLoading(false);
  }
}

const title = mode === 'register'
  ? t('auth.createAccountTitle')
  : mode === 'forgot'
  ? t('auth.forgotTitle')
  : mode === 'reset'
  ? t('auth.resetTitle')
  : t('auth.welcomeTitle');

const subtitle = mode === 'register'
  ? t('auth.registerSubtitle')
  : mode === 'forgot'
  ? t('auth.forgotSubtitle')
  : mode === 'reset'
  ? t('auth.resetSubtitle')
  : t('auth.loginIdentifierSubtitle');

return (
  <div className="flex min-h-screen w-full bg-[#0b0e14] text-white font-sans overflow-hidden relative">
    <div className="absolute top-4 right-4 z-50">
      <div className="relative">
        <button
          type="button"
          onClick={() => setIsLangMenuOpen((v) => !v)}
          className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg border border-gray-700/80 bg-[#151a23]/90 backdrop-blur-md text-xs font-
semibold text-gray-200 hover:border-gray-500 transition-all shadow-lg"
        >
          <Globe className="w-3.5 h-3.5 text-blue-400" />
          <span>{currentLangLabel}</span>
        </button>

        {isLangMenuOpen && (
          <div className="absolute top-full right-0 mt-1.5 w-32 rounded-xl bg-[#151a23] border border-gray-700 shadow-2xl p-1.5 space-y-0.5 z-50 text-xs">
            {
              ['tr', 'TR Türkçe'],
              ['en', 'GB English'],
              ['da', 'DK Dansk'],
            }.map(([lang, label]) => (
              <button
                key={lang}
                type="button"
                onClick={() => handleLanguageChange(lang as 'tr' | 'en' | 'da')}
                className={`w-full flex items-center justify-between px-2.5 py-1.5 rounded-lg font-medium transition-colors ${i18n.language.startsWith(lang)
                  ? 'bg-blue-600/20 text-blue-400'
                  : 'text-gray-300 hover:bg-gray-800'
                }`}
              >
                <span>{label}</span>
                {i18n.language.startsWith(lang) && <Check className="w-3 h-3" />}
              </button>
            )
          )
        )
      </div>
    </div>

    <div
      className="hidden lg:flex lg:w-3/5 relative bg-cover bg-center"
      style={{
        backgroundImage: `url('https://images.unsplash.com/photo-1611974789855-9c2a0a7236a3?q=80&w=1920&auto=format&fit=crop')`,
      }}
    >
      <div className="absolute inset-0 bg-gradient-to-r from-black/85 via-black/55 to-[#0b0e14]" />
      <div className="relative z-10 flex flex-col justify-between p-12">
        <div className="flex items-center space-x-3">
          <div className="p-2 bg-blue-600 rounded-lg shadow-md">
            <TrendingUp className="h-8 w-8 text-white" />
          </div>
          <span className="text-2xl font-bold tracking-wider">
            TRADING<span className="text-blue-500">PRO</span>
          </span>
        </div>
      </div>
    </div>
  </div>
)

```

```
</span>
</div>

<div className="max-w-xl space-y-5">
  <div className="inline-flex items-center gap-2 rounded-full border border-blue-500/20 bg-blue-500/10 px-3 py-1 text-xs text-blue-300">
    <ShieldCheck className="h-3.5 w-3.5" />
    {t('auth.secureAccess')}
  </div>
  <h1 className="text-4xl font-extrabold leading-tight tracking-wide">
    {t('auth.heroTitle', 'Gerçek Zamanlı Piyasa Verileri ve Gelişmiş Grafik Platformu')}
  </h1>
  <p className="text-gray-400 text-lg leading-relaxed">
    {t('auth.heroDescription', 'SignalR ile anlık mum verilerini takip edin, indikatörlerle analizinizi güçlendirin ve portföyünüzü tek ekrandan yönetin.')}
  </p>
</div>

<div className="text-sm text-gray-500">{t('auth.copyrightShort')}
```



```

        {mode === 'login' && (
          <
            <TextInput icon={<Lock />} type="password" name="password" label={t('auth.password')} value={formData.password}
            onChange={handleChange} placeholder="*****" />
            <div className="flex justify-end -mt-1">
              <button
                type="button"
                onClick={() => setAuthMode('forgot')}
                className="text-xs font-medium text-blue-400 hover:text-blue-300 transition"
              >
                {t('auth.forgotPassword')}
              </button>
            </div>
          </>
        )}

        {mode === 'reset' && (
          <
            <TextInput icon={<Lock />} type="password" name="newPassword" label={t('auth.newPassword')} value={formData.newPassword}
            onChange={handleChange} placeholder={t('auth.passwordMinPlaceholder')} minLength={8} />
            <TextInput icon={<Lock />} type="password" name="confirmPassword" label={t('auth.confirmNewPassword')}
            value={formData.confirmPassword} onChange={handleChange} placeholder={t('auth.confirmNewPasswordPlaceholder')} minLength={8} />
          </>
        )}

        <button
          type="submit"
          disabled={loading}
          className="w-full py-3 bg-blue-600 hover:bg-blue-500 active:bg-blue-700 text-white font-medium rounded-lg transition flex items-center justify-
center space-x-2 disabled:opacity-50 shadow-md"
        >
          <span>
            {loading
              ? t('auth.processing')
              : mode === 'register'
                ? t('auth.register')
                : mode === 'forgot'
                  ? t('auth.sendResetLink')
                  : mode === 'reset'
                    ? t('auth.updatePassword')
                    : t('auth.login')}
            </span>
          <ArrowRight className="h-4 w-4" />
        </button>
      </form>

      {(mode === 'login' || mode === 'register') && (
        <div className="text-center pt-2 border-t border-gray-800/70">
          <button
            type="button"
            onClick={() => setAuthMode(mode === 'register' ? 'login' : 'register')}
            className="text-sm text-gray-400 hover:text-blue-400 transition pt-4"
          >
            {mode === 'register'
              ? t('auth.hasAccount')
              : t('auth.noAccount')}
          </button>
        </div>
      )}
    </div>
  </div>
</div>
);
};

interface TextInputProps {
  name: string;
  label: string;
  value: string;
  onChange: (e: React.ChangeEvent<HTMLInputElement>) => void;
  type?: string;
  placeholder?: string;
  icon?: React.ReactNode;
  minLength?: number;
}

const TextInput: React.FC<TextInputProps> = ({
  name,
  label,
  value,
  onChange,
  type = 'text',
  placeholder,

```

```

    icon,
    minLength,
  }) => (
    <div>
      <label className="block text-xs font-medium text-gray-400 mb-1">{label}</label>
      <div className="relative">
        {icon && <span className="absolute left-3 top-2.5 h-4 w-4 text-gray-500 [&>svg]:h-4 [&>svg]:w-4">{icon}</span>}
        <input
          type={type}
          name={name}
          required
          minLength={minLength}
          placeholder={placeholder}
          value={value}
          onChange={onChange}
          autoComplete={type === 'password' ? 'current-password' : undefined}
          className={`w-full ${icon ? 'pl-9' : 'px-3'} pr-3 py-2 bg-#0b0e14 border border-gray-700 rounded-lg focus:outline-none focus:border-blue-500 text-sm text-white transition-colors`}
        />
      </div>
    </div>
  );

```

F19. src/Services/aiApi.ts

Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.

```

import { api } from './api';
import type { AiAnalysisResponseDto } from '../Types/ai';

const normalizeLanguage = (language?: string): 'tr' | 'en' | 'da' => {
  const value = (language || 'en').toLowerCase();
  if (value.startsWith('tr')) return 'tr';
  if (value.startsWith('da')) return 'da';
  return 'en';
};

export const aiApi = {
  getDetailedAnalysis: async (
    symbolId: number,
    interval: string,
    language?: string,
  ): Promise<AiAnalysisResponseDto> => {
    const normalizedLanguage = normalizeLanguage(language);

    const response = await api.post<AiAnalysisResponseDto>(
      '/AiAnalysis/analyze',
      {
        symbolId,
        interval,
        language: normalizedLanguage,
      },
      {
        headers: { 'Accept-Language': normalizedLanguage },
      },
    );

    return response.data;
  },
};

```

F20. src/Services/alertService.ts

Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.

```

import { api } from './api';
import type { Alert, CreateAlertRequest } from '../Types/alert';

export const alertService = {
  getMyAlerts: async (): Promise<Alert[]> => {
    const response = await api.get<Alert[]>('/alerts');
    return response.data;
  },

  getActiveAlertsBySymbol: async (symbolId: number): Promise<Alert[]> => {
    const response = await api.get<Alert[]>('/alerts/active', { params: { symbolId } });
    return response.data;
  },
};

```

```
createAlert: async (data: CreateAlertRequest): Promise<Alert> => {
  const response = await api.post<Alert>('/alerts', data);
  return response.data;
},

deleteAlert: async (id: string): Promise<void> => {
  await api.delete(`/alerts/${id}`);
},

toggleAlert: async (id: string): Promise<void> => {
  await api.patch(`/alerts/${id}/toggle`);
},
};
```

F21. src/Services/api.ts

Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.

```
import axios from 'axios';

const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || 'https://localhost:7143/api';

export const api = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Hjælper: Ryd sessionen, og naviger videre.
const handleLogout = () => {
  localStorage.removeItem('token');
  localStorage.removeItem('user');

  // Undgå en uendelig redirect-løkke, hvis vi allerede er på login-siden.
  if (window.location.pathname !== '/login' && window.location.pathname !== '/') {
    window.location.href = '/login';
  }
};

// 1. Request-interceptor
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// 2. Response-interceptor
api.interceptors.response.use(
  (response) => {
    // Marker globalt, at backend-forbindelsen er aktiv, når requesten lykkes.
    window.dispatchEvent(new CustomEvent('backend-connection-restored'));
    return response;
  },
  (error) => {
    // 🚨 SERVEREN SVARER IKKE (backend er nede / netværksfejl)
    if (!error.response) {
      window.dispatchEvent(new CustomEvent('backend-connection-error', {
        detail: { message: 'The server is unreachable. The backend service may be down.' }
      }));
      return Promise.reject(error);
    }

    const status = error.response?.status;
    const originalRequest = error.config;

    // Redirect kun ved 401 fra andre requests end /login eller /auth.
    const isAuthRequest = originalRequest.url?.includes('/login') || originalRequest.url?.includes('/auth');

    if (status === 401 && !isAuthRequest) {
      handleLogout();
    }

    return Promise.reject(error);
  }
);
```

```
}  
);
```

F22. src/Services/authService.ts

Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.

```
import { api } from './api';  
import type {  
  AuthResponse,  
  ForgotPasswordRequest,  
  LoginRequest,  
  MessageResponse,  
  RegisterRequest,  
  ResetPasswordRequest,  
  User,  
} from '../Types/auth';  
  
// Håndterer persist auth.  
const persistAuth = (response: AuthResponse) => {  
  if (!response.token) return;  
  
  localStorage.setItem('token', response.token);  
  localStorage.setItem(  
    'user',  
    JSON.stringify({  
      username: response.username,  
      email: response.email,  
      role: response.role,  
      firstName: response.firstName,  
      lastName: response.lastName,  
    })  
  );  
};  
  
export const authService = {  
  login: async (data: LoginRequest): Promise<AuthResponse> => {  
    const response = await api.post<AuthResponse>('/Auth/login', data);  
    persistAuth(response.data);  
    return response.data;  
  },  
  
  register: async (data: RegisterRequest): Promise<AuthResponse> => {  
    const response = await api.post<AuthResponse>('/Auth/register', data);  
    // Registrering skal ikke oprette en lokal session. Brugeren får vist en succesbesked  
    // og logger derefter selv ind.  
    return response.data;  
  },  
  
  forgotPassword: async (data: ForgotPasswordRequest): Promise<MessageResponse> => {  
    const response = await api.post<MessageResponse>('/Auth/forgot-password', data);  
    return response.data;  
  },  
  
  resetPassword: async (data: ResetPasswordRequest): Promise<MessageResponse> => {  
    const response = await api.post<MessageResponse>('/Auth/reset-password', data);  
    return response.data;  
  },  
  
  logout: () => {  
    localStorage.removeItem('token');  
    localStorage.removeItem('user');  
  },  
  
  getCurrentUser: (): User | null => {  
    const userStr = localStorage.getItem('user');  
    return userStr ? JSON.parse(userStr) : null;  
  },  
  
  isAuthenticated: (): boolean => !!localStorage.getItem('token'),  
  
  isAdmin: (): boolean => {  
    const user = authService.getCurrentUser();  
    const role = (user?.role || '').toLowerCase();  
    return role === 'admin' || role === 'superadmin';  
  },  
  
  isSuperAdmin: (): boolean => {  
    const user = authService.getCurrentUser();  
    return (user?.role || '').toLowerCase() === 'superadmin';  
  },
```

```
    },  
  };  
};
```

F23. src/Services/candleApi.ts

Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.

```
import { api } from './api';  
import type { CandleData } from '../Types/candle';  
  
export const candleApi = {  
  getHistoricalCandles: async (  
    symbolId: number,  
    interval: string = '1m',  
    limit: number = 500,  
    endTime?: number  
  ): Promise<CandleData[]> => {  
    if (!symbolId) return [];  
    const response = await api.get<CandleData[]>('/candles', {  
      params: { symbolId, interval, limit, endTime },  
    });  
    return response.data;  
  },  
};
```

F24. src/Services/chartDrawingService.ts

Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.

```
import { api } from './api';  
import type {  
  ChartDrawingDto,  
  CreateChartDrawingDto,  
  UpdateChartDrawingDto,  
} from '../Types/chartDrawing';  
  
export const chartDrawingService = {  
  getDrawings: async (symbolId: number, interval: string): Promise<ChartDrawingDto[]> => {  
    if (!symbolId || !interval) return [];  
    const response = await api.get<ChartDrawingDto[]>('/chart-drawings', {  
      params: { symbolId, interval },  
    });  
    return response.data;  
  },  
  
  createDrawing: async (dto: CreateChartDrawingDto): Promise<ChartDrawingDto> => {  
    const response = await api.post<ChartDrawingDto>('/chart-drawings', dto);  
    return response.data;  
  },  
  
  updateDrawing: async (drawingId: number, dto: UpdateChartDrawingDto): Promise<ChartDrawingDto> => {  
    const response = await api.put<ChartDrawingDto>('/chart-drawings/${drawingId}', dto);  
    return response.data;  
  },  
  
  deleteDrawing: async (drawingId: number): Promise<void> => {  
    await api.delete('/chart-drawings/${drawingId}');  
  },  
  
  clearDrawings: async (symbolId: number, interval: string): Promise<number> => {  
    const response = await api.delete<{ deletedCount: number }>('/chart-drawings', {  
      params: { symbolId, interval },  
    });  
    return response.data.deletedCount;  
  },  
};
```

F25. src/Services/exchangeApi.ts

Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.

```
import { api } from './api';  
import type { ExchangeResponseDto } from '../Types/symbol';  
  
export interface ExchangeUpsertDto {  
  code: string;  
  name: string;
```

```
    isActive: boolean;
  }

export const exchangeApi = {
  getActiveExchanges: async (): Promise<ExchangeResponseDto[]> => {
    const response = await api.get<ExchangeResponseDto[]>('/exchanges');
    return response.data;
  },

  getAdminExchanges: async (): Promise<ExchangeResponseDto[]> => {
    const response = await api.get<ExchangeResponseDto[]>('/exchanges/admin');
    return response.data;
  },

  createExchange: async (payload: ExchangeUpsertDto): Promise<ExchangeResponseDto> => {
    const response = await api.post<ExchangeResponseDto>('/exchanges', payload);
    return response.data;
  },

  updateExchange: async (id: number, payload: ExchangeUpsertDto): Promise<ExchangeResponseDto> => {
    const response = await api.put<ExchangeResponseDto>(`/exchanges/${id}`, payload);
    return response.data;
  },

  deleteExchange: async (id: number): Promise<void> => {
    await api.delete(`/exchanges/${id}`);
  },
};
```

F26. src/Services/preferenceService.ts

Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.

```
import { api } from './api';
import type { UserPreferences } from '../Types/preferences';

export const preferenceService = {
  getPreferences: async (): Promise<UserPreferences> => {
    const response = await api.get<UserPreferences>('/users/preferences');
    return response.data;
  },

  savePreferences: async (preferences: UserPreferences): Promise<void> => {
    await api.put('/users/preferences', preferences);
  },
};
```

F27. src/Services/signalRService.ts

Frontendens SignalR-klient. Håndterer forbindelse, reconnect, symbolgrupper og reference counting mellem eksempelvis chart og watchlist.

```
import * as signalR from '@microsoft/signalr';
import type { CandleData } from '../Types/candle';
import type { Alert } from '../Types/alert';

const SIGNALR_HUB_URL = import.meta.env.VITE_SIGNALR_HUB_URL || 'https://localhost:7143/hubs/market-data';

class SignalRService {
  private connection: signalR.HubConnection | null = null;
  private candleListeners: Set<(candle: CandleData) => void> = new Set();
  private alertListeners: Set<(alert: Alert) => void> = new Set();
  private isListening: boolean = false;
  private subscribedGroups: Set<string> = new Set();
  private groupRefCountes: Map<string, number> = new Map();

  public getConnection(): signalR.HubConnection | null {
    return this.connection;
  }

  public async startConnection(): Promise<void> {
    if (this.connection && this.connection.state === signalR.HubConnectionState.Connected) {
      return;
    }

    if (!this.connection) {
      this.connection = new signalR.HubConnectionBuilder()
        .withUrl(SIGNALR_HUB_URL, {
          accessTokenFactory: () => localStorage.getItem('token') || '',
        })
        .build();
    }
  }
}
```

```

    })
    .withAutomaticReconnect()
    .configureLogging(signalR.LogLevel.Information)
    .build();

this.connection.onreconnected(async () => {
    // SignalR-grupper er knyttet til forbindelsen. Tilslut alle aktive grupper igen efter reconnect.
    const groups = Array.from(this.subscribedGroups);
    for (const groupKey of groups) {
        const separator = groupKey.indexOf(':');
        if (separator <= 0) continue;
        const exchange = groupKey.slice(0, separator);
        const symbol = groupKey.slice(separator + 1);
        try {
            await this.connection?.invoke('SubscribeToSymbol', exchange, symbol);
        } catch (err) {
            console.error('[SignalR] Yeniden grup aboneliği başarısız:', groupKey, err);
        }
    }
});
}

if (!this.isListening) {
    // Lyt efter realtime candle-data.
    this.connection.on('ReceiveMarketData', (candle: CandleData) => {
        this.candleListeners.forEach((listener) => {
            try {
                listener(candle);
            } catch (err) {
                console.error('[SignalR] Mum dinleyici hatası:', err);
            }
        });
    });

    // Lyt efter realtime udløste prisalarmer.
    this.connection.on('ReceiveAlertTriggered', (alert: Alert) => {
        this.alertListeners.forEach((listener) => {
            try {
                listener(alert);
            } catch (err) {
                console.error('[SignalR] Alarm dinleyici hatası:', err);
            }
        });
    });

    this.isListening = true;
}

try {
    if (this.connection.state === signalR.HubConnectionState.Disconnected) {
        await this.connection.start();
        console.log('SignalR Bağlantısı Başarılı!');
    }
} catch (err) {
    console.error('SignalR Bağlantı Hatası:', err);
    setTimeout(() => this.startConnection(), 5000);
}

public async subscribeToSymbolGroup(exchangeCode: string, symbolName: string): Promise<void> {
    const exchange = exchangeCode?.trim().toUpperCase();
    const symbol = symbolName?.trim().toUpperCase();

    if (!exchange || !symbol) {
        console.warn('[SignalR] Geçersiz grup aboneliği atlandı.', { exchangeCode, symbolName });
        return;
    }

    const groupKey = `${exchange}:${symbol}`;
    const currentRefs = this.groupRefCounts.get(groupKey) ?? 0;
    this.groupRefCounts.set(groupKey, currentRefs + 1);

    // En anden chart/watchlist-forbruger bruger allerede serverabonnementet.
    if (currentRefs > 0 && this.subscribedGroups.has(groupKey)) return;

    if (!this.connection || this.connection.state !== signalR.HubConnectionState.Connected) {
        await this.startConnection();
    }

    if (this.connection && this.connection.state === signalR.HubConnectionState.Connected) {
        try {
            await this.connection.invoke('SubscribeToSymbol', exchange, symbol);
        }
    }
}

```

```

        this.subscribedGroups.add(groupKey);
        console.log(`${groupKey} grubuna katılım sağlandı.`);
    } catch (err) {
        const refs = (this.groupRefCounts.get(groupKey) ?? 1) - 1;
        if (refs <= 0) this.groupRefCounts.delete(groupKey);
        else this.groupRefCounts.set(groupKey, refs);
        console.error('Gruba katılırken hata:', err);
    }
}

public async unsubscribeFromSymbolGroup(exchangeCode: string, symbolName: string): Promise<void> {
    const exchange = exchangeCode?.trim().toUpperCase();
    const symbol = symbolName?.trim().toUpperCase();

    if (!exchange || !symbol) return;

    const groupKey = `${exchange}:${symbol}`;
    const currentRefs = this.groupRefCounts.get(groupKey) ?? 0;

    if (currentRefs > 1) {
        this.groupRefCounts.set(groupKey, currentRefs - 1);
        return;
    }

    this.groupRefCounts.delete(groupKey);
    if (!this.subscribedGroups.has(groupKey)) return;

    if (this.connection && this.connection.state === signalR.HubConnectionState.Connected) {
        try {
            await this.connection.invoke('UnsubscribeFromSymbol', exchange, symbol);
        } catch (err) {
            console.error('Gruptan ayrılırken hata:', err);
        }
    }

    this.subscribedGroups.delete(groupKey);
}

// Abonnement på candle-data
public subscribeToCandleUpdates(callback: (candle: CandleData) => void): void {
    this.candleListeners.add(callback);
}

public unsubscribeFromCandleUpdates(callback?: (candle: CandleData) => void): void {
    if (callback) {
        this.candleListeners.delete(callback);
    } else {
        this.candleListeners.clear();
    }
}

// Abonnement på udløste alarmer
public subscribeToAlertTriggered(callback: (alert: Alert) => void): void {
    this.alertListeners.add(callback);
}

public unsubscribeFromAlertTriggered(callback?: (alert: Alert) => void): void {
    if (callback) {
        this.alertListeners.delete(callback);
    } else {
        this.alertListeners.clear();
    }
}
}

export const signalRService = new SignalRService();

```

F28. src/Services/symbolApi.ts

Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.

```

import { api } from './api';
import type { SymbolResponseDto } from '../Types/symbol';

export const symbolApi = {
    getActiveSymbols: async (exchangeId: number): Promise<SymbolResponseDto[]> => {
        const response = await api.get<SymbolResponseDto[]>('/symbols/active', {
            params: { exchangeId },

```



```

    });
    return response.data;
  },

  searchSymbols: async (query?: string, exchangeId?: number): Promise<SymbolResponseDto[]> => {
    const response = await api.get<SymbolResponseDto[]>('/symbols', {
      params: {
        ...(query ? { query } : {}),
        ...(exchangeId ? { exchangeId } : {}),
      },
    });
    return response.data;
  },

  getSymbol: async (symbolId: number): Promise<SymbolResponseDto> => {
    const response = await api.get<SymbolResponseDto>(`/symbols/${symbolId}`);
    return response.data;
  },

  syncExchange: async (exchangeCode: string): Promise<{ message: string }> => {
    const response = await api.post<{ message: string }>(`/symbols/sync/${encodeURIComponent(exchangeCode)}`);
    return response.data;
  },

  toggleSymbolStatus: async (
    symbolId: number,
    isActive: boolean
  ): Promise<{ message: string; symbolId: number; exchangeCode: string; symbolName: string; isActive: boolean; historicalSyncQueued: boolean;
  realtimeRestartRequested: boolean }> => {
    const response = await api.put<{ message: string; symbolId: number; exchangeCode: string; symbolName: string; isActive: boolean; historicalSyncQueued:
  boolean; realtimeRestartRequested: boolean }>({
      '/symbols/${symbolId}/status',
      { isActive }
    });
    return response.data;
  },
};

```

F29. src/Services/systemHealthService.ts

Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.

```

import { api } from './api';
import type { SystemHealthDto } from '../Types/systemHealth';

export const systemHealthService = {
  getHealth: async (): Promise<SystemHealthDto> => {
    const response = await api.get<SystemHealthDto>('/admin/system-health');
    return response.data;
  },
};

```

F30. src/Services/userService.ts

Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.

```

import { api } from './api';

export interface UserManagementDto {
  id: number;
  username: string;
  email: string;
  role: string;
  firstName: string;
  lastName: string;
  isDeleted: boolean;
  deletionRequestedAt?: string | null;
}

export interface ChangePasswordDto {
  currentPassword: string;
  newPassword: string;
}

export interface UserPreferencesDto {
  theme?: string;
  language?: string;
  selectedLayout?: string;
  activeIndicatorsJson?: string;
}

```

```

    chartSettingsJson?: string;
  }

export const userService = {
  getMyProfile: async (): Promise<UserManagementDto> => {
    const response = await api.get<UserManagementDto>('/users/me');
    return response.data;
  },

  updateMyProfile: async (dto: Partial<UserManagementDto>): Promise<UserManagementDto> => {
    const response = await api.put<UserManagementDto>('/users/me', dto);
    return response.data;
  },

  changePassword: async (dto: ChangePasswordDto): Promise<{ message: string }> => {
    const response = await api.post<{ message: string }>('/users/change-password', dto);
    return response.data;
  },

  // 🗑 Slet brugerkonto (soft delete)
  deleteMyAccount: async (): Promise<{ message: string }> => {
    const response = await api.delete<{ message: string }>('/users/me');
    return response.data;
  },

  // 🗑 Admin/SuperAdmin sletter en konto permanent fra databasen (hard delete)
  hardDeleteUser: async (userId: number): Promise<{ message: string }> => {
    const response = await api.delete<{ message: string }>('/users/${userId}');
    return response.data;
  },

  getPreferences: async (): Promise<UserPreferencesDto> => {
    const response = await api.get<UserPreferencesDto>('/users/preferences');
    return response.data;
  },

  savePreferences: async (dto: UserPreferencesDto): Promise<{ message: string }> => {
    const response = await api.put<{ message: string }>('/users/preferences', dto);
    return response.data;
  },

  getAllUsers: async (): Promise<UserManagementDto[]> => {
    const response = await api.get<UserManagementDto[]>('/users');
    return response.data;
  },

  assignRole: async (dto: UserManagementDto): Promise<{ message: string }> => {
    const response = await api.put<{ message: string }>('/users/assign-role', dto);
    return response.data;
  },
};

```

F31. src/Services/watchlistService.ts

Frontend-service til API, authentication, realtime eller anden kommunikation uden at lægge netværkslogik direkte i komponenterne.

```

import { api } from './api';
import type { AddWatchlistItemDto, WatchlistResponseDto } from '../Types/watchlist';

export const watchlistService = {
  getMyWatchlists: async (): Promise<WatchlistResponseDto[]> => {
    const response = await api.get<WatchlistResponseDto[]>('/watchlists');
    return response.data;
  },

  createWatchlist: async (name: string): Promise<WatchlistResponseDto> => {
    const response = await api.post<WatchlistResponseDto>('/watchlists', JSON.stringify(name), {
      headers: { 'Content-Type': 'application/json' },
    });
    return response.data;
  },

  deleteWatchlist: async (watchlistId: number): Promise<void> => {
    await api.delete(`/watchlists/${watchlistId}`);
  },

  addSymbolToWatchlist: async (dto: AddWatchlistItemDto): Promise<void> => {
    await api.post('/watchlists/items', dto);
  },
};

```

```
removeSymbolFromWatchlist: async (watchlistId: number, symbolId: number): Promise<void> => {
  await api.delete(`/watchlists/${watchlistId}/items/${symbolId}`);
},
};
```

F32. src/Types/ai.ts

TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.

```
export interface AiAnalysisRequestDto {
  symbolId: number;
  interval: string;
  language?: string;
}

export interface AiAnalysisResponseDto {
  symbolId: number;
  symbol: string;
  exchange: string;
  analysisText: string;
  generatedAt: string;
}
```

F33. src/Types/alert.ts

TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.

```
export type AlertCondition = 'CROSSES_UP' | 'CROSSES_DOWN';

export interface Alert {
  id: string;
  userId: string;
  symbolId: number;
  symbol: string;
  exchange: string;
  targetPrice: number;
  condition: AlertCondition;
  isTriggered: boolean;
  isActive: boolean;
  note?: string;
  createdAt: string;
}

export interface CreateAlertRequest {
  symbolId: number;
  targetPrice: number;
  condition: AlertCondition;
  note?: string;
}
```

F34. src/Types/auth.ts

TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.

```
export interface User {
  username: string;
  email: string;
  role: string;
  firstName: string;
  lastName: string;
}

export interface AuthResponse {
  token: string;
  username: string;
  email: string;
  role: string;
  firstName: string;
  lastName: string;
}

export interface LoginRequest {
  identifier: string;
  password: string;
}

export interface RegisterRequest {
  firstName: string;
  lastName: string;
  username: string;
}
```

```
email: string;
password: string;
}

export interface ForgotPasswordRequest {
  identifier: string;
}

export interface ResetPasswordRequest {
  userId: number;
  token: string;
  newPassword: string;
}

export interface MessageResponse {
  message: string;
}
```

F35. src/Types/candle.ts

TypeScript-type/interfaces, som beskriver dataformen og giver compile-time typekontrol.

```
export interface CandleData {
  time: number;
  open: number;
  high: number;
  low: number;
  close: number;
  volume?: number;
  symbolId?: number;
  symbol?: string;
  exchange?: string;
  interval?: string;
  isClosed?: boolean;
}
export type ChartType = 'candles' | 'bars' | 'line' | 'area' | 'heikin-ashi';
```

F36. src/Types/chartDrawing.ts

TypeScript-type/interfaces, som beskriver dataformen og giver compile-time typekontrol.

```
export type DrawingTool = 'cursor' | 'trendLine' | 'horizontalLine' | 'rectangle' | 'fibonacci' | 'text' | 'eraser';
export type PersistedDrawingType = Exclude<DrawingTool, 'cursor' | 'eraser'>;

export interface DrawingPoint {
  time: number;
  price: number;
}

export interface DrawingStyle {
  color?: string;
  lineWidth?: number;
  fillOpacity?: number;
}

export type DrawingData =
  | { points: [DrawingPoint, DrawingPoint]; style?: DrawingStyle }
  | { price: number; style?: DrawingStyle }
  | { point: DrawingPoint; text: string; style?: DrawingStyle };

export interface ChartDrawingDto {
  id: number;
  symbolId: number;
  exchangeCode: string;
  symbolName: string;
  interval: string;
  drawingType: PersistedDrawingType;
  data: DrawingData;
  isVisible: boolean;
  createdAt: string;
  updatedAt: string;
}

export interface CreateChartDrawingDto {
  symbolId: number;
  interval: string;
  drawingType: PersistedDrawingType;
  data: DrawingData;
  isVisible?: boolean;
}
```

```
export interface UpdateChartDrawingDto {
  data: DrawingData;
  isVisible?: boolean;
}
```

F37. src/Types/chartSettings.ts

TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.

```
export interface ChartVisualSettings {
  // 1. Candle-krop og farver
  upColor: string;
  downColor: string;
  showBorders: boolean;
  showWicks: boolean;

  // 2. Indstillinger for lærred og gitter
  showGrid: boolean;
  gridColorDark: string;
  gridColorLight: string;

  // 3. Prislinjer
  showPriceLine: boolean;

  // 4. Alarmer og lyd
  soundEnabled?: boolean;
}

export const DEFAULT_CHART_SETTINGS: ChartVisualSettings = {
  upColor: '#089981',
  downColor: '#f23645',
  showBorders: false,
  showWicks: true,
  showGrid: true,
  gridColorDark: '#1e222d',
  gridColorLight: '#0f3fa',
  showPriceLine: true,
  soundEnabled: true,
};
```

F38. src/Types/indicator.ts

TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.

```
export type IndicatorType = 'SMA' | 'EMA' | 'RSI' | 'BB' | 'MACD';

export interface IndicatorConfig {
  id: string;
  type: IndicatorType;
  name: string;
  enabled: boolean;
  color: string;
  period?: number;
  fastPeriod?: number; // Til MACD.
  slowPeriod?: number; // Til MACD.
  signalPeriod?: number; // Til MACD.
  stdDev?: number; // Til Bollinger Bands.
}

export const DEFAULT_INDICATORS: IndicatorConfig[] = [
  { id: 'sma-20', type: 'SMA', name: 'SMA (20)', enabled: false, color: '#f59e0b', period: 20 },
  { id: 'ema-50', type: 'EMA', name: 'EMA (50)', enabled: false, color: '#3b82f6', period: 50 },
  { id: 'ema-200', type: 'EMA', name: 'EMA (200)', enabled: false, color: '#ec4899', period: 200 },
  { id: 'rsi-14', type: 'RSI', name: 'RSI (14)', enabled: false, color: '#8b5cf6', period: 14 },
  { id: 'bb-20', type: 'BB', name: 'Bollinger Bands (20, 2)', enabled: false, color: '#10b981', period: 20, stdDev: 2 },
];
```

F39. src/Types/preferences.ts

TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.

```
export type Language = 'tr' | 'en' | 'da';
export type Theme = 'dark' | 'light';
export type ChartType = 'candles' | 'bars' | 'line' | 'area' | 'heikin-ashi';
export type LayoutMode = '1x1' | '1x2' | '2x2';

export interface ChartIndicatorConfig {
  id: string;
  type: 'SMA' | 'EMA' | 'RSI' | 'MACD' | 'BB';
  name: string;
  enabled: boolean;
}
```

```

    color: string;
    period?: number;
    stdDev?: number;
    fastPeriod?: number; // Til MACD.
    slowPeriod?: number; // Til MACD.
    signalPeriod?: number; // Til MACD.
  }

  export interface ChartVisualSettings {
    upColor: string;
    downColor: string;
    showBorders: boolean;
    showWicks: boolean;
    showGrid: boolean;
    gridColorDark: string;
    gridColorLight: string;
    showPriceLine: boolean;
    soundEnabled: boolean;
  }

  export interface UserPreferences {
    theme: Theme;
    language: Language;
    lastSymbol: string;
    lastInterval: string;
    chartType: ChartType;
    layoutMode: LayoutMode;
    indicators: ChartIndicatorConfig[];
    visualSettings: ChartVisualSettings;
  }

  export const defaultPreferences: UserPreferences = {
    theme: 'dark',
    language: 'tr',
    lastSymbol: 'BTCUSDT',
    lastInterval: '15m',
    chartType: 'candles',
    layoutMode: '1x1',
    indicators: [
      { id: 'sma-20', type: 'SMA', name: 'SMA (20)', enabled: true, period: 20, color: '#59e0b' },
      { id: 'ema-50', type: 'EMA', name: 'EMA (50)', enabled: false, period: 50, color: '#3b82f6' },
      { id: 'ema-200', type: 'EMA', name: 'EMA (200)', enabled: false, period: 200, color: '#ec4899' },
      { id: 'rsi-14', type: 'RSI', name: 'RSI (14)', enabled: false, period: 14, color: '#8b5cf6' },
      { id: 'bb-20', type: 'BB', name: 'Bollinger Bands (20, 2)', enabled: false, period: 20, stdDev: 2, color: '#10b981' },
    ],
    visualSettings: {
      upColor: '#089981',
      downColor: '#f23645',
      showBorders: false,
      showWicks: true,
      showGrid: true,
      gridColorDark: '#1e222d',
      gridColorLight: '#0f3fa',
      showPriceLine: true,
      soundEnabled: true,
    },
  };

```

F40. src/Types/symbol.ts

TypeScript-type/interfaces, som beskriver dataformen og giver compile-time typekontrol.

```

export interface SymbolResponseDto {
  id: number;
  exchangeId: number;
  exchangeCode: string;
  exchangeName: string;
  name: string;
  baseAsset: string;
  quoteAsset: string;
  isActive: boolean;
}

export interface ExchangeResponseDto {
  id: number;
  code: string;
  name: string;
  isActive: boolean;
}

```

F41. src/Types/systemHealth.ts

TypeScript-type/interfaces, som beskriver dataformen og giver compile-time typekontrol.

```

export interface ExchangeHealthDto {
  exchangeCode: string;
  exchangeName: string;
  isActive: boolean;
  activeSymbolCount: number;
  lastCandleTimeUtc?: string | null;
  lastCandleAgeSeconds?: number | null;
  historicalSyncRunning: boolean;
  historicalSyncStartedAtUtc?: string | null;
  historicalSyncCompletedAtUtc?: string | null;
  realtimeConnected: boolean;
  realtimeConnectedAtUtc?: string | null;
  lastRealtimeMessageAtUtc?: string | null;
  lastError?: string | null;
  lastErrorAtUtc?: string | null;
  validationErrorCount: number;
  validationWarningCount: number;
  status: string;
}

export interface SystemHealthDto {
  status: string;
  databaseHealthy: boolean;
  timescaleHealthy: boolean;
  candlesHypertableHealthy: boolean;
  activeExchangeCount: number;
  activeSymbolCount: number;
  lastCandleTimeUtc?: string | null;
  lastCandleAgeSeconds?: number | null;
  validationErrorCount: number;
  validationWarningCount: number;
  startedAtUtc: string;
  uptime: string;
  checkedAtUtc: string;
  exchanges: ExchangeHealthDto[];
}

```

F42. src/Types/watchlist.ts

TypeScript-typer/interfaces, som beskriver dataformen og giver compile-time typekontrol.

```

export interface AddWatchlistItemDto {
  watchlistId: number;
  symbolId: number;
}

export interface WatchlistItemResponseDto {
  id: number;
  symbolId: number;
  symbol: string;
  exchange: string;
  addedAt: string;
}

export interface WatchlistResponseDto {
  id: number;
  name: string;
  items: WatchlistItemResponseDto[];
}

```

F43. src/Uutils/indicatorCalculators.ts

Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.

```

import type { CandlestickData, LineData, Time } from 'lightweight-charts';

// Simpelt glidende gennemsnit (SMA)
export const calculateSMA = (data: CandlestickData<Time>[], period: number): LineData<Time>[] => {
  const result: LineData<Time>[] = [];
  for (let i = period - 1; i < data.length; i++) {
    let sum = 0;
    for (let j = 0; j < period; j++) {
      sum += data[i - j].close;
    }
    result.push({
      time: data[i].time,
      value: sum / period,
    });
  }
  return result;
};

// Eksponentielt glidende gennemsnit (EMA)

```

```
export const calculateEMA = (data: CandlestickData<Time>[], period: number): LineData<Time>[] => {
  if (data.length < period) return [];
  const result: LineData<Time>[] = [];
  const multiplier = 2 / (period + 1);

  // Brug SMA til den første værdi.
  let initialSum = 0;
  for (let i = 0; i < period; i++) {
    initialSum += data[i].close;
  }
  let prevEma = initialSum / period;
  result.push({ time: data[period - 1].time, value: prevEma });

  for (let i = period; i < data.length; i++) {
    const currentClose = data[i].close;
    const currentEma = (currentClose - prevEma) * multiplier + prevEma;
    result.push({ time: data[i].time, value: currentEma });
    prevEma = currentEma;
  }

  return result;
};

// Bollinger Bands (øvre, midterste, nedre)
export const calculateBollingerBands = (
  data: CandlestickData<Time>[],
  period: number = 20,
  stdDevMultiplier: number = 2
) => {
  const upper: LineData<Time>[] = [];
  const middle: LineData<Time>[] = [];
  const lower: LineData<Time>[] = [];

  for (let i = period - 1; i < data.length; i++) {
    let sum = 0;
    for (let j = 0; j < period; j++) {
      sum += data[i - j].close;
    }
    const sma = sum / period;

    let varianceSum = 0;
    for (let j = 0; j < period; j++) {
      varianceSum += Math.pow(data[i - j].close - sma, 2);
    }
    const stdDev = Math.sqrt(varianceSum / period);

    middle.push({ time: data[i].time, value: sma });
    upper.push({ time: data[i].time, value: sma + stdDev * stdDevMultiplier });
    lower.push({ time: data[i].time, value: sma - stdDev * stdDevMultiplier });
  }

  return { upper, middle, lower };
};

// Relative Strength Index (RSI)
export const calculateRSI = (data: CandlestickData<Time>[], period: number = 14): LineData<Time>[] => {
  if (data.length <= period) return [];
  const result: LineData<Time>[] = [];

  let gains = 0;
  let losses = 0;

  for (let i = 1; i <= period; i++) {
    const change = data[i].close - data[i - 1].close;
    if (change >= 0) gains += change;
    else losses += Math.abs(change);
  }

  let avgGain = gains / period;
  let avgLoss = losses / period;

  let rs = avgLoss === 0 ? 100 : avgGain / avgLoss;
  let rsi = 100 - 100 / (1 + rs);
  result.push({ time: data[period].time, value: rsi });

  for (let i = period + 1; i < data.length; i++) {
    const change = data[i].close - data[i - 1].close;
    const gain = change > 0 ? change : 0;
    const loss = change < 0 ? Math.abs(change) : 0;

    avgGain = (avgGain * (period - 1) + gain) / period;
    avgLoss = (avgLoss * (period - 1) + loss) / period;
    let newRsi = 100 - 100 / (1 + avgGain / avgLoss);
    result.push({ time: data[i].time, value: newRsi });
  }

  return result;
};
```



```
    avgLoss = (avgLoss * (period - 1) + loss) / period;

    rs = avgLoss === 0 ? 100 : avgGain / avgLoss;
    rsi = 100 - 100 / (1 + rs);
    result.push({ time: data[i].time, value: rsi });
  }

  return result;
};
```

F44. src/hooks/useAlerts.ts

Genbrugelig React hook, der samler state og side effects for denne funktion.

```
import { useState, useEffect, useCallback } from 'react';
import { HubConnection } from '@microsoft/signalr';
import type { Alert, CreateAlertRequest } from '../Types/alert';
import { alertService } from '../Services/alertService';

export const useAlerts = (
  symbol: string,
  hubConnection: HubConnection | null,
  soundEnabled: boolean = true
) => {
  const [alerts, setAlerts] = useState<Alert[]>([]);
  const [triggeredAlert, setTriggeredAlert] = useState<Alert | null>(null);

  // Hent alarmer fra backend.
  const fetchAlerts = useCallback(async () => {
    try {
      const data = await alertService.getMyAlerts();
      setAlerts(data);
    } catch (err) {
      console.error('[useAlerts] Alarmlar alinamadi:', err);
    }
  }, []);

  useEffect(() => {
    fetchAlerts();
  }, [fetchAlerts]);

  // Realtidslistener til "ReceiveAlertTriggered" via SignalR.
  useEffect(() => {
    if (!hubConnection) return;

    // Behandler den relevante brugerhandling eller event.
    const handleTriggered = (alert: Alert) => {
      setTriggeredAlert(alert);

      // Afspil lyd, hvis lyd er aktiveret i brugerindstillingerne.
      if (soundEnabled) {
        try {
          const audio = new Audio('/sounds/alert.mp3');
          audio.volume = 0.8;
          audio.play().catch((err) => {
            console.warn('[useAlerts] Tarayıcı ses çalmayı engelledi:', err);
          });
        } catch {}
      }

      // Opdater state, og markér alarmen som udløst.
      setAlerts((prev) =>
        prev.map((a) =>
          a.id === alert.id ? { ...a, isTriggered: true, isActive: false } : a
        )
      );
    };

    hubConnection.on('ReceiveAlertTriggered', handleTriggered);

    return () => {
      hubConnection.off('ReceiveAlertTriggered', handleTriggered);
    };
  }, [hubConnection, soundEnabled]);

  // Opret ny alarm
  const createAlert = async (request: CreateAlertRequest) => {
    const created = await alertService.createAlert(request);
    if (created) {
      setAlerts((prev) => [created, ...prev]);
    }
  }
}
```

```

    return created;
  };

  // Slet alarm
  const deleteAlert = async (id: string) => {
    await alertService.deleteAlert(id);
    setAlerts((prev) => prev.filter((a) => a.id !== id));
  };

  // Aktiver/deaktiver alarm
  const toggleAlert = async (id: string) => {
    await alertService.toggleAlert(id);
    setAlerts((prev) =>
      prev.map((a) => (a.id === id ? { ...a, isActive: !a.isActive } : a))
    );
  };

  // Aktive og ikke-udløste alarmer for det valgte symbol.
  const activeSymbolAlerts = alerts.filter(
    (a) =>
      a.symbol.toUpperCase() === symbol.toUpperCase() &&
      a.isActive &&
      !a.isTriggered
  );

  return {
    alerts,
    activeSymbolAlerts,
    triggeredAlert,
    clearTriggeredAlert: () => setTriggeredAlert(null),
    createAlert,
    deleteAlert,
    toggleAlert,
    refreshAlerts: fetchAlerts,
  };
};

```

F45. src/hooks/useUserPreferences.ts

Genbrugelig React hook, der samler state og side effects for denne funktion.

```

import { useState, useEffect, useCallback, useRef } from 'react';
import { type UserPreferences, defaultPreferences } from '../Types/preferences';
import { preferenceService } from '../Services/preferenceService';

export const useUserPreferences = () => {
  const [preferences, setPreferences] = useState<UserPreferences>(() => {
    try {
      const local = localStorage.getItem('user_prefs');
      return local ? { ...defaultPreferences, ...JSON.parse(local) } : defaultPreferences;
    } catch {
      return defaultPreferences;
    }
  });

  const [loading, setLoading] = useState(true);
  const saveTimeoutRef = useRef<ReturnType<setTimeout> | null> >(null);

  // 1. Hent indstillinger fra serveren, når siden åbnes.
  useEffect(() => {
    let isMounted = true;

    preferenceService
      .getPreferences()
      .then((data) => {
        if (isMounted && data && Object.keys(data).length > 0) {
          const merged = { ...defaultPreferences, ...data };
          setPreferences(merged);
          localStorage.setItem('user_prefs', JSON.stringify(merged));
        }
      })
      .catch((err) => {
        console.warn('Kullanıcı tercihleri sunucudan alınamadı, yerel ayarlar kullanılıyor:', err);
      })
      .finally(() => {
        if (isMounted) setLoading(false);
      });
  });

  return () => {
    isMounted = false;
    if (saveTimeoutRef.current) clearTimeout(saveTimeoutRef.current);
  };
};

```

```

    }, []);

    // 2. Opdater indstillinger, og gem dem i databasen (debounced).
    const updatePreferences = useCallback((newPrefs: Partial<UserPreferences>) => {
      setPreferences((prev) => {
        const updated = { ...prev, ...newPrefs };
        localStorage.setItem('user_prefs', JSON.stringify(updated));

        // Brug debounce for at undgå mange API-kald ved hurtige klik.
        if (saveTimeoutRef.current) {
          clearTimeout(saveTimeoutRef.current);
        }

        saveTimeoutRef.current = setTimeout(() => {
          preferenceService.savePreferences(updated).catch((err) => {
            console.error('Tercihler sunucuya kaydedilemedi:', err);
          });
        }, 500);

        return updated;
      });
    }, []);

    return { preferences, updatePreferences, loading };
  };

```

F46. src/i18n.ts

Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.

```

import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import LanguageDetector from 'i18next-browser-languagedetector';
import tr from './locales/tr.json';
import en from './locales/en.json';
import da from './locales/da.json';

i18n
  .use(LanguageDetector)
  .use(initReactI18next)
  .init({
    resources: {
      tr: { translation: tr },
      en: { translation: en },
      da: { translation: da },
    },
    fallbackLng: 'en', // Brug engelsk som standard, hvis brugerens sprog ikke understøttes.
    supportedLngs: ['tr', 'en', 'da'], // Kun understøttede sprog.
    nonExplicitSupportedLngs: true, // Eksempel: Hvis 'tr-TR' eller 'tr-AZ' modtages, matches det automatisk med 'tr'.
    interpolation: {
      escapeValue: false,
    },
    detection: {
      // Kontroller først localStorage; hvis der ikke findes et valg, registreres browserens sprog via navigator.
      order: ['localStorage', 'navigator'],
      caches: ['localStorage'],
      lookupLocalStorage: 'tradingpro_lang',
      // Hvis browsersproget f.eks. er 'tr-TR', bruges kun de første to bogstaver ('tr').
      convertDetectedLanguage: (lng) => {
        if (!lng) return 'en';
        const shortLang = lng.toLowerCase().split('-')[0];
        return ['tr', 'en', 'da'].includes(shortLang) ? shortLang : 'en';
      },
    },
  });

export default i18n;

```

F47. src/main.tsx

Støttekode, der indgår i TradingPros samlede backend-, frontend- eller teststruktur.

```

import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './i18n';
import './index.css';
import App from './App.tsx';
import { ThemeProvider } from './Context/ThemeContext.tsx';

createRoot(document.getElementById('root')!).render(

```

```
<StrictMode>
  <ThemeProvider>
    <App />
  </ThemeProvider>
</StrictMode>
);
```

6. Komplet testkode

T1. Controllers/CandlesControllerTests.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```
using Microsoft.AspNetCore.Mvc;
using Moq;
using TradingApp.Controllers;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingAppTest.Controllers;

public class CandlesControllerTests
{
    // Kontrollerer, at et ugyldigt symbol-id bliver afvist med Bad Request.
    [Fact]
    public async Task GetCandles_InvalidSymbolId_ReturnsBadRequest()
    {
        var controller = new CandlesController(Mock.Of<ICandleService>());

        var result = await controller.GetCandles(0, "1m", 100, null);

        Assert.IsType<BadRequestObjectResult>(result.Result);
    }

    // Kontrollerer, at en for stor candle-limit bliver begrænset før servicekaldet.
    [Fact]
    public async Task GetCandles_ClampsLimitTo1000()
    {
        var service = new Mock<ICandleService>();
        service.Setup(x => x.GetCandlesAsync(1, "15m", 1000, null))
            .ReturnsAsync(Array.Empty<CandleResponseDto>());
        var controller = new CandlesController(service.Object);

        var result = await controller.GetCandles(1, "15m", 5000, null);

        Assert.IsType<OkObjectResult>(result.Result);
        service.Verify(x => x.GetCandlesAsync(1, "15m", 1000, null), Times.Once);
    }

    // Kontrollerer, at en gyldig request returnerer det forventede service-resultat.
    [Fact]
    public async Task GetCandles_ValidRequest_ReturnsServiceResult()
    {
        var expected = new[]
        {
            new CandleResponseDto(1, 100, 110, 90, 105, 5, 1, "BTCUSDT", "BINANCE", "1m")
        };
        var service = new Mock<ICandleService>();
        service.Setup(x => x.GetCandlesAsync(1, "1m", 100, null)).ReturnsAsync(expected);
        var controller = new CandlesController(service.Object);

        var result = await controller.GetCandles(1, "1m", 100, null);

        var ok = Assert.IsType<OkObjectResult>(result.Result);
        Assert.Same(expected, ok.Value);
    }
}
```

T2. Controllers/ExchangesControllerTests.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using TradingApp.Controllers;
using TradingAppLibrary.DTO;
```

```
using TradingAppLibrary.Models;
using TradingAppTest.Helpers;

namespace TradingAppTest.Controllers;

public class ExchangesControllerTests
{
    // Kontrollerer, at kun aktive exchanges returneres.
    [Fact]
    public async Task GetActiveExchanges_ReturnsOnlyActiveRows()
    {
        await using var db = TestDbContextFactory.Create();
        db.Exchanges.Add(new Exchange { Id = 2, Code = "OKX", Name = "OKX", IsActive = false });
        await db.SaveChangesAsync();
        var controller = new ExchangesController(db, new FakeExchangeServiceFactory("BINANCE", "OKX"));

        var result = await controller.GetActiveExchanges(CancellationToken.None);

        var ok = Assert.IsType<OkObjectResult>(result.Result);
        var exchanges = Assert.IsAssignableFrom<IEnumerable<ExchangeResponseDto>>(ok.Value).ToList();
        var exchange = Assert.Single(exchanges);
        Assert.Equal("BINANCE", exchange.Code);
    }

    // Kontrollerer, at en exchange med en understøttet adapter kan oprettes.
    [Fact]
    public async Task CreateExchange_WithSupportedAdapter_CreatesNormalizedExchange()
    {
        await using var db = TestDbContextFactory.Create();
        var controller = new ExchangesController(db, new FakeExchangeServiceFactory("BINANCE", "OKX"));

        var result = await controller.CreateExchange(
            new CreateExchangeDto(" okx ", " OKX Exchange ", true), CancellationToken.None);

        var created = Assert.IsType<CreatedAtActionResult>(result.Result);
        var dto = Assert.IsType<ExchangeResponseDto>(created.Value);
        Assert.Equal("OKX", dto.Code);
        Assert.Equal("OKX Exchange", dto.Name);
        Assert.True(await db.Exchanges.AnyAsync(e => e.Code == "OKX"));
    }

    // Kontrollerer, at en exchange uden backend-adapter bliver afvist.
    [Fact]
    public async Task CreateExchange_WithoutBackendAdapter_ReturnsBadRequest()
    {
        await using var db = TestDbContextFactory.Create();
        var controller = new ExchangesController(db, new FakeExchangeServiceFactory("BINANCE", "OKX"));

        var result = await controller.CreateExchange(
            new CreateExchangeDto("KRAKEN", "Kraken", true), CancellationToken.None);

        Assert.IsType<BadRequestObjectResult>(result.Result);
        Assert.False(await db.Exchanges.AnyAsync(e => e.Code == "KRAKEN"));
    }

    // Kontrollerer, at en eksisterende exchange-kode ikke kan oprettes igen.
    [Fact]
    public async Task CreateExchange_WhenCodeAlreadyExists_ReturnsConflict()
    {
        await using var db = TestDbContextFactory.Create();
        var controller = new ExchangesController(db, new FakeExchangeServiceFactory("BINANCE", "OKX"));

        var result = await controller.CreateExchange(
            new CreateExchangeDto("BINANCE", "Another Binance", true), CancellationToken.None);

        Assert.IsType<ConflictObjectResult>(result.Result);
    }

    // Kontrollerer, at exchange-koden ikke kan ændres, når der allerede findes symboler.
    [Fact]
    public async Task UpdateExchange_CannotChangeCodeAfterSymbolsExist()
    {
        await using var db = TestDbContextFactory.Create();
        db.Symbols.Add(new Symbol { Exchangeld = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" });
        await db.SaveChangesAsync();
        var controller = new ExchangesController(db, new FakeExchangeServiceFactory("BINANCE", "OKX"));

        var result = await controller.UpdateExchange(
            1, new UpdateExchangeDto("OKX", "Changed", true), CancellationToken.None);
    }
}
```

```

    Assert.IsType<ConflictObjectResult>(result.Result);
    Assert.Equal("BINANCE", (await db.Exchanges.FindAsync(1)).Code);
}

// Kontrollerer, at en exchange med eksisterende symboler ikke kan slettes.
[Fact]
public async Task DeleteExchange_WithSymbols_ReturnsConflictAndKeepsExchange()
{
    await using var db = TestDbContextFactory.Create();
    db.Symbols.Add(new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USD" });
    await db.SaveChangesAsync();
    var controller = new ExchangesController(db, new FakeExchangeServiceFactory("BINANCE", "OKX"));

    var result = await controller.DeleteExchange(1, CancellationToken.None);

    Assert.IsType<ConflictObjectResult>(result);
    Assert.NotNull(await db.Exchanges.FindAsync(1));
}

// Kontrollerer, at en exchange uden symboler kan slettes korrekt.
[Fact]
public async Task DeleteExchange_WithoutSymbols_ReturnsNoContent()
{
    await using var db = TestDbContextFactory.Create();
    db.Exchanges.Add(new Exchange { Id = 2, Code = "OKX", Name = "OKX", IsActive = false });
    await db.SaveChangesAsync();
    var controller = new ExchangesController(db, new FakeExchangeServiceFactory("BINANCE", "OKX"));

    var result = await controller.DeleteExchange(2, CancellationToken.None);

    Assert.IsType<NoContentResult>(result);
    Assert.Null(await db.Exchanges.FindAsync(2));
}
}

```

T3. Controllers/SymbolsControllerTests.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```

using Microsoft.AspNetCore.Mvc;
using Moq;
using TradingApp.Controllers;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingAppTest.Controllers;

public class SymbolsControllerTests
{
    // Kontrollerer, at et ugyldigt exchange-id bliver afvist.
    [Fact]
    public async Task GetSymbols_InvalidExchangeId_ReturnsBadRequest()
    {
        var controller = new SymbolsController(Mock.Of<ISymbolService>(), Mock.Of<IMarketDataSyncService>());

        var result = await controller.GetSymbols(exchangeId: 0);

        Assert.IsType<BadRequestObjectResult>(result.Result);
    }

    // Kontrollerer, at et manglende element returnerer Not Found.
    [Fact]
    public async Task GetSymbol_WhenMissing_ReturnsNotFound()
    {
        var symbols = new Mock<ISymbolService>();
        symbols.Setup(x => x.GetSymbolByIdAsync(99)).ReturnsAsync((SymbolResponseDto?)null);
        var controller = new SymbolsController(symbols.Object, Mock.Of<IMarketDataSyncService>());

        var result = await controller.GetSymbol(99);

        Assert.IsType<NotFoundObjectResult>(result.Result);
    }

    // Kontrollerer, at exchange-koden normaliseres før synkronisering.
    [Fact]
    public async Task SyncSymbols_NormalizesExchangeCodeAndCallsSyncService()
    {
        var sync = new Mock<IMarketDataSyncService>();
        var controller = new SymbolsController(Mock.Of<ISymbolService>(), sync.Object);
    }
}

```

```

var result = await controller.SyncSymbols(" okx ", CancellationToken.None);

Assert.IsType<OkObjectResult>(result);
sync.Verify(x => x.SyncExchangeSymbolsAsync("OKX", It.IsAny<CancellationToken>()), Times.Once);
}

// Kontrollerer, at statusændring på et manglende symbol returnerer Not Found.
[Fact]
public async Task ToggleStatus_WhenSymbolMissing_ReturnsNotFound()
{
    var symbols = new Mock<ISymbolService>();
    symbols.Setup(x => x.UpdateSymbolStatusAsync(42, true))
        .ReturnsAsync((SymbolStatusUpdateResultDto?)null);
    var controller = new SymbolsController(symbols.Object, Mock.Of<IMarketDataSyncService>());

    var result = await controller.ToggleStatus(42, new ToggleStatusDto(true));

    Assert.IsType<NotFoundObjectResult>(result);
}
}

```

T4. Controllers/WatchlistsControllerTests.cs

ASP.NET Core controller. Modtager HTTP-requests, validerer input og omsætter service-resultater til HTTP-statuskoder og DTO-responses.

```

using System.Security.Claims;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Moq;
using TradingApp.Controllers;
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingAppTest.Controllers;

public class WatchlistsControllerTests
{
    // Kontrollerer den forventede adfærd for dette testscenarie.
    private static WatchlistsController CreateController(IWatchlistService service, int userId = 7)
    {
        var controller = new WatchlistsController(service);
        controller.ControllerContext = new ControllerContext
        {
            HttpContext = new DefaultHttpContext
            {
                User = new ClaimsPrincipal(new ClaimsIdentity(
                    new[] { new Claim(ClaimTypes.NameIdentifier, userId.ToString()) }, "Test"))
            }
        };
        return controller;
    }

    // Kontrollerer, at watchlists hentes for den autentificerede bruger.
    [Fact]
    public async Task GetMyWatchlists_UsesAuthenticatedUserId()
    {
        var service = new Mock<IWatchlistService>();
        service.Setup(x => x.GetUserWatchlistsAsync(7))
            .ReturnsAsync(new[] { new WatchlistResponseDto(1, "Main", new()) });
        var controller = CreateController(service.Object);

        var result = await controller.GetMyWatchlists();

        Assert.IsType<OkObjectResult>(result.Result);
        service.Verify(x => x.GetUserWatchlistsAsync(7), Times.Once);
    }

    // Kontrollerer, at en watchlist uden navn bliver afvist.
    [Fact]
    public async Task CreateWatchlist_BlankName_ReturnsBadRequest()
    {
        var controller = CreateController(Mock.Of<IWatchlistService>());

        var result = await controller.CreateWatchlist(" ");

        Assert.IsType<BadRequestObjectResult>(result.Result);
    }
}

```

```
// Kontrollerer, at ugyldige id-værdier bliver afvist.
[Fact]
public async Task AddSymbolToWatchlist_InvalidIds_ReturnsBadRequest()
{
    var controller = CreateController(Mock.Of<IWatchlistService>());

    var result = await controller.AddSymbolToWatchlist(new AddWatchlistItemDto(0, 1));

    Assert.IsType<BadRequestObjectResult>(result);
}

// Kontrollerer, at et mislykket service-resultat returnerer Not Found.
[Fact]
public async Task RemoveSymbolFromWatchlist_WhenServiceReturnsFalse_ReturnsNotFound()
{
    var service = new Mock<IWatchlistService>();
    service.Setup(x => x.RemoveSymbolFromWatchlistAsync(1, 10)).ReturnsAsync(false);
    var controller = CreateController(service.Object);

    var result = await controller.RemoveSymbolFromWatchlist(1, 10);

    Assert.IsType<NotFoundObjectResult>(result);
}
}
```

T5. Helpers/FakeExchangeServiceFactory.cs

Test-double for exchange factory. Gør det muligt at kontrollere exchange-relateret controllerlogik uden at kontakte Binance eller OKX.

```
using TradingAppLibrary.DTO;
using TradingAppLibrary.Interfaces;

namespace TradingAppTest.Helpers;

internal sealed class FakeExchangeServiceFactory : IExchangeServiceFactory
{
    private readonly HashSet<string> _supported;

    public FakeExchangeServiceFactory(params string[] supported)
    {
        _supported = supported.Select(x => x.Trim().ToUpperInvariant()).ToHashSet();
    }

    // Henter exchange.
    public IExchangeService GetExchange(string exchangeCode)
    {
        var code = exchangeCode.Trim().ToUpperInvariant();
        if (!_supported.Contains(code))
            throw new NotSupportedException(code);

        return new FakeExchangeService(code);
    }

    private sealed class FakeExchangeService : IExchangeService
    {
        public FakeExchangeService(string code) => ExchangeCode = code;

        public string ExchangeCode { get; }
        public DateTime HistoricalDataStartUtc => new(2017, 1, 1, 0, 0, 0, DateTimeKind.Utc);

        // Henter symbols.
        public Task<IReadOnlyList<ExchangeSymbolDto>> GetSymbolsAsync(CancellationToken cancellationToken = default) =>
            Task.FromResult<IReadOnlyList<ExchangeSymbolDto>>(Array.Empty<ExchangeSymbolDto>());

        // Henter top popular symbols.
        public Task<IReadOnlyList<string>> GetTopPopularSymbolsAsync(int topCount = 50, CancellationToken cancellationToken = default) =>
            Task.FromResult<IReadOnlyList<string>>(Array.Empty<string>());

        // Henter historical candles.
        public Task<IReadOnlyList<ExchangeCandleDto>> GetHistoricalCandlesAsync(
            string symbol, string interval, int limit = 500, long? startTime = null,
            CancellationToken cancellationToken = default) =>
            Task.FromResult<IReadOnlyList<ExchangeCandleDto>>(Array.Empty<ExchangeCandleDto>());

        // Behandler stream real time candles.
        public Task StreamRealTimeCandlesAsync(
            IReadOnlyCollection<string> symbols,
            string interval,
```



```
Func<ExchangeCandleDto, Task> onCandleReceived,  
CancellationToken cancellationToken) => Task.CompletedTask;  
}  
}
```

T6. Helpers/TestDbContextFactory.cs

Opretter en isoleret EF Core InMemory-database for hver test. Det gør testene uafhængige af den rigtige PostgreSQL/TimescaleDB-database.

```
using Microsoft.EntityFrameworkCore;  
using TradingAppLibrary.Data;  
  
namespace TradingAppTest.Helpers;  
  
internal static class TestDbContextFactory  
{  
    // Opretter den relevante operation.  
    public static AppDbContext Create(string? databaseName = null)  
    {  
        var options = new DbContextOptionsBuilder<AppDbContext>()  
            .UseInMemoryDatabase(databaseName ?? $"TradingAppTest_{Guid.NewGuid():N}")  
            .EnableSensitiveDataLogging()  
            .Options;  
  
        var context = new AppDbContext(options);  
        context.Database.EnsureCreated();  
        return context;  
    }  
}
```

T7. Repositories/CandleRepositoryTests.cs

Tester repository-logik for candle-rækkefølge, lazy loading med endTime og update-vs-duplicate adfærd.

```
using TradingAppLibrary.Models;  
using TradingAppLibrary.Repositories;  
using TradingAppTest.Helpers;  
  
namespace TradingAppTest.Repositories;  
  
public class CandleRepositoryTests  
{  
    // Kontrollerer den forventede adfærd for dette testscenarie.  
    private static Candle Candle(int symbolId, DateTime openTime, decimal close) => new()  
    {  
        SymbolId = symbolId,  
        Interval = "1m",  
        OpenTime = openTime,  
        Open = close - 1,  
        High = close + 1,  
        Low = close - 2,  
        Close = close,  
        Volume = 10  
    };  
  
    // Kontrollerer, at de seneste candles returneres i kronologisk rækkefølge.  
    [Fact]  
    public async Task GetCandlesAsync_OneMinute_ReturnsLatestLimitInAscendingOrder()  
    {  
        await using var db = TestDbContextFactory.Create();  
        var symbol = new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" };  
        db.Symbols.Add(symbol);  
        await db.SaveChangesAsync();  
        var start = new DateTime(2026, 8, 20, 10, 0, 0, DateTimeKind.Utc);  
        for (var i = 0; i < 5; i++) db.Candles.Add(Candle(symbol.Id, start.AddMinutes(i), 100 + i));  
        await db.SaveChangesAsync();  
  
        var repo = new CandleRepository(db);  
        var result = (await repo.GetCandlesAsync(symbol.Id, "1m", 3)).ToList();  
  
        Assert.Equal(3, result.Count);  
        Assert.Equal(start.AddMinutes(2), result[0].OpenTime);  
        Assert.Equal(start.AddMinutes(4), result[2].OpenTime);  
    }  
  
    // Kontrollerer, at lazy loading kun returnerer candles før endTime.  
    [Fact]  
    public async Task GetCandlesAsync_EndTime_ReturnsOnlyCandlesBeforeEndTime()  
    {  
        await using var db = TestDbContextFactory.Create();
```

```

var symbol = new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" };
db.Symbols.Add(symbol);
await db.SaveChangesAsync();
var start = new DateTime(2026, 8, 20, 10, 0, 0, DateTimeKind.Utc);
for (var i = 0; i < 4; i++) db.Candles.Add(Candle(symbol.Id, start.AddMinutes(i), 100 + i));
await db.SaveChangesAsync();
var endMs = new DateTimeOffset(start.AddMinutes(2)).ToUnixTimeMilliseconds();

var repo = new CandleRepository(db);
var result = (await repo.GetCandlesAsync(symbol.Id, "1m", 100, endMs)).ToList();

Assert.Equal(2, result.Count);
Assert.All(result, c => Assert.True(c.OpenTime < start.AddMinutes(2)));
}

// Kontrollerer, at en eksisterende candle opdateres i stedet for at blive duplikeret.
[Fact]
public async Task AddOrUpdateCandleAsync_UpdatesExistingCandleInsteadOfDuplicating()
{
    await using var db = TestDbContextFactory.Create();
    var symbol = new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" };
    db.Symbols.Add(symbol);
    await db.SaveChangesAsync();
    var time = new DateTime(2026, 8, 20, 10, 0, 0, DateTimeKind.Utc);
    var repo = new CandleRepository(db);
    await repo.AddOrUpdateCandleAsync(Candle(symbol.Id, time, 100));

    var replacement = Candle(symbol.Id, time, 125);
    replacement.Interval = "15m"; // Repository-lagringen skal normalisere til 1m.
    await repo.AddOrUpdateCandleAsync(replacement);

    var all = db.Candles.Where(c => c.SymbolId == symbol.Id).ToList();
    var saved = Assert.Single(all);
    Assert.Equal("1m", saved.Interval);
    Assert.Equal(125, saved.Close);
}
}

```

T8. Repositories/SymbolRepositoryTests.cs

Tester normalisering, aktive symbols, exchange-filtrering og case-insensitive symbolopslag.

```

using TradingAppLibrary.Models;
using TradingAppLibrary.Repositories;
using TradingAppTest.Helpers;

namespace TradingAppTest.Repositories;

public class SymbolRepositoryTests
{
    // Kontrollerer, at symbolfelter normaliseres før lagring.
    [Fact]
    public async Task AddAsync_NormalizesSymbolFields()
    {
        await using var db = TestDbContextFactory.Create();
        var repo = new SymbolRepository(db);

        var symbol = new Symbol
        {
            ExchangeId = 1,
            Name = " btcusdt ",
            BaseAsset = " btc ",
            QuoteAsset = " usdt ",
            IsActive = true
        };

        await repo.AddAsync(symbol);

        var saved = await repo.GetByIdAsync(symbol.Id);
        Assert.NotNull(saved);
        Assert.Equal("BTCUSDT", saved!.Name);
        Assert.Equal("BTC", saved.BaseAsset);
        Assert.Equal("USDT", saved.QuoteAsset);
    }

    // Kontrollerer, at kun aktive symboler fra aktive exchanges returneres.
    [Fact]
    public async Task GetAllActiveSymbolsAsync_ReturnsOnlyActiveSymbolsFromActiveExchange()
    {
        await using var db = TestDbContextFactory.Create();
        db.Exchanges.Add(new Exchange { Id = 2, Code = "OKX", Name = "OKX", IsActive = false });
    }
}

```

```

db.Symbols.AddRange(
    new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT", IsActive = true },
    new Symbol { ExchangeId = 1, Name = "ETHUSDT", BaseAsset = "ETH", QuoteAsset = "USDT", IsActive = false },
    new Symbol { ExchangeId = 2, Name = "BTC-USDT", BaseAsset = "BTC", QuoteAsset = "USDT", IsActive = true });
await db.SaveChangesAsync();

var repo = new SymbolRepository(db);
var result = (await repo.GetAllActiveSymbolsAsync()).ToList();

var symbol = Assert.Single(result);
Assert.Equal("BTCUSDT", symbol.Name);
Assert.Equal("BINANCE", symbol.Exchange.Code);
}

// Kontrollerer, at symbolsøgning ikke returnerer symboler fra en anden exchange.
[Fact]
public async Task SearchAsync_WithExchangeId_DoesNotLeakSymbolsFromOtherExchange()
{
    await using var db = TestDbContextFactory.Create();
    db.Exchanges.Add(new Exchange { Id = 2, Code = "OKX", Name = "OKX", IsActive = true });
    db.Symbols.AddRange(
        new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" },
        new Symbol { ExchangeId = 2, Name = "BTC-USDT", BaseAsset = "BTC", QuoteAsset = "USDT" });
    await db.SaveChangesAsync();

    var repo = new SymbolRepository(db);
    var result = (await repo.SearchAsync(2, "btc")).ToList();

    var symbol = Assert.Single(result);
    Assert.Equal(2, symbol.ExchangeId);
    Assert.Equal("BTC-USDT", symbol.Name);
}

// Kontrollerer, at opslaget er uafhængigt af store og små bogstaver.
[Fact]
public async Task GetByNameAsync_IsCaseInsensitiveAndTrimsInput()
{
    await using var db = TestDbContextFactory.Create();
    db.Symbols.Add(new Symbol
    {
        ExchangeId = 1,
        Name = "BTCUSDT",
        BaseAsset = "BTC",
        QuoteAsset = "USDT"
    });
    await db.SaveChangesAsync();

    var repo = new SymbolRepository(db);
    var result = await repo.GetByNameAsync(1, " btcusdt ");

    Assert.NotNull(result);
    Assert.Equal("BTCUSDT", result!.Name);
}
}

```

T9. Repositories/UserRepositoryTests.cs

Tester login-opslag, soft delete, preferences-upsert og filtrering af slettede brugere.

```

using TradingAppLibrary.Models;
using TradingAppLibrary.Repositories;
using TradingAppTest.Helpers;

namespace TradingAppTest.Repositories;

public class UserRepositoryTests
{
    // Kontrollerer den forventede adfærd for dette testscenarie.
    private static User CreateUser(string username = "ozan", string email = "ozan@test.com") => new()
    {
        Username = username,
        Email = email,
        PasswordHash = "hash",
        FirstName = "Ozan",
        LastName = "Korkmaz"
    };

    // Kontrollerer, at loginopslag virker med både brugernavn og e-mail.
    [Fact]
    public async Task GetByLoginIdentifierAsync_FindsByUsernameOrEmailCaseInsensitively()
    {

```

```

    await using var db = TestDbContextFactory.Create();
    var repo = new UserRepository(db);
    await repo.AddAsync(CreateUser());

    Assert.NotNull(await repo.GetByLoginIdentifierAsync(" OZAN "));
    Assert.NotNull(await repo.GetByLoginIdentifierAsync(" Ozan@Test.Com "));
}

// Kontrollerer, at sletning udføres som soft delete.
[Fact]
public async Task DeleteAsync_PerformsSoftDelete()
{
    await using var db = TestDbContextFactory.Create();
    var repo = new UserRepository(db);
    var user = CreateUser();
    await repo.AddAsync(user);

    await repo.DeleteAsync(user.Id);

    // Kontroller de gemte soft-delete-flags ved at omgå det globale query-filter.
    // Adfærden for det globale filter testes separat i GetAllUsersAsync_ExcludesSoftDeletedUsers.
    var deleted = await db.Users.IgnoreQueryFilters().SingleAsync(u => u.Id == user.Id);
    Assert.True(deleted.IsDeleted);
    Assert.NotNull(deleted.DeletionRequestedAt);
}

// Kontrollerer, at brugerindstillinger oprettes eller opdateres uden dubletter.
[Fact]
public async Task SavePreferencesAsync_UpsertsPreferences()
{
    await using var db = TestDbContextFactory.Create();
    var repo = new UserRepository(db);

    await repo.SavePreferencesAsync(new UserPreferences { UserId = "5", Settings.Json = "{\"theme\":\"dark\"}"});
    await repo.SavePreferencesAsync(new UserPreferences { UserId = "5", Settings.Json = "{\"theme\":\"light\"}"});

    var saved = await repo.GetPreferencesByUserIdAsync("5");
    Assert.NotNull(saved);
    Assert.Equal("{\"theme\":\"light\"}", saved!.Settings.Json);
    Assert.Single(db.UserPreferences);
}

// Kontrollerer, at sletning udføres som soft delete.
[Fact]
public async Task GetAllUsersAsync_ExcludesSoftDeletedUsers()
{
    await using var db = TestDbContextFactory.Create();
    var repo = new UserRepository(db);
    var active = CreateUser("active", "active@test.com");
    var deleted = CreateUser("deleted", "deleted@test.com");
    await repo.AddAsync(active);
    await repo.AddAsync(deleted);
    await repo.DeleteAsync(deleted.Id);

    var users = (await repo.GetAllUsersAsync()).ToList();

    var user = Assert.Single(users);
    Assert.Equal(active.Id, user.Id);
}
}

```

T10. Repositories/WatchlistRepositoryTests.cs

Tester indlæsning af watchlist-relationer, fjernelse af enkelte symbols og sletning af tilhørende items.

```

using Microsoft.EntityFrameworkCore;
using TradingAppLibrary.Models;
using TradingAppLibrary.Repositories;
using TradingAppTest.Helpers;

namespace TradingAppTest.Repositories;

public class WatchlistRepositoryTests
{
    // Kontrollerer den forventede adfærd for dette testscenarie.
    private static User User() => new()
    {
        Username = $"user_{Guid.NewGuid():N}",
        Email = $"{{Guid.NewGuid():N}}@test.com",
        PasswordHash = "hash"
    };
}

```

```
// Kontrollerer, at watchlisten indlæses med items, symbol og exchange.
[Fact]
public async Task GetWatchlistsByUserIdAsync_LoadsItemsSymbolAndExchange()
{
    await using var db = TestDbContextFactory.Create();
    var user = User();
    db.Users.Add(user);
    var symbol = new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT", IsActive = true };
    db.Symbols.Add(symbol);
    await db.SaveChangesAsync();
    var watchlist = new Watchlist { UserId = user.Id, Name = "Crypto" };
    db.Watchlists.Add(watchlist);
    await db.SaveChangesAsync();
    db.WatchlistItems.Add(new WatchlistItem { WatchlistId = watchlist.Id, SymbolId = symbol.Id });
    await db.SaveChangesAsync();

    var repo = new WatchlistRepository(db);
    var result = (await repo.GetWatchlistsByUserIdAsync(user.Id)).ToList();

    var loaded = Assert.Single(result);
    var item = Assert.Single(loaded.Items);
    Assert.Equal("BTCUSDT", item.Symbol.Name);
    Assert.Equal("BINANCE", item.Symbol.Exchange.Code);
}

// Kontrollerer, at kun det valgte symbol fjernes fra watchlisten.
[Fact]
public async Task RemoveItemAsync_RemovesOnlyRequestedSymbol()
{
    await using var db = TestDbContextFactory.Create();
    var user = User();
    db.Users.Add(user);
    var s1 = new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" };
    var s2 = new Symbol { ExchangeId = 1, Name = "ETHUSDT", BaseAsset = "ETH", QuoteAsset = "USDT" };
    db.Symbols.AddRange(s1, s2);
    await db.SaveChangesAsync();
    var watchlist = new Watchlist { UserId = user.Id, Name = "Main" };
    db.Watchlists.Add(watchlist);
    await db.SaveChangesAsync();
    db.WatchlistItems.AddRange(
        new WatchlistItem { WatchlistId = watchlist.Id, SymbolId = s1.Id },
        new WatchlistItem { WatchlistId = watchlist.Id, SymbolId = s2.Id });
    await db.SaveChangesAsync();

    var repo = new WatchlistRepository(db);
    await repo.RemoveItemAsync(watchlist.Id, s1.Id);

    var remaining = await db.WatchlistItems.Where(x => x.WatchlistId == watchlist.Id).ToListAsync();
    var item = Assert.Single(remaining);
    Assert.Equal(s2.Id, item.SymbolId);
}

// Kontrollerer, at tilhørende watchlist-items fjernes sammen med watchlisten.
[Fact]
public async Task DeleteAsync_CascadeDeletesWatchlistItems()
{
    await using var db = TestDbContextFactory.Create();
    var user = User();
    db.Users.Add(user);
    var symbol = new Symbol { ExchangeId = 1, Name = "BTCUSDT", BaseAsset = "BTC", QuoteAsset = "USDT" };
    db.Symbols.Add(symbol);
    await db.SaveChangesAsync();
    var watchlist = new Watchlist { UserId = user.Id, Name = "Delete me" };
    db.Watchlists.Add(watchlist);
    await db.SaveChangesAsync();
    db.WatchlistItems.Add(new WatchlistItem { WatchlistId = watchlist.Id, SymbolId = symbol.Id });
    await db.SaveChangesAsync();

    var repo = new WatchlistRepository(db);
    await repo.DeleteAsync(watchlist.Id);

    Assert.Null(await db.Watchlists.FindAsync(watchlist.Id));
    // EF InMemory håndhæver ikke relationelle cascade-regler. Den vigtige repository-adfærd
    // her er, at parent-posten fjernes; relationel cascade dækkes af skemaet hos den rigtige databaseprovider.
}
}
```